

出版说明

近年来，我国的高等教育特别是计算机学科教育，进行了一系列大的调整 and 改革，急需一批门类齐全、具有国际先进水平的计算机经典教材，以适应当前我国计算机科学的教学需要。通过使用国外先进的经典教材，可以了解并吸收国际先进的教学思想和教学方法，使我国的计算机科学教育能够跟上国际计算机教育发展的步伐，从而培育出更多具有国际水准的计算机专业人才，增强我国计算机产业的核心竞争力。为此，我们从国外知名的出版集团 Pearson 引进这套“国外经典教材·计算机科学与技术”教材。

作为全球最大的图书出版机构，Pearson 在高等教育领域有着不凡的表现，其下属的 Prentice Hall 和 Addison Wesley 出版社是全球计算机高等教育的龙头出版机构。清华大学出版社与 Pearson 出版集团长期保持着紧密友好的合作关系，这次引进的“国外经典教材·计算机科学与技术”教材大部分出自 Prentice Hall 和 Addison Wesley 两家出版社。为了组织该套教材的出版，我们在国内聘请了一批知名的专家和教授，成立了一个专门的教材编审委员会。

教材编审委员会的运作从教材的选题阶段即开始启动，各位委员根据国内外高等院校计算机及相关专业的现有课程体系，并结合各个专业的培养方向，从 Pearson 出版的计算机系列教材中精心挑选针对性强的题材，以保证该套教材的优秀性和领先性，避免出现“低质重复引进”或“高质消化不良”的现象。

为了保证出版质量，我们为这套教材配备了一批经验丰富的编辑、排版、校对人员，制定了更加严格的出版流程。本套教材的译者，全部来自于对应专业的高校教师或拥有相关经验的 IT 专家。每本教材的责编在翻译伊始，就定期不间断地与该书的译者进行交流与反馈。为了尽可能地保留与发扬教材原著的精华，在经过翻译、排版和传统的三审三校之后，我们还请编审委员或相关的专家教授对文稿进行审读，以最大程度地弥补和修正在前面一系列加工过程中对教材造成的误差和瑕疵。

由于时间紧迫和受全体制作人员自身能力所限，该套教材在出版过程中很可能还存在一些遗憾，欢迎广大师生来电来信批评指正。同时，也欢迎读者朋友积极向我们推荐各类优秀的国外计算机教材，共同为我国高等院校计算机教育事业贡献力量。

清华大学出版社

国外经典教材·计算机科学与技术

编 审 委 员 会

主任委员:

孙家广

清华大学教授

副主任委员:

周立柱

清华大学教授

委员(按姓氏笔画排序):

王成山

天津大学教授

王 珊

中国人民大学教授

冯少荣

厦门大学教授

冯全源

西南交通大学教授

刘乐善

华中科技大学教授

刘腾红

中南财经政法大学教授

吉根林

南京师范大学教授

孙吉贵

吉林大学教授

阮秋琦

北京交通大学教授

何 晨

上海交通大学教授

吴百锋

复旦大学教授

李 彤

云南大学教授

沈钧毅

西安交通大学教授

邵志清

华东理工大学教授

陈 纯

浙江大学教授

陈 钟

北京大学教授

陈道蓄

南京大学教授

周伯生

北京航空航天大学教授

孟祥旭

山东大学教授

姚淑珍

北京航空航天大学教授

徐佩霞

中国科学技术大学教授

徐晓飞

哈尔滨工业大学教授

秦小麟

南京航空航天大学教授

钱培德

苏州大学教授

曹元大

北京理工大学教授

龚声蓉

苏州大学教授

谢希仁

中国人民解放军理工大学教授

译者序

我们常常会惊叹于中震撼人心的电影场景和无比逼真的游戏画面，在享受这些前所未有的视觉冲击的同时，难免会让人联想到到底是什么力量在幕后提供支撑。毋庸置疑，这一切都应归功于计算机三维图形学领域所取得的最新成果。

目前，以科学计算可视化、计算机动画和虚拟现实技术为主题的三维图形技术已经在军事、航空、航天、医学、地质勘探、文化艺术以及艺术造型等领域得到了广泛的应用。每种快速发展的技术一定拥有强大工具的支持，正所谓“工欲善其事，必先利其器”。计算机图形学当然也不例外。OpenGL 是一个优秀的三维图形硬件的软件接口，同时也是一个跨平台的、开放性的三维图形和模型库。

自 1992 年 OpenGL 正式成为适于各种计算机环境下的三维应用程序接口以来，它便得到计算机行业巨头的极大关注和支持，先后涌现了大批支持 OpenGL 的图形卡，为计算机硬件的迅速更新起到了推波助澜的作用。它的出现，极大地增强了图形工作者的生产力，使他们从以往充满艰辛的工作中解脱出来，站在更高的抽象层次来思考和解决问题。计算机图形学的发展推动了 OpenGL 的持续发展和不断更新，可以说 OpenGL API 的每个新版本都体现了当时图形学发展的最高水平。目前，OpenGL 在专业绘图和科研领域仍然是占据统治地位的 API，在游戏开发领域也具有极其重要的位置。

本书的作者 Edward Angel 是一位有着多年丰富图形学教学经验的教授，除了在大学授课之外，他还曾经给来自世界各地的数以千计的科学家、工程师和程序员讲授计算机图形学方面的课程，并多次在计算机图形学领域的最高级别会议 SIGGRAPH 上讲授 OpenGL。正是这种授课对象的极端复杂性促使作者对传统的授课理念进行深入的反思和创新，因而大胆地在课堂中采用自顶向下、面向编程的方法来讲授计算机图形学。这种方式使得该门课程一开始便充满了趣味性，充分调动了学生的学习兴趣，使他们一开始就可以动手编写出具有一定难度的三维程序。如此一来，很容易使他们产生成就感并激发进一步深入学习的热情。事实证明，这种方式收到了非常良好的效果。令我们感到高兴的是，作者把他的这种教学理念也完全体现在本书的内容编排之中。相信读者朋友在阅读本书时也会从这种理念中获得愉悦的学习体验。

阅读本书，并不需要太多的先修知识，您只需具有一定的图形学基础，并且熟悉 C 语言的基本语法以及具备基本的线性代数基础(如向量、矩阵的基本运算)即可。本书的突出特点是实践性较强、自成体系，是迅速了解 OpenGL 编程的捷径之一。但须知，只有在实践中不断总结、不断加强巩固和加深对相关的理论的理解，才能在图形学之路上渐行渐远，

修炼成真正的高手，从而真正做到以不变应万变。

在译者翻译本书的过程中，OpenGL 正在酝酿着新一轮的变革，OpenGL 3.0 标准即将出台，新版本按照面向对象的思想将 API 完全重新组织。这无疑是一件好事，但其间必然会经历一番“阵痛”，因此正在学习“旧版本”OpenGL 的您大可不必因此而惊慌失措，因为以往的 API 虽然是面向过程的，但基本思想不会变，以往的知识积累仍然大有用武之地。

作为译者，本人在翻译过程中力求忠实于原作。书中所涉及的所有专业术语，都查阅了大量国内外相关文献，并请教了许多专家学者，力求找到最准确、最完美的译法，并在可能引起歧义或冲突的地方做了适当调整。同时，在专业术语第一次出现或在容易混淆的环境中出现时，都注上了该术语的英文原文，以便读者进行对比理解。尽管如此，由于计算机图形学发展迅速，每年都有大量新名词涌现，许多都尚未规范化，如果读者朋友对哪些术语的译法有不同的见解，非常乐意与我一同探讨。

特别感谢清华大学出版社第三事业部。在与他们合作的过程中，始终非常愉快。特别要感谢文开琪编辑，是她给了我许多非常有价值的建议，让我受益颇多。本书翻译工作的顺利完成，离不开她的帮助和指点。

翻译过程中，好友陈正华、段恺、郭山、李登、李硕、廖超、刘永、王汉奎、严严、朱云峰、李德武、姜媛媛、李黎明、夏琳等均参与其中，为我提供相当多的帮助和建议。限于译者水平，书中错误和不妥在所难免，恳请读者朋友们批评指正，欢迎发送邮件到 duanfeis@gmail.com。

关于译著，本书特别在书后保留原书主题索引和函数索引，并在译著的相应地方用粗体方括号标注其原书页码，以期进一步帮助读者查阅和掌握相关主题。

最后祝愿大家在精彩无限的图形编程之旅中，一帆风顺，日益精进！

段 菲

2008 年 3 月于北京清华园

献给 **Rose Mary**

前言

在 2000 年当我编写本书的第 1 版时,我希望它既可作为我的图形学教科书 *Interactive Computer Graphics: A Top-Down Approach Using OpenGL*(第 4 版, Addison-Wesley 于 2005 年出版)的配套教材,也希望它能够自成体系,为已经掌握了一些计算机图形学知识,而且希望了解 OpenGL 的开发人员提供一本 OpenGL 入门指南。对于第二种读者群,我并不是想要取代 OpenGL 的标准参考书:“红宝书”(OpenGL Programming Guide, 第 5 版, Addison-Wesley 出版)和“蓝宝书”(OpenGL 1.4 Reference Manual, 第 4 版, Addison-Wesley 出版)。相反,我希望奉献一本能够使程序员无需参考这些价值不菲的参考书便能快速入门的书。

我认为本书的第 1 版达到了我所期望的目标。很多情况下,我惊奇地发现,第二类读者群(至少以我所接收到的反馈来看)的数量超过了我的预期。我还惊奇地发现,本书已经作为一本独立的教科书为各种层次的图形学课所采用。

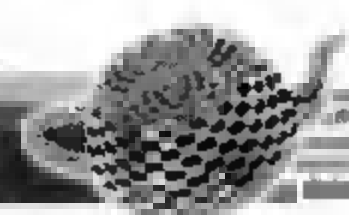
当我开始考虑编写本书的第 2 版时,我不得不做出一些艰难的决定,因为我必须对一本我认为已经非常成功的书籍进行修订。我希望这本书仍然保持简捷的风格,以将价格维持在一个较低的层次上,但我也希望对一些用户的反馈做出反应,同时希望体现图形学和 OpenGL API 的最新进展。为了实现这些看上去自相矛盾的愿望,本书的第 2 版与第 1 版相比,篇幅只增加了 20%。在第 2 版中,我添加了一些新例子。另外我们专门增加了一章用于介绍可编程图形流水线,该工具代表了计算机图形学的一个主要进展。

可编程流水线的进一步发展促成了 OpenGL 2.0,该版本中包含 OpenGL 着色语言(OpenGL Shading Language, GLSL),它是促使我编写本书第 3 版的主要动机。我所面临的挑战是在短短一章中通过一些简单而生动的示例介绍 GLSL 的一些核心内容。

本书遵循了自顶向下的教学理念,这一理念是我在 *Interactive Computer Graphics* 一书中引入的。这种方法所基于的思想是如果学生能够尽快地开始编写一些有意义的应用程序,则他们在学习现代计算机图形学时就能达到最好的效果。OpenGL API 非常有助于这一方法的实现。全世界的许多大中专院校都采用这种方法,充分表明这种教学理念的成功。

使用过 *Interactive Computer Graphics* 一书的学生都会有这样的认识:虽然这本书大量使用了 OpenGL,但它绝非一本 OpenGL 编程指南,更不是一本用户手册。所以,在这本教材中,关于 OpenGL 的信息实际上是不完整的。并非所有的 OpenGL 函数都被涵盖,书中也没有函数及其参数的详细清单。对学生来说,前面的形式问题不大,但后面的形式确实会带来一些不便。于是,本书的诞生就填补了这个缺憾,这样学生们就不必去购买非常有参考价值但却价格不菲的“红宝书”(OpenGL Programming Guide)和“蓝宝书”(OpenGL 1.4 Reference Manual)。

大学里选修图形学课程的学生只是图形学社区中规模很小的一个群体,如果放在整个编程社区,则比例更小。许多人至少都对计算机图形学产生过一时的兴趣,而且也编写过



图形程序。对他们来说,使用 OpenGL 是涉足图形学领域的一种非常诱人的方式。本书编写的第二个动机是为读者学习 OpenGL 提供一条捷径。在我看来,即使对那些最终将在 Windows 平台上使用 DirectX 进行开发的人,从研究和使用 OpenGL 开始,并借助 OpenGL 来掌握一些图形学概念,这种学习方式更容易,效果也更好。对于那些像我们一样长期从事科学应用,而且需要在跨平台环境中工作的读者,OpenGL 是一个绝佳的选择。

本书自始至终几乎都没有介绍我那本图形学教材中所用到的数学原理。所以在很多章节中,例如第 9 章,我们更关注编程实现的细节,而没有做任何数学推导。第 5 章介绍了如何使用旋转、平移及比例变换,但是并未对其中涉及的矩阵做任何推导。本书的章节编排顺序大致与我那本图形学教材一致,但同时也遵循了 OpenGL 的循序渐进的学习规律。我们首先从研究二维问题入手,这是第 2 章的内容,紧接着在第 3 章中我们转而研究程序的交互性,在第 4 章和第 5 章中,我们对三维程序进行了研究。第 6 章则介绍光照和材质。第 7 章和第 8 章介绍如何使用 OpenGL 来显示离散实体,我们首先讨论了像素和位图,接着又对纹理映射进行了详细的讨论。第 9 章介绍曲线和曲面。第 10 章介绍 OpenGL 着色语言(GLSL),这种语言提供了一种 C 风格的编程方式,借助它我们可在应用程序中对可编程显卡进行编程。第 11 章涉及 OpenGL 的一些高级特性。

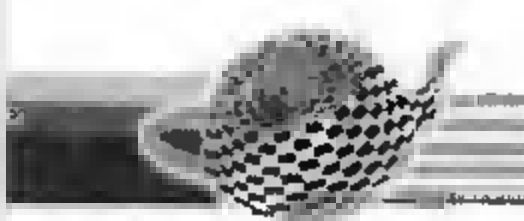
本书中既有完整的程序代码,也有代码片段。您在阅读本书时将发现,一旦您编写了少量 OpenGL 应用程序之后,许多代码都会在后续的程序中重复出现。所以,在介绍完前面几个例子后,我们将大部分重复代码都省略。读者可从我的个人主页 www.cs.unm.edu/~angel 下载所有示例的完整代码。

在本书的编写过程中,得到了很多人的大力支持。我要感谢现在或曾经在 Silicon Graphics 公司工作的许多人在学习 OpenGL 中给予我的热忱帮助。特别要感谢 Mark Kilgard 和 Mason Woo,当我在新墨西哥大学所开的图形学课程中最初采用 OpenGL 时,他们给予了我许多帮助。Mason、Kathleen Danielson、Dave Shreiner 以及 Vicki Shreiner 曾邀请我与他们在过去 9 年中,在 SIGGRAPH 一同讲授 OpenGL 的入门课程,正是通过这种形式的讲授迫使我对 OpenGL API 进行更深入的学习,并帮助我获得了如何教授 OpenGL 的一些心得。Mark、Nate Robins 以及 Brian Paul,他们通过构建 GLUT 库、Mesa 实现以及许多 OpenGL 示例程序,对我们这些使用和讲授 OpenGL 的人提供了非常大的帮助。本书第 1 版从编写到出版只经历了很短的时间,目的是满足图形学社区的一些需求。我非常感激那些对本书提出意见和印刷错误的读者。在编写本书第 2 版时,Mark Kilgard(NVIDIA)和 Dave Shreiner(SGI)对我的手稿进行了细致的审阅。他们给予了我太多的恩惠,因为他们对 OpenGL 开展的研究工作不知给予了我多少启示。现在我仍在非常快乐地享受着他们的恩惠。在我学习和研究 GLSL 时,Takeshi Hakamata 也给予了我莫大的帮助,我都欠他们一份人情。

我陆续在 Addison-Wesley 出版公司出版了 7 本著作,我时常惊叹于该公司工作人员的能力和专业技术水准。来自 Addison-Wesley 的我的编辑 Matt Goldstein、Maite Suarez-Rivas 以及 Peter Gordon,他们都是我的好友,我们曾经在一起共进午餐。我的妻子和好搭档 Rose Mary Molnar,与我一同经历了另外一本书的出版周期,她理应获得更多的嘉奖。

目 录

第 1 章 绪论.....1	
1.1 OpenGL API.....2	
1.2 关于 OpenGL 的三种观点.....3	
1.2.1 程序员的观点.....3	
1.2.2 OpenGL 状态机.....4	
1.2.3 OpenGL 绘制流水线.....4	
1.3 OpenGL 的组成.....5	
1.4 OpenGL 的版本和扩展.....6	
1.5 语言.....6	
1.6 编程约定.....6	
1.7 编译.....8	
1.8 资源.....9	
1.9 本书的适用对象.....10	
1.10 全书概览.....11	
第 2 章 OpenGL 中的二维编程.....13	
2.1 一个简单的示例程序.....14	
2.2 GLUT.....15	
2.3 事件循环和回调函数.....16	
2.4 矩形的绘制.....17	
2.5 修改 GLUT 中的默认值.....19	
2.6 OpenGL 中的颜色.....20	
2.6.1 颜色的设置.....20	
2.6.2 颜色和状态.....21	
2.7 OpenGL 和 GLUT 坐标系之间的差异.....21	
2.8 二维取景.....22	
2.9 视口.....23	
2.10 坐标系与变换.....23	
2.11 simple.c(第 2 版).....24	
2.12 图元及其属性.....26	
2.12.1 点.....27	
2.12.2 直线.....27	
2.12.3 启用 OpenGL 特性.....29	
2.12.4 填充的图元.....29	
2.12.5 矩形.....31	
2.12.6 多边形的点划模式.....31	
2.13 多边形类型.....31	
2.14 颜色插值.....34	
2.14.1 离散处理与边标记.....34	
2.14.2 离散化与细分.....35	
2.15 文本.....43	
2.16 查询与错误.....44	
2.17 状态的保存.....46	
2.18 编程练习.....47	
第 3 章 交互与动画.....49	
3.1 重绘回调函数.....50	
3.2 Idle 回调函数.....51	
3.3 一个旋转的矩形.....52	
3.4 双缓存.....54	
3.5 键盘的使用.....54	
3.6 鼠标回调函数的使用.....56	
3.7 鼠标的移动.....59	
3.8 菜单.....60	
3.9 NULL 回调函数.....62	
3.10 子窗口与多窗口.....62	
3.11 例程: single_double.c.....63	
3.12 显示列表.....66	
3.12.1 多个显示列表.....67	
3.12.2 显示列表与文本.....68	
3.12.3 显示列表与对象.....69	
3.13 拾取和选择模式.....69	



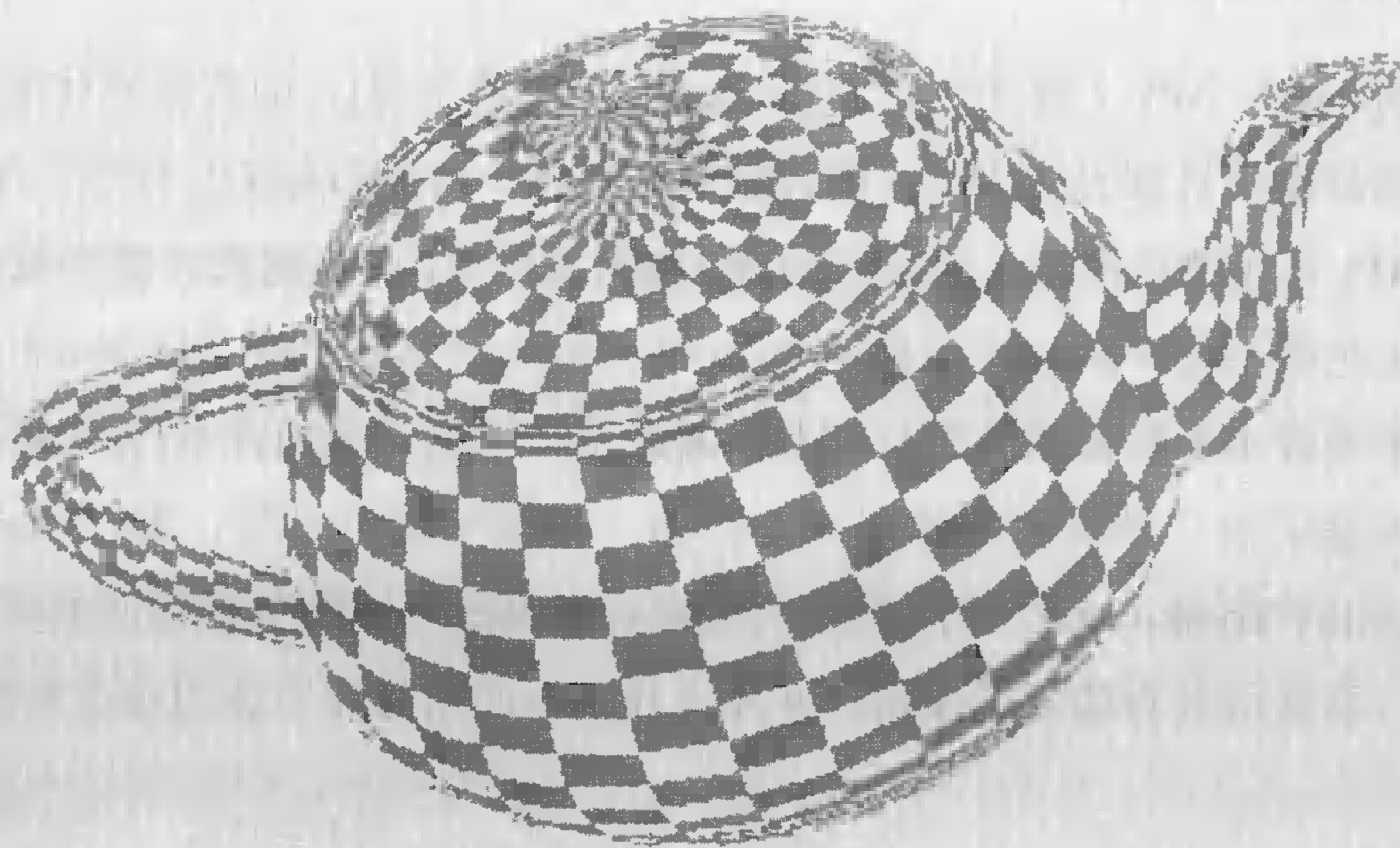
3.14 编程练习	74	6.2.2 镜面反射	122
第 4 章 三维编程基础	75	6.2.3 环境反射	123
4.1 摄像机与对象	76	6.2.4 发射光	123
4.2 OpenGL 中的正交投影	79	6.3 OpenGL 中的光照	123
4.3 观察一个立方体	80	6.4 光源的指定	124
4.4 摄像机的定位	81	6.5 材质的指定	128
4.5 对象的生成	84	6.6 旋转立方体的明暗计算	130
4.5.1 数组的使用	85	6.7 对明暗计算的控制	133
4.5.2 顶点数组	86	6.8 平滑着色	134
4.6 消隐	88	6.9 法线的处理	135
4.7 GLU 与 GLUT 对象	89	6.10 透明度	135
4.7.1 GLU 二次曲面	90	6.11 编程练习	139
4.7.2 GLUT 对象	92	第 7 章 离散图元	141
4.8 透视投影	94	7.1 像素和位图	142
4.9 编程练习	96	7.2 位图	143
第 5 章 几何变换	97	7.2.1 位图的显示	144
5.1 线性变换	98	7.2.2 位图和几何图元的融合	146
5.2 齐次坐标	98	7.2.3 颜色与模板	147
5.3 模型-视图变换与投影变换	99	7.3 绘制模式	148
5.4 平移	99	7.4 像素的读/写	151
5.5 旋转变换	102	7.4.1 像素的写操作	152
5.6 比例变换	103	7.4.2 像素的读取	152
5.7 一个旋转的立方体	104	7.4.3 像素的复制	153
5.8 直接设置矩阵	106	7.5 缓存的选择	154
5.9 变换与坐标系	109	7.6 像素存储模式	155
5.10 基于变换的建模	110	7.7 PPM 图像的显示	155
5.10.1 实例化	110	7.8 灰度图像的使用	162
5.10.2 层次模型	111	7.9 像素映射	162
5.11 编程练习	118	7.10 像素的缩放	164
第 6 章 光照与材质	119	7.11 OpenGL 中的图像处理	165
6.1 光照与材质之间的交互	120	7.12 编程练习	166
6.2 Phong 模型	121	第 8 章 纹理映射	167
6.2.1 漫反射	122	8.1 什么是纹理映射	168
		8.2 纹理图的创建	169

8.3 纹理坐标.....	172	10.4 GLSL 语言基础.....	221
8.4 纹理参数.....	173	10.4.1 限定符.....	221
8.5 一个带有纹理的旋转立方体.....	175	10.4.2 运算符.....	223
8.6 将纹理映射到表面.....	179	10.4.3 控制结构.....	224
8.7 边界与尺寸调整.....	180	10.4.4 内置函数.....	224
8.8 多级渐进纹理.....	180	10.4.5 采样器.....	225
8.9 纹理坐标的自动生成.....	182	10.5 建立与应用程序之间的接口.....	226
8.10 纹理对象.....	186	10.5.1 着色器对象的创建.....	226
8.11 用于图像操作的纹理图.....	187	10.5.2 着色器的读取和编译.....	227
8.12 编程练习.....	190	10.5.3 编译和连接.....	228
第 9 章 曲线与曲面.....	191	10.5.4 错误检查.....	229
9.1 参数曲线.....	192	10.5.5 将数据送入着色器.....	229
9.2 参数曲面.....	194	10.6 顶点着色器示例.....	230
9.3 贝塞尔曲线和曲面.....	195	10.7 片段着色器示例.....	232
9.4 一维 OpenGL 求值器.....	196	第 11 章 总结与展望.....	237
9.5 二维求值器.....	198	11.1 OpenGL 各版本及扩展.....	238
9.6 一个交互式例程.....	199	11.1.1 OpenGL 1.1 版本.....	238
9.7 其他类型的曲线.....	201	11.1.2 OpenGL 1.2 版本.....	238
9.7.1 B 样条.....	202	11.1.3 OpenGL 1.3 版本.....	239
9.7.2 NURBS 曲线.....	206	11.1.4 OpenGL 1.4 版本.....	239
9.8 犹他壶.....	206	11.1.5 OpenGL 1.5 版本.....	239
9.9 法向量与明暗.....	211	11.1.6 OpenGL 2.0 版本.....	240
9.10 为表面映射纹理.....	213	11.1.7 OpenGL 2.1 版本.....	240
9.11 编程练习.....	214	11.2 OpenGL 扩展.....	240
第 10 章 OpenGL 着色语言.....	215	11.3 一些附加的 OpenGL 特性.....	240
10.1 对流水线的回顾.....	216	11.4 其他缓存.....	241
10.2 着色器与着色语言.....	217	11.4.1 累积缓存.....	241
10.2.1 RenderMan.....	217	11.4.2 模板缓存.....	242
10.2.2 Cg 与 GLSL.....	218	11.4.3 片段测试.....	243
10.3 GLSL.....	218	11.5 编写可移植、高效而又健壮的 代码.....	243
10.3.1 执行模型.....	219	索引.....	245
10.3.2 一个简单的顶点着色器.....	219		
10.3.3 一个简单的片段着色器.....	220		

第 1 章

绪 论

OpenGL 是一套应用程序编程接口(Application Programming Interface, API)。借助该 API, 程序员可编写出对图形硬件具有访问能力的程序。对程序员来说, OpenGL 有两个重要优点。首先, OpenGL 非常接近底层硬件, 使得用 OpenGL 编写的程序具有较高的运行效率; 其次, OpenGL 易于掌握和使用。在本章中, 我们将对 OpenGL 进行概述: 它适于(或不适于)做哪些工作、它的组织结构以及我们如何对其进行表示。



1.1 OpenGL API

在现代计算机的诸多应用中, 计算机图形学是一个很重要的方面。无论是在我们访问网页、玩互动游戏还是使用 CAD 软件包设计房屋, 我们无一不在与计算机图形学打交道。随着硬件和软件的速度以及复杂性的日益增长, 我们所使用的图形程序也“水涨船高”。这些应用程序的开发者创建应用程序时需要借助标准的软件接口。这样的接口使程序员可以不必为实现那些许多应用程序所共有的标准功能一次次重复编写代码, 而且使程序员在编写应用程序时不必关心图形硬件的细节。这样, 我们的开发效率越来越高, 与此同时, 程序的可移植性也得到了提升。程序员可以借助一套具有精心定义的接口的函数来与图形系统进行交互, 我们将这套函数称为应用程序编程接口(Application Programming Interface, API)。

近年来涌现出许多图形 API。其中的一部分例如 GKS 和 PHIGS, 已经上升到国际标准的水平, 其他一部分 API 在特定的领域也得到了广泛应用。其中的大多数都惨遭夭折。另外一部分 API, 如微软公司的 DirectX, 只局限于特定的平台。OpenGL 来自于一个称为 GL(Graphics Library 的缩写)的接口, 该接口最初是为 SGI 公司的硬件开发的。【1】

GL 的简单易用和功能强大得到了广泛的认可。它构成了 OpenGL 的基石, 如今 OpenGL 已为多种图形硬件所支持。OpenGL 包含 200 多个可用于构建应用程序的函数。使用 OpenGL 编写的程序可被移植到任何支持该接口的计算机。在几乎所有的计算机和操作系统中都有 OpenGL 的相应实现。这些实现方式跨越了从纯软件实现到充分发挥目前最先进的硬件的加速功能的实现。一个典型的 OpenGL 应用程序可运行在具有任意实现方式的平台中, 所要做的只是将针对该系统的 OpenGL 库重新进行编译。此外, OpenGL 还具有高度稳定性, 这就保证了用 OpenGL 编写的程序具有很长的生命期。即便 API 随着硬件的发展而不断演化, 情况仍然如此。

OpenGL API 主要关注绘制(rendering, 也称呈现); 即依据物体的规格参数以及属性, 借助虚拟摄像机和光照生成一幅该物体的图像。由于 OpenGL 程序的平台无关性, 所以 OpenGL API 不包含任何输入或窗口函数。原因很简单, 因为这两者都严重依赖于特定的平台。然而, 无论图形程序运行在何种平台上(Windows, Linux 还是 Macintosh), 都不可避免地要和操作系统或本地窗口系统进行交互。面对这种情况, 我们并不打算屈从于编写平台相关的代码, 而是采取一种折衷的策略——借助一个简单的工具集, 即 OpenGL 实用工具集(OpenGL Utility Toolkit, GLUT)。该工具集在标准编程环境中都有相应的实现, 其 API 包含大多数窗口系统所共有的标准操作, 并允许我们在应用程序中使用键盘和鼠标。

1.2 关于 OpenGL 的三种观点

虽然从某个层面上来看, OpenGL 只是一个具有访问硬件能力的函数库, 如果能够从三种关联的方式来看待 OpenGL, 还是很有意义的。理解这些观点将有助于我们更好地理解 OpenGL 和图形系统是如何发挥功能的, 从而能更轻松地创建自己的应用程序。

1.2.1 程序员的观点

一般而言, 大多数图形应用程序主要包括如下三要素:

- 指定绘制对象
- 描述对象的属性
- 定义这些对象被观察的方式

现代图形系统支持两种不同的对象类型: **几何类型和图像类型**。**几何对象**通常存在于三维空间中, 即**包括如直线、点、多边形等简单对象**, 也包括如二次曲面或更一般的曲线和曲面的复杂实体。从图形学发展的历史看, 计算机图形系统只与几何对象打交道, 本书的绝大多数内容也主要关注这类对象。然而, 随着硬件技术的迅速发展, 图形系统目前也完全能够对图像进行处理。**图像实际上就是像素的矩形数组**, 其中的每个元素可用来表示像素的颜色、灰度或其他属性。

【2~3】

大多数 API 将对象的定义(例如是直线, 还是多边形)与其显示方式进行了分离。这样, 一条绿色的实线就与一条红色的虚线属于同一对象类型, 仅仅是外观上存在差异。虽然这种分离貌似违背了面向对象的思想, 但这样做与硬件的实际工作方式更为接近, 而且使开发人员可将程序的逻辑流程与指定描述表面属性的诸多参数的冗长细节隔离开来。

计算机图形学遵循了现代物理和工程模型, 因为图形学将对象的描述和行为与其显示方式进行了分离。为了生成一幅图像(image)或图片(picture), 我们必须同时对对象和形成图片的摄像机或视点方位进行描述。在 OpenGL 中, 有一小部分函数专门用于对虚拟摄像机进行定位和对其进行描述。

虽然所有产生图形输出的程序必须包含以上三个要素, 如果某个程序是交互式的, 它还必须包含一些输入函数。最后, 所有的程序必须具有一些针对本地操作系统和窗口环境的初始化和终止的函数。

以上观点给出了一种 OpenGL 函数的分类方法, 在下一节中, 我们将对 OpenGL 的函数进

行分类。但是，这并没有揭示 OpenGL 的实际工作原理。另外两种观点可为我们带来一些启发。

1.2.2 OpenGL 状态机

现在我们换一种角度，将 OpenGL 看作是一个具有输入和输出的状态机(state machine)，如图 1.1 所示。其中输入为几何对象的描述信息(例如线段和多边形)和离散对象(如位数组)，它们都是通过 OpenGL 函数调用来指定的。该状态机的输出为一个离散图元的集合或构成图像的像素集合。在输入和输出之间，是一台能够获取对象的描述信息、并将这些信息转化为一幅图像的机器。这台机器处理输入的方式取决于其内部状态。按照这种观点，OpenGL 具有两种类型的函数，即那些指定输入对象的函数以及那些修改机器状态的函数。状态修改函数包括那些指定颜色、观察条件、材质属性以及许多其他变量的函数。状态决定了输入的处理方式。

【3~4】



图 1.1 作为状态机的 OpenGL

1.2.3 OpenGL 绘制流水线

从实现角度，我们可以不同的方式来看待 OpenGL 程序的行为。OpenGL 的基础是一个所谓的流水线模型(pipeline model)。图 1.2 展示了一个简单的流水线。所有的实时系统和商业显卡都包含一个硬件流水线。几何图元，如多边形，可在程序中通过描述空间位置或顶点来指定其形状而由应用程序生成。这些顶点流经一系列模块，每个模块在图元经过时对其实施一种或多种操作。几何模块负责对流经的图元实施一种或多种变换，包括旋转、平移、缩放以及相对于 OpenGL 摄像机为图元进行定位。该模块也被用来决定某一对象对虚拟摄像机的可见性。位于绘制流水线末端的帧缓存(frame buffer)包含在显示设备上将要显示的像素信息。位于几何处理器(geometry processor)和帧缓存之间的是光栅化器(rasterizer)，它负责生成片段(fragment)，或者说那些可能位于几何对象中的像素以及一个片段处理器(fragment processor)，该处理器可对这些片段上色，并判别这些片段是否被其他对象所阻挡以及它们在图像中是否真的可见。

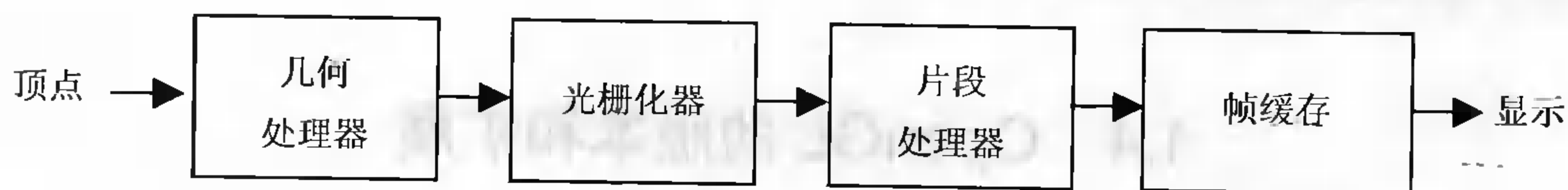


图 1.2 简单的图形流水线

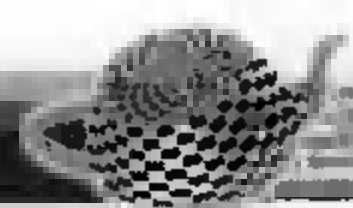
我们可将该绘制流水线视为 OpenGL 状态机的某种特定实现。更重要的是，流水线模型与目前绝大多数图形硬件系统的底层工作方式极为接近。流水线模型同时也强调了处理图像时的相对独立性，即每个图元都可独立地决定自己的颜色和可见性，而不依赖于任何其他图元。所以，我们往往需要在流水线体系的高吞吐量和集成依赖于图元间相互关系的全局效果(如阴影)之间做一取舍。

1.3 OpenGL 的组成

OpenGL 包含 200 多个函数。依据功能对这些函数进行分类无疑是有意义的。

- **图元函数(primitive function)**指定要生成屏幕图像的图元。包括两种类型：可在二维、三维或四维空间进行定义的几何图元，如多边形；离散实体，如位图。【4】
- **属性函数(attribute function)**负责控制图元的外观。这类函数定义了颜色、线型、材质属性、光照以及纹理。
- **观察函数(viewing function)**指定摄像机的属性。OpenGL 提供一个虚拟摄像机，我们可相对于由图元函数定义的对象设置该摄像机的位置和朝向。我们也可以控制摄像机的镜头参数，以便制造出广角或长焦效果。
- **控制函数(control function)**允许我们启用或禁用各种 OpenGL 特性，如消隐、纹理映射和光照等。
- **查询函数(query function)**允许我们决定某一特定系统的性能以及 OpenGL 状态变量的取值。
- **输入与窗口函数**不属于 OpenGL 核心库，但是由于它们在交互式程序中的重要地位，这类函数已被包含在 GLUT 库中。这些函数使我们能够对屏幕中的窗口进行控制并使用鼠标和键盘。

注意，以上各类函数除第一类外都是状态修改函数，但是仅凭这些函数自身无法改变显示效果。



1.4 OpenGL 的版本和扩展

直到 2006 年, OpenGL 一直由体系结构评审委员会(ARB, Architecture Review Board)所掌控, 该委员会的许多成员来自像 SGI、IBM 和 Microsoft 这样的公司。在 2006 年, ARB 由 OpenGL 工作组(OpenGL Working Group)所取代, 该工作组目前是 Kronos Group 标准化组织的一部分。OpenGL 目前的最新版本是 2.1。OpenGL 非常稳定, 使大部分用先前版本开发的程序具有极好的向后兼容性。OpenGL 的性能反映了当前大多数图形系统中的可用功能, 即使某种特定平台缺乏实现某种特性的硬件支持, 它们仍然可用软件方式来替代实现。对于本书的大部分内容, 我们仅用到了 1.2 版所提供的功能。在第 10 章中, 我们将介绍 OpenGL 着色语言(OpenGL Shading Language, GLSL), 它是 OpenGL 最近几个版本的关键特性之一。目前也有一些 OpenGL 的专用版本, 如 OpenGL ES, 该版本是专为嵌入式系统设计的。

许多高端用户会接触到一些具有特殊性能的设备, 或者有利用一些非通用的特定功能的需求。这种情况下, OpenGL 仍然是开发人员的极佳选择。开发人员可以借助 OpenGL 扩展(OpenGL extension)来应对以上情况, 该扩展目前已进入审核阶段, 但尚未成为官方制定的 OpenGL 规范的一部分。例如, 图像处理方面的扩展就是 OpenGL 非常流行的一个扩展方向, 它主要体现在对像素操作的扩展上(第 7 章将集中讨论这部分内容), 只是该扩展常常需要借助高端图形工作站的专用硬件特性。

【5】

1.5 语 言

我们将使用 OpenGL 的 C 语言绑定, 这是目前最流行的做法。另外还有一个官方的 Fortran 语言绑定版本。很遗憾, 对于其他较为流行的编程语言, 如 Java, 虽然可以在互联网上找到一些非官方的 Java 绑定, 但目前尚未推出官方绑定。对于一些脚本语言, 如 Python 和 Perl, 目前也有一些非官方绑定。

1.6 编 程 约 定

OpenGL 函数包含在两个库中, 这两个库通常称为 `gl` 和 `glu`(或 `GL` 和 `GLU`)。GL 库又称为 OpenGL 核心库, 它包含全部必需的 OpenGL 函数。GLU 库, 即 OpenGL 实用库(OpenGL Utility Library), 它包含许多用核心库中的函数编写的新函数, 这些函数会为用户带来极大的便利。核心库中的函数都以 `gl` 为前缀, 例如 `glVertex3f()`, 而实用库中的函数都以 `glu` 为前缀, 例如 `gluOrtho2D()`。

OpenGL 不是面向对象的，而且我们打算使用其 C 语言绑定。所以，该 API 就无法利用面向对象语言(如 C++)的一些特性，如重载。当然，开发人员完全可以使用 C++来编制面向对象的应用程序，并与 OpenGL 的 C 实现进行链接即可。为了支持 C 程序员所使用的各种数据类型，许多 OpenGL 函数都具有多种形式。例如，函数 `glVertex3f()`接收 `float` 类型的参数，而 `glVertex3i()`函数以 `int` 类型为输入参数。我们将使用记号 `glVertex*()`来泛指所有形式的顶点函数。

许多 OpenGL 函数的参数只取自一些较小的离散整数集。为了避免使用一些略显“诡异”的数字，OpenGL 为这些值定义了若干宏，这些宏定义可从 `gl.h` 和 `glu.h` 中找到。前缀 `GL_` 和 `GLU_` 定义了某个宏来自哪个包含文件，例如 `GL_LINES` 和 `GL_LIGHT0`。

OpenGL 程序中通常使用类 C 数据类型，包括 `GLfloat`，`Gldouble` 和 `GLint` 等。包含文件 `gl.h` 定义了这些基本类型，与标准的 C 类型 `float`，`double` 和 `int` 相一致。虽然这些类型的定义与您所设想的一致，如：

```
#define GLfloat float
```

但在 OpenGL 函数中为了保证良好的可移植性，通常都使用这种“新”数据类型。例如，以下函数：

```
void glVertex3f(GLfloat x, GLfloat y, GLfloat z)
```

用三个 `GLfloat` 类型的数定义了三维空间中的一个顶点。可能偶尔会看到一些不太熟悉的类型如 `GLclampf`(表示取值限定在 0.0~1.0 的浮点数)和 `GLsizei`(表示当前版本中可用的非负大整数)。

【6】

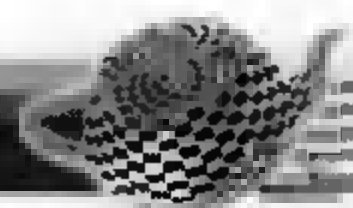
由于有时我们希望通过指针传递顶点数据，为此，OpenGL 为许多顶点函数提供了以指针为形参的替代函数。这些函数在声明时，在参数列表前往往带有字母 `v`。例如，对于函数 `glVertex3fv()`，我们就可对其进行如下调用：

```
GLfloat point[3];
glVertex3fv(point);
```

本书中，在对函数进行描述时，通常使用尖括号(<>)和词“`type`”来指定函数的形式。例如，我们可将各种形式的 `glVertex()`函数表示为：

```
glVertex<234><sifd>(type coords, ...);
glVertex<234><sifd>v(type * coords);
```

使用该函数时，我们必须选择维数(2、3 或 4)和数据类型(short : s, integer : i, float : f 或 double : d)。type 应与所选择的数据类型一致。对于指针形式的函数，我们需要传递类型正确的指针。例如，下面是一些合法的函数调用示例：



```
GLint ix, iy;
GLfloat x, y, z, point[3];

glVertex2i(ix, iy);
glVertex2f(x, y);
glVertex3f(x, y, z);
glVertex3fv(point);
```

在最后一种形式的调用中, `point` 必须为如下形式的数组:

```
float point[3];
```

本书中, 描述每个函数的细节时都采用了如下描述框的形式:

```
void glFunctionName(GLint param)
```

该函数并不存在, 但 `param` 的值可以为 `GL_TRUE` 或 `GL_FALSE`.

该描述框会在某一函数首次引入时出现。同时, 在描述框中有时也会有一些提示或提示信息, 例如:

提示 请阅读此格式提示框中的内容。

【7】

1.7 编 译

一般来说, 编译 OpenGL 程序不会有太大的问题。在 Unix 系统下, 典型的命令行编译的形式为:

```
cc myapp.c -o myapp -lgl -lglu lglut -lm -lx11
```

在 Linux 下, 几乎可以使用完全相同的命令行来编译, 惟一的区别在于必须指定 X 库的位置。通常需要增加一个表示路径的标记, 例如:

```
-L/usr/X11R6/lib
```

还可以将如下 4 行编译命令写入可在 Unix 和 Linux 下工作的一个简单的 makefile:

```
CC = cc
LDLIBS = -lglut -lGL -lGLU -lx11 -lm -L/usr/X11R6/lib
.c:
    $(CC) $@.c $(LDLIBS) -o $@
```

makefile 允许您通过命令行 `make progname` 来编译一个单独文件, 该命令首先读取文件 `progname.c`, 最终生成二进制文件 `progname`。有时, 您可能需要在加载行添加 `-lXmu`。

如果您的 X 库安装在非标准位置, 编译时可能还必须增加标记 `-L`; 如果您的 OpenGL 包含

文件也处在非标准位置, 编译时必须增加标记-I。

如果使用的是 Windows 平台, 并且使用了 Visual C++, 则需要创建一个控制台应用程序。为了顺利编译, 应确保文件 OpenGL32.dll 和 glu32.dll 已经位于 system 文件夹中。相应的 lib 文件应设为..\\vc\\lib。包含文件应位于..\\vc\\include\\gl 中。对于其他的编译器(如 Borland), 也需要完成一系列类似的设置。

为了编译本书中的例程, 还需要从互联网下载 GLUT 文件。可从 www.opengl.org 下载该文件。对于 Windows 系统, 文件 glut.h、glut32.lib 和 glut32.dll 应与其他 OpenGL 文件的位置一致。大多数 Linux 版本中都已经提供 GLUT 库。

如果您工作在 Mac OSX 平台下, 那么需要做些小的改动。文件 glut.h 需要按照如下方式包含:

```
#include <GLUT/glut.h>
```

而非像大多数其他平台所使用的如下形式:

```
#include <GL/glut.h>
```

【8】

注意, glut.h 已经包含 gl.h 和 glu.h, 所以在您的程序中几乎不再需要任何多余的包含文件。对于 OSX, makefile 中的链接行应包含以下形式:

```
-framework OpenGL -framework -GLUT
```

1.8 资源

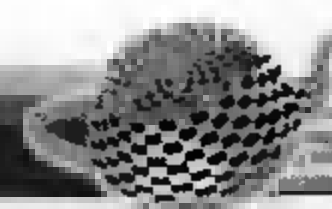
有关 OpenGL 的资源非常丰富。首先, 有许多相关书籍。其中最重要也最为人们所熟知的当属“红宝书”和“蓝宝书”, 它们分别是:

- OpenGL Architecture Review Board, *OpenGL Programming Guide* (5th Edition), Addison-Wesley, 2005
- OpenGL Architecture Review Board, *OpenGL Reference Manual* (4th Edition), Addison-Wesley, 2004

其中, 红宝书中的例程都利用了 GLUT 库。

如果您想在 X 窗口系统中进行开发, 不妨参考 Kilgard, M., *OpenGL Programming for the X Windows System*, Addison-Wesley, 1996。

如果您正在开发 Windows 或 Mac 平台下的应用程序, 请参考 Wright, R, B. Lipchak,



OpenGLSuperBible, (4th Edition), Sams, 2004。

本书假定您对计算机图形学已有一些了解。如果您还需要一本使用 OpenGL 的计算机图形学入门书籍, 请参考 Angel, E., *Interactive Computer Graphics, A Top-Down Approach with OpenGL*(4th Edition), Addison-Wesley, 2006。

计算机图形学的标准参考书我们推荐 Foley, J., A. Van Dam, S. Feiner, J. Hughes, *Computer Graphics, Principles and Practice*(2nd Edition), Addison-Wesley, 1990。

您也可从许多站点获取相关资源。最好的莫过于 OpenGL.org 站点(www.opengl.org), 该网站有许多相关书籍、代码、文件和文档的链接。关于本书的支持, 可从 Addison-Wesley 的网站 www.awl.com 或作者的主页 www.cs.unm.edu/~angel 获得。

如果您的系统中没有 OpenGL, 可以使用一个称为 Mesa 的开源版本, 该版本的源代码和库随大多数 Linux 版本一起发布。Mesa 也可从 www.mesa3d.org 获取。您可从很多地方获取 GLUT 库。OpenGL 网站上有相关的链接。它通常包含在大多数 Linux 版本中并与 Mesa 绑定在一起。

Nate Robins 提供了一些非常有帮助的 OpenGL 教学程序, 这些程序清楚地解释了取景、变换、纹理映射以及少量其他主题。这些例程都是交互式的, 使您有机会尝试各种各样的 OpenGL 函数和参数。这些例程可从 www.xmission.com/~nate/tutors.html 获取。【9】

另外还有许多很好的演示程序。红宝书中的所有例程都是可获取的, 这些例程已经包含在 Mesa 中。在 Mesa 和 GLUT 中还有一些附加的演示和测试程序。

1.9 本书的适用对象

在购买本书之前, 希望您能仔细阅读一下“前言”。但如果您是随手翻到这里的, 我们还想提醒您本书适用的读者。这是有关 OpenGL API 的一本入门书。虽然在接下来的 10 章中我们涉及许多内容, 但我们并不打算涵盖所有的 API。如果您需要全面的介绍和文档, 请参阅红宝书和蓝宝书。如果您想立即上手, 本书是您的理想选择。

我们假定您已具有一些计算机图形学的基本知识。我们尽量避免所有的数学推导, 那应该是读者在学习计算机图形学时应做的工作。除了以上提到的两本书外, 还有一些非常优秀的计算机图形学教科书。如果您正在学习计算机图形学, 并打算使用 OpenGL, 本书将会很有帮助。

1.10 全书概览

在第2章中，我们从研究二维问题开始。虽然在 OpenGL 中，二维图形学是三维图形学的一种特例，但从分析二维问题入手相对容易一些。在这章中，我们将介绍所有的 OpenGL 图元及其属性。

第3章主要关注交互式程序。虽然输入函数并非 OpenGL 核心的一部分，我们仍可用 OpenGL 来开发交互式应用程序。我们将用 GLUT 库来编写一些具有可移植性的简单交互式程序。

第4章和第5章介绍了 OpenGL 的三维计算机图形学。我们将内容分为两部分。在第4章中，我们将介绍如何由基本的 OpenGL 图元构建三维对象。我们还将介绍由 GLU 库和 GLUT 库所提供的一些附加三维对象。我们将讨论如何通过指定 OpenGL 摄像机的参数以及如何相对于该摄像机为场景中的对象定位，以生成这些对象的二维图像。在第5章中，我们将介绍 OpenGL 强大的变换功能。我们将展示如何用变换来创建具有任意大小、朝向和位置的基本对象的实例。我们也将看到如何利用变换来建立对象各部件之间的关系的模型。

在第6章中，我们将借助 Phong 光照模型来介绍光照-材质的交互作用，该光照模型是用 OpenGL 实现的。我们还将讨论如何在 OpenGL 中处理半透明表面。【11】

第7章和第8章主要关注 OpenGL 在位和像素操作方面的性能。在第7章中，我们将讨论位模式的直接显示以及如何在 OpenGL 的各种缓存间移位。在第8章中，我们将介绍纹理映射，该特性允许我们将 OpenGL 的几何和位操作特性结合起来。

在第9章中，我们对平面多边形的讨论将告一段落，转而开始学习如何使用 OpenGL 的曲线和曲面。

第10章介绍了 OpenGL 着色语言(GLSL, OpenGL Shading Language)，该特性使程序可以访问最新的显卡中的可编程流水线(Programmable Pipeline)。

第11章回顾了 OpenGL 的各个版本，并介绍了 OpenGL 的其他一些特性。

本书中既有代码片段，也有完整的 OpenGL 示例程序。完整的程序可从作者的主页中获取。

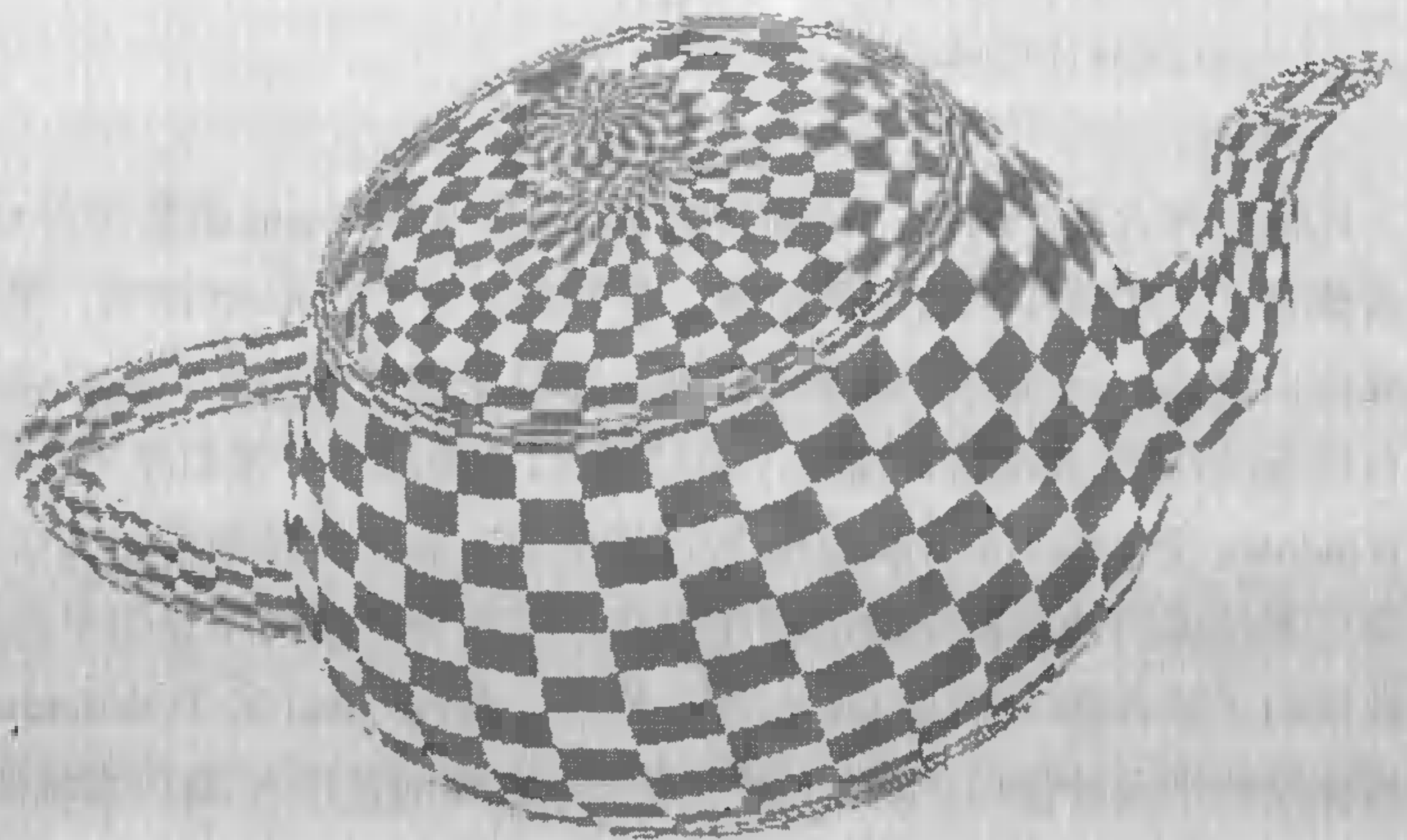
最后想说的是，图形编程乐趣无穷。愿您在学习图形学的过程中获得充分的享受！

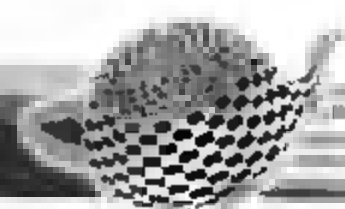
第 2 章

OpenGL 中的二维编程

OpenGL 是一个三维图形系统。从开发者的角度看，OpenGL 图元可用来描述三维世界中的三维对象。二维问题(即当所有的物体共面时)是三维问题的一个特例。尽管如此，该特例仍然相当重要，因为许多应用程序都是二维的。对于这些类型的应用程序，直接在二维中开展工作相对容易一些，这也是 OpenGL 支持的。我们从探讨这类简单问题入手，因为它们提供了一条直接切入 OpenGL 编程的捷径。

在本章中，为了帮助您理解 OpenGL 程序的基本要素，我们将剖析一段非常简单的程序。之后，我们会通过不断引入一些其他的 OpenGL 函数以对该程序进行完善和功能扩展。最后，我们将介绍完整的 OpenGL 基本图元集。





2.1 一个简单的示例程序

我们的第一个程序 `simple.c`，将在黑色背景中绘制一个白色矩形。虽然该程序大量使用许多参数的默认值，仍然有利于阐明大多数 OpenGL 程序的结构。

```
/*simple.c*/

#include <GL/glut.h>
/* for Mac OSX use #include <glut/glut.h> */

void display()
{
    glClear(GL_COLOR_BUFFER_BIT);

    glBegin(GL_POLYGON);
        glVertex2f(-0.5, -0.5);
        glVertex2f(-0.5, 0.5);
        glVertex2f( 0.5, 0.5);
        glVertex2f( 0.5, -0.5);
    glEnd();

    glFlush();
}

int main(int argc, char ** argv)
{
    glutInit(& argc, argv);
    glutCreateWindow("simple");
    glutDisplayFunc(display);
    glutMainLoop();
}
```

【13~14】

上述程序包含两个函数：`main()`和`display()`。其中`main`函数负责 OpenGL 的初始化，`display`函数定义了所要绘制的图形实体。首先我们来研究`main`函数。虽然 OpenGL 没有包含输入或窗口命令，任何用户程序都必须与窗口系统进行交互，而且交互式程序必须能够接受来自设备(如鼠标或键盘)的输入。但是窗口系统的用户接口是平台相关的。一个为 Microsoft Windows 平台编写的 OpenGL 程序操纵窗口和使用鼠标的方式必然与在 Linux 平台下为 X 窗口系统编写的同样功能的程序存在很大差异。我们可使用平台相关的库中所包含的最少数量的“粘合剂”(例如 GLX 之于 X 窗口系统，wgl 之于 Windows，agl 之于 Macintosh)来搭建这些平台的接口。图 2.1 展示了一个典型的使用 X 窗口系统的 Linux 平台下的 OpenGL

库及其与应用程序之间的接口。对于 Windows 或 Macintosh 平台，也有一张类似的组织架构图。应用程序既可以使用 OpenGL、GLU 以及 GLX 库中的函数，也可以直接使用 X 库或 X 工具集中的函数。当然，任何对 X 特定库的函数调用都会使应用程序难以移植到其他平台中。

另外还有一种折衷的方法是借助 GLUT 库，这样可使同一个程序能够在不同的平台下重新编译和运行。

【14】

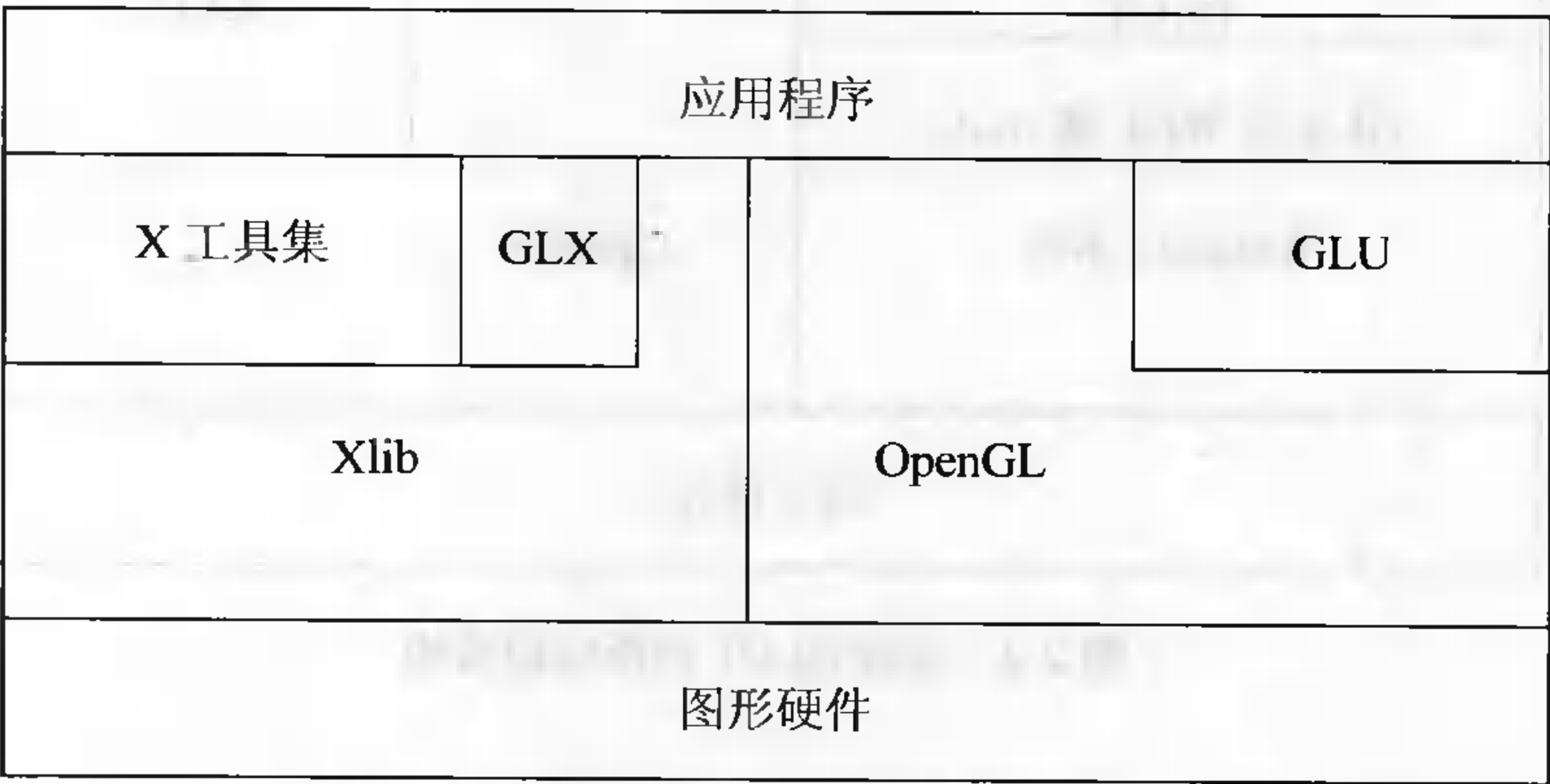


图 2.1 Linux 平台下实现的库组织结构

2.2 GLUT

OpenGL 实用工具集(GLUT, OpenGL Utility Toolkit)是一个体现了几乎所有现代窗口系统所共有的功能的函数库。GLUT 在所有流行的平台中都有相应的实现，这就使得用 GLUT API 编写的交互式程序(包含输入和窗口的程序)无需修改源代码即可在不同平台下进行编译。该库的组织结构如图 2.2 所示。该示意图中的应用程序仅使用了 GL、GLU 和 GLUT 库。依据平台的不同，GLUT 将使用 glx, wgl 或 agl。我们在整本书中都将使用 GLUT。还有一些工具集构筑在 GLUT 的基础上，它们可以提供一些附加的功能。但是，如果对程序的性能有较高的要求或者想要使用某一特定工具集，最好将程序的交互部分设计为平台相关的。

【15】

`void glutInit(int * argc, char ** argv);`
初始化 GLUT，应在其他 GLUT 函数和 OpenGL 函数之前进行调用。`glutInit()`接收来自 main 函数的参数，程序可以具体实现相关的方式来使用这些参数。

从 main 函数中的第一个函数 `glutInit` 开始，我们看到的几个函数名均以 `glut` 为前缀，实际上所有的 GLUT 函数都以 `glut` 为前缀。虽然我们可以通过 main 函数传递命令行参数，但在 GLUT 内部，对这些参数的解释是依赖于具体实现的。在我们的示例程序中，并不打算使用任何与具体实现相关的参数。函数 `glutCreateWindow()` 将一个窗口以默认尺寸(300×300 像素)显示在屏幕的默认位置上(屏幕的左上角)。该函数的参数使我们可为窗口创建一个标题。

`int glutCreateWindow(char * title)`
在屏幕上创建一个窗口，该窗口的标题由参数 `title` 给出。在多窗口情况下，该函数返回一个标识所创建窗口的整数。

在后面的章节中，我们将会看到如何修改 GLUT 中的默认参数。

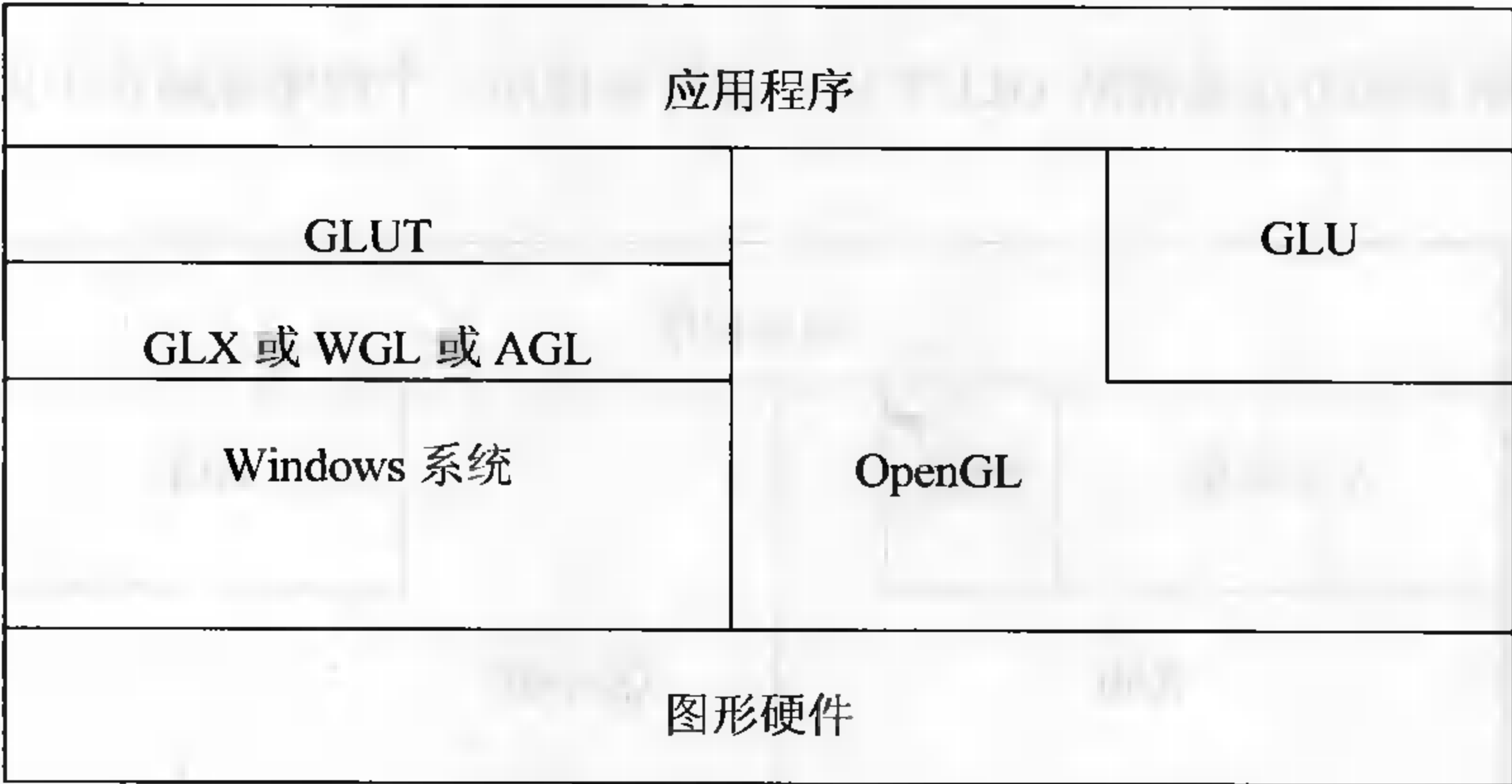


图 2.2 带有 GLUT 的库组织结构

2.3 事件循环和回调函数

大多数交互式程序需要对各种离散事件做出适当的响应。事件通常包括鼠标事件(如移动鼠标或单击鼠标)、键盘事件(如按键)和窗口事件(如用户对窗口尺寸的调整或窗口之间的遮挡)。按照事件处理编程的惯例，通常将事件放入事件队列(event queue)，并由窗口系统来对事件进行处理。事件依照事件队列的顺序依次得到处理。**开发人员所要做的仅仅是编写一些定义了程序对具体事件响应方式的回调函数(callback function)。**在 GLUT 中，只能处理一些基本事件。程序往往需要定义自己的回调函数，对于少数事件，可以采用默认的回调函数或不做任何处理(在这种情况下没有定义回调函数的事件将被忽略)。

在程序 `simple` 中，仅有一个回调函数，即显示回调函数(display callback)。只要 OpenGL 确定窗口需要进行重绘，该函数便会被调用。需要重绘的一种情况是当窗口首次被打开时。所以，如果将 OpenGL 绘制命令放在显示回调中，我们就可确保绘制至少会完成一次。请注意回调函数的形式，它是一个无参数的函数，需要用函数 `glutDisplayFunc` 来指定或注册(register)。

【16】

`void glutDisplayFunc(void (* func)(void))`
函数 `func()` 在每次需要对窗口重绘时被调用。

如果我们需要为显示回调函数传入一些值，可在程序中借助全局变量。在回调函数注册完毕之后，程序通过执行函数 `glutMainLoop` 而进入事件循环。一旦进入该循环，除非借助回

调函数或一些外部干预(如按下一个“终止”键),否则无法跳出该循环。main 函数中的任何位于该函数之后的代码将永远无法得到执行的机会。

一些编译器也许会坚持认为正确的 C 需要主函数有一个返回值,OpenGL main 函数应该以如下代码作为结尾:

```
glutMainLoop();
return 0;
}
```

我们并不打算采用这种形式,因为我们不希望插入一些根本得不到执行的代码,即便从技术角度来看也许是正确的。

提示 GLUT 回调函数的形式是固定的。所以,若要在不同的函数之间进行值传递,可能需要借助全局变量。

```
void glutMainLoop()
```

使程序进入无限事件处理循环。该函数的调用应作为 main 函数最后一条语句出现。

2.4 矩形的绘制

现在我们已经有了自己的 main 函数,还需要定义自己的显示回调函数,不妨将其命名为 display。首先,注意我们包含了文件 glut.h,该文件通常存放在名为 GL 的目录下,这正是 OpenGL 的标准包含文件所在的路径。该文件包含了 GLUT 函数的原型以及 OpenGL 程序中需要用到的各种宏定义。该文件还包含如下两行指令:

```
#include <GL/gl.h>
#include <GL/glu.h>
```

目的在于包含 OpenGL 和 GLU 函数及常量的相似定义。

用于指定几何对象的基本实体是顶点,即空间中的某一个位置。简单的几何对象,如直线和多边形,可通过顶点集合来指定。通过使用函数 glVertex*()函数的各种变体,OpenGL 使得我们可以在二维、三维和四维空间中指定顶点的位置。【17~18】

```
void glVertex<234><sifd>(type xcoordinate, type ycoordiante, ...)
```

```
void glVertex<234><sifd>v(type * coordinates)
```

指定顶点在二维、三维或四维空间中的位置坐标,坐标值的类型可以是 short(s)、int(i)、float(f)或 double(d)。如果函数名的末尾出现了 v,则 type 为一个指向指定类型的坐标数组的指针。

我们将使用记号 glVertex*()来表示所有这些变体函数。例如,glVertex2f(x,y)在二维空间中

定义了一个坐标为(x, y)的顶点, 其中 x 和 y 都为浮点型; glVertex2fv(p)指定了一个以数组 p 的前两个元素为位置坐标的顶点, 即(p[0], p[1])。

由于顶点可用来定义各种各样的对象, 我们必须通知 OpenGL 顶点列表究竟定义了何种类型的对象, 并且要标明列表的起始和终止位置。可借助函数 glBegin()和 glEnd()来完成这种指定。

void glBegin(GLenum mode)

指定 mode 类型对象的起始点。mode 可取 GL_POINTS、GL_LINES 或 GL_POLYGON

void glEnd()

指定顶点列表的终止点。

提示 在顶点列表的末尾不要忘记包含 glEnd()。

利用这三个函数, 我们可以定义一个矩形:

```
glBegin(GL_POLYGON);
    glVertex2f(-0.5, -0.5);
    glVertex2f(-0.5, 0.5);
    glVertex2f(0.5, 0.5);
    glVertex2f(0.5, -0.5);
glEnd();
```

OpenGL 将绘制的图像放在一个称为颜色缓存(color buffer)的内存区域中, 该缓存通常存在于显卡中。颜色缓存是构成帧缓存(frame buffer)的多种类型的缓存之一。颜色缓存的内容可以显示在屏幕中。在后面的内容中, 我们将看到其他类型的缓存。在绘制矩形前, 应利用函数 glClear()清除我们所要使用的颜色缓存。【18】

void glClear(GLbitfield mask)

清空 mask 指示的所有缓存中的位。其中 mask 可由定义在 gl.h 中的常量通过逻辑或运算构成。GL_COLOR_BUFFER_BIT 指颜色缓存。

出于效率的考虑, OpenGL 将绘制命令做了缓冲处理, 我们可通过调用 glFlush()函数强制绘制器将结果立即输出。

void glFlush()

强制执行之前缓存的所有 OpenGL 命令。

图 2.3 展示了本程序的输出结果。虽然该程序显然是输出了一幅图像, 但仍有几个需要阐明的问题, 具体如下。

- 如何改变图像的尺寸?
- 如果改变图像在屏幕中显示的位置?

- 为什么白色矩形占据了窗口区域一半的面积？
- 为什么背景是黑色的而矩形是白色的？我们如何使用其他颜色？
- 是否可以一些不同于窗口系统所提供的关闭按钮的方式终止程序？
- 如何定义更为复杂的对象？

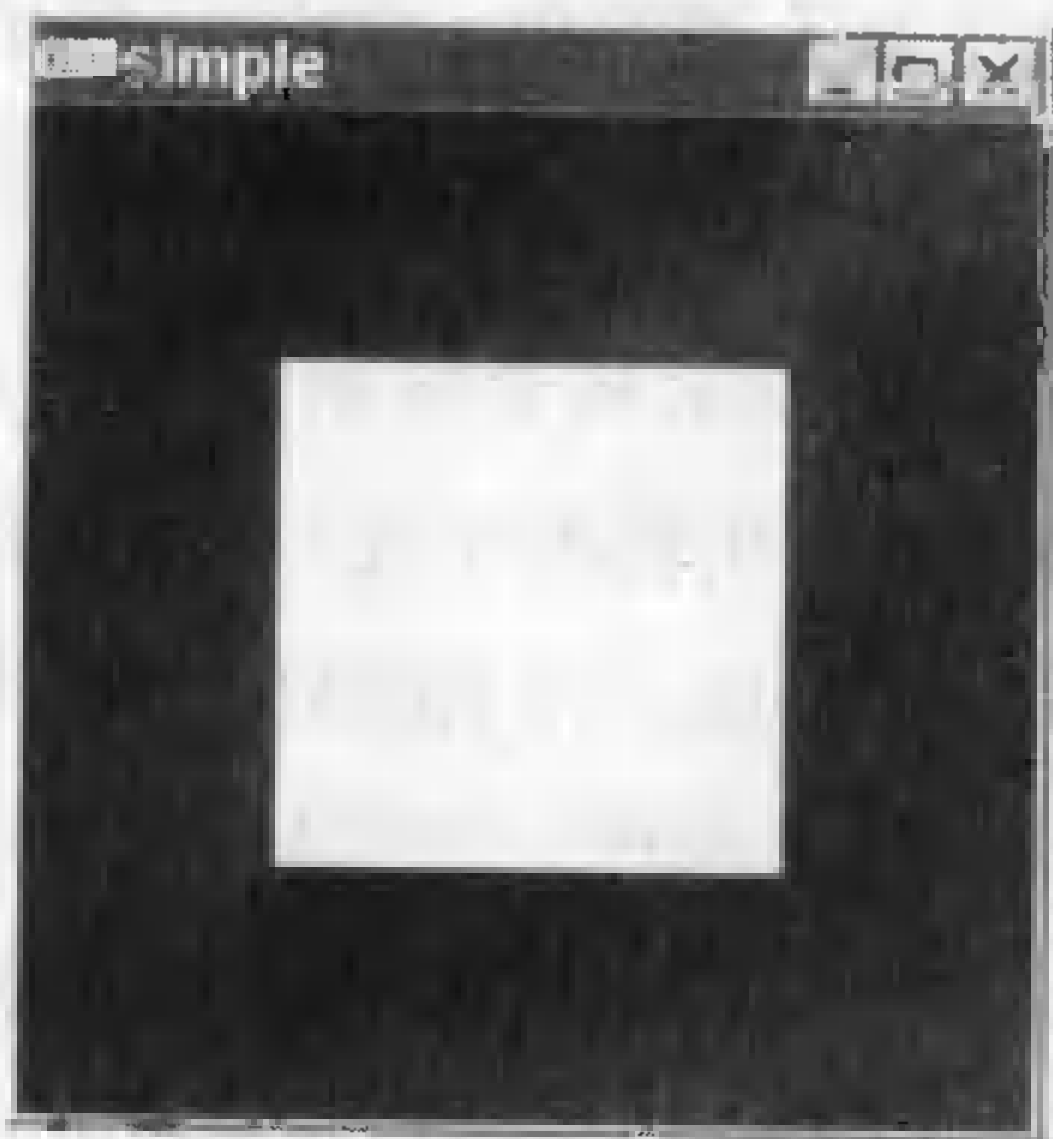


图 2.3 simple.c 程序的输出结果

【19】

2.5 修改 GLUT 中的默认值

首先，我们来添加少量 GLUT 函数以更好地控制显示在屏幕中的图像。函数 `glutInitDisplayMode()`、`glutInitWindowSize()` 以及 `glutInitWindowPosition()` 指定用户所需的窗口类型、尺寸及位置。通常每种实现都支持与屏幕窗口相关的各种属性。应用程序可以借助函数 `glutInitDisplayMode()` 来请求它所需要的窗口类型。所要指定的最常见的窗口属性是希望使用的颜色类型或是否需要双缓存(参见 3.4 节)。GLUT 中的默认值为 RGB 颜色和单缓存，当然也可通过如下方式显式指定：

```
glutInitDisplayMode(GLUT_RGB | GLUT_SINGLE);
```

```
void glutInitDisplayMode(unsigned int mode)
```

请求 `mode` 所表示的显示模式。`mode` 的值可以是一些选项如颜色模式 (`GLUT_RGB`, `GLUT_INDEX`) 和颜色缓存的缓存模式 (`GLUT_SINGLE`, `GLUT_DOUBLE`) 用逻辑或组合而成。

函数 `glutInitWindowSize()` 指定了屏幕中窗口的初始尺寸，函数 `glutInitWindowPosition()` 给出了窗口的初始位置。

```
void glutInitWindowSize(int width, int height)
```

指定屏幕中窗口的初始宽度和高度，单位为像素。

```
void glutInitWindowPosition(int x, int y);
```

指定窗口的左上角相对于屏幕左上角的位置，单位为像素。

2.6 OpenGL 中的颜色

OpenGL 支持两种基本的颜色模型：RGB(或 RGBA)模式和颜色索引模式。在 RGB 模式中，每种颜色都是红、绿、蓝三种颜色组成的三元组。人眼将这些主要的颜色(或颜色分量)进行叠加或混合，就形成了我们所看到的各种颜色。加性模型适合于监视器和投影设备。印刷业通常使用减色模型，相关内容在大多数图形学教材中都可找到。

如果我们用实数来指定颜色，则 0.0 表示某个分量没有“贡献”，1.0 表示该分量的值达到上限。所以，RGB 三元组(1.0, 0.0, 0.0)就表示亮红色，(0.5, 0.5, 0.0)就表示暗黄色，(1.0, 1.0, 1.0)表示白色，(0.0, 0.0, 0.0)表示黑色。在 RGBA 模型中，我们还将使用第 4 个颜色分量——A 或 alpha，该分量用于表示不透明度。若某种颜色的不透明度为 1.0，则表明该颜色是完全不透明的，即我们无法“看透”它，而大小为 0.0 的不透明度表明该颜色是完全透明的。我们目前暂不需要不透明度，在后面的章节中我们再详细讨论。如果我们用整型数来表示颜色，则该颜色的取值范围为 0 到该类型可取的最大值。例如，如果我们将颜色值类型设为无符号字节型，则颜色值的范围为 0~255。注意，将颜色值指定为某种精度并不意味着物理显示设备也支持同样的精度。所有即便显示设备只支持两种颜色(即前景色和背景色)，也可以将颜色的每种分量的类型指定为实数型。【20~21】

提示 显示设备的颜色精度可能比用函数 `glColor*()` 所指定的精度低。

在颜色索引模式下，颜色由指向一个由红、绿、蓝三种颜色的值构成的表的索引来指定。在该模式下，我们将所有许可的颜色值组织成一个表格，该表格通常包含 256 种颜色。在过去，颜色索引模式在图形系统中得到了广泛应用。如今，随着价格低廉的高密度存储器的出现，这种模式已经不那么常用了。此外，在颜色索引模式下，用户与窗口系统交互时往往需要关注更多的细节(相较于 RGB 颜色模式)。

2.6.1 颜色的设置

在 simple 程序中，我们使用了颜色的默认值。清屏的颜色默认情况下为黑色而绘制颜色(即用于填充多边形的颜色)默认值为白色。这些颜色可借助函数 `glColor*()` 和 `glClearColor()` 来修改。

```
void glColor3<b i f d ub us ui>(type r, type g, type b)
void glColor3<b i f d ub us ui>v(type * color)
void glColor4<b i f d ub us ui>(type r, type g, type b, type a)
void glColor4<b i f d ub us ui>v(type * color)
```

用标准类型来指定 RGB 或 RGBA 颜色值。如果函数名的末尾出现 v，则颜色值由 color 所指向的数组指

定, 该数组中每个元素的类型均为 `type`。

```
void glClearColor(GLclampf r, GLclampf g, GLclampf b, GLclampf a)
```

指定用于清空颜色缓存的 RGBA 颜色值。

【22】

提示 用于清除操作的颜色必须指定为 RGBA 颜色, 且每个分量的类型必须为 `GLclampf`。

2.6.2 颜色和状态

在 OpenGL 中, 颜色也是 OpenGL 状态的一部分。我们可以认为存在一种当前绘制颜色(current drawing color)和当前清除颜色(current clear color), 其中前者由函数 `glColor*()` 来指定, 后者由 `glClearColor()` 来指定。所以, 颜色与对象并不存在依附关系, 而是作为 OpenGL 的内部状态。用于绘制对象的颜色即为当前绘制颜色。虽然在程序中看起来好像颜色与对象和顶点结合在一起, 实际上在程序定义顶点时, OpenGL 是依据当前状态来查找当前颜色的。程序中哪些地方修改了当前颜色, 程序员必须特别留意。在后面的学习中, 我们将了解如何将颜色与我们所要绘制的对象以更紧密的方式进行绑定。

借助函数 `glColor*()`, 我们可用标准 C 的数据类型来设置 RGB 或 RGBA 颜色值。对于当前颜色, OpenGL 实际上仅保留一种内部形式, 即 RGBA 形式, 每个颜色分量通常是浮点类型的。如果我们使用的是 `glColor3*()`, 实际上取得效果与将 RGBA 颜色的 Alpha 分量设为最大值一致。由函数 `glClearColor()` 所指定的清除颜色必须指定为 RGBA 颜色, 而且每个分量的类型必须为 `GLclampf`。

提示 务必始终保持对当前状态的跟踪, 例如颜色的改变。

【22~23】

2.7 OpenGL 和 GLUT 坐标系之间的差异

在用 OpenGL 编程时, 往往会用到各种坐标系。通常, 用户用对象坐标(object coordinate)来描述几何形状, 这也是应用程序所使用的坐标。对于二维应用程序, 该坐标系中 x 轴的正方向水平向右, y 轴正方向竖直向上。所以, 如果我们将本页的左下角设为原点, 则该页面中所有点的 x , y 轴坐标均为正值。大多数窗口系统所使用的坐标系中的 y 轴正方向都是竖直向下的。在这样的坐标系中, 如果我们想使 x 和 y 坐标只出现正值, 可将坐标系原点设在屏幕的左上角。这种方向的定义是有渊源的, 因为大多数 CRT 显示系统在形成过程中都按照从上到下, 从左到右的顺序, 行数和列数都是从左上角算起的。由于 GLUT 经常与窗口系统进行交互, 它所采取的是第二种形式的坐标系, 所以我们应该将屏幕的左上角视为原点, 像素坐标的正方向为向右和向下。对于 `glutInitWindowPosition()`, 稍微有些不同。在后面的内容中, 当我们使用来自鼠标的输入时, 需要同时考虑两种坐标系, 这很容易产生混淆。

提示 对于二维问题，在 OpenGL 中， x 轴和 y 轴的正方向分别为向右和向上。对于 GLUT 中使用的输入函数和窗口系统， x 轴和 y 轴的正方向分别为向右和向下。

2.8 二维取景

在 simple 程序的第 1 版中，我们使用了默认的取景条件(viewing condition)。在 OpenGL 中，二维问题是二维问题的一个特例。二维对象的空间坐标的形式为 (x, y) ，但是依据 OpenGL 的观点，它们其实是三维坐标 (x, y, z) 中 $z=0$ 时的情形(实际上，OpenGL 是在四维空间中工作的。三维空间是四维空间的一个特例，但是我们尚不必在意这一点)。所以，二维取景问题(如哪些对象以何种尺寸出现在屏幕中)是三维取景问题的一个特例。但是，由于我们希望从二维程序入手，所以我们可以独立地编写简单的二维取景程序。

在取景问题中，我们所使用的基本模型称为合成摄像机模型(synthetic camera model)。该模型模拟了形成对象集合的图像的观察者(观测者或摄像师)以及为了生成一幅图像我们需要在计算机中完成的工作。在第 4 章中，我们将详细探讨该模型。在二维问题中，该模型如图 2.4 所示。

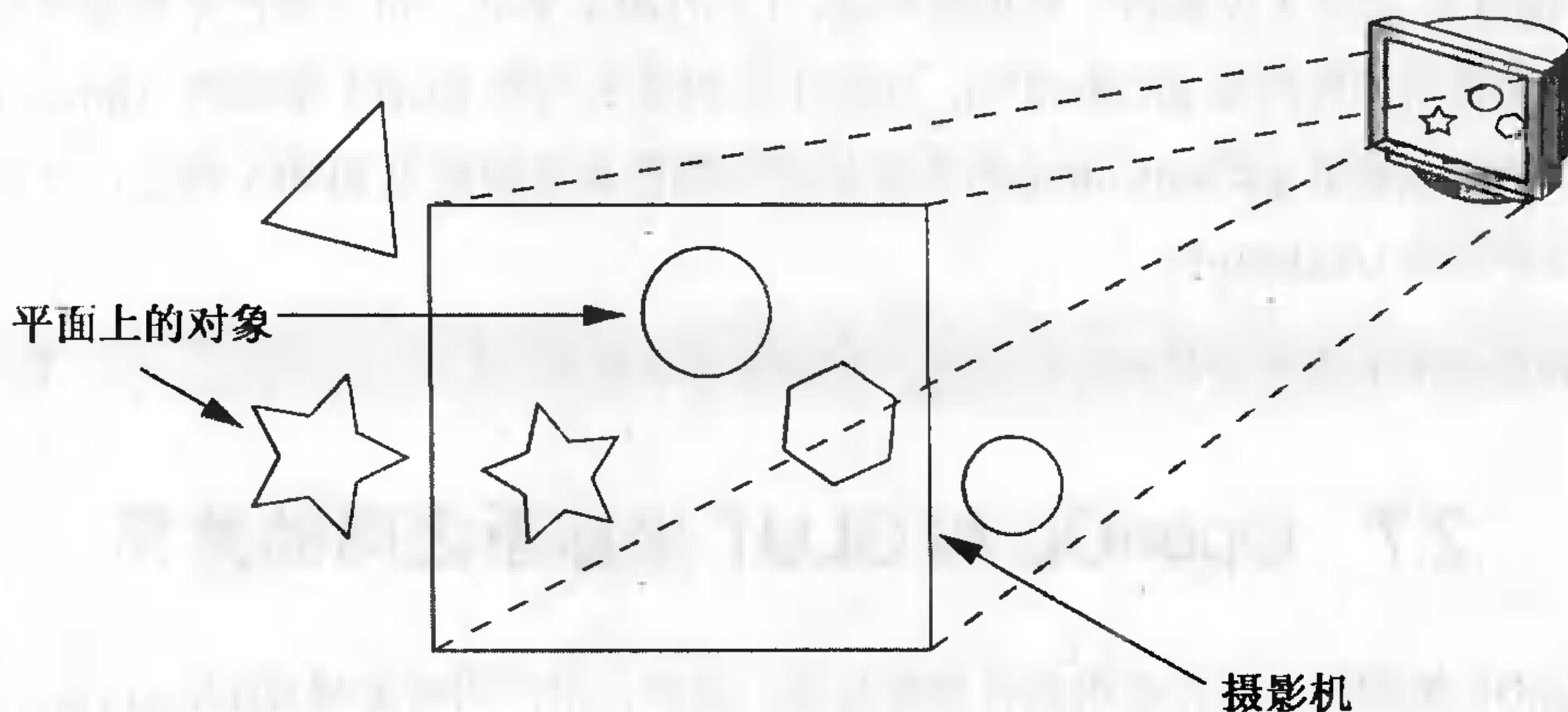


图 2.4 二维取景

在二维中，可通过指定或计算一个顶点集，在程序中使用一些代码的组合——如 `glVertex*()`，`glBegin()` 和 `glEnd()`——来定义我们想要的对象。我们可将代码想象成在一张无限大的纸上来对对象进行描述。取景这一步就是确定这张虚拟纸的哪一部分对于合成摄像机模型是可见的，从而可以出现在屏幕中。如果我们假定摄像机与 x 轴和 y 轴都对齐，则我们只需通过 x 和 y 坐标的最大值和最小值来指定一个矩形区域。这一工作可借助函数 `gluOrtho2D()` 来完成。

`void gluOrtho2D(GLdouble left, GLdouble right, GLdouble bottom, GLdouble top)`
指定一个二维矩形裁剪区域，其左下角坐标为 $(left, bottom)$ ，右上角坐标为 $(right, top)$ 。 【23】

前缀 `glu` 表明该函数属于 `GLU` 库(由于该函数是三维函数 `glOrtho()` 的一个特例)。由 `glOrtho2D()` 所定义的区域称为裁剪窗口(或取景窗口)。位于该窗口内的对象是可见的, 落在该窗口之外的对象则不可见, 我们称其“被裁剪”。

2.9 视 口

到目前为止, 我们一直都是整个屏幕窗口来显示裁剪窗口中的所有对象。我们也可通过使用视口(viewport)来限制 OpenGL 仅在屏幕窗口的任一部分进行绘制。视口是屏幕窗口中的一个矩形区域, 其尺寸的度量单位为像素。我们可借助函数 `glViewport` 来对视口进行设置。

```
void glViewport(GLint x, GLint y, GLsizei w, GLsizei h)
```

将视口的宽度设为 `w`, 高度设为 `h`, 左下角坐标为 `(x, y)`, 单位均为像素。默认情况下, 视口为整个初始窗口。

视口的一种应用是对窗口进行划分, 使得不同类型的信息可被绘制到窗口的不同部分。例如, 在 CAD 应用程序中, 我们可能需要为菜单和命令留出一些视口, 而为构建设计留出另外一些视口。这些部分都可以有各自独立的取景条件, 这些条件可由各视口通过分别调用 `gluOrtho2D` 函数来指定。借助互不重叠的若干视口, 我们可在单个窗口中创建一幅复杂的图像, 与此同时可将代码的复杂性降至最低。

在第 3 章中, 当用户改变窗口形状时, 我们将用视口来控制图像的外观。

【24~25】

2.10 坐标系与变换

到目前为止, 我们已经在函数中使用了两种坐标系。第一个坐标系称为对象坐标系(object coordinate)或世界坐标系(world coordinate)^①。开发人员在程序中所使用的坐标系正是该坐标系。每个程序员都可自由决定选取何种单位, 然后在如 `glVertex*()` 的 OpenGL 函数中指定相应单位的数值。这样, 程序员在 VLSI(超大规模集成电路)设计中就可以微米为单位, 而在天文问题中以光年为单位。第二个坐标系称为窗口坐标系或屏幕坐标系, 该坐标系中的度量单位为像素。窗口坐标的许可范围由物理显示器的属性以及应用程序所选择的显示器中的区域两方面因素决定。

作为绘制过程的一部分, OpenGL 会自动实现对象到窗口坐标系中的变换^②。所需要的信息

① 从技术角度讲, 在 OpenGL 中, 对象是在对象坐标系中指定的, 并且可以通过坐标变换将其转换至世界坐标系中。

② 作为绘制流水线的一部分, OpenGL 在内部还使用了一些其他的坐标系。其中的一些坐标系对于应用程序是不可见的; 对于另外一些坐标系, 如摄像机坐标系, 我们将在第 5 章讨论三维应用程序时用到。

仅仅是屏幕中显示窗口的尺寸以及用户希望显示多大范围的对象空间。前者由 `glutInitWindowSize()` 函数决定(可能会根据与用户的交互而改变), 而后者由函数 `gluOrtho2D()` 设定。

OpenGL 中所需的坐标系变换由两个矩阵决定, 即模型视图矩阵(model-view matrix)和投影矩阵(projection matrix), 这些矩阵也是 OpenGL 的状态的一部分。我们将在第 5 章中详细讨论这些矩阵。但是, 即使是在最简单的程序中, 我们也需要使用一个简单的投影矩阵。函数 `gluOrtho2D()` 用于为二维应用程序指定一个投影矩阵。设置这两种矩阵的典型步骤包括以下三步。

1. 指定我们希望修改的矩阵。
2. 将矩阵设为单位矩阵。
3. 修改当前矩阵为用户所期望的矩阵。

如果想要以增量式来修改已有矩阵, 第 2 步便不需要。所以, 如果我们想设定一个二维裁剪窗口, 使其左下角位于 $(-1.0, -1.0)$, 其右上角位于 $(1.0, 1.0)$ (这些是 OpenGL 的默认值), 我们可执行以下函数:

```
glMatrixMode(GL_PROJECTION);
glLoadIdentity();
gluOrtho2D(-1.0, 1.0, -1.0, 1.0);
```

【25~26】

void glMatrixMode(GLenum mode)

指定哪个矩阵将会受到后续变换函数的影响。mode 通常取 `GL_MODELVIEW` 或 `GL_PROJECTION`。

void glLoadIdentity()

将当前矩阵初始化为单位矩阵。

由于这些矩阵是 OpenGL 状态的一部分, 所以无论何时指定一个图元, OpenGL 都会使用这些状态的当前值。这些矩阵几乎可在程序的任何部分进行修改。在我们的例子中, 尚无用户的交互, 我们可在程序的初始化阶段将矩阵一次性设好。在第 3 章中, 我们会依据用户事件, 如屏幕窗口的尺寸调整, 来修改变换矩阵。

2.11 simple.c(第 2 版)

现在我们将前面所做的各种修改整合一下。整合后运行的效果与第 1 版几乎完全相同, 但是第 2 版的结构更加通用, 而且具有更复杂的二维应用程序的一些特点。

```
/*simple.c second version*/
/*This program draws a white rectangle on a black background.*/
```



```

#include<GL/glut.h> /* glut.h includes gl.h and glu.h */

void display()
{
    /* clear window*/
    glClear(GL_COLOR_BUFFER_BIT);

    /* draw unit square polygon */
    glBegin(GL_POLYGON);
        glVertex2f(-0.5, -0.5);
        glVertex2f(-0.5, 0.5);
        glVertex2f(0.5, 0.5);
        glVertex2f(0.5, -0.5);
    glEnd();

    /* flush GL buffers */

    glFlush();
}

void init()
{
    /* set clear color to black */
    glClearColor(0.0, 0.0, 0.0, 0.0);

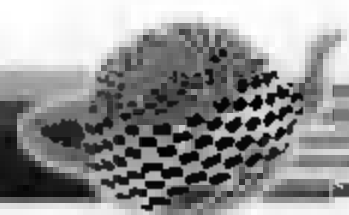
    /* set fill color to white */
    glColor3f(1.0, 1.0, 1.0);

    /* set up standard orthogonal view with clipping */
    /* box as cube of side 2 centered at origin */
    /* this is default view and these statement could be removed.*/

    glMatrixMode(GL_PROJECTION);
    glLoadIdentity();
    gluOrtho2D(-1.0, 1.0, -1.0, 1.0);
}

int main(int argc, char** argv)
{
    /* initialize mode and open a window in upper-left corner
    of screen */
    /* window title is name of program (argv[0])*/
    glutInit(&argc, argv);

```

```
glutInitDisplayMode(GLUT_SINGLE | GLUT_RGB);  
glutInitWindowSize(500, 500);  
glutInitWindowPosition(0, 0);  
glutCreateWindow("simple");  
glutDisplayFunc(display);  
init();  
glutMainLoop();  
}
```

【26~27】

第2版展示了对于全书所有程序我们所使用的基本结构。该程序主要包含以下4个部分。

1. 一个 `main()` 函数。该函数负责初始化 GLUT，将窗口显示在屏幕上，注册回调函数以及进入主循环。
2. 一个 `init()` 函数。该函数负责状态变量的初始化。
3. 一个显示回调函数 `display()`。该函数包含的代码描述了我们所要绘制的对象。
4. 其他处理输入和窗口事件的回调函数。

虽然选择其他结构也是可行的，但以上结构具有一些突出的优点。采用这种结构可使不同程序的 `main` 函数几乎完全相同。而差别仅体现在特定程序所使用的回调函数和菜单上。`init()` 函数允许我们将大量详细的状态信息和期望的参数值放在一处，与几何对象的绘制代码(通常位于显示回调中)以及动画和交互逻辑(通常位于响应的回调函数中)实现了分离。

2.12 图元及其属性

图元是我们在图形系统中所使用的基本实体。我们已经看到多边形是由一个顶点集合定义的。多边形也是一种几何图元。即它存在于某个空间中，即二维或者三维用户自定义空间中，并可由图形系统进行绘制。后面我们将看到 OpenGL 允许我们对几何图元实施一系列变换，以便对图元进行平移、尺寸调整和重新指定朝向。在 OpenGL 中也存在一些非几何图元，例如位图和像素矩形，对它们的处理方式存在很大的差异。我们将在第7章对这类图元进行介绍。

在 OpenGL 中，有三类基本类型的图元：点、直线段和多边形。更复杂的对象可由这些基本图元来构建，或者我们也可以使用 OpenGL 的曲线和曲面(参见第9章)。这些基本的几何图元都是由若干顶点定义的。所以，它们的指定方式与 `simple.c` 中多边形的指定方法完全相同，差别仅在于 `glBegin()` 函数中的参数。

每种图元都有自己的称为属性(attribute)的性质(property)。属性决定了图元由 OpenGL 显示

的方式。例如，在前面的例程中，多边形被绘制成白色的。线段必须以绿色进行绘制，线条宽度可粗可细。在第6章中，我们将介绍一些更为复杂的属性——材质属性，它可决定光线与图元的交互方式。

不要忘记虽然我们已经将属性概念化为对象的一部分，如红色直线、蓝色点等，但实际上 OpenGL 将属性看作其状态的一部分。所以，当我们在 `simple.c` 中生成一个白色多边形时，多边形之所以为白色是因为当前颜色作为 OpenGL 状态的一部分，是白色的。在第3章中，我们将了解如何更容易地将属性与对象进行绑定。 【28】

2.12.1 点

点是最基本的图元。调用 `glBegin()` 时应将参数取为 `GL_POINTS`。每个顶点指定了一个点，位于裁剪窗口之内的点将依据点尺寸属性和当前颜色进行显示，其中点尺寸属性是借助函数 `glPointSize()` 来设定的。

```
void glPointSize(GLfloat size)
```

对点尺寸状态变量进行设定。尺寸的度量单位为屏幕中的像素，默认值为 1.0。

下面的示例中，我们仍使用与上例中完全相同的 4 个顶点，但将每个顶点设为不同的颜色：

```
glPointSize(2.0);
glBegin(GL_POINTS);
    glColor3f(1.0, 1.0, 1.0);
    glVertex2f(-0.5, -0.5);
    glColor3f(1.0, 0.0, 0.0);
    glVertex2f(-0.5, 0.5);
    glColor3f(0.0, 0.0, 1.0);
    glVertex2f(0.5, 0.5);
    glColor3f(0.0, 1.0, 0.0);
    glVertex2f(0.5, -0.5);
glEnd();
```

提示 `glPointSize()` 不能放置在 `glBegin()` 和 `glEnd()` 函数对之间。

2.12.2 直线

在 `glBegin()` 和 `glEnd()` 中定义一条或多条直线段时，`glBegin()` 函数的类型参数可取以下三个值(参见图 2.5)。

- `GL_LINES`: 在 `glBegin()` 和 `glEnd()` 之间的每个连续顶点对指定了一条线段。所以，以下代码：



```
glBegin(GL_LINES);  
    glVertex2f(-0.5, -0.5);  
    glVertex2f(-0.5, 0.5);  
    glVertex2f(0.5, 0.5);  
    glVertex2f(0.5, -0.5);  
glEnd();
```

指定了两条直线段, 第1条线段的起点为(-0.5, -0.5), 终点为(-0.5, 0.5), 第2条线段的起点为(0.5, 0.5), 终点为(0.5, -0.5)。

- **GL_LINE_STRIP**: 顶点集定义了一系列首尾相接的线段。所以如下代码:

```
glBegin(GL_LINE_STRIP);  
    glVertex2f(-0.5, -0.5);  
    glVertex2f(-0.5, 0.5);  
    glVertex2f(0.5, 0.5);  
    glVertex2f(0.5, -0.5);  
glEnd();
```

指定了三条线段, 第1条线段的起点为(-0.5, -0.5), 终点为(-0.5, 0.5), 第2条线段的起点为(-0.5, 0.5), 终点为(0.5, 0.5), 第3条线段的起点为(0.5, 0.5), 终点为(0.5, -0.5)。

- **GL_LINE_LOOP**: 除了以 **GL_LINE_STRIP** 的方式连接线段外, 最后一个顶点还将与第一个顶点相连。所以, 以下代码将指定一个矩形:

```
glBegin(GL_LINE_LOOP);  
    glVertex2f(-0.5, -0.5);  
    glVertex2f(-0.5, 0.5);  
    glVertex2f(0.5, 0.5);  
    glVertex2f(0.5, -0.5);  
glEnd();
```

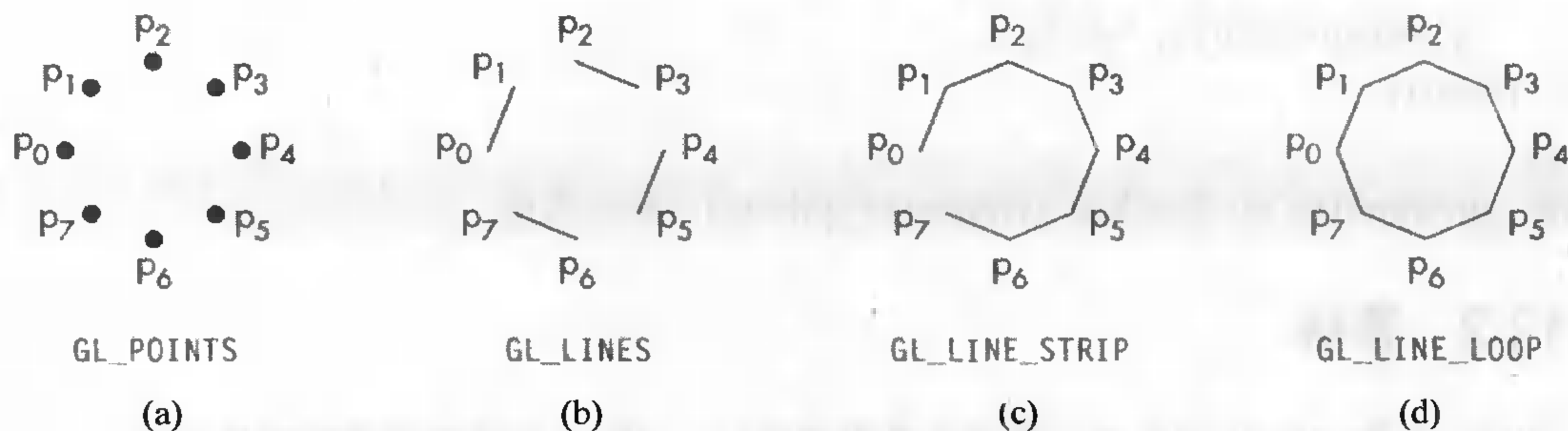


图 2.5 点和直线类型

直线段的属性包括颜色、线宽以及模式(又称为点划模式, 该属性允许我们创建虚线和点划线)。

【29~30】


```
void glLineWidth(GLfloat width)
```

设置显示线宽，单位为像素。默认值为 1.0。

```
void glLineStipple(GLint factor, GLushort pattern)
```

为所绘制的直线定义了一个 16 位模式。如果模式中的某一位为 1，则直线上相应的像素被绘制。如果为 0，则相应像素不被绘制。pattern 中的 1 和 0 的连续组将被重复绘制 factor 次，factor 的取值范围为 1~256。为了绘制一条完整的线段，点划模式需要重复必要的次数。pattern 中位的使用是从最低位开始的。

例如，以下命令：

```
glColor3f(1.0, 1.0, 0.0);
glLineWidth(2.0);
glLineStipple(3, 0xcccc);
```

将绘制颜色设为黄色，将线宽定义为两个像素，并定义了一种虚线点划模式，其中每 6 个像素为一组，任意相邻两组中一组不被上色，另一组以黄色进行绘制。

2.12.3 启用 OpenGL 特性

点划模式是 OpenGL 的许多必须专门启用的特性之一。OpenGL 绘制器具有很多绘制功能，如光照、消隐、纹理映射等，每种特性都将影响绘制处理的速度。应用程序可独立地启用或禁用这些特性。一些特性，如光照，可能只在程序中的某一部分需要而在另一部分不需要，所以如果当该项特性不再需要时将其禁用，会使程序更加高效。

```
void glEnable(GLenum feature)
void glDisable(GLenum feature)
```

启用或禁用 OpenGL 选项 feature。

直线的点划模式通过如下方式启用：

```
glEnable(GL_LINE_STIPPLE);
```

【31】

提示 在您想要使用某种特性之前不要忘记首先启用该特性。如果某项特性没有被启用，即便设定了相关参数也是徒劳的。

高性能显卡的特征之一是许多特性(如光照和纹理映射等)，都是以硬件运算方式来完成而非软件方式。所以，启用这些特性可能并不会给程序的性能带来很大的影响。

2.12.4 填充的图元

在前面的例子(simple.c)中，我们用到的多边形图元就是填充图元的一个例子。具有内部区域的图元在显示时都可以某种颜色或模式进行填充。共有 6 种填充图元，其类型参数如图 2.6 所示。

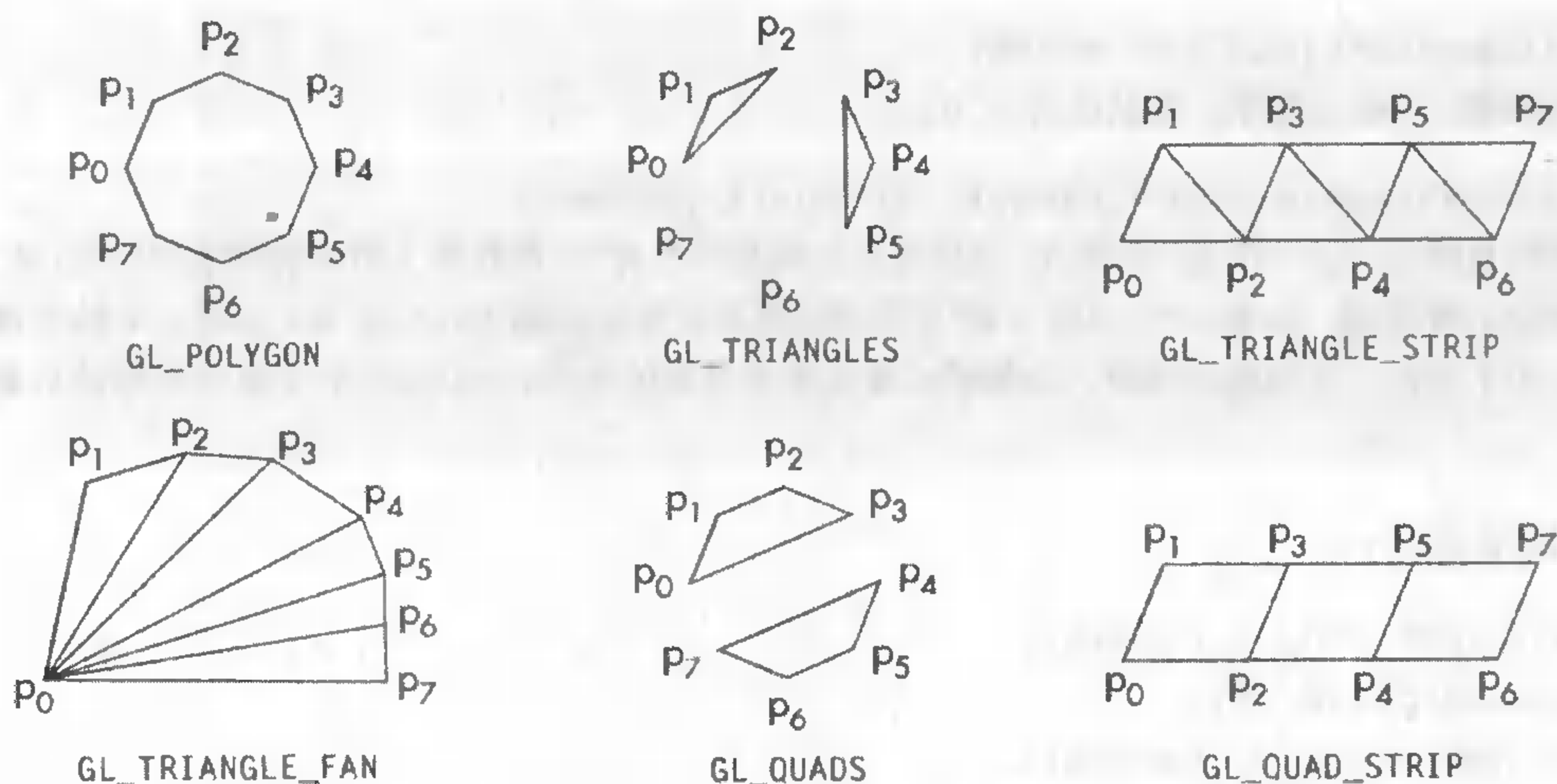


图 2.6 填充的类型

- **GL_POLYGON** 由 `glBegin()` 和 `glEnd()` 之间的 `glVertex*()` 调用序列定义一个多边形。
- **GL_TRIANGLE** 将 `glBegin()` 和 `glEnd()` 之间每三个连续的顶点划为一组，并以三角形来表示该组。多余的顶点将被忽略。
- **GL_TRIANGLE_STRIP** `glBegin()` 之后的前三个顶点定义了第一个三角形。每个后续顶点都与之前的两个顶点构成一个新的三角形。所以，当第一个多边形定义好后，其余多边形只需要调用一次 `glVertex*()` 即可。
- **GL_TRIANGLE_FAN** 前三个顶点定义了第一个三角形，每个后续顶点都与第一个顶点和它的前一个顶点构成一个新三角形。
- **GL_QUADS** 每 4 个连续的顶点定义一个四边形。
- **GL_QUAD_STRIP** 头 4 个顶点定义一个四边形。每个后续顶点对都与其之前的一对顶点构成一个新四边形。

这些多边形类型具有两个优点。首先，一个 OpenGL 的特定实现可能会用专门的软件或硬件来绘制三角形或四边形，所取得的绘制速度比通常的多边形快很多。其次，许多 CAD 应用程序都会生成一些具有公共边的三角形或四边形。条带图元使得我们仅调用极少量的 OpenGL 函数就完成这类图元的定义。而如果我们按照孤立多边形来定义这类图元，所需调用的 OpenGL 函数将会成倍增加。在第 4 章中，我们将介绍顶点数组，该方法可以减少定义具有许多公共顶点的复杂对象时函数调用的次数。

2.12.5 矩形

OpenGL 提供了一个函数, `glRect*`(), 用于绘制二维的填充矩形。

```
void glRect<sifd>(type x1, type y1, type x2, type y2)
```

```
void glRect<sifd>v(type * v1, type * v2)
```

用矩形的左上角和右下角的 x 坐标和 y 坐标或指向包含坐标值的数组的指针来指定一个长和宽分别与 x 轴和 y 轴平行的二维矩形。坐标值的类型为标准数据类型。

2.12.6 多边形的点划模式

所有的填充类型在绘制过程中都被处理成多边形, 因而具有相同的属性。显示多边形的最简单的方式就是用一种纯色对其进行填充。正如我们在 `simple.c` 中看到的, 可用 `glColor*`() 来获取一个纯色的多边形。我们也可通过启用多边形点划模式来对多边形进行填充, 启用的方法为:

```
glEnable(GL_POLYGON_STIPPLE);
```

模式可由 `glPolygonStipple()` 来设定。

```
void glPolygonStipple(const Glubyte * mask)
```

设置多边形的点划模式。`mask` 为 32×32 的位模式。

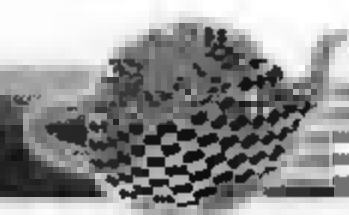
【32~33】

这种模式与直线的点划模式用法相同, 只是维数为 2, 且与窗口对齐。所以, 如果我们通过改变多边形的顶点而将其旋转, 并进行重绘, 点划模式不会发生任何旋转。

2.13 多边形类型

对于非填充图元, 如直线和直线环, 无论顶点是在二维还是三维空间进行定义, 都不存在任何困难。而对于填充图元, 情况则迥然不同。让我们来考虑图 2.7 所示的边发生了交叉的多边形, 要确定哪些点位于多边形内部, 哪些点位于多边形外部, 具有很大的随意性。除非多边形为简单多边形(即那些边互相不交叉的多边形), 否则将以一种不可预料的方式被绘制。OpenGL 并不检查某一多边形是否为简单多边形, 这个任务需要开发人员在程序中自行完成。

二维多边形还有另外一个重要特征: 所有的顶点都共面, 所以, 对于简单多边形, 其内部区域是有明确定义的。在三维空间中, 任意一个总数多于 3 的顶点集完全可以不共面, 因而其内部区域可能无法定义。再次强调, 不同的 OpenGL 实现可能会以不同的方式对这个三维顶点集进行绘制。如果上述情形是一个潜在的问题, 开发者必须在程序中对其进行



处理。

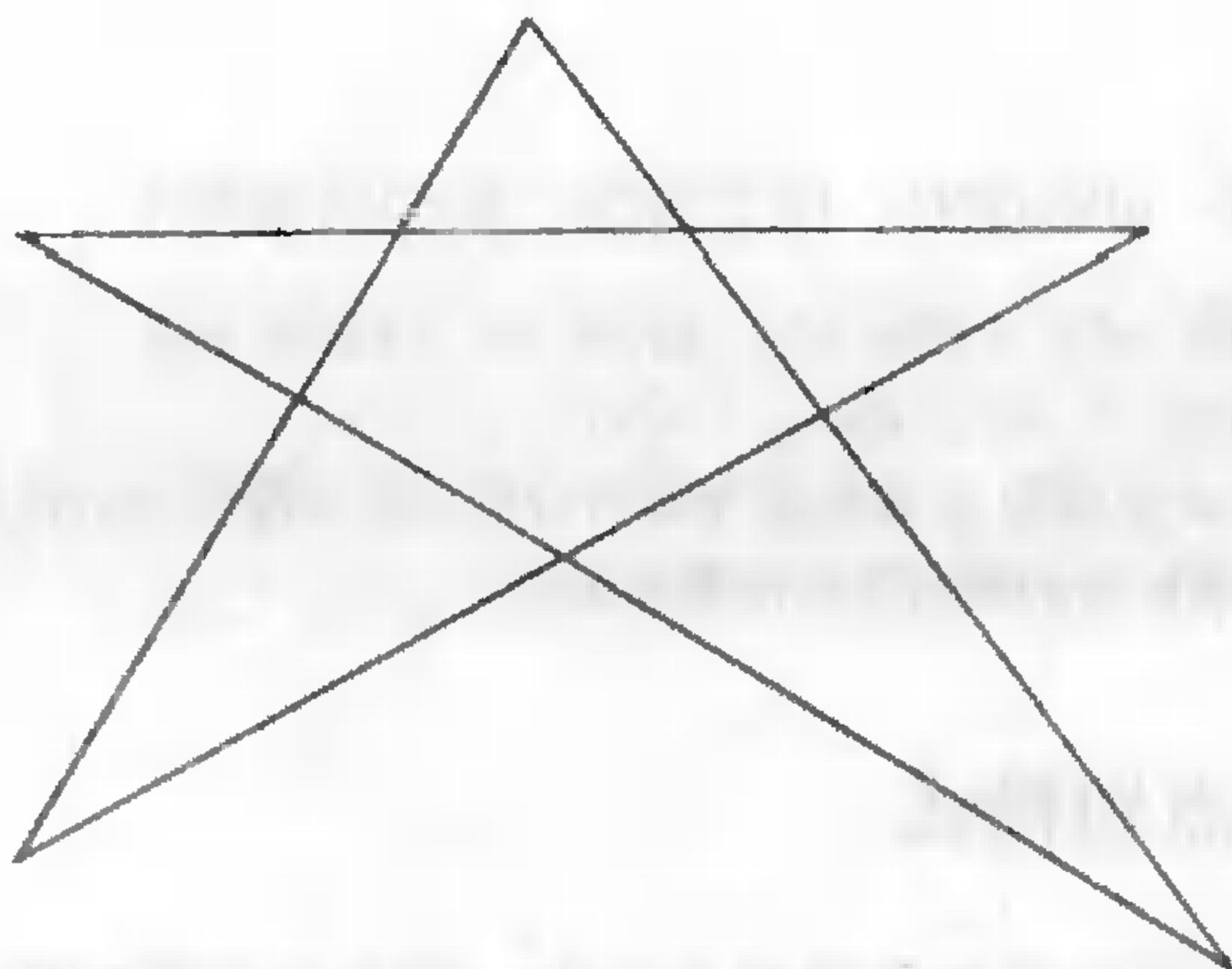


图 2.7 非简单多边形

第三个问题是，即便当所有的顶点共面，绘制一个具有许多顶点的多边形仍然可能会给实现带来一些问题。凸对象具有如下性质：若连接该对象内的任意两点，则连接这两点的整个线段都位于该对象内部(参见图 2.8)。凸多边形的绘制就十分容易。由于三角形总是凸的而且都是平面的，图形系统通常最适合处理三角形单元。

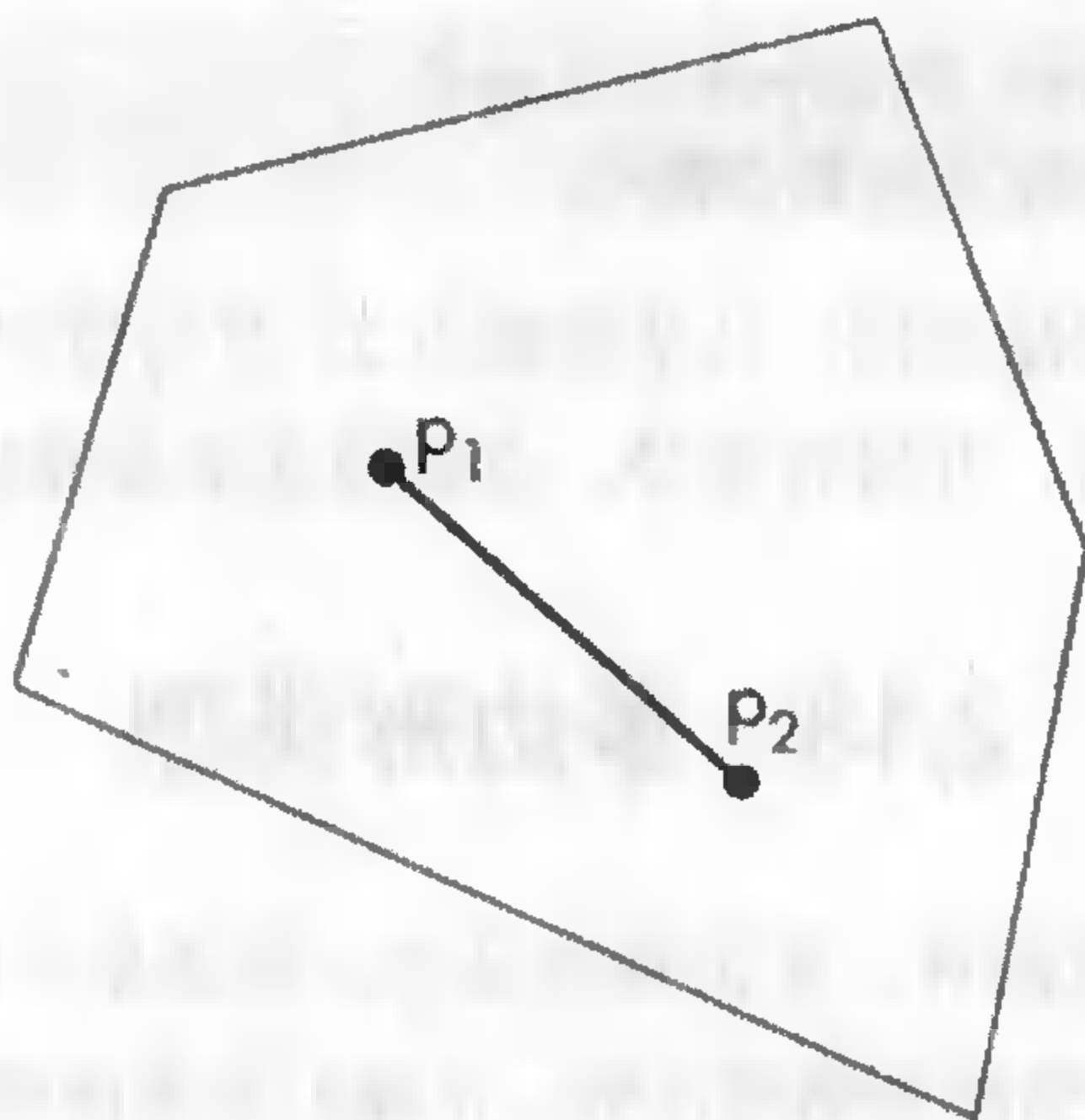


图 2.8 凸多边形

在 OpenGL 中，一个多边形可以三种不同的方式进行显示：填充方式、边线方式或与顶点对应的点集方式。此外，由于二维多边形实际上是三维多边形限制在 $z=0$ 平面上的特例，二维多边形实际上具有两个面——正面和背面。对于这些面，OpenGL 可部分或全部绘制。【34】

默认情况下，若沿着观察方向看，多边形顶点的绕序为逆时针方向，则该面为正面，否则为背面。这些定义对于凸多边形是有意义的。对于非凸多边形，要想为其定义正面和背面相当困难。OpenGL 从未对这类多边形作出任何承诺。

借助于函数 `glPolygonMode()`，我们可以通知 OpenGL 以何种方式对表面进行绘制。

```
void glPolygonMode(GLenum face, GLenum mode)
```

指定表面 (`GL_FRONT`、`GL_BACK` 或 `GL_FRONT_AND_BACK`) 如何进行绘制 (`GL_POINT`、`GL_LINE` 或 `GL_FILL`)。默认情况下对正面和背面都进行填充。

我们也可以不对表面中的某一个或全部进行绘制，方法是借助函数 `glCullFace()` 将正面朝向或背面朝向多边形或全部多边形进行裁剪。

```
void glCullFace(GLenum mode)
```

在绘制时忽略由 `mode` (`GL_FRONT`、`GL_BACK` 或 `GL_FRONT_AND_BACK`) 所指定的表面。

裁剪必须通过如下方式启用：

```
glEnable(GL_CULL_FACE);
```

OpenGL 还允许我们通过函数 `glFrontFace()` 修改正面朝向和背面朝向的定义。

【35】

```
void glFrontFace(GLenum mode)
```

允许将逆时针 (`GL_CCW`) 方向或顺时针 (`GL_CW`) 方向定义为正面朝向。

假定我们想在显示多边形时除了进行颜色填充外，还想将其各边显示出来。例如我们想用黄色来填充多边形，而将其各边用红色来填充。在 OpenGL 中，多边形的各边是多边形内部的一部分，所以仅凭一次绘制就同时将边和多边形内部表现出来是不可能的。我们可以将该多边形绘制两次，第一次绘制时将多边形模式设为 `GL_FILL`，并将颜色设为红色，第二次绘制时将模式设为 `GL_LINE`，将颜色设为黄色，细节如下：

```
glPolygonMode(GL_FRONT_AND_BACK, GL_FILL);
glColor3f(1.0, 0.0, 0.0);
square();
glPolygonMode(GL_FRONT_AND_BACK, GL_LINE);
glColor3f(1.0, 1.0, 0.0);
square();
```

在上述代码中，多边形是在函数 `square()` 中定义的。这里还存在一些潜在的问题。即便边是在填充之后进行绘制的，但这两次绘制的多边形是叠加在一起的，绘制器的一些较小的数值误差会使黄色的边被红色的填充色所覆盖。我们可通过调用函数 `glPolygonOffset` 要求 OpenGL 将边线向外稍微做些偏移。该偏移量是由两个内部常量对两个参数进行加权而构成的线性组合。所以，依据应用程序的单位来设置这些参数并不简单。尽管如此，如果您在没有使用偏移量的情况下尚未取得理想效果，请不妨多尝试几组参数，或许会得到较好的结果(只有当需要进行消隐处理时，多边形偏移量才会派上用场)。

```
void glPolygonOffset(GLfloat factor, GLfloat units)
```

设置多边形的偏移量。该偏移量可正可负，可为任意模式的多边形启用。

多边形偏移量可为三种多边形模式分别启用。例如：

```
glPolygonOffset(1.0, 1.0);
glEnable(GL_POLYGON_OFFSET_LINE);
```

指定了一个正的偏移量，这对我们前面提到的例子是适宜的。

【36】

2.14 颜色插值

用于绘制多边形的颜色由当前颜色状态决定。我们在两次 `glVertex*()` 调用之间改变颜色时，修改了当前状态，但是在概念上我们是将新颜色与新顶点建立了关联。在 OpenGL 中，以这种方式指定颜色时，我们称其为顶点颜色。假定现有一条如下定义的线段：

```
glBegin(GL_LINES);
    glColor3f(1.0, 0.0, 0.0);
    glVertex2f(1.0, 0.0);
    glColor3f(0.0, 0.0, 1.0);
    glVertex2f(0.0, 1.0);
glEnd();
```

上述代码中，两个顶点分别被定义为红色和蓝色。但是 OpenGL 应该以何种颜色来绘制位于这些顶点之间的点呢？默认情况下，OpenGL 将使用平滑着色(smooth shading)。在这种模式下，OpenGL 将依据两端顶点的颜色通过插值计算出中间点的颜色值。所以，当沿着这条线段扫描时，我们将发现颜色将由最初的红色逐渐过渡到蓝色，中间将历经各种明暗的紫红色。

对于多边形，着色的原理是相同的，只是插值公式有些区别，因为我们必须依据多边形各顶点颜色计算出多边形的内部区域中每一点的颜色值。通常，OpenGL 用一组三角形来表现多边形，而且使用的是一个较简单的二维插值公式。对顶点的属性进行插值将涉及后续章节的内容，例如纹理映射以及使用材质属性。

OpenGL 也允许我们用最后一个顶点的颜色来确定整个图元的属性。这样，我们就可生成一条蓝色实线或一个绿色多边形，即使定义每个顶点时使用了不同的颜色值。该模式称为平面着色(flat shading)，是通过调用函数 `glShadeModel()` 对明暗模型进行设置而设定的。

```
void glShadeModel(GLenum mode)
```

将明暗模型设为平滑模式(GL_SMOOTH)或平面模式(GL_FLAT)。着色模式点默认值为平滑模式。

2.14.1 离散处理与边标记

假定您想编写一个能够生成非凸、非平面多边形。怎样来实现呢？您可能对于 OpenGL 的

期望值很高,因为通常情况下它总是能生成一些结果。或者您也可在程序中将多边形分解(或离散化)为若干三角形单元(或三角面片)。但是当您希望只显示边线时,就会发现一个问题。不妨考虑图 2.9 所示的多边形,当我们将其分解为 5 个三角形单元时^①,如果对这 5 个三角形单元进行填充时,显示是没有任何问题的。然而如果我们将多边形模式设为边模式时,我们实际上想显示的是对应于原始多边形的各边,而非由离散化产生的一些新边。解决方案是,借助函数 `glEdgeFlag*()` 函数,我们可决定那些边将被显示。如果该标记设为 `GL_TRUE`,则每个顶点都被认为是所要显示的线段的起点;若该标记被设为 `GL_FALSE`,则构成“虚假”边的顶点将不被显示。

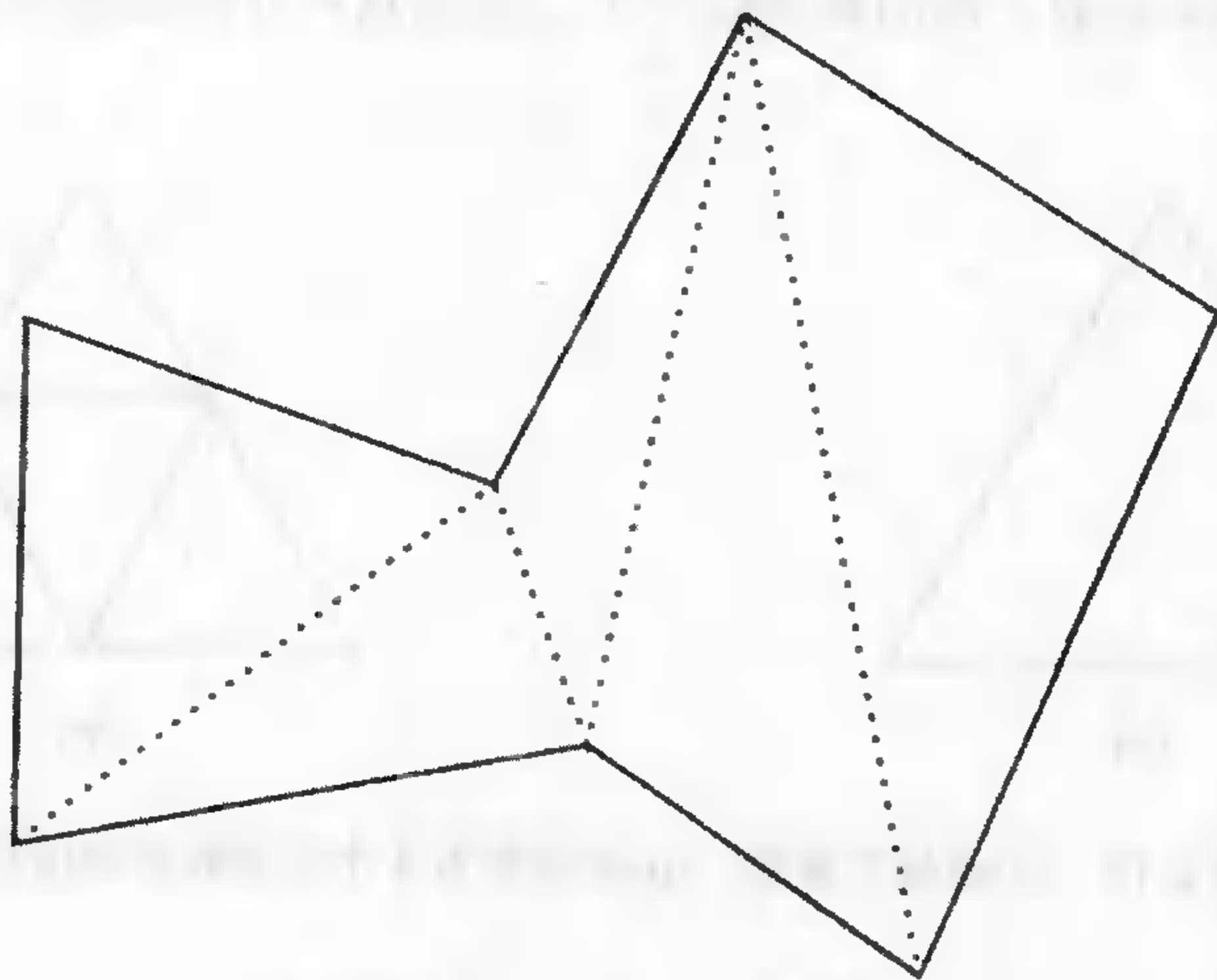


图 2.9 多边形的离散化

```
void glEdgeFlag(GLboolean flag)
```

```
void glEdgeFlagv(GLboolean * flag)
```

对边标记(`GL_TRUE` 或 `GL_FALSE`)进行设定,以确定当多边形模式为 `GL_LINE` 时,后续顶点是否为应显示的边的起点。

现在来介绍第 3 种方案。GLU 库提供了一个离散化器(tessellator)。该离散化器可决定如何对一个多边形进行分解,并负责处理一些重要问题,如创建具有一致朝向的三角以及设定边标记。使用离散化器时,需要大量的附加函数以及选项,所以我们不打算对其进行进一步的讨论。

【38】

2.14.2 离散化与细分

在第 4 章中,我们将介绍一些二次曲面(如圆盘或锥面),在第 9 章中,我们将讨论一些为

^① 注意,有很多种方法可用于分解(或离散化)多边形。

OpenGL 所支持的更一般的曲线和曲面。最终，大多数曲线都将由若干直线段来逼近表示，而曲面将由一系列多边形来逼近表示。我们可用目前为止我们已掌握的手段来将一个简单多边形离散化，以及生成一个扭曲的二维对象。

现在来考虑如图 2.10(a)所示的三角形。我们可将该三角形每条边的中点用线段连接，从而将原三角形分解为 4 个三角形，如图 2.10(b)所示。如果对于每个小三角形重复同样的分解过程，则每次分解都会生成 4 倍于原来数量的三角形。如果我们用递归来实现该细分过程，则代码非常简单。

现在我们来分析 triangle 函数，该函数接收三个二维顶点，并将这些顶点构成的多边形绘制出来：

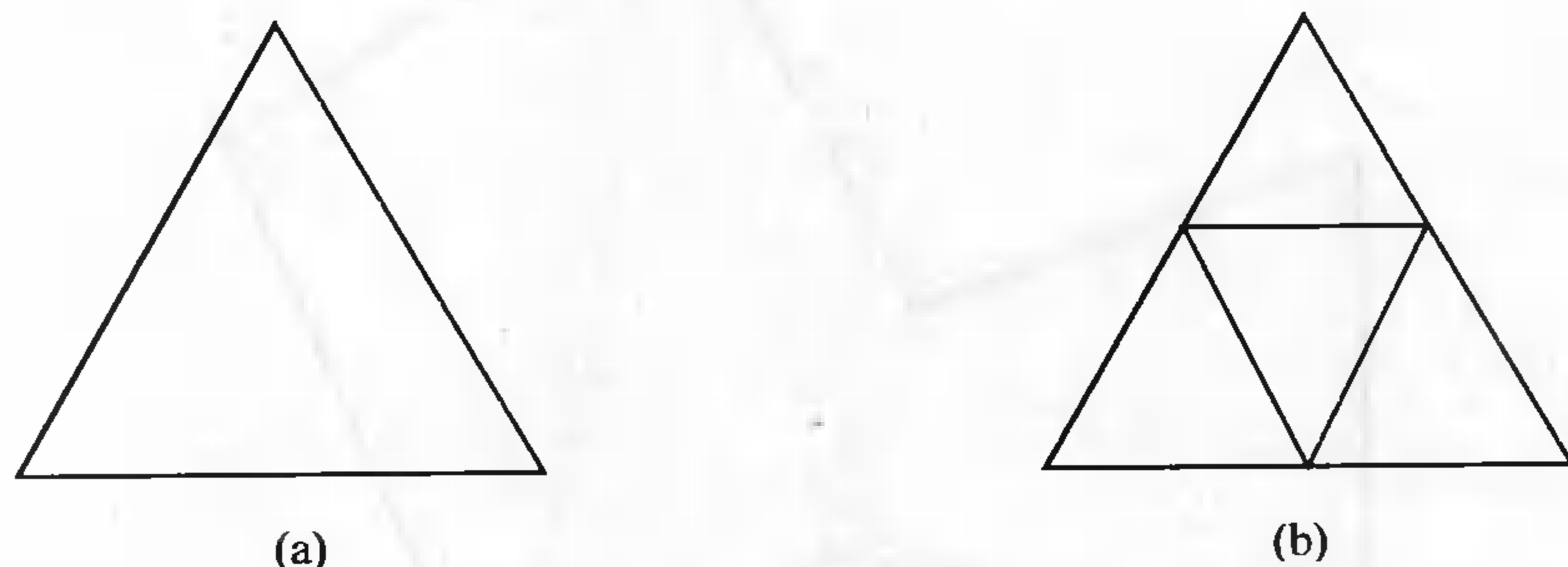


图 2.10 (a)原始三角形；(b)被细分为 4 个三角形后的结果

```
void triangle(GLfloat *a, GLfloat *b, GLfloat *c)
{
    glBegin(GL_TRIANGLES);
        glVertex2fv(a);
        glVertex2fv(b);
        glVertex2fv(c);
    glEnd();
}
```

我们可从一个单个三角形开始，将其递归地细分 m 次，最终由下列代码将所有的小三角形绘制出来：

```
void divide_triangle(GLfloat *a, GLfloat *b, GLfloat *c, int m)
{
    GLfloat v[3][2];
    int j;
    if(m > 0)
    {
        for (j = 0; j < 2; j++) v[0][j] = (a[j] + b[j]) / 2;
        for (j = 0; j < 2; j++) v[1][j] = (a[j] + c[j]) / 2;
```



```

for (j = 0; j < 2; j++) v[2][j] = (b[j] + c[j]) / 2;
divide_triangle(a, v[0], v[1], m-1);
divide_triangle(v[0], b, v[2], m-1);
divide_triangle(v[1], v[2], c, m-1);
divide_triangle(v[0], v[1], v[2], m-1);
}
else(triangle (a, b, c));
}

```

我们可用如下显示回调函数生成 $4n$ 个三角形:

```

GLfloat v[3][2] = {...}; /* specify vertices */
void display(void)
{
    glClear(GL_COLOR_BUFFER_BIT);
    divide_triangle(v[0], v[1], v[2], n);
    glFlush();
}

```

图 2.11 展示了 4 次细分后的结果, 前提是我们已将多边形模式做了如下设置:

```
glPolygonMode(GL_FRONT, GL_LINE);
```

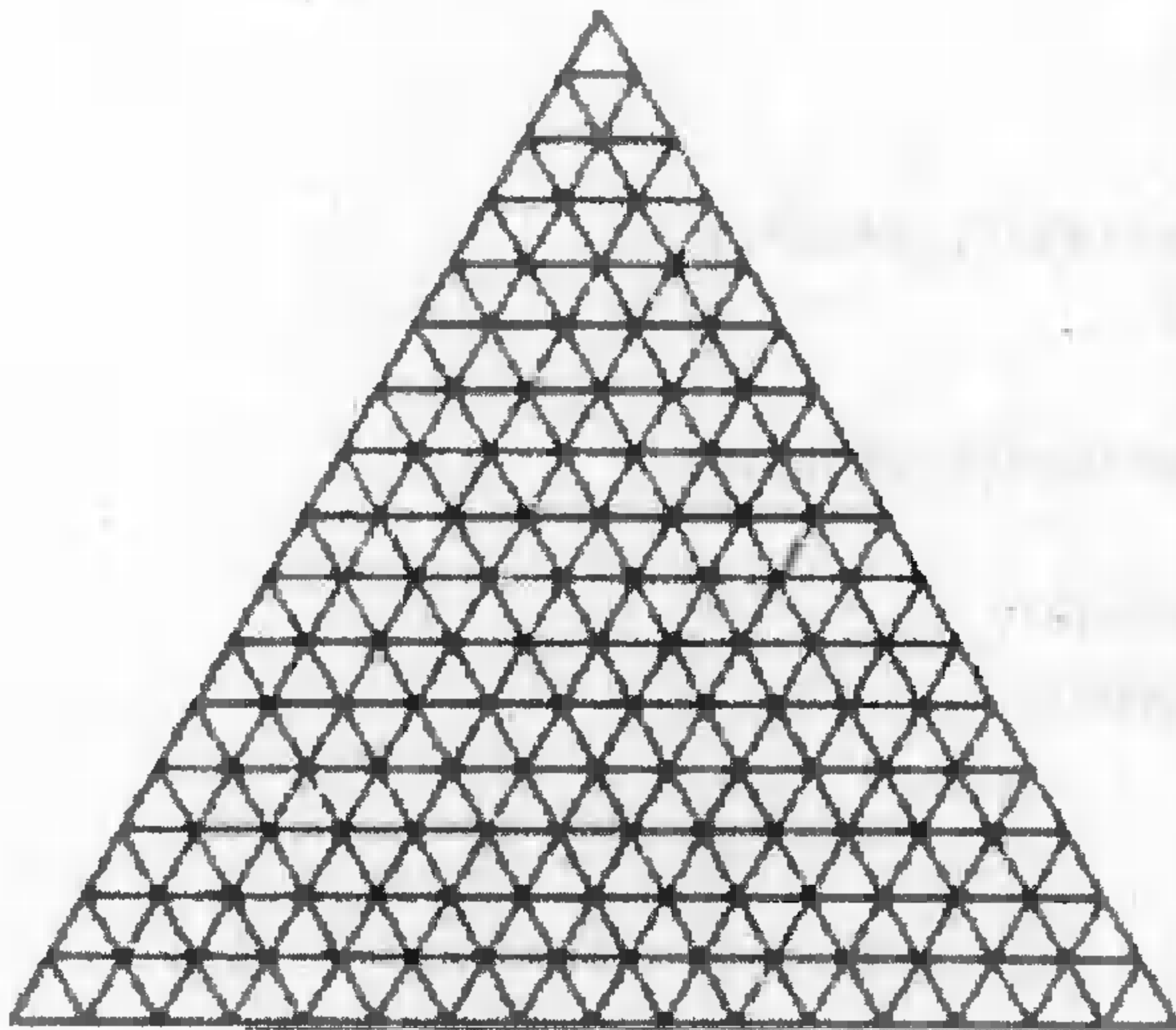


图 2.11 4 次细分后的三角形

【39~40】

现在假定我们只想显示整个集合的外边界。我们很希望借助函数 `glEdgeFlag()` 来实现这一目标。我们意识到, 如果能够像图 2.12 那样生成 8 种不同的边的颜色, 问题便迎刃而解。图中的黑色边表示该边的边标记应设为 `true`, 而灰色边表明该边的边标记应设为 `false`。

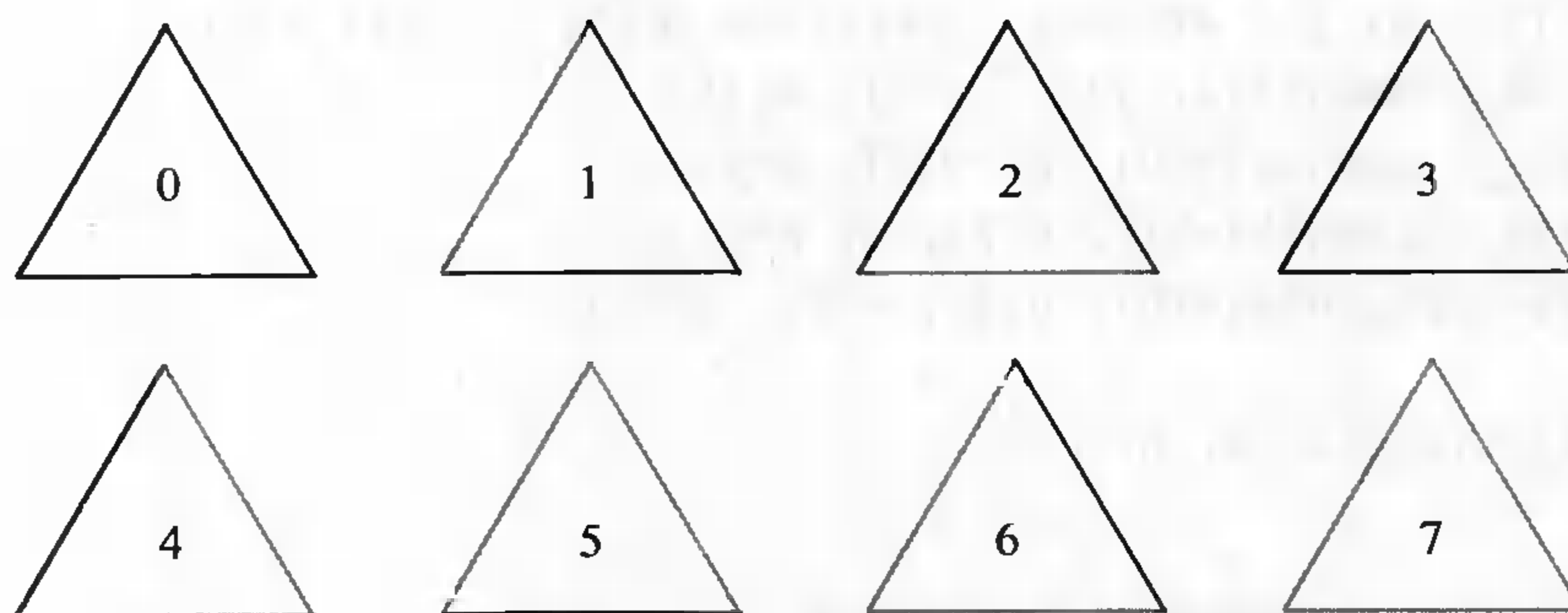


图 2.12 一个三角形所有可能的边标记

现在让我们来重新改写一下 `triangle` 函数，为其包含一个类型参数，以表明所要创建的三角形类型：

```
void triangle(GLfloat *a, GLfloat *b, GLfloat *c, int type)
{
    glBegin(GL_POLYGON);
    switch(type)
    {
        case(2):
        case(4):
        case(6):
        case(7):
            glEdgeFlag(GL_FALSE);
            break;
        default:
            glEdgeFlag(GL_TRUE);
    }
    glVertex2fv(a);
    switch (type)
    {
        case(3):
        case(4):
        case(5):
        case(7):
            glEdgeFlag(GL_FALSE);
            break;
        default:
            glEdgeFlag(GL_TRUE);
    }
    glVertex2fv(b);
    switch(type)
    {
```



```

    case(1):
    case(5):
    case(6):
    case(7):
        glEdgeFlag(GL_FALSE);
        break;
    default:
        glEdgeFlag(GL_TRUE);
    }
    glVertex2fv(c);
glEnd();
}

void divide_triangle(GLfloat *a, GLfloat *b,
                    GLfloat *c, int m, int k)
{
    /* triangle subdivision using vertex numbers */
    GLfloat v[3][2];
    int j, flag[4];
    if(m > 0)
    {
        for(j = 0 ; j < 2 ; j++)
        {
            v[1][j] = (a[j] + b[j]) / 2;
            v[2][j] = (b[j] + c[j]) / 2;
            v[0][j] = (a[j] + c[j]) / 2;
        }

        switch(k)
        {
            case (0):
                flag[0] = 3;
                flag[1] = 1;
                flag[2] = 2;
                break;
            case (1):
                flag[0] = 5;
                flag[1] = 1;
                flag[2] = 6;
                break;
            case (2):
                flag[0] = 4;
                flag[1] = 6;
                flag[2] = 2;
                break;
        }
    }
}

```




```
        case (3):
            flag[0] = 3;
            flag[1] = 5;
            flag[2] = 4;
            break;
        case (4):
            flag[0] = 4;
            flag[1] = 7;
            flag[2] = 4;
            break;
        case (5):
            flag[0] = 5;
            flag[1] = 5;
            flag[2] = 7;
            break;
        case (6):
            flag[0] = 7;
            flag[1] = 6;
            flag[2] = 6;
            break;
        case (7):
            flag[0] = 7;
            flag[1] = 7;
            flag[2] = 7;
            break;
    }

    flag[3] = 7;
    divide_triangle(a, v[0], v[2], m - 1, flag[0]);
    divide_triangle(v[0], b, v[1], m - 1, flag[1]);
    divide_triangle(v[2], v[1], c, m - 1, flag[2]);
    divide_triangle(v[0], v[1], v[2], m - 1, flag[3]);
}

else(triangle(a, b, c, k));
}

void display(void)
{
    glClear(GL_COLOR_BUFFER_BIT);
    divide_triangle(v[0], v[1], v[2], n, 0);
    glFlush();
}
```

如果您完成了该程序并运行，您将看到只有原始三角形的真正外边界被显示，而离散网格是不可见的。所以，您可能会对我们对原始多边形离散化的目的产生疑问。不妨来考虑一

种二维扭曲(twist)。如果我们将一个点 (x, y) 绕着原点旋转 θ 度, 则其新位置为(参见图 2.13):

$$x' = x \cdot \cos \theta - y \cdot \sin \theta$$

$$y' = x \cdot \sin \theta + y \cdot \cos \theta$$

【41~43】

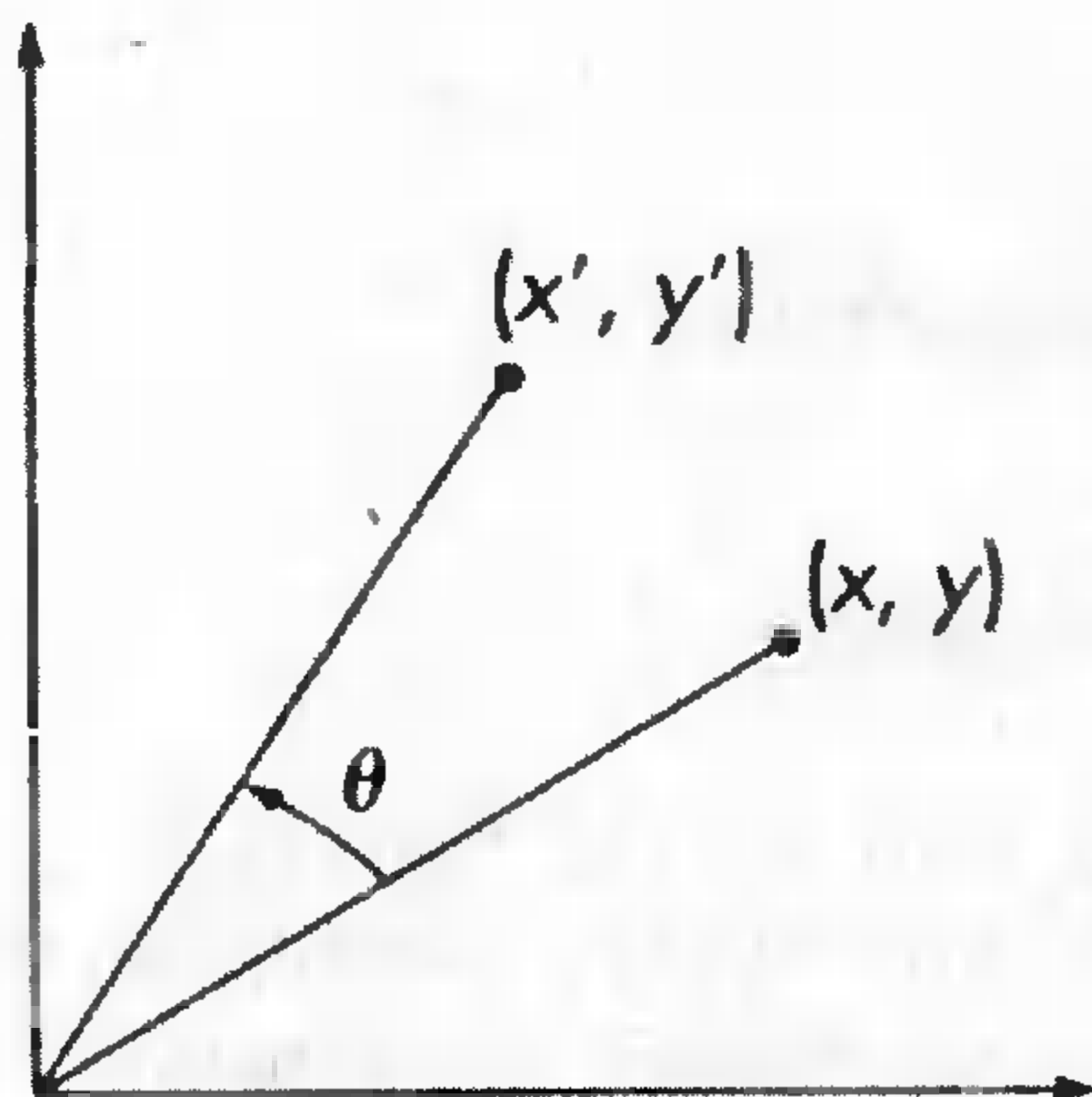


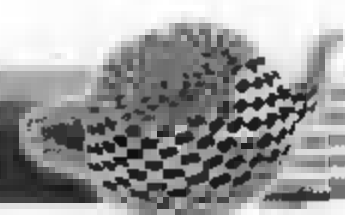
图 2.13 将一点绕原点旋转

如果我们令旋转的角度由该点到原点的距离确定, 我们就是在对对象进行扭曲而非简单的旋转。在上述的离散化程序中, 所需的修改仅局限于三角形的显示:

```
#include <math.h>

float twist;

void triangle(GLfloat *a, GLfloat *b, GLfloat *c, int type)
/* display one triangle */
{
    GLfloat v[2];
    double d;
    glBegin(GL_POLYGON);
    switch(type)
    {
        case (2):
        case (4):
        case (6):
        case (7):
            glEdgeFlag(GL_FALSE);
            break;
        default:
            glEdgeFlag(GL_TRUE);
    }
    d = sqrt(a[0] * a[0] + a[1] * a[1]);
    v[0] = cos(twist * d) * a[0] - sin(twist * d) * a[1];
    v[1] = sin(twist * d) * a[0] + cos(twist * d) * a[1];
```

```
glVertex2fv(v);
switch(type)
{
case (3):
case (4):
case (5):
case (7):
    glEdgeFlag(GL_FALSE);
    break;
default:
    glEdgeFlag(GL_TRUE);
}
d = sqrt (b[0] * b[0] + b[1] * b[1]) ;
v[0] = cos(twist * d) * b[0] - sin(twist * d) * b[1];
v[1] = sin(twist * d) * b[0] + cos(twist * d) * b[1];
glVertex2fv(v);
switch(type)
{
case (1):
case (5):
case (6):
case (7):
    glEdgeFlag(GL_FALSE);
    break;
default:
    glEdgeFlag(GL_TRUE);
}
d = sqrt(c[0] * c[0] + c[1]*c[1]);
v[0] = cos(twist * d) * c[0] - sin(twist * d) * c[1];
v[1] = sin(twist * d) * c[0] + cos(twist * d) * c[1];
glVertex2fv(v);
glEnd();
}
```

运行后得知, 现在结果有了显著的不同。图 2.14 展示了两种不同水平的扭曲。实际上, 在创建扭曲对象时, 我们已经借助离散化生成的网格。离散化的三角形单元数越多, 变形后的轮廓越光滑。注意, 如果我们仅仅是想创建如图 2.14 所示的图像, 我们只需要对构成原始三角形的三条边进行细分, 而不必去关心边标记。但是, 离散化方法极具通用性, 并且可以直接推广到三维表面。

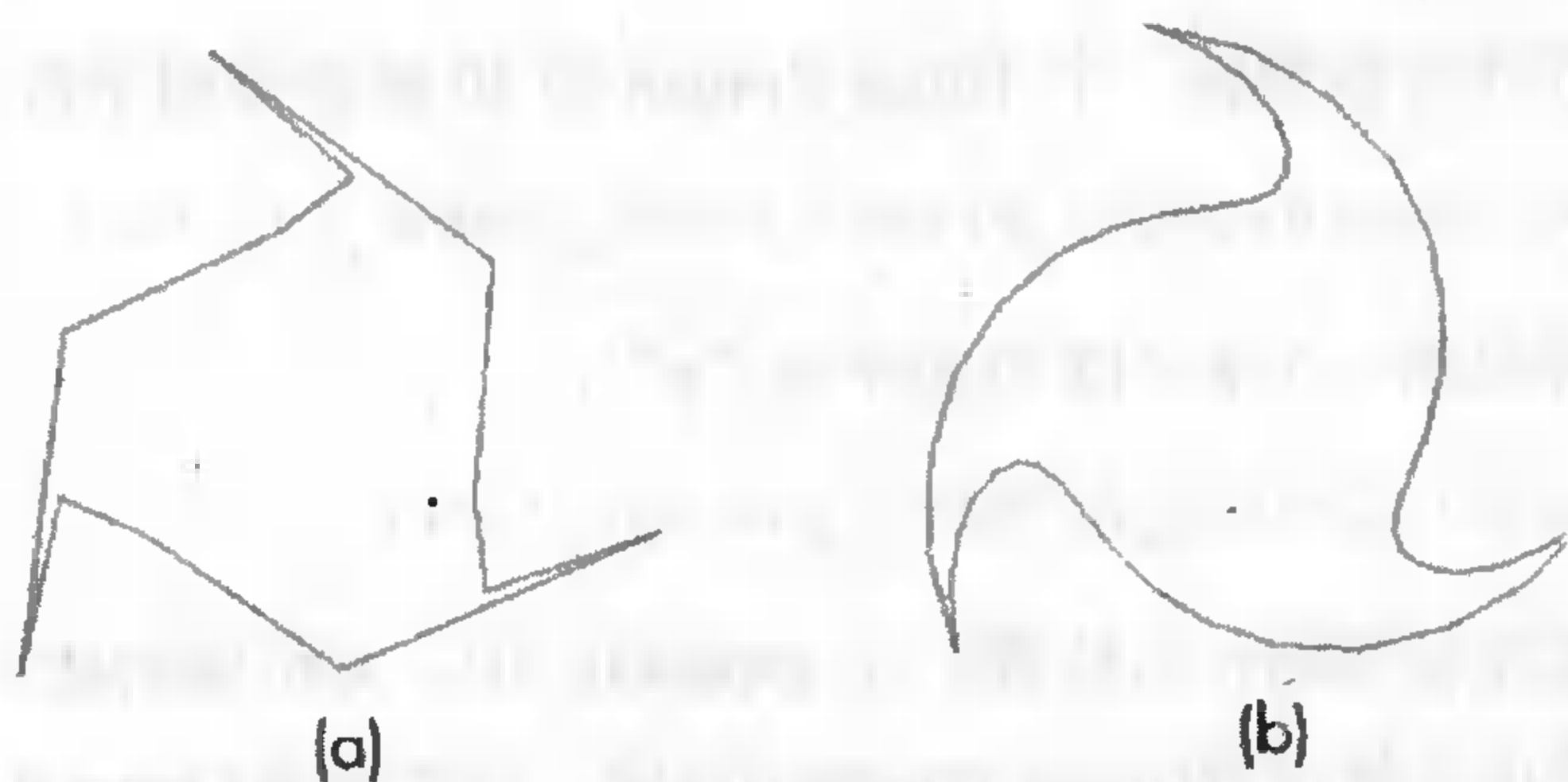


图 2.14 扭曲后的离散三角形: (a)两次细分; (b)4 次细分

【44~45】

2.15 文 本

文本不是 OpenGL 图元的一部分。这可能有点不易理解, 毕竟文本在应用程序(例如标注程序)中的重要性有目共睹。但是, 不要忘记, OpenGL 提供了一个图元的最小集合, 该集合为开发人员提供开发的基本构件。我们可以尝试依据我们已经了解的图元构建一个字符集, 但这个任务听起来并无多大的吸引力。

通常, 生成文本时有几个问题必须解决。假定我们想创建一种字体(font), 即一种给定尺寸和风格的字符集, 例如 10 磅的 Times-Roman 粗体。生成这类字符时有两种主要的形式: 位图字体(bitmap character)和笔划字体(stroke character)。位图字体存储在一个位的矩阵模式中, 其中当某一位为 1 时, 该位将得到显示; 否则不被显示。在第 7 章中, 我们将学习如何定义和输出位图。这类字符的输出速度极快, 因为每个字符的显示只需要一次 OpenGL 函数调用。但是不能对位图字符进行缩放操作。此外, 即便当窗口尺寸调整改变了几何图元的比例, 位图字符仍然保持原来的尺寸。笔划字符是由标准的 OpenGL 图元(如直线、多边形和曲线)创建的。这些图元可通过第 5 章介绍的变换进行修改。但是, 由于笔划字符占据更多的存储空间, 所以绘制速度要慢一些。您可选择生成任何一种字符, 两种类型都受 OpenGL 支持。

一个更简单的方法是使用大多数窗口系统所提供的字体。程序的可移植性可能因此而降低, 但是针对不同的系统进行字体间的切换并不困难。我们可通过系统相关的函数来获取这样的字体。

另外还有第 3 种选择, 即利用 GLUT 提供的少量位图字体和笔划字体。我们可借助函数 `glutBitmapCharacter()` 来获取一个位图字符, 该函数使用了 X 窗口系统中某些字体的模式。

```
void glutBitmapCharacter(void * font, int char)
```

按 font 指定的字体绘制字符 char, 其中字符 char 由其 ASCII 码给出。

【46】

所以，我们按如下方式可以得到一个 Times-Roman 的 10 磅的位图字符 “a”：

```
glutBitmapCharacter(GLUT_BITMAP_TIMES_ROMAN_10, 'a');
```

按如下方式调用则可得到一个 8×13 位的字符 “a”：

```
glutBitmapCharacter(GLUT_BITMAP_8_BY_13, 'a');
```

但是这个字符将出现在屏幕的什么位置？在 OpenGL 中，对位图的处理方式不同于几何图元。位图的出现位置由光栅位置(raster position)决定，后者也是 OpenGL 状态的一部分。这个位置决定了下一幅位图的左下角在屏幕中的位置。可用函数 `glRasterPos*()` 对光栅位置进行设定。

```
void glRasterPos<234><sifd>(type x, type y, type z, type w)
```

```
void glRasterPos<234><sifd>v(type * array)
```

指定光栅位置。该位置将通过当前模型视图矩阵和投影矩阵变换为窗口坐标。

所以，我们可以对象坐标来设定光栅位置。当前光栅位置会自动产生一个偏移量，所以下一个字符不会被绘制到先前绘制的字符之上。GLUT 提供了帮助函数 `glutBitmapWidth()`，借助该函数，我们可在程序中确定字符的宽度，字符宽度可以确定如何在需要时修改光栅位置。

```
int glutBitmapWidth(void * font, int char)
```

返回由 font 指定的 GLUT 字体的字符 char 的像素宽度。

与笔划字体相应的函数为 `glutStrokeCharacter()` 和 `glutStrokeWidth()`。

```
void glutStrokeCharacter(void * font, int char)
```

绘制字符 char，该字符由其 ASCII 码给出，笔划字体由 font 指定。

```
void glutStrokeWidth(void * font, int char)
```

返回 char 的位宽(width in bits)，char 的字体为由 font 命名的 GLUT 字体。

字体 `GLUT_STROKE_MONO_ROMAN` 和 `GLUT_STROKE_ROMAN` 分别为等宽和等距的笔划字体。但它们的尺寸单位并非像素。它们的尺寸在世界坐标系中大约为 100×100 单位。因为这两种字体是有其他几何图元定义的，将流经几何流水线。所以，为笔划字符定位的通常方式是借助 OpenGL 变换，例如比例变换和平移变换，这些内容我们将在第 5 章展开讨论。

【47~48】

2.16 查询与错误

到目前为止，我们始终在假设一切都很完美；一切程序都能够达到预期效果。现在我们回到现实中。我们时常犯这样或那样的错误，程序的运行结果可能也并不如我们所预料的那

样；或者程序也许能够运行，但什么也显示不出来。为此，OpenGL 提供了一些错误检查机制。OpenGL 也允许我们获取状态的任一部分的值。

OpenGL 状态可通过 6 个函数 `glGetBooleanv()`、`glGetIntegerv()`、`glGetFloatv()`、`glGetDoublev()`、`glGetPointerv()` 以及 `glIsEnabled()` 来进行查询。前面 5 个函数将查询结果由应用程序传入的指针返回。应用程序需要知道它所查询的参数。我们即可得到作为状态的一部分的当前值，也可得到系统参数。例如：

```
GLfloat color_array[4];
glGetFloatv(GL_CURRENT_COLOR, color_array);
```

将当前 RGBA 值保存在指定数组中，该数组应能够容纳 4 个值。以下调用：

```
GLint bits;
glGetIntegerv(GL_RED_BITS, & bits);
```

返回用于实现中的颜色所使用的红色位数。另外，在 OpenGL 中还有少量其他的查询函数。我们将在后续章节中进行介绍。

```
void glGetBooleanv(GLenum name, GLboolean * value)
```

```
void glGetIntegerv(GLenum name, GLint * value)
```

```
void glGetFloatv(GLenum name, GLfloat * value)
```

```
void glGetDoublev(GLenum name, GLdouble * value)
```

```
void glGetPointerv(GLenum name, GLvoid ** value)
```

将由 `name` 指定的系统参数或状态参数的当前值写入用户变量中。

【48】

函数 `glIsEnabled(GLenum feature)` 允许应用程序检查某个特性是否已通过调用函数 `glIsEnabled()` 而启用。

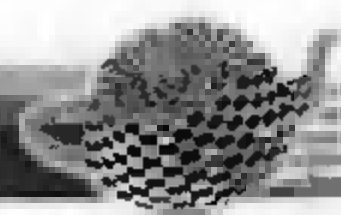
```
GLboolean glIsEnabled(GLenum feature)
```

取决于 `feature` 是否被启用，返回 `GL_TRUE` 或 `GL_FALSE`。

错误检查可分两部分进行。我们可以借助函数 `glGetError()` 检查是否出现错误，该函数的返回值为错误类型，如果没有发现错误，则返回 `GL_NO_ERROR`。该函数只关心自初始化起直至该函数被调用时所发生的错误。当该函数被调用时，错误标记被复位为 `GL_NO_ERROR`。关于某个特定错误的字符串信息，可通过函数 `gluGetErrorString()` 获取。

```
GLenum glGetError()
```

返回自初始化或最近一次调用 `glGetError()` 起，最近发生的错误的类型。如果没有错误发生，则返回 `GL_NO_ERROR`。



```
GLubyte * gluErrorString(GLenum error)
```

返回一个与 `glGetError()` 返回的错误对应的字符串信息。

GLUT 中的错误报告机制是平台相关的。一般说来,如果您请求一个未被支持的功能,程序将会终止,并发出一条错误消息。GLUT 提供了一个函数 `glutGet()` 可供应用程序获取关于 GLUT 状态的信息。

```
int glutGet(GLenum state)
```

返回指定的 GLUT 状态变量的状态。

我们可获取信息的范围跨越了颜色缓存的深度(`GLUT_WINODW_BUFFER_SIZE`)直至当前显示模式是否被支持(`GLUT_DISPLAY_MODE_POSSIBLE`)。

2.17 状态的保存

OpenGL 状态决定了图元的绘制方式。实际上我们对属性和其他状态变量所做出的全部修改,例如模型视图矩阵和投影矩阵,都会改变当前状态。改变了这些变量的程序可将大部分时间用于状态的修改,许多修改都需要对这些变量重新进行计算。例如,我们常常需要计算摄像机参数或我们希望使用的颜色。OpenGL 采取的方式并非重新计算原先我们所使用的值,而是提供了两种类型的堆栈,我们可将当前状态保存在堆栈中,以便后期使用。 【49~50】

矩阵堆栈(matrix stack)可用来保存投影矩阵和模型视图矩阵。每种堆栈只能容纳相应类型的矩阵。可用函数 `glPushMatrix()` 和 `glPopMatrix()` 使矩阵入栈或出栈。所使用的矩阵由当前矩阵模式(`GL_MODELVIEW` 或 `GL_PROJECTION`)决定。

```
void glPushMatrix()
```

```
void glPopMatrix()
```

依据当前矩阵模式使矩阵入栈或出栈。

矩阵堆栈有两个主要应用。在第 5 章中构建层次模型时,我们将使用堆栈来遍历用来描述这些模型的树型数据结构。应用程序用到了模型视图矩阵。第 2 种应用则涉及投影矩阵。为确定所需的投影矩阵,常常需要花费大量的计算或用户输入。假定要临时将镜头推向近景,我们可以一种非常简单的方式通过修改当前投影矩阵来做到这一点。现在的问题在于当我们想回到先前的视图时,并不想(或根本无法)重新计算原来的投影矩阵。将其保存在堆栈中,该问题便迎刃而解。所以,我们常常会看到这样的代码:

```
glMatrixMode(GL_PROJECTION);
/* set projection matrix */
/* draw scene */
glPushMatrix();
/* change the projection matrix */
```



```
/* draw scene */
glPopMatrix();
```

注意，入栈和出栈操作必须成对使用；一次出栈必须与一次入栈对应。在层次系统中，如果这对操作没有正确的匹配，将使堆栈处于一种不可预知的状态。

提示 为了能够返回所期望的状态，入栈和出栈必须成对出现。

OpenGL 依据相关性将其属性划分为 20 组。例如，所有的多边形属性都位于 GL_POLYGON_BIT 组中。所有的直线属性都位于组 GL_LINE_BIT 中。我们可将多组属性或全部属性(GL_ALL_ATTRIBUTE_BITS)通过函数 glPushAttrib()压入属性堆栈中，恢复这些状态则使用函数 glPopAttrib()。【50~51】

```
void glPushAttrib(GLbitfield mask)
```

```
void glPopAttrib()
```

使属性组入栈或出栈。mask 可由组的标识符通过逻辑或运算组合而成。

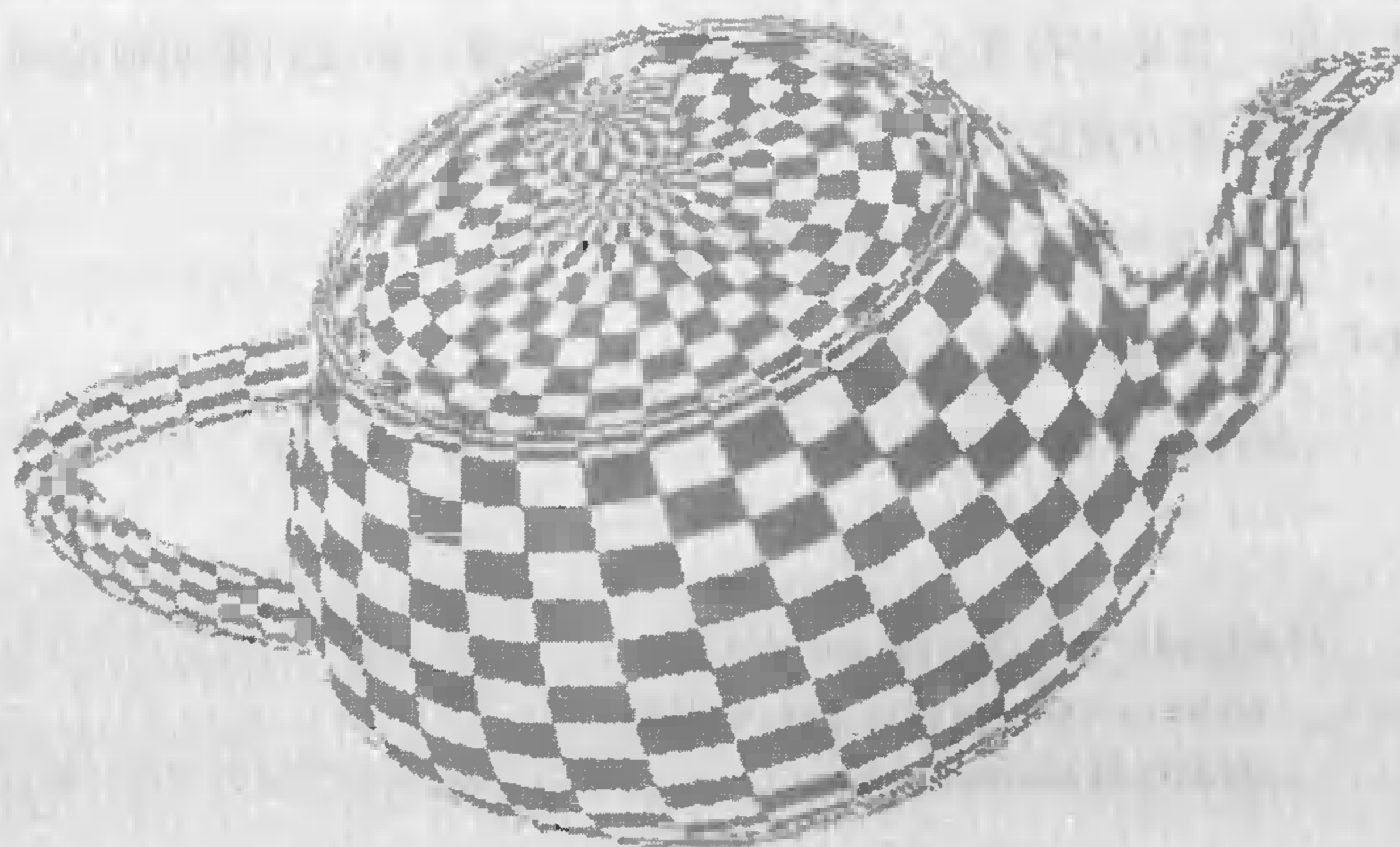
2.18 编程练习

1. 编写一个程序，绘制一个用直线或多边形逼近表示的圆。一种方法是利用简单的三角法计算出圆周上各点的位置。另外一种方法是将一个简单正多边形(如等边三角形)逐步细分为精细的多边形。
2. 重新编写我们用于演示离散化的细分程序，使得每次进行细分时，中间的三角形都被忽略。结果可以填充多边形的形式显示，也可以三角形的边来表示。所形成的对象为一个简单的分形图案——Sierpinski 垫圈。
3. 从一个底边与 x 轴平行，有三条线段构成的三角形开始，递归地对另外两条边进行细分，每次都应将新顶点移动一个随机小量。最后形成的形状看起来应该像山脉的轮廓。
4. 获取一些二维数据。编写一个可将这些数据绘制成多边形的程序(连接连续数据点的线段)。程序应该能够对这些数据实施比例变换，以使图形大小与窗口尺寸相匹配。试着增加坐标轴和标题。
5. 使用习题 1 的结果依据一些数据绘制饼图。

第 3 章

交互与动画

在接下来的内容中，我们准备为程序添加一些动态元素。为了实现这个目标，我们创建或改变对象的方式必须能够使得显示过程比较平滑。我们可能还想通过使用菜单、拾取对象或使用鼠标来指定位置以实现用户与程序之间的交互。虽然这些功能并未由 OpenGL 直接提供，所幸的是，OpenGL 实用工具集(GLUT)提供了我们需要的这些附加函数。



3.1 重绘回调函数

在第2章中, 我们看到, 一个典型的 OpenGL 主程序都会进入一个事件处理的无限循环。在该循环内部, 程序可依据程序员编写的回调函数(如键盘回调函数和鼠标回调函数)来对检测到的离散事件做出响应。

最常见的一种交互操作就是调整屏幕的尺寸。通常, 在用户用鼠标拖动窗口的一角或某一边框时, 该操作将被触发。这种行为会产生一个窗口事件, 该事件的处理将由重绘(reshape)回调函数负责。返回给该回调函数的信息包括窗口的新长度和新宽度(单位为像素)。所以, 程序员需要先定义一个回调函数, 然后在 main() 函数中用函数 glutReshape() 将其注册。

```
void glutReshapeFunc(void (* f)(int width, int height))
```

当用户用鼠标改变窗口的尺寸时, 该函数即被调用。新窗口的 height 和 width 将返回给函数 f()。在执行 f() 之后, 显示回调函数将被自动调用。 **【53】**

在窗口首次创建时, 重绘回调函数将被调用。所以, 该函数是放置观察函数的理想之地。由于窗口的长度和宽度将返回给该回调函数, 当用户改变窗口的尺寸和形状时, 我们就可对观察条件进行修改。在 GLUT 中有一个默认的重绘回调函数, 但是这样并未发掘出重绘的巨大潜力, 所以我们一般不用默认函数, 而是根据需要自行创建, 以便取得各种不同的效果。

假定窗口最初由 glutInitWindowPosition() 函数指定为方形, 且用户对窗口的尺寸进行调整后, 窗口的尺寸变小, 不再是正方形。当窗口变小时, 程序员必须确定同一对象应以较小的尺寸进行显示还是只显示对象中的一个较小区域。程序员还必须确定在窗口不再是正方形的情况下, 对象的形状是否不必改变。显然, 这些决策都依赖于具体的应用程序。现在我们来考虑这种情形: 我们想看到同样的对象, 但是要确保其形状不发生扭曲, 即, 圆仍然为圆, 只是直径变小。我们可通过改变窗口和视口来实现这项任务。典型的重绘回调函数是按如下方式定义的:

```
glutReshapeFunc(myreshape);
```

其中 myreshape() 的定义如下:

```
GLsizei ww, hh;
void myreshape(GLsizei w, GLsizei h)
{
    /*adjust clipping window*/
    glMatrixMode(GL_PROJECTION);
    glLoadIdentity();
```



```

    if(w<=h)
        gluOrtho2D(-0.2, 2.0, -2.0*(GLfloat)h/(GLfloat)w,
        2.0*(GLfloat)h/(GLfloat)w);
    else
        gluOrtho2D(-2.0*(GLfloat)w/(GLfloat)h,
        2.0*(GLfloat)w/(GLfloat)h, -2.0, 2.0);
    glMatrixMode(GL_MODELVIEW);
    /*adjust viewport*/
    glViewport(0, 0, w, h);

    /*set global size for use by drawing routine*/
    ww = w;
    hh = h;
}

```

【54】

该函数的第一部分改变了投影矩阵。按照惯例，我们首先加载一个单位矩阵。纵横比(aspect ratio, 定义为窗口宽度与高度的比值)用于确定一个裁剪窗口。如果屏幕窗口是方形的，则裁剪窗口的尺寸为 4.0×4.0 单位，且其中心位于对象坐标系的原点。如果屏幕窗口不是方形的，我们将裁剪窗口的最短边设为 4.0 单位。我们还对视口进行了设置，目的是为能够使用整个新的屏幕窗口。注意，新窗口和裁剪窗口的纵横比是相同的，这就防止了物体的形状发生扭曲。接下来我们将矩阵模式设为模型视图模式。这一步不是必须的，但常会带来一些便利。因为程序修改模型视图矩阵的概率总是大于修改投影矩阵。注意，ww 和 hh 是在回调函数外部作为全局变量定义的，这样就使得窗口的尺寸为其他函数所利用。

现在来考虑，如果显示回调函数绘制一个边长为 2.0 个单位的正方形(如同我们在前一章的简单例程 simple)，会发生什么情况？如果窗口初始时为正方形，我们将看到一个正方形位于窗口中央，其边长为窗口长度的一半。现在假定我们重新调整屏幕窗口的尺寸，使得窗口的纵横比为 5。重绘回调函数将修改裁剪窗口的尺寸，其高度仍为 4.0 个单位，但其宽度现在只有 2.0 个单位。我们的 2.0×2.0 正方形仍然会出现在窗口的中央，而且仍然保持原来的形状——方形。对视口的修改是必要的，因为这样可使得无论屏幕窗口的尺寸如何变化，由 gluOrtho2D()设置的整个裁剪窗口都将是可见的。显示回调函数将被自动调用，这就使得屏幕窗口尺寸的变动不但会改变整个窗口的尺寸，而且会使由显示回调函数定义的几何对象得到重绘。

3.2 Idle 回调函数

假定我们想让前一章中绘制的正方形运动起来。一种简单方法是使用 GLUT 库自带的空闲回调函数(idle function)。该空闲回调函数由函数 glutIdleFunc()来指定，注册应在 main()函数

中完成。空闲回调函数标识了这样一个函数：当事件队列中没有事件需要处理时(即事件队列为空时)，它才有机会得到执行。

```
void glutIdleFunc(void (* f)(void))
```

当事件队列为空时，函数 $f()$ 将得到执行。

下面是一个极其简单的例子。假定我们在前一章程序 simple.c 的 main 函数中添加以下代码：

```
glutIdleFunc(myidle);
```

并如下定义 myidle() 函数：

```
void myidle()
{
    glutPostRedisplay();
}
```

【55~56】

新函数 glutPostRedisplay() 所执行的功能类似于直接调用显示回调函数(simple.c 中的 display() 函数)，但该函数允许实现在对何时真正需要调用显示回调函数而作出决策时，变得更加“智能化”。在 GLUT 将遍历整个事件循环时，必然会检索到许多要求窗口重绘的事件。如果每次都去直接调用显示回调函数，窗口必然会被多次绘制。而使用 glutPostRedisplay() 之后，就使得在遍历消息队列的整个过程中，只对窗口重绘一次。一般说来，在屏幕需要重绘时，使用 glutPostRedisplay() 而非直接调用显示回调函数是非常明智的。

```
void glutPostRedisplay()
```

请求在当前回调函数返回之后，执行显示回调函数。

如果我们对 simple.c 程序作出上述修改，在程序运行时尽管我们有可能注意到计算机在后台做了更多的工作以及屏幕看起来有些闪烁，但我们很难看出修改前后有什么差别。我们所做的其实就是请求计算机在空闲时便对屏幕进行重绘。但是由于我们在显示回调中并未做任何修改，所以计算机始终是一次次地绘制完全相同的物体。所以，我们首先进行清屏(这是造成屏幕闪烁的原因)，然后再绘制矩形。相较于前面的程序(只对正方形绘制一次)，修改之后的程序占用的系统资源更多。一种对回调函数更切实的使用方式为修改一些绘制正方形时所使用的数据。

3.3 一个旋转的矩形

假定我们将正方形的 4 个顶点定义为一个圆周上的 4 个等间隔点，如图 3.1 所示。对于角度 θ ，则正方形的 4 个顶点分别为 $(\cos \theta, \sin \theta)$ ， $(-\sin \theta, \cos \theta)$ ， $(-\cos \theta, -\sin \theta)$ 以及 $(\sin \theta, -\cos \theta)$ 。

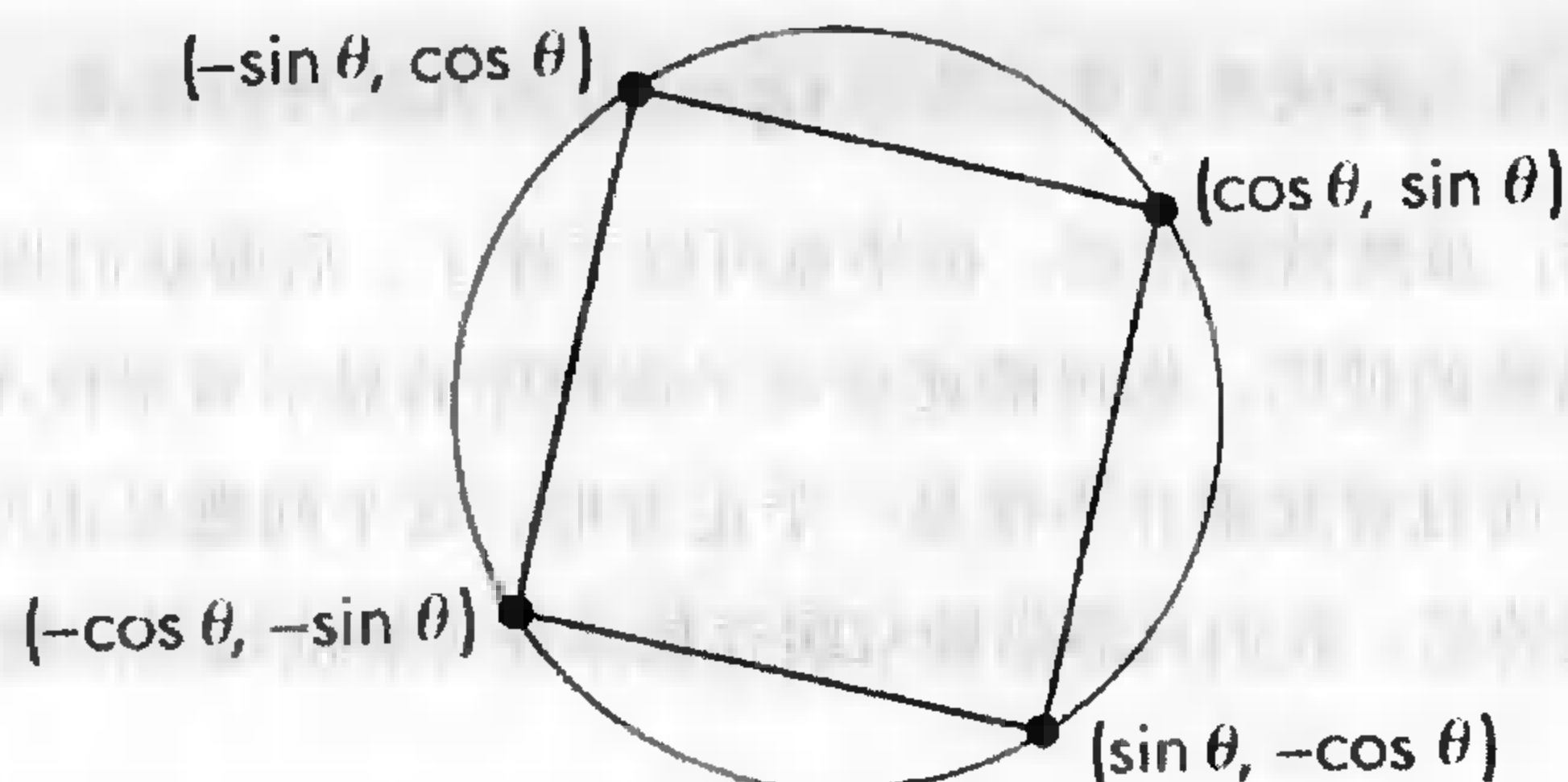


图 3.1 单位圆内的正方形

如果我们将角度的修改作为空闲回调函数的一部分，然后再重新显示，则每次重绘之后，正方形都会出现在一个新位置上。现在我们来分析如下空闲回调函数：

```
void myidle()
{
    theta += 2.0;
    if(theta > 360.0) theta -= 360.0;
    glutPostRedisplay();
}
```

【56~57】

每次 `myidle()` 被调用时， θ 的值增加 2 度。现在，我们所要做的仅仅是重写显示回调函数，使得它能够用 θ 来表示顶点的位置变化。另外，顶点定义的位置我们也需要关注。因为回调函数的形式已由 GLUT 限定了，我们必须将 θ 定义为一个全局变量，以便使显示回调函数和空闲回调函数都能够使用 θ 值。

```
#include <math.h>
#define DEG_TO_RAD 0.017453 /*degrees to radians*/
GLfloat theta = 0.0;
void mydisplay()
{
    glClear(GL_COLOR_BUFFER_BIT);
    glBegin(GL_POLYGON):
        glVertex2f(sin(DEG_TO_RAD*theta)),
            cos(DEG_TO_RAD*theta);
        glVertex2f(-sin(DEG_TO_RAD*theta)),
            cos(DEG_TO_RAD*theta);
        glVertex2f(-sin(DEG_TO_RAD*theta)),
            -cos(DEG_TO_RAD*theta);
        glVertex2f(sin(DEG_TO_RAD*theta)),
            -cos(DEG_TO_RAD*theta);
    glEnd();
    glFlush();
}
```


提示 数学库中的三角函数使用的是弧度而 OpenGL 函数使用的是度。

程序作出上述修改后,虽然效率很低,但毕竟可以工作了。后面我们将用 OpenGL 中的旋转操作来取代三角函数的使用。您可能还会对于该程序的显示效果很不满意。运行时窗口可能会一直在闪烁,而且看起来并不像是一个正方形。这个问题是由应用程序与显示过程的交互造成的。幸运的是,我们只需借助双缓存技术便可解决这类问题。

3.4 双 缓 存

大多数图形显示设备在工作时都以固定的频率进行重绘或刷新。该过程要求显示硬件获取颜色缓存中的内容,并用这些值来决定屏幕上的图形窗口的颜色。对于应用程序来说,屏幕的刷新过程是不可控的,实际上,二者是不同步(或异步)的。所以,用户程序在为颜色缓存创建新值的同时,显示过程取出这些值并加以显示。用于显示的值并不单单是由某个特定角度 θ 确定的正方形的值,而是多次执行显示回调函数后形成的一些颜色值的组合。

虽然我们无法将这两个过程同步,但借助双缓存技术,我们能够保证只显示一个完整绘制的正方形。其主要思想是使用两个颜色缓存——前台缓存和后台缓存,而非一个。前台缓存是显示硬件所显示的内容,而后台缓存则是应用程序进行绘制的地方。在后台缓存中绘制完整个正方形后,我们所要做的仅仅是交换这两个缓存中的内容。在显示回调函数中用函数 `glutSwapBuffers()` 取代函数 `glFlush()` 即可。

void glutSwapBuffers()
交换后台缓存和前台缓存中的内容。

注意, `glutSwapBuffers()` 是一个 GLUT 函数,而非 OpenGL 函数。因为交换过程涉及与窗口系统的交互。我们必须在初始化阶段请求双缓存模式,因为在默认情况下显示模式为单缓存模式。我们可通过初始化语句来完成进行设定,假定我们同时还希望采用 RGB 颜色模式:

```
glutInitDisplayMode(GLUT_DOUBLE | GLUT_RGB);
```

【58~59】

3.5 键盘的使用

现在我们开始使用两种最重要的交互设备——鼠标和键盘。对于键盘,每次当我们按下或释放一个键时都会产生事件。GLUT 带有一个键盘回调函数,该回调函数可通过 `glutKeyboardFunc()` 来指定。当某一个键被按下时,该回调函数便会被调用。虽然许多系统允许程序响应键的释放事件,GLUT 选择了忽略该时间。


```
void glutKeyboardFunc(void * f(unsigned char key, int x, int y))
```

指定当任何键被按下时，函数 `f()` 将被调用。所按下的键连同光标的位置一并传入 `f()`。注意，光标位置的单位为像素，是从窗口的左上角开始度量的。

所按下的键作为 `unsigned char` 类型返回给键盘回调函数。通常，键盘回调函数使用一个简单的控制结构如 `if` 或 `switch` 来确定回调函数应进行处理的键是否被按下，以及如果被按下，应如何处理。例如，我们希望当用户按下 `Q`、`q` 或 `Esc` 键时，能够终止程序。在主函数中，我们注册一个键盘回调函数：

```
glutKeyboardFunc(mykey);
```

然后如下定义该回调函数：

```
void mykey(unsigned char key, int x, int y),
{
    if(key == 'Q' || key == 'q' || key == '\27') exit(0);
}
```

提示 所有的 GLUT 回调函数都返回一个鼠标位置，该位置是以屏幕坐标的形式给出，坐标系的原点位于平面左上角。

提示 `glutKeyboardFunc()` 只响应按键消息，键的释放消息将被忽略。

【59】

键盘回调函数在某个键被按下的同时也会返回鼠标的位置。我们可利用这些值使按键产生一些不同的效果。例如，在一个画图程序中，我们希望鼠标位于绘图区外时，能够通过按键来终止程序。还可用键盘回调函数向屏幕输出一些文本。注意，由于鼠标的位置是相对于屏幕坐标系给出的，所以位置坐标的单位为像素，而且坐标值从屏幕左上角开始算。

特殊键

对于各种特殊键(例如大多数键盘上都有的功能键和方向键)的用法，GLUT 有自己的规定。这些键是通过 `glutSpecialFunc()` 所指定的回调函数来处理的。

```
void glutSpecialFunc(void (* f)(int key, int x, int y))
```

指定特殊键的回调函数。当键 `key` 被按下时，函数 `f()` 将得到执行。同时鼠标位置 `(x, y)` 也将被返回。

每个特殊键都有一个在 `glut.h` 中定义的字符串来指定。所以，在回调函数内部，我们可能会看到如下形式的代码：

```
if(key == GLUT_KEY_F1)    ... /* function key 1 */
if(key == GLUT_KEY_UP)    ... /* up arrow key */
```

我们也可以借助于函数 `glutGetModifier()` 来使用修饰键(modifier key)——`Ctrl`、`Alt` 和 `Shift`。

该函数可在鼠标或键盘回调函数中进行调用。

```
int glutGetModifiers()
```

如果键在鼠标或键盘事件产生时，键被按下，则该函数返回 GLUT_ACTIVE_SHIFT, GLUT_ACTIVE_CTRL, 或 GLUT_ACTIVE_ALT 的逻辑与结果。

所以，为了能够让 Ctrl + c 或 Ctrl + C 终止一段程序，我们可在键盘回调函数中加入下列代码：

```
if(glutGetModifiers() == GLUT_ACTIVE_CTRL) &&
    ((key == 'c') || (key == 'C'))    exit(0);
```

如果我们只有单键或双键鼠标，却希望使用一些三键鼠标才具有的功能时，这些修饰符就显得很有用了。例如，对于单键鼠标，我们可用鼠标单击、Ctrl + 鼠标单击以及 Shift + 鼠标单击来实现三键鼠标的功能。 【60~61】

3.6 鼠标回调函数的使用

对于鼠标，我们用 glutMouseFunc() 来注册鼠标回调函数。

```
void glutMouseFunc(void (*f)(int button, int state, int x, int y))
```

表明三件事：鼠标回调函数 f() 返回鼠标在窗口中的位置、其状态 (GLUT_UP 或 GLUT_DOWN) 以及鼠标事件的来源 (GLUT_LEFT_BUTTON, GLUT_MIDDLE_BUTTON 或 GLUT_RIGHT_BUTTON)。

使用鼠标最简单也最通用的方法就是将鼠标在屏幕上的位置返回给应用程序。当鼠标的某一键被按下或释放时，就会产生一个鼠标事件。回调函数将以像素为单位返回鼠标的位置，位置的度量以窗口的左上角为参考点。该回调函数同时将返回引发该事件的按键，以及该按键在引发该事件后的状态。在我们的例子中，只考虑由按键引发的事件，所以我们要检查的状态为 GLUT_DOWN。所以，如果想用鼠标左键来结束程序，我们可在主函数中注册一个鼠标回调函数：

```
glutMouseFunc(mymouse);
```

然后，我们就可以像下面这样使用该回调函数：

```
void mymouse(int button, int state, int x, int y)
{
    if(state == GLUT_DOWN && button == GLUT_LEFT_BUTTON)
        exit(0);
}
```

鼠标回调函数可用于交互式地控制所绘对象。假定我们想绘制一个矩形，构成一条对角线的两个角点由鼠标右键的连续两次单击确定，且该矩形各边与屏幕对齐。鼠标左键仍用于

程序的退出。

我们必须面对的第一个问题是确定完成绘制的代码应放在程序的哪个部分：即，我们应将绘制该矩形的 OpenGL 函数放在哪里？有两种简单的策略。第一，我们将所有的工作都在鼠标回调函数中完成。这样，当用户首次按下右键时，我们将该位置保存起来；第二次我们按下右键时，将该矩形绘制出来。这样，该鼠标回调函数看起来会像下面这样：

```
int hh; /*global for viewport height*/

void mymouse(int button, int state, int x, int y)
{
    static bool first = true;
    static int xx, yy;
    if(state == GLUT_DOWN && button == GLUT_LEFT_BUTTON)
        exit(0);
    if(state == GLUT_DOWN && button == GLUT_RIGHT_BUTTON)
    {
        if(first)
        {
            xx = x;
            yy = hh - y;
            first = !first;
        }
        else
        {
            first = !first;
            glClear(GL_COLOR_BUFFER_BIT);
            glBegin(GL_POLYGON);
                glVertex2i(xx, yy);
                glVertex2i(xx, hh - y);
                glVertex2i(x, hh - y);
                glVertex2i(x, yy);
            glEnd();
        }
    }
}
```

【61~62】

在前面的代码中，有几点需要特别注意。最重要的当属对由鼠标回调函数所返回的 y 坐标所作的转换。这种转换是必须的，因为返回给鼠标回调函数的 y 值是在屏幕坐标系中度量的，而屏幕坐标系的原点位于屏幕左上角。用于裁剪窗口和几何对象规格的值是以世界坐标定义的，而世界坐标系的原点位于左下角。但是，为了能够进行坐标变换，我们必须直到屏幕窗口的高度，而且在程序执行过程中，该高度值将会随用户的交互而发生改变。该

问题的解决方案是将窗口高度(hh)设为全局变量, 这样该值就会随重绘回调函数不断自动更新。

提示 将窗口尺寸设为全局变量以便将鼠标位置正确地转换为世界坐标。

【62】

在前面的例子中, 顶点的指定使用了整数形式的 `glVertex2i()`。用于定位顶点的值是从鼠标回调函数中获取的, 所以都是整型数。稍作思考就会发现, 我们在重绘回调函数中定义的裁剪窗口实在太小了。本例的一个更好的选择是借助重绘回调函数使裁剪窗口与屏幕窗口的尺寸相匹配, 如下所示:

```
int ww, hh; /*globals for viewport height and width*/

void myReshape(GLsizei w, GLsizei h)
{
    glMatrixMode(GL_PROJECTION);
    glLoadIdentity();
    gluOrtho2D(0.0, (GLfloat)w, 0.0, (GLfloat) h);
    glMatrixMode(GL_MODELVIEW);
    glViewport(0, 0, w, h);
    ww = w;
    hh = h;
}
```

由于我们已经编写了上述代码, 便不再需要显示回调函数; 即所有的工作都在鼠标回调函数中完成。在实际应用中, 由于 GLUT 坚持每个程序都应该具有一个显示回调函数, 我们可以编写一个函数体为空的显示回调:

```
void mydisplay() {}
```

一种更通用的策略是将绘制函数统一放在显示回调函数中; 将其他回调函数统一用于状态的修改。虽然在许多情况下, 我们并不是总能做到这一点, 但在本例中我们可以通过这两个回调函数很容易做到:

```
GLint x1, y1, x2, y2;
int hh;

void mymouse(int x, int y, int button, int state)
{
    static bool first = true;
    int xl, yy;
    if(state == GLUT_DOWN && button == GLUT_LEFT_BUTTON)
        exit(0);
    if(state == GLUT_DOWN && button == GLUT_RIGHT_BUTTON)
```



```

{
    if(first)
    {
        x1 = x;
        y1 = hh - y;
        first = !first;
    }
    else
    {
        first = !first;
        x2 = x;
        y2 = hh - y;
    }
    glutPostRedisplay();
}
}

void mydisplay()
{
    glClear(GL_COLOR_BUFFER_BIT);
    glBegin(GL_POLYGON);
    glVertex2i(x1, y1);
        glVertex2i(x1, hh - y2);
        glVertex2i(x2, hh - y2);
        glVertex2i(x2, y1);
    glEnd();
}

```

【63~64】

3.7 鼠标的移动

当鼠标移动时，无论键是否被按下，也会引发一些事件。如果鼠标在键被按下的同时发生移动，所产生的事件称为移动事件(move event)。如果鼠标在移动时没有键被按下，则称此时引发的事件为被动移动事件(passive move event)。当鼠标进入或离开窗口时，将产生进入事件(entry event)。移动回调函数由 `glutMotionFunc()` 指定，被动移动回调函数由 `glutPassiveMotionFunc()` 指定。这两种回调函数都将返回鼠标的位置。

```

void glutMotionFunc(void (* f)(int x, int y))
void glutPassiveMotionFunc(void (* f)(int x, int y))

```

指定移动和被动移动回调函数。鼠标的位置(x, y)将返回给这两类回调函数。

当鼠标移动一个系统相关的小量时，将会引发一些事件。移动回调的一个应用是在拖动鼠标的同时绘制曲线。只要有一个键被按下，我们就可在回调函数中通过一行代码来对折线

进行扩展:

```
int points[2][100];
int i = 0;
void mymotion(int x, int y)
{
    .
    .
    .
    points[0][i] = x;
    points[1][i] = h - y;
    i++;
    .
    .
    .
}
```

在上述代码中, 鼠标的垂直位置被转换到 OpenGL 坐标系中。当鼠标被释放之后, 可用该数组存储的数据来绘制一段折线。

当鼠标进入或离开 OpenGL 窗口时将产生一个进入事件。该类事件的回调函数可用函数 `glutEntryFunc()` 来注册。

```
void glutEntryFunc(void (* f)(int state))
```

指定进入回调函数。返回的状态为 GLUT_ENTERED 或 GLUT_LEFT。

3.8 菜 单

交互式程序的特点是具有类型非常详尽的交互, 绝不只是到目前为止我们所讨论过的键盘和鼠标。所有的现代窗口系统都支持一个窗口小组件集, 这些组件是用户可与之进行交互的特殊类型的窗口。典型的小组件包括菜单、按钮、滑杆以及对话框。大多数小组件都由系统特定的工具集提供, 这些工具集往往能够充分利用窗口系统的全部性能。由于 GLUT 简单而且可移植, 它只提供了一个规模较小的组件集。当然, 由于组件(如按钮或表盘)可被视作图形对象, 所以我们也可用 OpenGL 和 GLUT 来创建任意特定类型的组件。

GLUT 的确提供了一种重要的组件——菜单。通常这些菜单都被实现为弹出式(pop-up)菜单, 它往往在鼠标的某一键被按下时出现。定义一个菜单需要三个步骤。首先, 必须确定菜单中有哪些选项, 即菜单的每行将显示哪些字符串。其次, 必须为菜单中的每一行关联一种特定操作。最后, 我们还必须将每个菜单与鼠标按键建立关联。菜单的工作机制与其他回调函数非常类似。当用户释放使菜单弹出的鼠标按键时, 一个表明鼠标所在行的标识符即

被传递给菜单回调函数。

【65】

菜单通常在主函数或主函数调用的初始化函数中被创建。GLUT 允许创建级联菜单，方法是令一个菜单项指向一个子菜单。顶级菜单由函数 `glutCreateMenu()` 创建，该函数将为菜单指定回调函数，并将返回一个标识该菜单的整数值。

```
int glutCreateMenu(void (* f)(int value))
```

创建一个使用回调函数 `f()` 的顶级菜单，`f()` 将由菜单项传入一个整数值 `value`。`glutCreateMenu` 将为所创建的菜单返回一个惟一的整型标识符。

所创建的菜单将称为当前菜单，随后的代码将会为其指定菜单项。当前菜单可由函数 `glutSetMenu()` 改变。

```
void glutSetMenu(int id)
```

将当前菜单设为标识符为 `id` 的菜单。

菜单项可借助函数 `glutAddMenuEntry()` 加入当前菜单中。每个菜单项都包含两部分内容：一个用于显示的菜单项字符串以及被选中时所返回的一个整数值。

```
void glutAddMenuEntry(char * name, int value)
```

为当前菜单增加一个名称为 `name` 的菜单项；`value` 将被返回给菜单回调函数。

最后，我们可借助函数 `glutAttachMenu()` 将当前菜单依附到鼠标的某一个键上。

```
void glutAttachMenu(int button)
```

将当前菜单依附到一个指定的鼠标按钮上 (`GLUT_RIGHT_BUTTON`、`GLUT_MIDDLE_BUTTON` 或 `GLUT_LEFT_BUTTON`)。

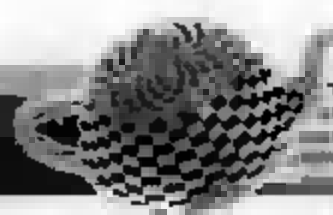
假定我们想用鼠标右键弹出一个有两个菜单项的菜单：一个可清屏，另一个将结束程序。我们可在 `init()` 函数或 `main()` 函数中对该菜单进行设置：

```
id = glutCreateMenu(mymenu);
glutAddMenuEntry("Clear Screen", 1);
glutAddMenuEntry("Exit", 2);
glutAttachMenu(GLUT_RIGHT_BUTTON);
```

【66】

注意，由于本例中只有一个菜单，所以不需要其标识符。现在我们来编写菜单回调函数 `mymenu()`。

```
void mymenu(int value)
{
    if(value == 1)
    {
        glClear(GL_COLOR_BUFFER_BIT);
        glutSwapBuffers(); /* or glFlush() */
    }
}
```

```
if(value == 2) exit(0);  
}
```

子菜单

我们可为一个菜单添加若干子菜单。子菜单拥有自己的名称，该名称作为父菜单中的一项而出现。当用户将鼠标移动到该项上时，子菜单将自动弹出。为子菜单添加菜单项需要借助函数 `glutAddSubMenu()`。

```
void glutAddSubMenu(char * name, int menu)
```

增加一个子菜单项 `name` 作为当前菜单的下一项。`menu` 为当该子菜单被创建时所返回的子菜单 `id`。

创建子菜单的方式与主菜单大体相同，但是我们必须首先创建子菜单，以便当创建主菜单时能够将该子菜单的标识符传给 `glutAddSubMenu()` 函数。

3.9 NULL 回调函数

回调函数可在程序执行期间重新定义，方法是在一个恰当的函数中命名一个新的回调函数。有时，我们只想注销一个回调函数。例如，假定在程序的一些执行点上，我们不再想要一个已定义的空闲回调函数。我们只需将 `NULL` 作为新回调函数的名称，并将其传入回调函数注册函数中即可。

```
glutIdleFunc(NULL);
```

3.10 子窗口与多窗口

在许多应用程序中，无论是交互式程序还是非交互式程序，我们可能都需要使用多窗口。例如，在一个 CAD 应用程序中，我们可能希望一个窗口负责创建对象，另一个负责操作，另一个用于显示警告信息。我们也许希望窗口能够在程序运行期间能够创建或销毁。函数 `glutCreateWindow()` 将为每个创建的窗口返回一个整型标识符。函数 `glutDestroyWindow()` 允许我们将窗口销毁。

【67~68】

```
int glutCreateWindow(char * name)
```

创建一个顶级窗口 `name`，并为其返回一个标识符。

```
void glutDestroyWindow(int id)
```

销毁标识符为 `id` 的顶级窗口。

当一个窗口被创建后，它就成为当前窗口。通常，图元都将被绘制到当前窗口中。可借助

函数 `glutSetWindow()` 来改变当前窗口。

```
void glutSetWindow(int id)
```

将当前窗口设为标识符为 `id` 的窗口。

每个窗口都可以拥有自身的属性(称为窗口的上下文)。所以,如果在创建窗口之前,我们先调用函数 `glutInitDisplayMode(properties)`,则所创建的窗口将具有特定的属性。我们可以使用如 `glutInitWindowSize()` 之类的函数来影响当前窗口。同样,每个窗口都拥有自己的回调函数。所以,我们可有多个窗口,每个窗口都依据应用程序的需要而具有不同的属性。

我们也可以创建任何窗口的子窗口,包括顶级窗口和子窗口。子窗口也可拥有自己的上下文。每个子窗口都可通过函数 `glutCreateSubWindow()` 定义为其父窗口的一个子矩形。

```
int glutCreateSubWindow(int parent, int x, int y,  
int width, int height)
```

为 `parent` 创建一个子窗口,并返回其标识符。子窗口的原点位于 `(x, y)`,其宽度和高度分别为 `width` 和 `height`,单位为像素。

当子窗口被创建时,它就称为当前窗口。函数 `glutPostWindowRedisplay()` 用来通知某个特定窗口,需要进行重新显示。

```
void glutPostWindowRedisplay(int winid)
```

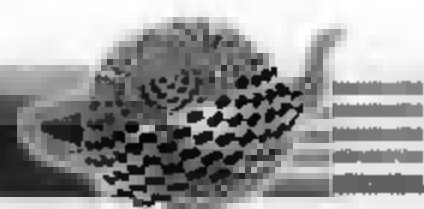
通知标识符为 `winid` 的窗口重新显示。

【68】

3.11 例程: single_double.c

下面的示例程序将综合我们之前所介绍的许多概念。它将通过在两个具有不同属性的窗口中分别显示一个旋转的正方形来演示单缓存和双缓存。我们还可随意终止该正方形的旋转。我们先从主函数开始分析。

```
int singleb, doubleb; /*window ids*/
int main(int argc, char**argv)
{
    glutInit(&argc, argv);
    /*create a single buffered window*/
    glutInitDisplayMode(GLUT_SINGLE|GLUT_RGB);
    singleb = glutCreateWindow("single buffered");
    myinit();
    glutDisplayFunc(displays);
    glutReshapeFunc(myReshape);
    glutIdleFunc(spinDisplay);
    glutMouseFunc(mouse);
```

```

glutKeyboardFunc(mykey);

/*creat a double buffered window to right*/

glutInitDisplayMode(GLUT_DOUBLE|GLUT_RGB);
glutInitWindowPosition(310, 0);
doubleb = glutCreateWindow("double buffered");
myinit();
glutDisplayFunc(displayd);
glutReshapeFunc(myReshape);
glutIdleFunc(spinDisplay);
glutMouseFunc(mouse);
glutCreatMenu(quit_menu);
glutAddMenuEntry("quit", 1);
glutAttachMenu(GLUT_RIGHT_BUTTON);

/*enter event loop*/

glutMainLoop();
}

```

注意，每个窗口都为自己定义了若干回调函数。虽然有一些回调函数为这两个窗口所共享，每个窗口的回调函数都可在后续阶段彼此独立地进行修改。该旋转正方形可按先前的那种方式定义，即用空闲回调函数 `SpinDisplay()` 使其不断旋转。 【69~70】

```

#define DEG_TO_RAD 0.017453

void display()
{
    glClear(GL_COLOR_BUFFER_BIT);
    glBegin(GL_POLYGON);
        glVertex2f(sin(DEG_TO_RAD*theta),
                    cos(DEG_TO_RAD*theta));
        glVertex2f(-sin(DEG_TO_RAD*theta),
                    cos(DEG_TO_RAD*theta));
        glVertex2f(-sin(DEG_TO_RAD*theta),
                    -cos(DEG_TO_RAD*theta));
        glVertex2f(sin(DEG_TO_RAD*theta),
                    -cos(DEG_TO_RAD*theta));
    glEnd();
    glutSwapBuffers();
}

void spinDisplay(void)

```



```

{
    /*increment angle*/
    theta += 2.0
    if (theta > 360.0) theta -= 360.0;

    /*draw single buffer window*/
    glutSetWindow(Singleb);
    glutPostWindowRedisplay(singleb);

    /*draw double buffer window*/
    glutSetWindow(doubleb);
    glutPostWindowRedisplay(doubleb);
}

```

【70~71】

这两个窗口使用了相同的鼠标回调函数。该函数允许用户将空闲回调函数设为 NULL(以后再恢复旋转)而同时终止两个窗口中的旋转。虽然这两个窗口使用了相同的回调函数,每个窗口都维护其自身的状态。所以,如果将一个窗口中的旋转终止,我们并未改变另一个窗口的状态。为了改变其他窗口的回调函数,我们必须首先在其他窗口内单击鼠标。

```

void mouse(int btn, int state, int x, int y)
{
    if(btn == GLUT_LEFT_BUTTON && state == GLUT_DOWN)
        glutIdleFunc(SpinDisplay);
    if(btn == GLUT_MIDDLE_BUTTON && state == GLUT_DOWN)
        glutIdleFunc(NULL);
}

```

重绘回调函数对裁剪窗口进行了设置:

```

void myReshape(int w, int h)
{
    glViewport(0, 0, w, h);
    glMatrixMode(GL_PROJECTION);
    glLoadIdentity();
    gluOrtho2D(-2.0, 2.0, -2.0, 2.0);
    glMatrixMode(GL_MODELVIEW);
    glLoadIdentity();
}

```

为了说明回调函数中的另外两个,我们同时使用了键盘回调函数和菜单回调函数来终止程序。但是每个回调函数只被赋予一个窗口,所以每个窗口可以使用一种不同的方法来退出。

```

void mykey(unsigned char key, int x, int y)
{

```



```

    if(key == 'Q' || key == 'q') exit(0);
}

void quit_menu(int id)
{
    if(id == 1) exit(0);
}

```

【71】

3.12 显示列表

OpenGL 中的标准绘制模式称为立即模式(immediate mode)。只要各种图元在程序中被定义,就立即被传入 OpenGL 流水线。此时这些图元就不处于系统中;只有它们保存在颜色缓存中的图像会被显示在屏幕上。当情况发生变化,屏幕需要重绘时,我们必须重新生成这些图元(这部分工作通常在显示回调函数中完成),然后再次将这些图元传入流水线。依据所绘场景的复杂程度,这个重新显示的过程可能会相当耗时,尤其是当应用程序位于网络的一端(客户端),而绘制器和显示器(图形服务器)位于网络的另一端时。在这种情况下,所有的图元,无论其是否会出现于屏幕上,在被传入流水线之前,必须接受连接速度可能非常缓慢的事实。

在保留模式(retained mode)下,图元集合及其他信息就可作为对象保存在服务器中,这就避免了代价较高的传输和重新生成的问题。OpenGL 定义这些对象的机制是依靠显示列表(display list)。可将显示列表视为一种类型的图形文件,在该文件中我们可以放置绘制和状态更新命令。使用显示列表通常包含 4 个步骤:打开显示列表、为其命名、添加命令以及关闭显示列表。显示列表以一种内部格式将其内容保存起来,这样在重新显示时可以取得较快的速度。当我们创建一个显示列表时,我们实际上是在说对其进行编译。显示列表驻留在服务器中,它可通过简单的 OpenGL 调用来执行。

假定我们想要创建一个显示列表,其中定义了一个红色正方形。创建显示列表之前,需要首先调用 `glNewList()` 函数,然后像往常那样定义一个正方形,定义完毕之后,还需调用 `glEndList()` 函数。

```

#define RED_SQUARE 1
glNewList(RED_SQUARE, GL_COMPILE);
    glPushAttrib(GL_CURRENT_BIT);
    glColor3f(1.0, 0.0, 0.0);
    glRectf(-1.0, -1.0, 1.0, 1.0);
    glPopAttrib();
glEndList();

```


标记 `GL_COMPILE` 表示通知 OpenGL 将显示列表放在服务器中，但其内容不被显示。如果我们想让显示列表在创建时即被显示，只需使用标记 `GL_COMPILE_AND_EXECUTE`。注意，我们将当前属性(它包含了当前颜色值)做了入栈和出栈操作。我们必须这样做以避免由于设置新颜色而造成的状态改变，并进而影响后续绘制的结果。我们常常通过启动一个显示列表来防止由于状态改变造成的不可预见的副作用，方法是在显示列表的起始处将矩阵和当前状态入栈，而在显示列表的末尾执行出栈操作。虽然对矩阵和属性执行入栈和出栈操作能够免受不希望的状态变动的影响，但这种操作会降低程序的运行效率。【72】

```
void glNewList(GLuint name, GLenum mode)
```

创建一个新的显示列表，其 ID 为无符号整型数 `name`。`mode` 的值 (`GL_COMPILE`、`GL_COMPILE_AND_EXECUTE`) 决定了命令被编译到显示列表中时是否被执行。

```
void glEndList()
```

结束显示列表的定义。

我们可借助函数 `glCallList()` 来执行显示列表。

```
void glCallList(GLuint name)
```

执行显示列表 `name`。

大多数 OpenGL 函数都可被放入显示列表中。在所有我们目前已接触的函数中，只有那些返回状态变量的函数(如 `glGet*()`)不允许放在显示列表中。显示列表可由其他显示列表在内部进行调用，利用这一点可用来创建层次模型。

显示列表一经创建便不可进行修改。如果我们希望改变显示列表的内容，可以先创建一个满足要求的显示列表，然后用函数 `glDeleteLists()` 将当前显示列表删除即可。

```
void glDeleteLists(GLuint first, GLsizei number)
```

删除自 `first` 开始的 `number` 个显示列表。

3.12.1 多个显示列表

通常我们会同时使用多个显示列表。我们可借助函数 `glCallLists()` 批量执行多个显示列表。

显示列表的标识是相对于列表基数算起的，列表基数可通过函数 `glListBase()` 来设置，并存储在一个数组中。

```
void glListBase(GLuint offset)
```

设置 `glCallLists()` 所使用的偏移量。默认值为 0。

```
void glCallLists(GLsizei num, GLenum type, GLvoid * lists)
```

依据存储在类型为 `type` 的数组 `lists` 中的整数值执行 `num` 个显示列表。当前偏移量被叠加到 `lists` 中的每个整数上，这样就得到了所要执行的显示列表的 ID。【73】

使用多个显示列表时，我们需要为新列表分配一些未被使用的 ID。函数 `glGenLists()` 返回一个连续的未经使用的整数集合，可用来定义新的显示列表。

GLuint glGenLists(GLsizei n)

返回可用于新显示列表的 `n` 个连续整数标识中的第一个。

3.12.2 显示列表与文本

如果能够为每个字符使用一个显示列表，则文本就可更高效地得到绘制。无论我们是在使用笔划字体或位图字体，字符的描述都位于服务器中。为了生成显示设备上的一个字符串，我们可为每个字符进行一次函数调用。

假定我们决定使用 GLUT 的 10 磅大小的 Times-Roman 位图字体。我们需要 256 个显示列表 (假定我们甚至为非打印字符都生成了显示列表)。可用以下方式获取所需的 ID:

```
base = glGenLists(256);
```

现在我们来设置显示列表:

```
for(i = 0; i < 256; i++)
{
    glNewList(base + i, GL_COMPILE);
    glutBitmapCharacter(GLUT_BITMAP_TIMES_ROMAN_10, i);
    glEndList();
}
```

由于我们正在使用一种 GLUT 位图字体，光栅位置将自动向右移动的距离为字符的宽度。

接下来我们对列表基数进行设置:

```
glListBase(base);
```

这样就可用每个显示列表的 ASCII 码实现引用。现在假定我们有一个字符串:

```
char * text;
```

我们可用如下单次函数调用将该字符串绘制:

```
glCallLists((GLint)strlen(text), GL_UNSIGNED_BYTE, text);
```

这里我们使用标准函数 `strlen()` 来获取该字符串的长度，该长度也反映了我们必须执行的显示列表的数目。接下来，字符串就可被视作一个无符号字节型数组，该数组中的每个元素就是为得到所要执行的显示列表的 ID 而需叠加在列表基数上的数(每个字符的 ASCII 码)。【74~75】

如果我们使用的是笔划字体，程序则没有什么本质的不同，只是我们需要处理绘制完每个字符后，需要向右移动的距离。这个问题的最佳处理方法是通过使用 OpenGL 变换，这部

分内容我们将在第5章介绍。

3.12.3 显示列表与对象

虽然在许多情况下，我们的显卡性能已经足够好，以至于我们很难察觉显示列表较之立即模式所带来的速度上的提高。显示列表为用户提供了一种比立即模式更符合面向对象的思想的程序构建方式。显示列表的这种包含对象属性的能力(如正方形的红色)使得颜色可以与几何实体绑定在一起，即便 OpenGL 自身认为颜色是状态的一部分。显示列表更强大的地方在于可在一个给定的显示列表内执行其他的显示列表，这就提供了一种创建层次对象的方法。例如，让我们来考虑如何创建一个卡通风格的人脸对象。每张脸都有两只眼、两只耳朵、一张嘴和一个鼻子。则绘制人脸的程序看起来大致如下：

```
glNewList(FACE, GL_COMPILE);
/*position first eye*/
    glCallList(EYE);
/*position second eye*/
    glCallList(EYE);
/*position first ear*/
    glCallList(EAR);
/*position second ear*/
    glCallList(EAR);
/*position nose*/
    glCallList(NOSE);
/*position mouth*/
    glCallList(MOUTH);
glEndList();
```

注意，我们可通过修改任意一个定义了这些部件的显示列表而对人脸作出改动。在调用显示列表之前只要将人脸位置定好，则其余部件也就相应得到了确定。

3.13 拾取和选择模式

现在我们来讨论一种在许多交互式程序中具有基础性地位的一种操作——拾取(picking)。拾取是对屏幕中对象进行定位的操作。该操作为 OpenGL 程序员提出了一些问题。首先，我们必须对对象进行界定。我们已经了解的一种方式就是通过显示列表来定义。但是还有其他方式。我们可以创建某种类型的标签系统，以便为程序的各部分分别赋予一个标签。其次，我们必须精确地对“拾取目标”进行定义。应对这些问题需要讲究一些策略。我们单击的位置究竟是在构成对象的图元上，还是对象附近的位置？如果对象之间出现了交叠



该如何处理？针对这些问题，我们可以提出多种解决方案，只是也许其中一些是系统相关的。【75~76】

一个更有难度的问题是如何在 OpenGL 流水线内部实现拾取。流水线系统中存在的问题是我们无法从鼠标单击的位置直接向后定位到与鼠标的屏幕位置最近的那个图元。开发人员已经找到了该问题的多种解决方案。一些方法基于鼠标的位置用查询表来为对象定位。另外一些方法则进行多次绘制，并读取帧缓存中的内容，这部分内容我们将在第 7 章中进行讨论。

OpenGL 提供了一种有些复杂的处理方法——选择模式(selection mode)。在这种模式下，进行拾取操作时所要付出的额外代价就是在每次拾取时都需要进行一次额外的绘制。选择模式的基本思想就是场景中的对象可被绘制，但不能绘制到正在被显示的颜色缓存中。当我们绘制对象时，OpenGL 通过判定对象是否位于某个特定的裁剪体中来跟踪那些对象被绘制到选择区中，该裁剪体不必与绘制和显示时所使用的裁剪体相同。如果我们在使用双缓存，我们可将后台缓存既用于形成所要显示的图像，也用于拾取，但必须注意前台缓存和后台缓存交换的时机。

为了实现拾取，需要多个步骤以及大量的函数调用。现在我们来研究每一个步骤，并将其放在一个简单程序中。函数 `glRenderMode()` 允许我们选择三种模式中的一种：绘制模式(`GL_RENDER`，绘制到颜色缓存)、选择模式(`GL_SELECT`)或反馈模式(`GL_FEEDBACK`)。反馈模式可用于获取被绘制图元的列表。由于该模式的应用场合很有限，在这里我们不再对其展开论述。

GLint glRenderMode(GLenum mode)

选择以下三种模式之一：绘制模式(`GL_RENDER`)、选择模式(`GL_SELECT`)或反馈模式(`GL_FEEDBACK`)。该函数的返回值可用于确定选择模式下的命中次数或反馈模式下的图元数量。

当我们进入选择模式并绘制一幅场景时，每个被绘制到裁剪体内部的图元都会生成一条称为命中(hit)的消息，该消息存处于一个称为名称堆栈(name stack)的缓存中。可用函数 `glSelectBuffer()` 来指定存放返回选择数据时所使用的数组。共有 4 个函数分别用于名称堆栈的初始化、信息的入栈和出栈以及对堆栈中的元素进行操作。我们所产生的信息称为命中列表(hit list)，在绘制结束之后可检查该命中列表以检索拾取操作所需要的信息。【76~77】

void glSelectBuffer(GLsizei n, GLuint *buff)

指定将选择数据存入容量为 `n` 的数组 `buff` 中。

void glInitNames()

初始化名称堆栈。

void glPushName(GLuint name)

将 `name` 压入名称堆栈。


```
void glPopName()
```

将栈顶名称从名称堆栈中弹出。

```
void glLoadName(GLuint name)
```

用 `name` 取代名称堆栈中的栈顶元素。

一般说来, 我们所希望识别的每一个对象都是一个图元集, 每个集合中所有的元素我们都为其赋予了相同的整数标识符。在绘制对象之前, 我们要将其名称加载(即取代栈顶名称)到名称堆栈中。不能将名称加载到一个空的名称堆栈中, 所以我们通常在初始化名称堆栈时, 将一个未经使用的名称入栈, 如下所示:

```
glInitNames();
```

```
glPushName(0);
```

提示 使用选择模式时, 务必确保名称堆栈已用一个未被使用的名称初始化。

通常使用鼠标回调函数来进入或退出选择模式。返回绘制模式时, 函数 `glRenderMode()` 将返回已经处理的命中次数。然后我们对命中列表进行检查。也可在鼠标回调函数中对裁剪体进行修改, 以便在所期望的区域能够被命中, 通常这类区域都在鼠标位置附近。

我们可以两种方式对裁剪体进行设置。可借助函数 `gluOrtho2D()`(或使用其他观察函数)对视域体进行简单的设置。可能我们希望用 `glPushMatrix()` 将当前裁剪体保存起来。然后所有落在新裁剪体内部的图元都将产生一次命中, 而无论此时鼠标位置在何处。这种做法对选择是适用的, 但是当我们想拾取时, 只希望那些绘制在光标附近的对象被命中。 【77】

假定我们想要得到被绘制在一个以光标为中心的用户自定义的小矩形中的所有对象, 该矩形的尺寸可以反映拾取灵敏度的大小。该矩形是视口中的一个很小的子区域。给定视口、光标的位置、该矩形的尺寸以及观察参数, 我们便可求出新的观察参数以使所有位于新裁剪窗口中的对象都被绘制到整个视口中。所以, 如果我们恰当地选择新的裁剪窗口以使仅在光标附近的对象位于该窗口中, 就可对可被拾取的对象的一个子集产生命中。从数学角度看, 新窗口和视口的计算用到了一些比例变换的知识, 另外还涉及了对投影矩阵求逆的运算。我们可借助 GLU 函数 `gluPickMatrix()` 来让 OpenGL 替我们完成上述计算。在选择模式下, 该函数应该在 `gluOrtho2D()` 函数之前进行调用。

```
void gluPickMatrix(GLdouble x, GLdouble y, GLdouble w,
GLdouble h, GLint *vp)
```

创建一个用于拾取的投影矩阵, 该矩阵可将绘制区域限制在一个以窗口坐标 (x, y) 为中心的 $w \times h$ 的视口 `vp` 的子区域中。

假定我们想在初始化阶段或重绘回调函数中为绘制模式设置观察条件, 则鼠标回调函数的形式为:


```

#define size 500
#define N 3
void mouse(int button, int state, int x, int y)
{
    GLuint nameBuffer[SIZE];
    GLint hits;
    GLint viewport[4];
    if(button == GLUT_LEFT_BUTTON && state == GLUT_DOWN)
    {
        /*initialize the name stack*/
        glInitNames();
        glPushName(0);
        glSelectBuffer(SIZE, nameBuffer);

        /*set up viewing for selection mode*/
        glGetIntegerv(GL_VIEWPORT, viewport);
        glMatrixMode(GL_PROJECTION);

        /*save original viewing matrix*/
        glPushMatrix();
        glLoadIdentity();

        /*N*N pick area around cursor*/
        /*must invert mouse y to get in world coordinates*/
        gluPickMatrix((GLdouble)x, (GLdouble)
            (viewport[3] - y), N, N, viewport);

        /*same clipping window as in reshape callback*/
        gluOrtho2D(xmin, xmax, ymin, ymax);
        glRenderMode(GL_SELECT);
        draw_objects(GL_SELECT);
        glMatrixMode(GL_PROJECTION);

        /*restore viewing matrix*/
        glPopMatrix();
        /*return to normal render mode*/
        hits = glRenderMode(GL_RENDER);
        /*process hits from selection mode rendering*/
        processHits(hits, nameBuff);
        /*normal render*/
        glutPostRedisplay();
    }
}

```



```

void display()
{
    glClear(GL_COLOR_BUFFER_BIT);
    draw_objects(GL_RENDER);
    glFlush();
}

```

【78~79】

注意，我们必须调用直接绘制这些对象的函数而不能在鼠标回调函数中使用函数 `glutPostRedisplay()`。因为 GLUT 在每次遍历事件循环时只执行一次显示回调函数。下面是一个绘制了两个有部分交叠的矩形的简单示例：

```

void drawObjects(Glenum mode)
{
    if(mode == GL_SELECT) glLoadName(1);
    glColor3f(1.0, 0.0, 0.0);
    glRectf(-0.5, -0.5, 1.0, 1.0);
    if(mode == GL_SELECT) glLoadName(2);
    glColor3f(0.0, 0.0, 1.0);
    glRectf(-1.0, -1.0, 0.5, 0.5);
}

```

注意，在本例中，我们只需改变名称堆栈中的栈顶元素。如果有一个层次对象，其各部件可能都在光标附近，我们可使用函数 `glPushName()`，以使命中一次便可在名称堆栈中增加多个名称。对于具有许多部件的对象，所有靠近光标的部件的名称在这些部件命中之后都将压入名称堆栈。

最后，我们需要编写能够对名称堆栈进行检查的函数。在我们的程序中，将输出每次鼠标单击后有多少名称被存入名称堆栈以及被拾取对象的名称。

命中缓存为每次命中保存了一条记录。所以，每一个绘制在光标附近的对象都将产生一条记录。如果命中列表中没有任何名称(即没有任何图元被绘制在光标附近)，则说明当我们进行名称堆栈的初始化时，命中记录为 0。否则，我们将找到三种类型的信息，这些信息都以整数的形式存储。首先，当产生一次命中时，名称堆栈中有名称的数量。在本例中，该值只可能为 1。接下来是两个整数，它们分别为命中图元给出了经过比例变换后的最小深度和最大深度值。由于我们是在二维空间中考虑问题，这些值并不能为我们提供有价值的信息，故可将其忽略。而对于三维应用程序，我们可用这些值来确定被拾取的前方对象。这三个整数之后是名称堆栈中的各个元素。在本例中，我们将检索到红色或蓝色矩形的标识符(1 或 2)。下面的函数会将上述信息输出：

```

void processHits(Glint hits, GLuint buffer[])

```



```

{
    unsigned int i, j;
    GLuint names, *ptr;
    printf("hits = %d\n", hits);
    ptr = (GLuint*)buffer;
    /*loop over number of hits*/

    for(i = 0; i < hits; i++)
    {
        names = *ptr;

        /*skip over number of names and depths*/
        ptr += 3;

        /*check each name in record*/
        for(j = 0; j < names; j++)
        {
            if(*ptr == 1)printf("red rectangle\n");
            else printf("blue rectangle\n");
            /*go to next bit record*/
            ptr++;
        }
    }
}

```

【79~81】

该拾取方法有些复杂，但总是可行的。通常使用的一种替代方法是用不同的颜色来绘制不同的物体，单击时读取光标所在位置的颜色值。一般说来，所有的拾取方法都要求对场景进行额外的绘制。

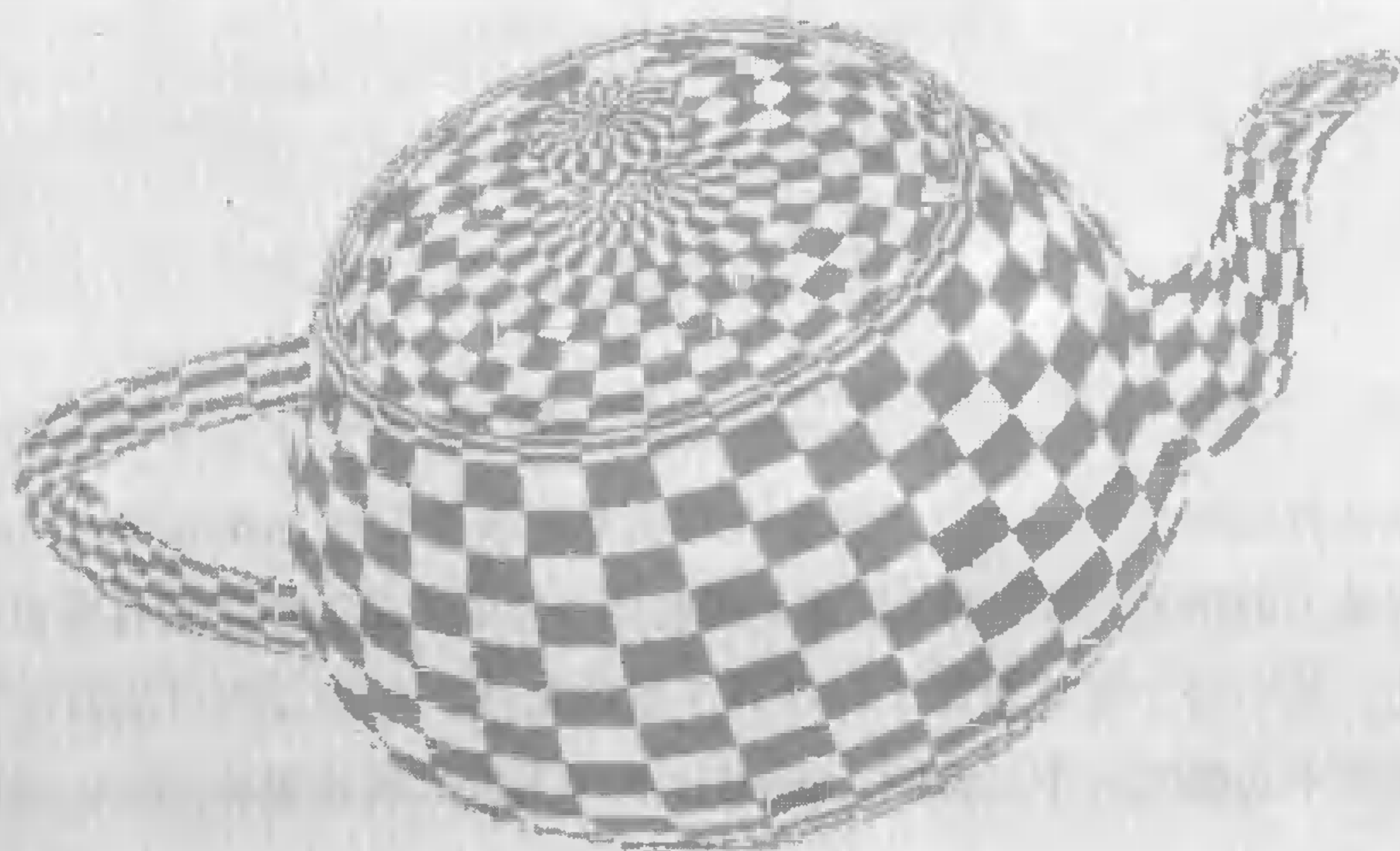
3.14 编程练习

1. 创建一个简单的绘制程序，使得可用鼠标来创建一些简单的形状，如线段、矩形和三角形，并可通过菜单来选择各种模式，并允许用户改变绘制颜色。
2. 通过使用鼠标移动回调函数为习题 1 中的程序赋予自由形式的绘制功能，使得用户可利用该功能来绘制对曲线逼近表示的多边形。
3. 使用显示列表编写一个允许用户创建人脸图形(见 3.12 节)的交互程序。菜单中可以包含绘制所需的不同部件，例如鼻子或眼睛。
4. 从第 2 章中任选一个程序，将其修改为交互式的。例如，对于所有使用了三角形的程序，允许用户用鼠标给出各顶点。
5. 使用 GLUT 和 OpenGL 创建一些附加组件(widget)，如表盘(dial)和按钮。

第 4 章

三维编程基础

现在我们以一种简单的方式来介绍 OpenGL 三维编程。我们首先来介绍观察场景的正交(orthogonal)摄像机，其定义方式与二维程序中的摄像机定义非常类似。我们将使用与二维程序中完全相同的图元，并将介绍一些包含在 GLU 和 GLUT 库中的预定义对象。最后，我们将讨论透视投影。



4.1 摄像机与对象

将三维图形系统形成图像的过程概念化的基本模式称为合成摄像机模型(synthetic camera model)。借助该模型,我们可以模拟现实世界中的成像系统(如摄像机和人的视觉系统)的功能。无论观察的主体是摄像机还是我们的视觉系统,合成摄像机模型都是建立在为形成图像所做观察的基础上。我们需要两个独立的实体:一个几何对象集以及这些对象的观察者。每个实体都可彼此独立地进行指定。

由摄像机和几何对象集(虽然它们本身是三维对象)所生成的图像是二维的。将对象的规格和摄像机整合的过程称为投影(projection),该过程是由 OpenGL 在其流水线内部完成的。所以,应用程序只需要指定摄像机和场景中的对象。图 4.1 展示了该模型。称为投影线(projector)的直线穿过图像中的每一点,并经过摄像机的光心(投影中心)。经过对象上某一点的投影线与焦平面的交点即为该点在图像中的投影点所在的位置。图 4.2 将图 4.1 中的视图进行了适当的调整。在该视图中,焦平面被移到了摄像机的前方,这就避免了由透镜引起的图像倒置。

【83】

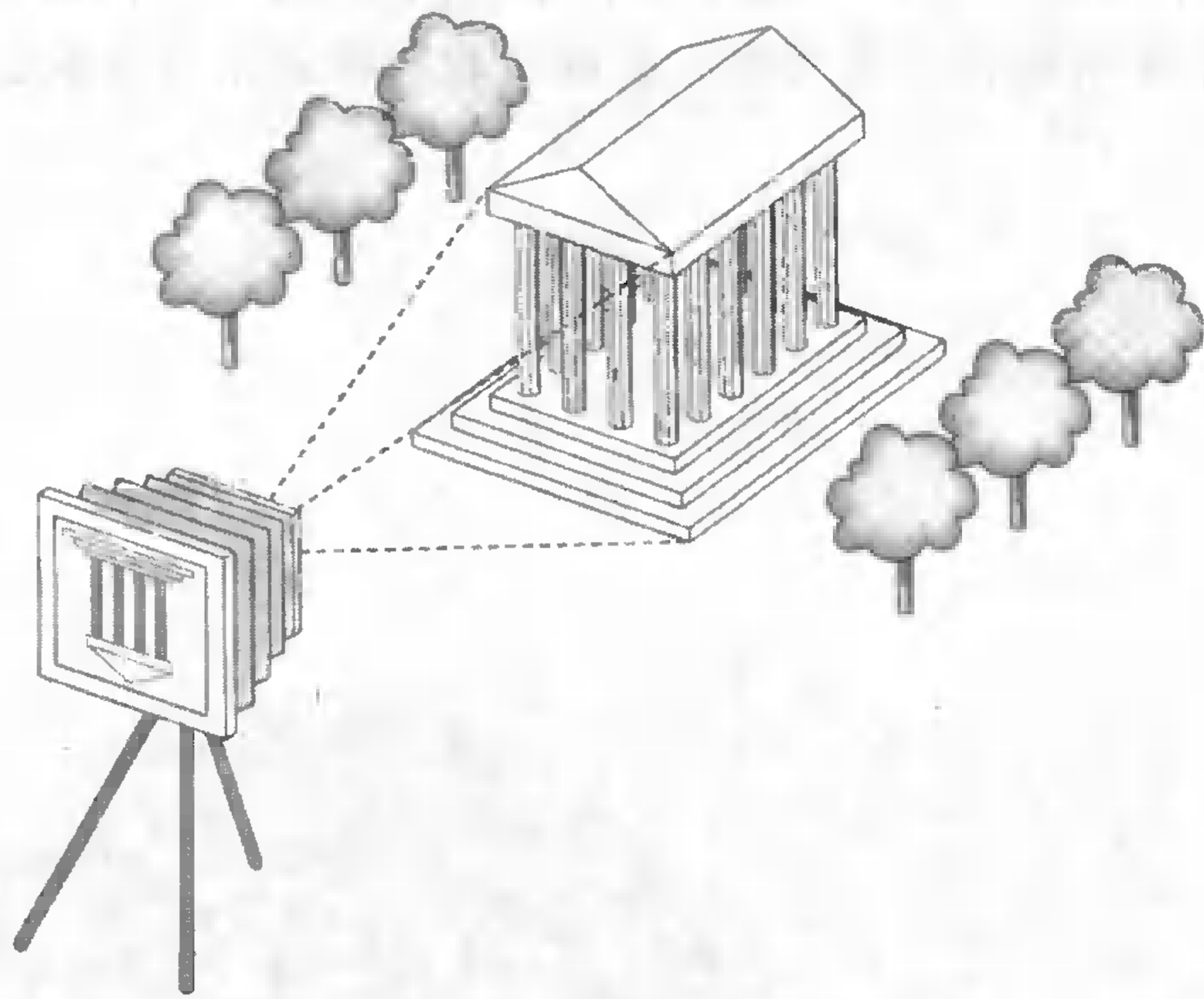


图 4.1 投影

在计算机图形学中,位于摄像机的焦平面或投影平面(projection plane)中的图像即为我们在屏幕上的所见。与之等价,我们可将图 4.2 中所示的焦平面视作已以像素为单位进行了离散化,其中每个像素对应于一条或多条投影线。投影过程可从数学原理上进行描述,但我们完全不必编写一个三维的 OpenGL 程序来专门实现投影功能。我们只需要指定摄像机和场

景中的物体。OpenGL 将在其实现内部替我们完成该项工作。

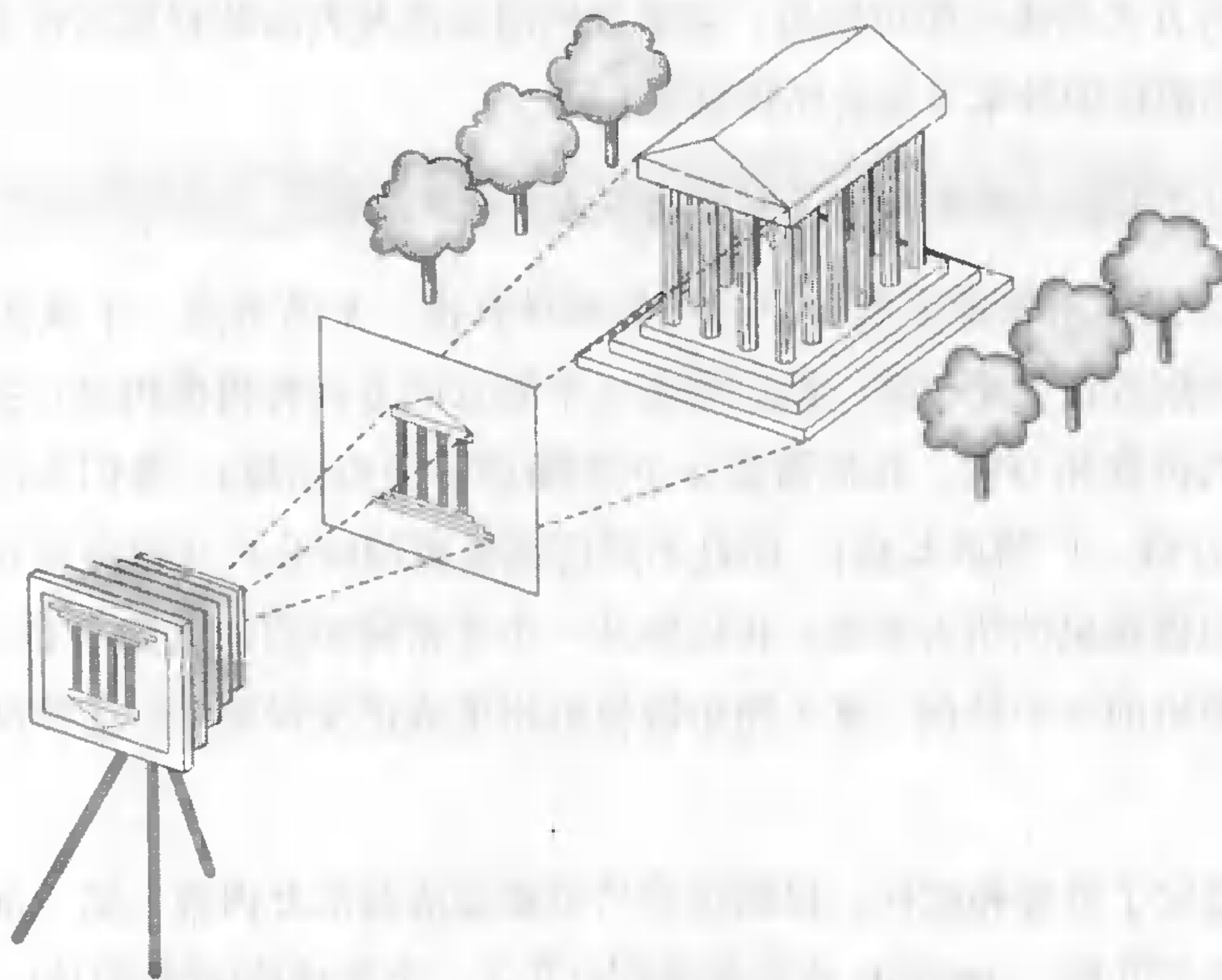


图 4.2 投影平面移至摄像机前方后形成的投影

在 OpenGL 中，三维对象是用 `glVertex*()` 的三维版本来指定的，例如 `glVertex3f(x, y, z)` 和 `glVertex3fv(array)`，我们将使用 OpenGL 的变换来创建更加复杂的对象。所以，在三维空间中的一个三角形可以这样指定：

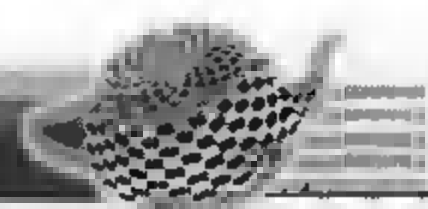
```
glBegin(GL_POLYGON)
    glVertex3f(0.0, 1.0, 0.0);
    glVertex3f(0.0, 0.0, 1.0);
    glVertex3f(1.0, 0.0, 0.0);
glEnd();
```

只要这三个顶点不在同一条直线上，就可以确定一个平面，由此该三角形的内部区域就有了明确的界定。如果我们在代码中添加另外一个顶点：

```
· glBegin(GL_POLYGON);
    glVertex3f(0.0, 1.0, 0.0);
    glVertex3f(0.0, 0.0, 1.0);
    glVertex3f(1.0, 0.0, 0.0);
    glVertex3f(1.0, 1.0, 2.0);
glEnd();
```

【84~85】

这 4 个顶点有可能不共面。如果遇到这种情形，在数学上就不可能得到一个正确定义的多边形。实际上 OpenGL 并不对顶点的合法性进行检查，它所作的只是在光栅化阶段将顶点集离散化为若干多边形。所以，如果在指定顶点时，若颜色始终没有发生变化，最后形成



的多边形呈现为纯红色。但是,如果为每个顶点都指定一种新颜色,OpenGL 将依据对四边形实施离散化的方式来进行颜色插值。如果我们用光照及表面的材质属性来确定多边形的内部颜色,则离散化的效果可能会比较显著。

提示 由三个以上的顶点指定的多边形可能不是平面多边形。

指定摄像机的方式并不像指定场景中的对象那样直接。不妨考虑一下现实世界中的摄像机,我们可将其摆放在某个位置,然后沿着三个独立的方向将摄像机进行旋转。这样,为了描述摄像机的位置和方向,我们需要 6 个参数(或 6 个自由度)。我们还需要对摄像机的透镜进行设置(法线、广角或长焦),而且不同的摄像机的底片尺寸也有差异。OpenGL 允许我们指定虚拟摄像机的所有参数。我们将从一个非常简单的摄像机入手,二维摄像机实际上是三维摄像机的一个特例。这个简单摄像机将生成正交投影图,这种投影方式是最简单的一种。 【85~86】

一旦对摄像机指定了位置和朝向,投影过程只需要完成两部分内容。第一部分是确定哪些对象可见,哪些不可见。OpenGL 的观察函数定义了一个视域体(或裁剪体),以模拟真实摄像机通过透镜所能观察到的空间。所有位于该视域体外的物体将不会出现在图像中,我们称这些对象被“裁剪”掉了。在现实世界中,我们看到只有那些位于摄像机前方的物体才会被观察到,而无论它们距离摄像机有多远。在计算机图形学中,我们人为地增加了一个前裁剪面(近裁剪面)和后裁剪面(远裁剪面),如图 4.3 所示。所以我们只能看到在一个有限体积内的对象。这样,该裁剪体的形状就是一个被截掉头部的金字塔形,我们称其为视域体(或 frustum, 平截头体)。

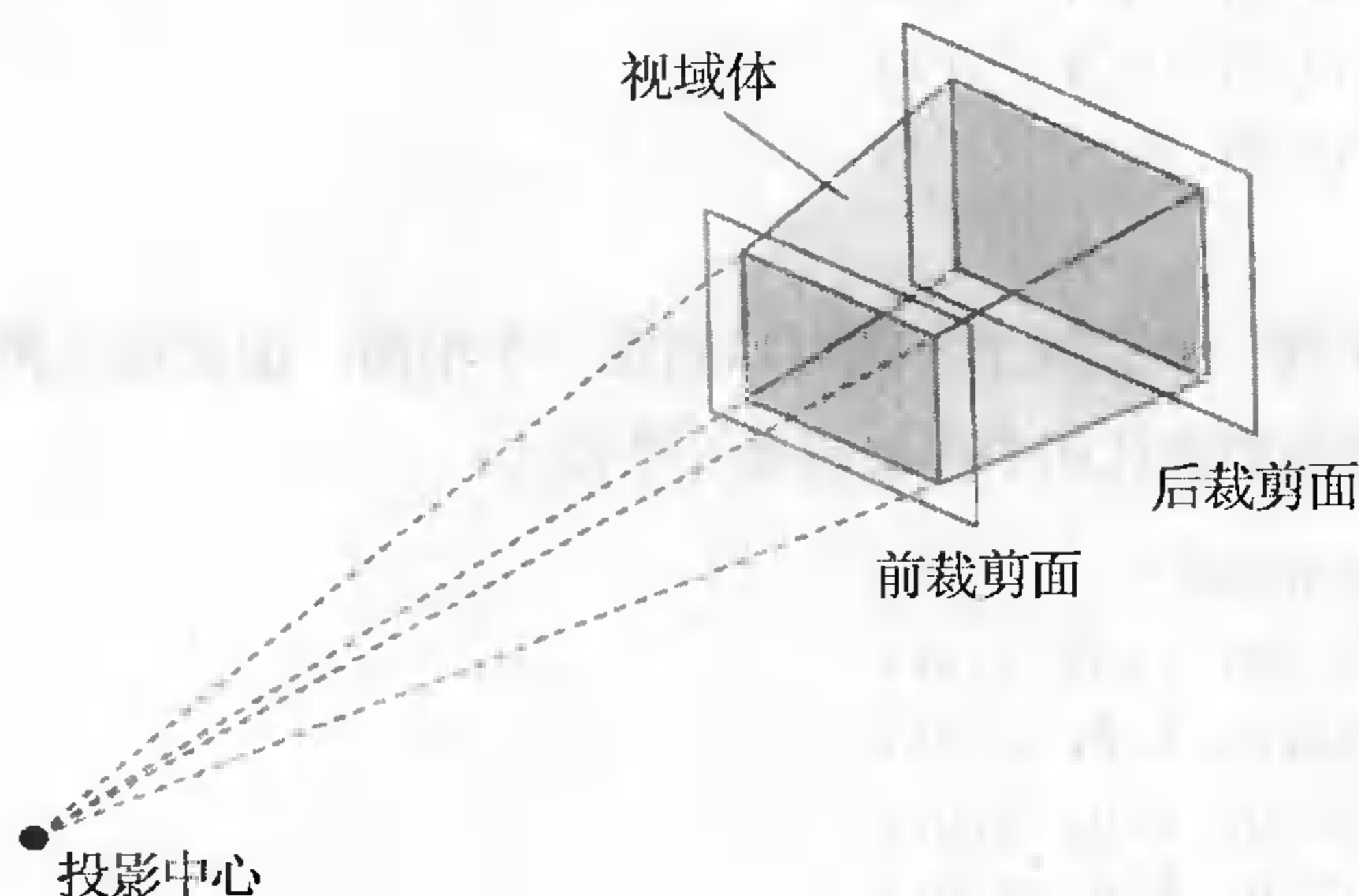


图 4.3 观察视域体

投影过程的第 2 部分内容是确定位于裁剪体内的对象上的某个特定位置的图像在投影平面中的位置。

4.2 OpenGL 中的正交投影

我们可将正交投影(orthographic projection 或 orthogonal projection)看作模拟长焦镜头拍摄图像的方式。这种类型的图像看起来缺乏立体感,但具有保持距离和形状的特点。依据图 4.2 所示的合成摄像机,当我们将摄像机移到距离拍摄对象无穷远处,但投影平面在对象附近时,就可得到正交投影图像。当物距无穷大时,所有的投影线都彼此平行,投影中心将被投影方向所取代。此时视域体演变为一个直平行六面体(right parallelepiped)——即一个如图 4.4 所示的长方体。

【86】

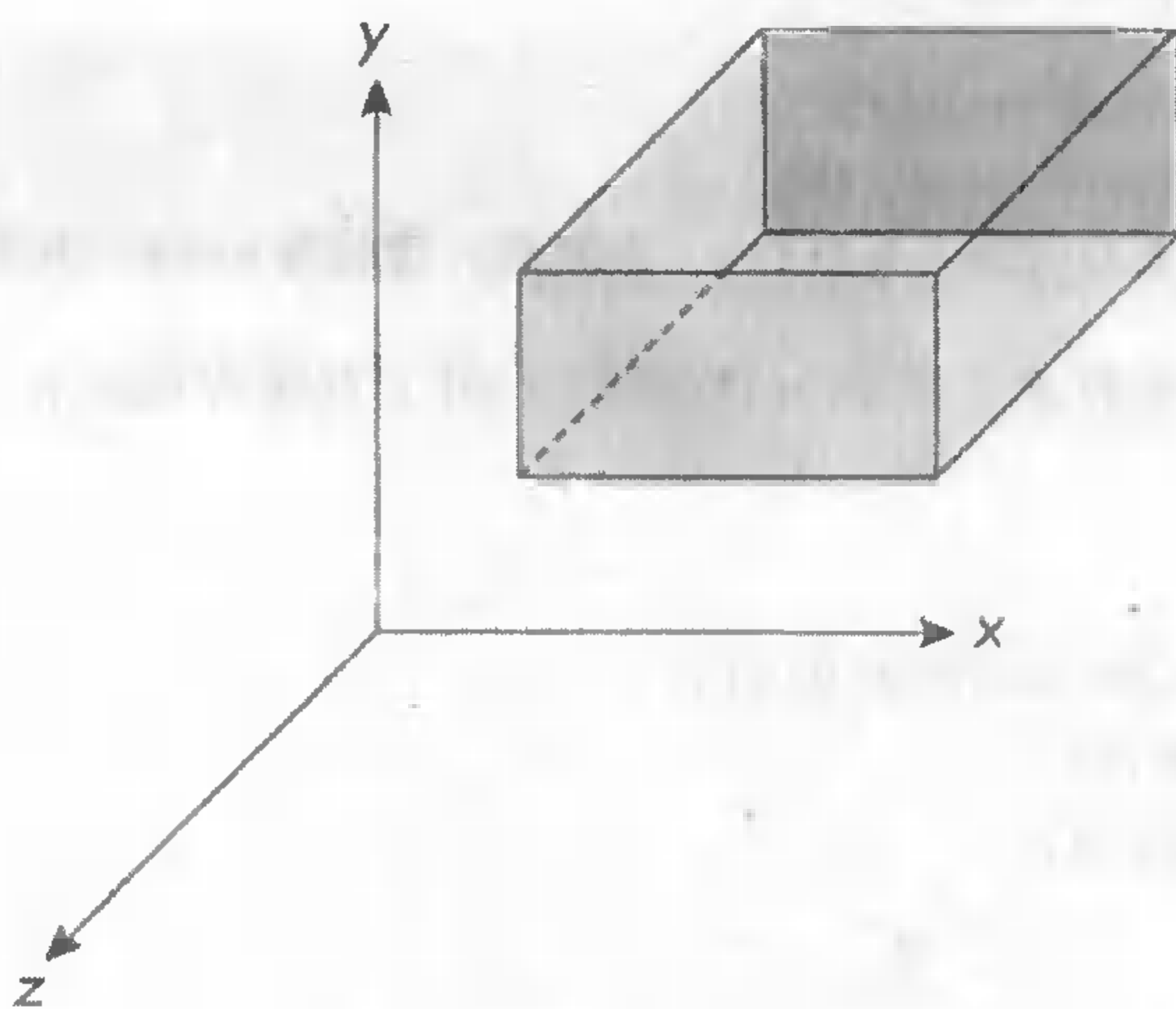


图 4.4 正交视域体

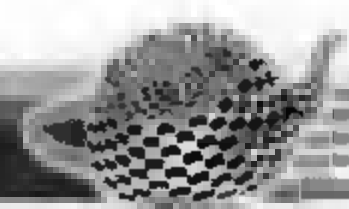
基本的正交观察函数为 `glOrtho()`, 我们可利用该函数来设置合适的观察矩阵(取景变换矩阵)。此时, 视域体如图 4.4 所示。注意, `gluOrtho2D()` 实际上是通过将 `glOrtho()` 函数中的 `near` 和 `far` 参数分别设为 -1 和 +1 而引出的。

```
void glOrtho(GLdouble left, GLdouble right, GLdouble bottom, GLdouble top,
GLdouble near, GLdouble far)
```

设定一个正交投影矩阵, 并定义了一个形状为直平行六面体的视域体。距离是依据到摄像机的远近来度量的, 所以这些参数必须满足以下约束 `right > left, top > bottom, far > near`。

用一个正交摄像机实现观察特别简单。假定我们想要修改投影矩阵, 函数 `glOrtho()` 将以递增方式修改当前矩阵, 所以我们通常在指定当前矩阵类型后, 立即加载一个单位矩阵。所以, 要构建一个边长为 2 的立方体形状的裁剪体, 且其中心与原点重合, 则可以这样做:

```
glMatrixMode(GL_PROJECTION);
glLoadIdentity();
glOrtho(-1.0, 1.0, -1.0, 1.0, -1.0, 1.0);
```

4.3 观察一个立方体

我们可编写一个简单的三维程序来模拟我们的第一个二维程序，方法是通过在显示回调函数中定义一个对象。稍后，我们将讨论对象的建模。现在，我们暂且先使用由 GLUT 提供的对象，一个线框模型的立方体。该线框立方体由线段构成。【87】

实立方体则是由多边形构建的。GLUT 还提供了一些函数可用于定义球体、圆锥、圆环面、规则多面体(如十二面体、八面体、四面体、二十面体)及一个著名的图形对象——犹他壶。在本章的最后，我们将来研究这些对象。

```
void glutWireCube(GLdouble size)
```

```
void glutSolidCube(GLdouble size)
```

分别用来生成线框立方体和实立方体，其中心位于原点处，边长为 `size`，且各边与坐标轴平行。

这样，我们就可用回调函数来显示立方体(我们使用了双缓存模式):

```
void display()
{
    glClear(GL_COLOR_BUFFER_BIT);
    glutWireCube(0.5);
    glutSwapBuffers();
}
```

以及基本的 `init()`和 `main()`:

```
void init()
{
    glClearColor(0.0, 0.0, 0.0, 0.0);
    glColor3f(1.0, 1.0, 1.0);
    glMatrixMode(GL_PROJECTION);
    glLoadIdentity();
    glOrtho(-1.0, 1.0, -1.0, 1.0, -1.0, 1.0);
}
```

```
int main(int argc, char** argv)
```

```
{
    glutInit(&argc, argv);
    glutInitDisplayMode(GLUT_SINGLE|GLUT_RGB);
    glutInitWindowSize(500, 500);
    glutInitWindowPosition(0, 0);
    glutCreateWindow("simple");
    glutDisplayFunc(display);
    init();
}
```



```
glutMainLoop();
}
```

【88】

4.4 摄像机的定位

在我们的例子中存在一个问题，即摄像机只朝向该立方体的一个面，所以我们也只能看到该立方体的一个面。如果我们想看到多边形的其他面，以便了解其三维结构，我们必须移动摄像机或被观察对象。在 OpenGL 中，这些操作是彼此等价的。相对一个固定摄像机移动物体或相对固定物体移动摄像机可以生成完全相同的图像。在计算机图形学中，这些视图之间的差别比现实世界中的要小，如我们绕着建筑物行走，看到建筑的其他面时的效果与要求某人将建筑物旋转之间的差别还是很显著的。这两种方法的数学等价性是通过模型—视图矩阵来实现的，该矩阵的名称表达了这种等价性。

给定视点(或观察点)，将立方体旋转，并使其偏离原点，这是有意义的。我们可用三种方式来实现这一过程。第一种方法是计算立方体各顶点的新位置。但是，由于我们使用的立方体来自 GLUT，所以无法直接访问这些顶点。第二种方法是使用变换。我们可将立方体旋转，并将其平移到所期望的新位置上。但是，我们尚未开始讨论变换，而且为形成所期望的视图的参数计算涉及许多技巧。为此，OpenGL 提供了一个很有用的函数——`gluLookAt()`，该函数位于 GLU 库中，它为多种观察条件提供了简单的用户接口。

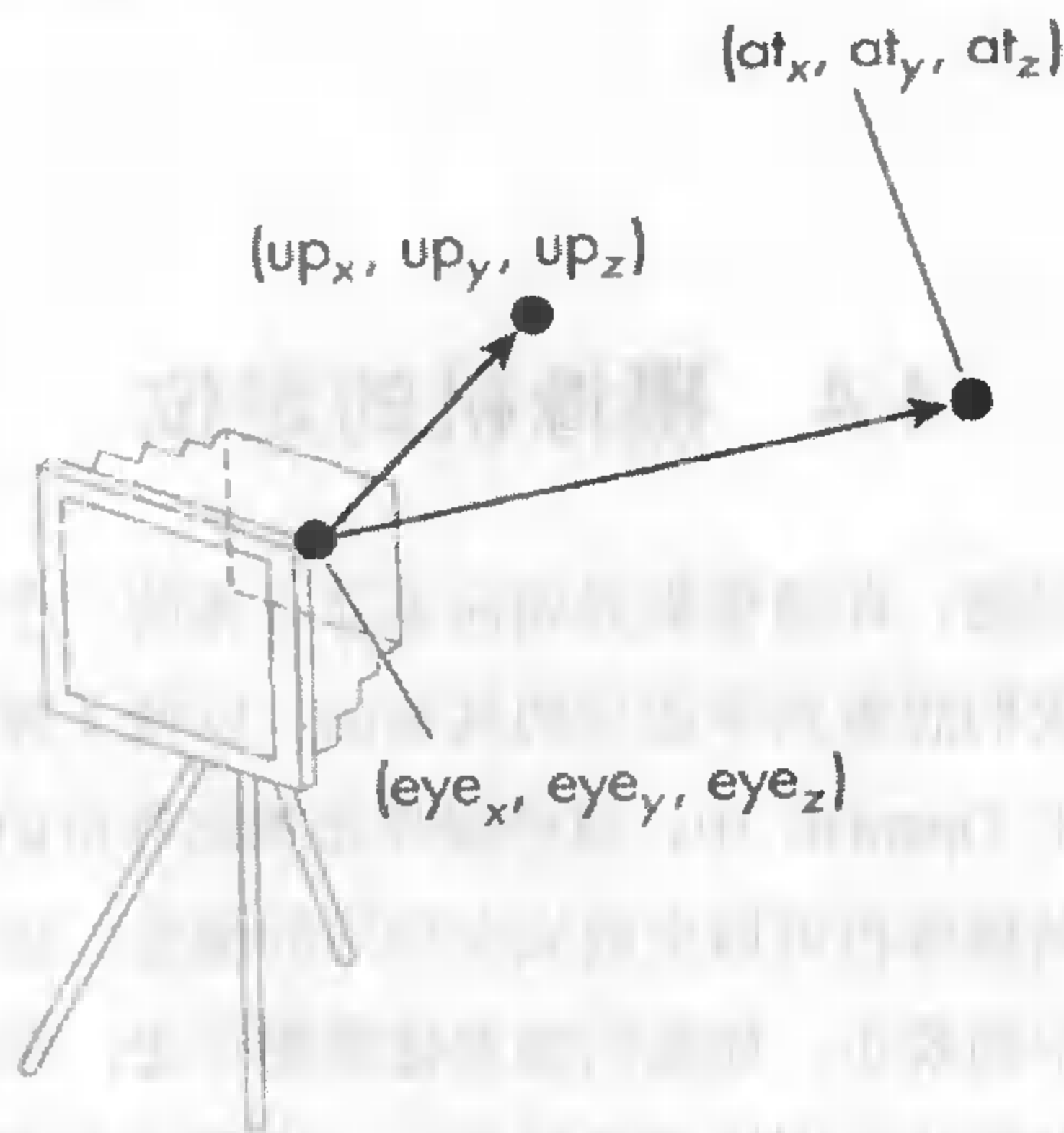
假定现有一些位置已知的对象。为了获得所期望的视图，我们可在对象坐标系中设置摄像机的位置和朝向。我们可决定摄像机的位置(称为视点，即 `eye point`)并通过该摄像机所指向的点来指定其朝向。该情形如图 4.5 所示。注意，仅凭这两个点并不能完全将摄像机固定，因为此时我们仍然可将摄像机绕着视点和被观察点构成的直线进行旋转，而不会影响其他的参数。显然，我们还需要第三个输入，即在图像中我们认为是向上的方向——向上向量。从技术角度看，为了获得向上向量，OpenGL 会将该向量投影到虚拟摄像机的背面。只要视点和被观察点构成的向量不与向上向量平行，就不会有任何问题。为简便起见，通常将向上向量取为(0, 1, 0)或对象坐标系中的 y 轴方向。

【89~90】

```
void gluLookAt(GLdouble eyex, GLdouble eyey, GLdouble eyez,
               GLdouble atx, GLdouble aty, GLdouble atz,
               GLdouble upx, GLdouble upy, GLdouble upz)
```

确定了一个可用于为摄像机定位和定向的矩阵，涉及的参数包括摄像机的位置(视点 `eye`)、被观察点(点 `at`)以及所期望的 `up` 方向。

由 `gluLookAt()` 所确定的矩阵将被应用于当前矩阵。所以，我们必须首先确保当前矩阵模式已被正确地设置(通常为模型—视图矩阵)，并将该矩阵初始化。

图 4.5 用向量 **at** 来指定摄像机

假定我们想要得到立方体的一个等轴测视图(isometric view), 该立方体的中心与原点重合, 且其各边与世界坐标系各轴平行。一个等轴测视图(如图 4.6 所示)相对于立方体的各顶点是对称的。获取这种视图的一个简单方法是将摄像机放在一条通过原点与点(1, 1, 1)的直线上。这样, 我们就可用下列代码来为摄像机定位和定向:

```
glMatrixMode(GL_MODELVIEW);
glLoadIdentity();
gluLookAt(1.0, 1.0, 1.0, 0.0, 0.0, 0.0, 0.0, 1.0, 0.0);
```

函数 `gluLookAt()` 的具体实现种包含了为构建该视图所需的变换和旋转。

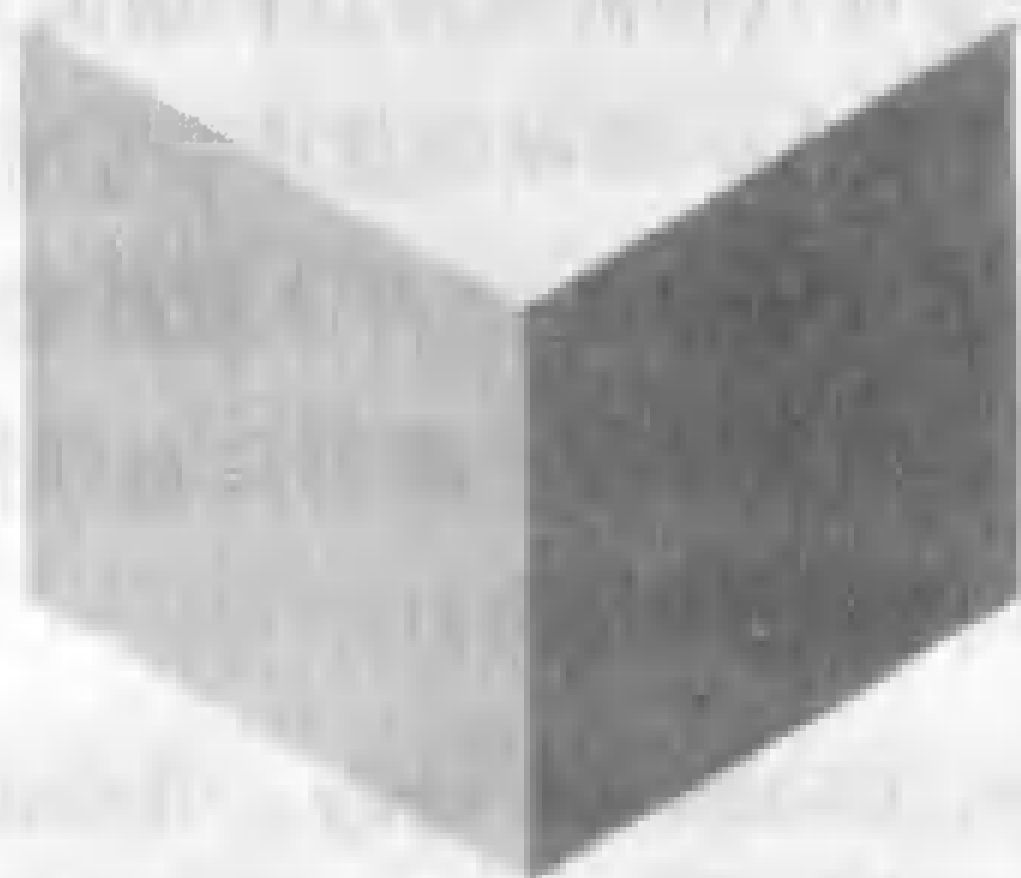


图 4.6 一个立方体的等轴测视图

【90】

一个立方体显示程序

现在我们将前面的内容整合起来, 就得到了一个最简单的三维程序。


```

#include <GL/glut.h>

void display()
{
    glClear(GL_COLOR_BUFFER_BIT);
    glMatrixMode(GL_MODELVIEW);
    glLoadIdentity();
    gluLookAt(1.0, 1.0, 1.0, 0.0, 0.0, 0.0, 0.0, 1.0, 0.0);
    glutWireCube(0.5);
    glutSwapBuffers();
}

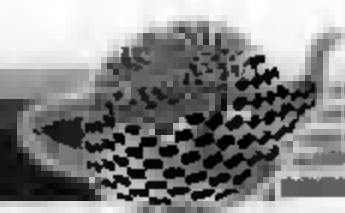
void reshape(int w, int h)
{
    glViewport(0, 0, w, h);
    glMatrixMode(GL_PROJECTION);
    glLoadIdentity();
    glOrtho(-4.0, 4.0, -4.0, 4.0, -4.0, 4.0);
}

void init()
{
    glClearColor(1.0, 1.0, 1.0, 1.0);
    glColor3f(0.0, 0.0, 0.0);
}

int main(int argc, char** argv)
{
    glutInit(&argc, argv);
    glutInitDisplayMode(GLUT_DOUBLE|GLUT_RGB);
    glutInitWindowSize(500, 500);
    glutInitWindowPosition(0, 0);
    glutCreateWindow("cube");
    glutReshapeFunc(reshape);
    glutDisplayFunc(display);
    init();
    glutMainLoop();
}

```

注意，我们使用了默认的颜色值，所以在 `init()` 函数中我们省去了对颜色函数的调用。该程序不是交互式的，所以双缓存并不是必需的，我们也可在 `init()` 中为摄像机定位。但是，我们所选择的程序结构也许更好些，这在当我们想把上述程序扩展成三维交互程序时尤为明显。



虽然将视点(摄像机)放在一条通过原点与点(1, 1, 1)的直线上可以获得这种等轴测视图, 如果我们改变视点的方位, 我们可能必须对 `glOrtho()` 中的 `near` 和 `far` 参数进行修改, 因为这些距离是相对摄像机坐标系的原点定义的。

提示 由 `glOrtho()` 设置的裁剪体是相对于摄像机坐标系的原点定义的。所以应有 `near < far`。

提示 移动摄像机时, 也就是相对于物体移动近裁剪面和远裁剪面。

4.5 对象的生成

现在重复前面的例子, 即重新构建我们自己的立方体。这次, 我们将使用多边形, 并为每个面赋予一种不同的颜色。我们将立方体的中心定义在原点, 并将其边长指定为 2。在代码的开始部分, 我们可以这样写:

```
void cube()
{
    glColor3f(1.0, 0.0, 0.0);
    glBegin(GL_POLYGON)
        glVertex3f(-1.0, -1.0, -1.0);
        glVertex3f(-1.0, 1.0, -1.0);
        glVertex3f(-1.0, 1.0, 1.0);
        glVertex3f(-1.0, -1.0, 1.0);
    glEnd();

    :
    the other faces
}

void display()
{
    glClear(COLOR_BUFFER_BIT);
    cube();
}
```

【92】

提示 于将对多边形进行填充, 所以必须确保从立方体外部对其进行观察时, 多边形各顶点的绕序为逆时针方向。这样我们方可对立方体各面的朝向(正面和背面)作出合理的解释。

为定义该立方体, 需要在 `cube()` 函数内进行 42 次函数调用。如果我们在第一个 `glBegin()` 内使用类型 `GL_QUADS` 而非 `GL_POLYGON`, 情况会稍微得到改善。这样我们就可省去所有

后续的 `glBegin()` 和 `glEnd()` 调用(最后一个 `glEnd()` 除外), 因为每连续 4 个顶点都将确定一个四边形。

4.5.1 数组的使用

在上述代码中, 如果我们将颜色和顶点存放在数组中, 将进一步改善程序的结构。该结构不但会使代码更清晰, 而且会为交互式程序带来更大的灵活性。

下面我们将图 4.7 中的各顶点进行编号出来。

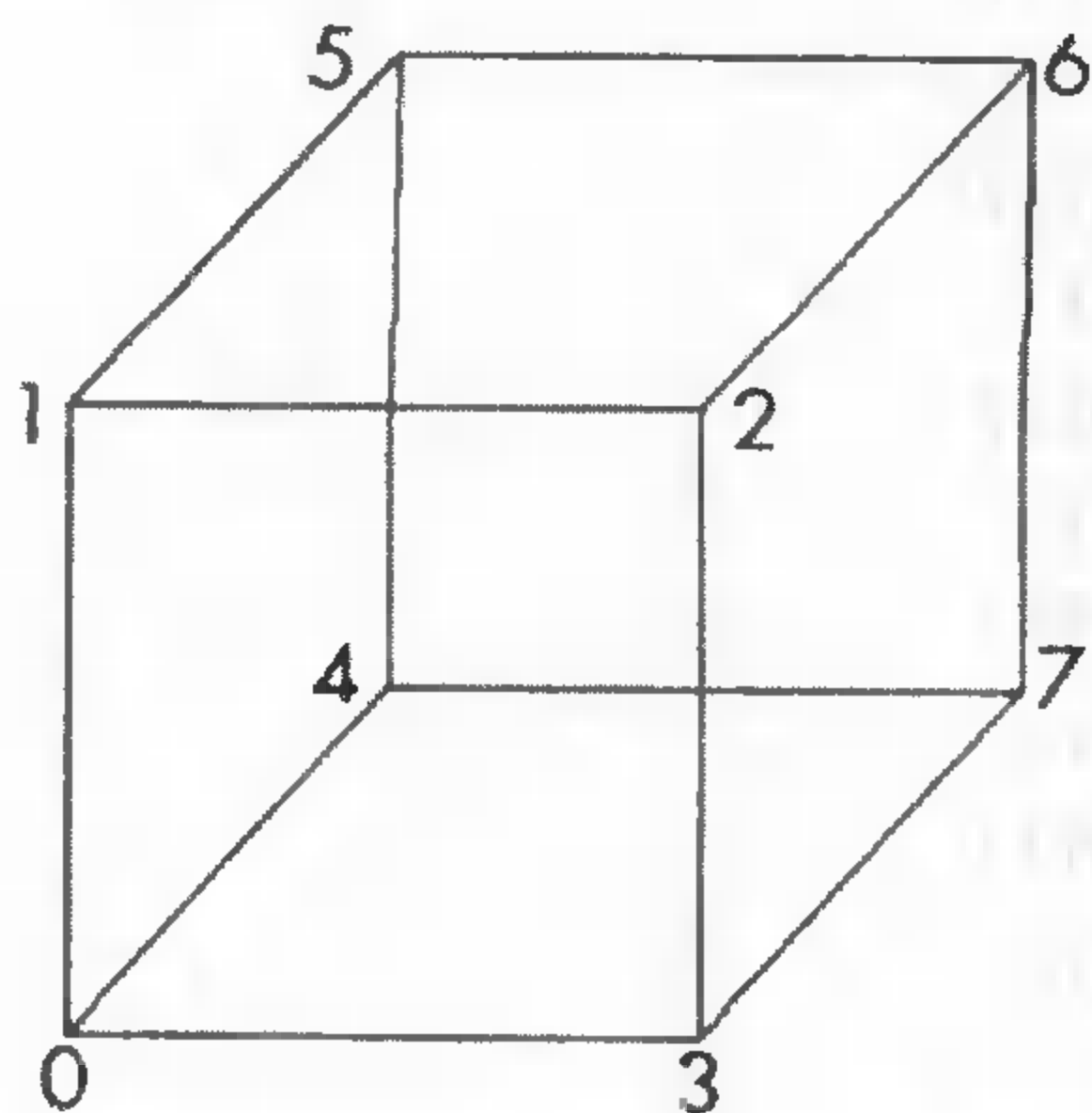


图 4.7 立方体各顶点的编号

所需的数组如下:

```
GLfloat vertices[][3] = {{-1.0, -1.0, 1.0}, {-1.0, 1.0, 1.0},
    {1.0, 1.0, 1.0}, {1.0, -1.0, 1.0}, {-1.0, -1.0, -1.0},
    {-1.0, 1.0, -1.0}, {1.0, 1.0, -1.0}, {1.0, -1.0, -1.0}};
```

```
GLfloat colors[][3] = {1.0, 0.0, 0.0}, {0.0, 1.0, 1.0},
    {1.0, 1.0, 0.0}, {0.0, 1.0, 0.0}, {0.0, 0.0, 1.0},
    {1.0, 0.0, 1.0}};
```

在前面的代码中, 我们只需用 `glColor3fv()` 取代 `glColor3f()`, 用 `glVertex3fv()` 取代 `glVertex3f()` 即可, 但这样并未对原程序作出实质性的改变。我们将增加一个依据顶点索引绘制多边形的简单函数。【93】

```
void polygon(int a, int b, int c, int d)
{
    glBegin(GL_POLYGON);
        glVertex3f(vertices[a]);
        glVertex3f(vertices[b]);
        glVertex3f(vertices[c]);
        glVertex3f(vertices[d]);
```



```

    glEnd();
}

```

该函数从第一个顶点开始，为多边形赋予了一种颜色，但是我们可以很容易地修改这个选项。现在我们修改 `cube()` 函数如下：

```

void cube()
{
    glColor3fv(colors[0]);
    polygon(0, 3, 2, 1);
    glColor3fv(colors[2]);
    polygon(2, 3, 7, 6);
    glColor3fv(colors[3]);
    polygon(3, 0, 4, 7);
    glColor3fv(colors[0]);
    polygon(1, 2, 6, 5);
    glColor3fv(colors[4]);
    polygon(4, 5, 6, 7);
    glColor3fv(colors[5]);
    polygon(5, 4, 0, 1);
}

```

虽然上述代码的调用并不能帮助我们减少函数调用的次数，但是起码它的结构比以前清晰不少。^①这样做的一个优点是每个顶点的位置只出现一次。在交互式程序中，在程序运行中，顶点可能会被用户修改，使对顶点的修改变得极其容易。

4.5.2 顶点数组

OpenGL 提供了一个称为顶点数组(vertex array)的工具，它扩展了数组的使用方式，可以显著减少绘制立方体时的函数调用的次数。其主要思想是存储在数组中的信息可被存储在客户端(应用程序)中，并可通过单次函数调用来访问。我们可将这些信息以一种保持原先结构的方式(例如为绘制立方体各顶点的调用顺序)来存储。【94~95】

OpenGL 为 6 种类型的数组提供了支持：顶点、颜色、颜色索引、法线、纹理坐标以及边标记。稍后会有一些其他类型的数组。使用顶点数组需要三个步骤。首先，像其他 OpenGL 特性一样，必须将其启用。其次，必须指定数组的格式。这通常是程序初始化阶段的一部分。最后，用该数组来绘制场景。

在我们的例子中，只用到了颜色和顶点数组。启用方式如下：

① 注意，我们可将颜色设置放在函数 `polygon()` 内，这样会使本例简短一些，但会增加我们后续一些例子的实现难度。


```
glEnableClientState(GL_COLOR_ARRAY);
glEnableClientState(GL_VERTEX_ARRAY);
```

数组的形式由下列代码给出:

```
glVertexPointer(3, GL_FLOAT, 0, vertices);
glColorPointer(3, GL_FLOAT, 0, colors);
```

```
void glEnableClientState(GLenum array)
```

```
void glDisableClientState(GLenum array)
```

启用或禁用数组类型 `GL_VERTEX_ARRAY`、`GL_COLOR_ARRAY`、`GL_INDEX_ARRAY`、`GL_NORMAL_ARRAY`、`GL_TEXTURE_COORD_ARRAY` 或 `GL_EDGE_FLAG_ARRAY`。

```
void glVertexPointer(GLint dim, GLenum type, GLsizei stride,
GLvoid *array)
```

```
void glColorPointer(GLint dim, GLenum type, GLsizei stride,
GLvoid *array)
```

提供数组中的信息。数据存储在数组中, `dim` 为数组的维数(2、3 或 4), `type` 表明数据的存储类型(`GL_SHORT`、`GL_INT`、`GL_FLOAT`、`GL_DOUBLE`), `stride` 是相邻两个数据值之间的字节数(0 意味着数组中的数据是紧密排列的)。

第一个值(3)表明所使用的数据是三维的。第 2 和第 3 个参数表明数据是浮点类型的, 并紧密排列在由第 4 个参数指定的数组中。

我们需要一个新数组以一种我们所期望使用的顺序来存储顶点的索引值。下面的数组包含必要的信息:

```
GLubyte cubeIndices[] = {0, 3, 2, 1, 2, 3, 7, 6, 0, 4, 7, 3,
1, 2, 6, 5, 4, 5, 6, 7, 0, 1, 5, 4};
```

现在 we 可通过函数 `glDrawElements()` 来绘制该立方体。

【95】

```
void glDrawElements(GLenum mode, GLsizei n, GLenum type,
void * indices)
```

使用数组 `indices` 中的 `n` 个索引来绘制类型为 `mode` 的元素。该数组可为以下类型(`GL_UNSIGNED_BYTE`、`GL_UNSIGNED_SHORT` 或 `GL_UNSIGNED_INT`)。

如果我们将每个面进行独立的绘制, 我们可用如下循环:

```
for(i = 0; i < 6; i++)
glDrawElements(GL_POLYGON, 4, GL_UNSIGNED_BYTE, cubeIndices);
```

但是, 如果使用类型 `GL_QUADS`(每 4 个连续顶点确定一个新四边形)会将程序进一步简化。所以, 按如下方式进行一次函数调用已足够了:

```
glDrawElements(GL_QUADS, 24, GL_UNSIGNED_BYTE, cubeIndices);
```

这里还有一个不易察觉的问题。当我们执行 `glDrawElements()` 时, 所有被启用的数组将被绘

制。所以，上述绘制过程等价于下列形式的函数调用：

```
glColor3fv(color(cubeIndices[0]));
glVertex3fv(vertex(cubeIndices[0]));

glColor3fv(color(cubeIndices[1]));
glVertex3fv(vertex(cubeIndices[1]));
:
```

所以，上述代码看起来每个顶点使用了一种不同的颜色。这里，我们实际上做了一个颜色数组与顶点数组维数相同的假设，实际上这是有问题的。在第 2 章中我们曾了解到，默认的明暗模型是平滑类型的。如果我们为每个顶点赋予不同的颜色值，在默认情况下，OpenGL 将用这些顶点的颜色值在每个多边形的表面内部进行插值。我们可通过请求平面着色模式来避免该问题：

```
glShadeModel(GL_FLAT);
```

现在，虽然每个顶点都可以使用颜色数组中的值，但在绘制时，对于每个多边形，只有最后一个颜色值将被使用。所以每次用顶点数组绘制立方体时，我们只需要进行一次函数调用，相对于我们原先的 42 次函数调用显然已取得了显著的改进。

我们也可用显示列表来减少绘制立方体的代价。如果将整个立方体放在一个显示列表中，我们只需调用一次 `glCallList()` 函数。具体采用那种形式还要依据应用程序的需求来决定。 【96】

4.6 消 隐

接下来我们必须面对的一个问题是如何确保在所绘制的图像中只有观察者能够看见的表面是可见的。在第 2 章中，我们看到了一个这种消隐问题的一个特例的解决方案，即我们可通过启用剔除功能并声明我们希望将背面剔除而将背面朝向的多边形剔除掉：

```
glEnable(GL_CULL_FACE);
glCullFace(GL_BACK);
```

但是，这种策略只适用于凸对象。虽然立方体是凸的，我们还是需要一种更通用的方法。

到目前为止，我们始终是在跟多边形打交道，这些多边形都是以不透明或实体的形式绘制的。在现实世界中，包含这些对象的场景的观察者不可能看到位于某些多边形后方的那部分多边形。在一些情况下，如对于凸对象，我们只是选择不将背面朝向的多边形进行绘制。我们也可考虑使用画家算法(*painter's algorithm*)，其思想是如果我们依据到摄像机的距离将多边形进行排序，并按照自后向前的顺序对这些多边形进行绘制，则位置靠前的多边形会

将位于其后的多边形遮挡。暂且抛开效率问题和这种方法的适用条件不说,该方法的问题在于它不适用于流水线绘制器。在应用程序中,多边形(以及其他类型的图元)可以任意的顺序生成。当绘制模式为立即模式(即 OpenGL 的模式绘制模式)时,每个多边形一经定义就被立即送入流水线,所以我们用于消隐的方法必须与顶点的定义顺序无关。

OpenGL 所使用的消隐方法称为 z 缓存算法(z-buffer algorithm)或深度缓存算法。该算法需要借助额外的存储空间来保存在绘制过程中绘制器已经绘制的多边形的深度信息。这个额外的存储空间称为 z 或深度缓存。虽然用户并不需要了解该算法的细节,但在显示过程的一开始,我们必须先启用该算法,并将深度缓存清空。在大多数程序中,当对颜色缓存执行清空操作时,也应对深度缓存进行清空。由于这些操作必须由用户来完成,这是在绘制过程中用户所必须了解的关于该算法的惟一信息。与其他运算,如绘制直线或填充多边形,用户不需要任何有关该算法实现的具体细节。

在初始化阶段,我们必须先请求一个深度缓存。这样,一个典型的 `glutInitDisplayMode()` 的调用形式为:

```
glutInitDisplayMode(GLUT_RGB | GLUT_DOUBLE | GLUT_DEPTH);
```

可通过如下方式启用深度缓存:

```
glEnable(GL_DEPTH_TEST);
```

最后,当我们清空颜色缓存时,通常也将深度缓存清空:

```
glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);
```

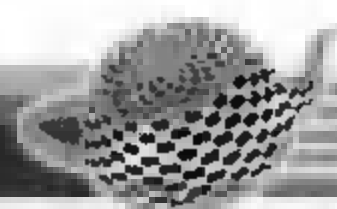
【97~98】

对于大多数程序来说,以上是我们必须做的全部内容。对于更加复杂的情形,例如在使用半透明的材质时,我们可能还需要少量的函数做辅助以便更好地控制深度缓存,但目前我们尚不必关心这些问题。

提示 要实现消隐,必须启用深度缓存并对其进行清空。

4.7 GLU 与 GLUT 对象

我们可借助目前我们已接触过的简单图元集来构建自己的对象。在第 9 章中,我们将介绍曲线和曲面,这样我们就可脱离由直线和多边形构成的平面世界。但是在许多情况下,我们需要一些较为复杂的对象,却又不希望引入处理曲线和表面所需的复杂机制。在许多程序中,二次曲面占有相当重要的地位:椭圆球体(包括球体)、圆锥以及圆柱体。GLUT 和 GLU 都提供了一些绘制这类对象的函数。这些函数所做的实际上是用多边形来逼近表示这些对象,并允许用户指定需要使用的多边形数量。但是,从应用程序开发者的角度看,他



可以通过单次函数调用来实现对每个对象的访问，这就避免了在程序中对这些对象的创建步骤。

提示 GLU 二次曲面和 GLUT 对象是用多边形构造的，所以如果用户没有指定足够数量的多边形，则看起来可能不那么光滑。

4.7.1 GLU 二次曲面

GLU 提供了三种类型的二次曲面：球体、圆柱体以及圆盘。这些图元的存储方式比前面所有图元都复杂。特别是，我们可用理想的绘制风格、不同风格的法线(在第 6 章介绍光照计算时我们将进行介绍)以及纹理坐标来创建这些对象。由于这些对象所需要的数据结构比简单图元更复杂，对其进行定义时需要花费不少的精力。

上述二次曲面都可通过函数 `gluNewQuadric()` 和 `gluDeleteQuadric()` 来创建或删除。创建之后将返回一个指向所创建对象的指针，以便我们将信息输入其数据结构中。 【98】

GLUQuadricObj*gluNewQuadric()

创建一个新的二次曲面对象，并返回其指针。

void gluDeleteQuadric(GLUQuadricObj*obj)

删除二次曲面对象 obj。

我们可以 4 种方式来绘制二次曲面对象：仅显示用于逼近二次曲面的多边形的各顶点，显示多边形的各边、填充多边形或以某种模式(轮廓模式)绘制顶点之间的连线(共面的多边形除外)。这些风格可借助函数 `gluQuadricDrawStyle()` 来设定。

void gluQuadricDrawStyle(GLUQuadricObj * obj, GLenum style)

为二次曲面对象 obj 设置绘制风格 (GLU_POINT、GLU_LINE、GLU_FILL、GLU_SILHOUETTE)。

我们可让 OpenGL 通过函数 `gluQuadricNormals()` 为明暗处理生成法线，以及通过函数 `gluQuadricTexture()` 来生成纹理坐标。

void gluQuadricNormals(GLUQuadricObj*obj, GLenum mode)

将二次曲面对象 obj 的法线模式指定为 mode：无法线 (GL_NONE)，该值为默认值；每个多边形一个法线 (GLU_FLAT)；或每个定点一个法线 (GLU_SMOOTH)。

void gluQuadricTexture(GLUQuadricObj*obj, GLboolean mode)

指定二次曲面对象 obj 是否应具有纹理坐标。默认值为 GL_FALSE，即不需要纹理坐标。

我们可用以下函数来创建各种二次曲面的线框模型，创建的效果如图 4.8 所示：`gluSphere()`，`gluCylinder()`，`gluDisk()` 或 `gluPartialDisk()`。 【99】

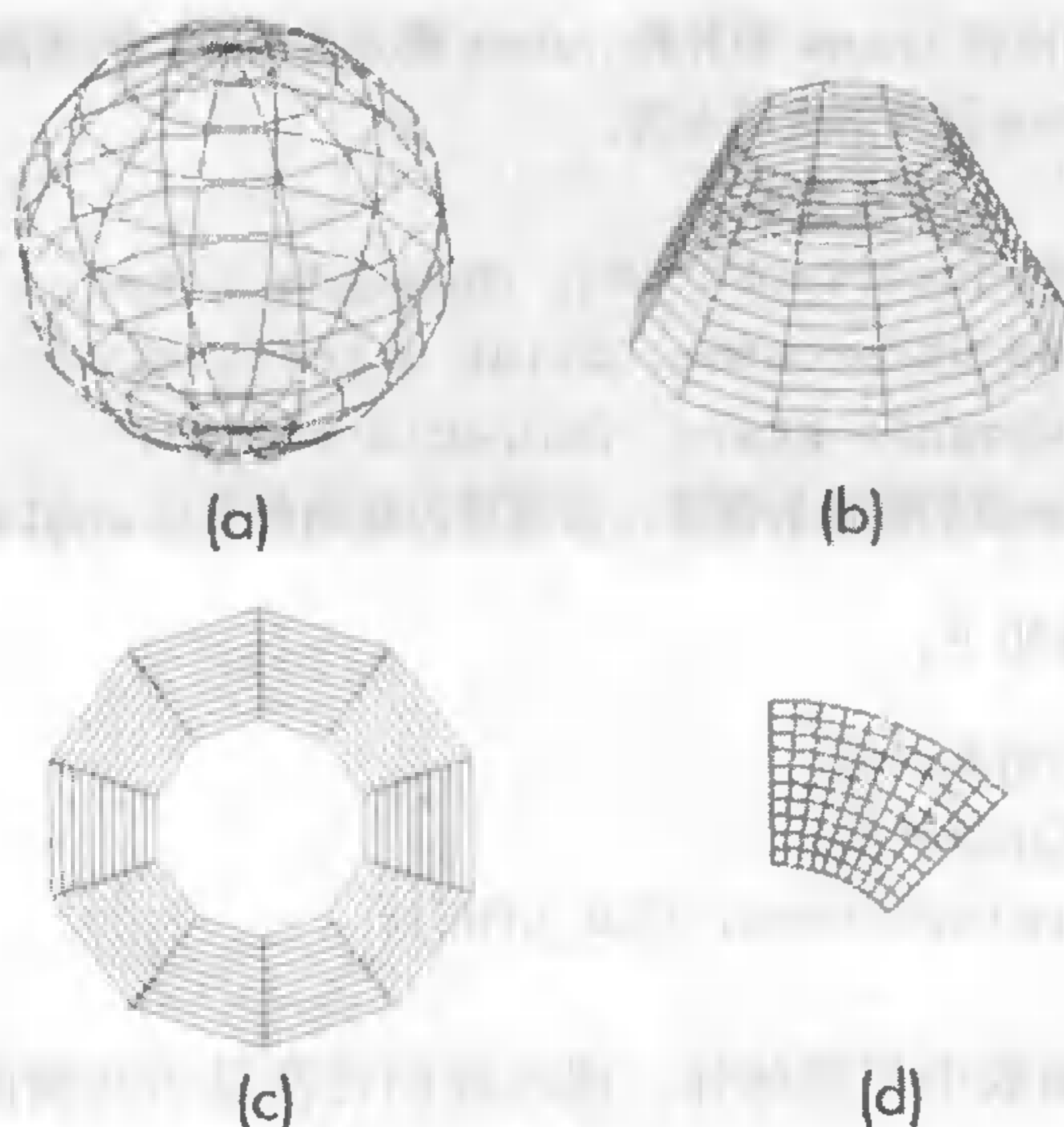


图 4.8 GLU 二次曲面: (a)球体; (b)圆柱体; (c)圆盘; (d)局部圆盘

球体是通过用经线和纬线创建近似多边形来近似表示的。我们可用 OpenGL 变换(参见第 5 章)实现球体的移动、旋转及比例变换(可变为椭圆球体)。

```
void gluSphere(GLUquadricObj*obj, GLdouble radius,  
GLint slices, GLint stacks)
```

将二次曲面对象 `obj` 按中心位于原点, 半径为 `radius` 的球体进行绘制。该球体是用 `slices` 行经线和 `stacks` 行纬线组成的多边形来近似的。

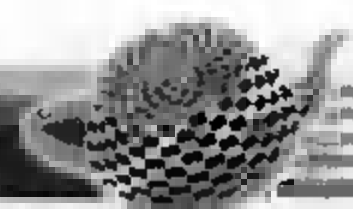
圆柱体是沿着 z 轴方向定义的。其底面总是 $z = 0$, 其高度、顶部直径、底部直径都通过 `gluCylinder()` 函数来指定。用来对圆柱逼近表示的多边形由 z 轴方向的层数 `stacks` 和每层的片段(直线段)数目 `slices` 确定。与球体一样, 我们可用 OpenGL 变换实现对圆柱体的移动、方向的改变以及比例变换。

```
void gluCylinder(GLUquadricObj*obj, GLdouble base, GLdouble top,  
GLdouble height, GLdouble slices, GLdouble stacks)
```

将二次曲面对象 `obj` 指定为居中的圆柱体, 其轴线与 z 轴平行, 其底面为 $z = 0$, 顶面为 $z = \text{height}$ 。该圆柱拥有自己的底面和顶面半径。圆柱沿着 z 轴方向被细分为 `stacks` 层, 绕着 z 轴方向被细分为 `slices` 段。

圆盘是平面物体, 且其中心处有一个孔洞。圆盘都是在平面 $z = 0$ 中创建的。圆盘由内和外圆的半径确定。显示时圆盘是以同心圆环的形式出现的, 该同心圆环沿着径向被切割, 与饼图非常类似。当从一个圆盘中移除一个楔形块后就形成了局部圆盘。该楔形是由其起始角和楔形角来指定的, 其中角度是依据 y 轴正方向来度量的。【100】

```
void gluDisk(GLUquadricObj*obj, GLdouble inner, GLdouble outer,  
GLint slices, GLint rings)
```

在平面 $z = 0$ 上指定一个由内径 `inner` 和外径 `outer` 确定的圆盘。构成圆盘的多边形面元是由 `rings` 个同心圆及围绕圆心的 `slices` 次分割所确定的。

```
void gluPartialDisk(GLUQuadricObj*obj, GLdouble inner,
                   GLdouble outer, GLint slices, GLint rings,
                   GLdouble start, GLdouble angle)
```

指定一个移除了顶角为 `angle` 度的楔形的圆盘，该楔形的起始角度是 `angle`。

创建一个球体的典型代码如下：

```
GLUQuadricObj * mySphere;
mySphere = gluNewQuadric();
gluQuadricDrawStyle(mySphere, GLU_LINE);
```

我们可在 `init()` 或 `main()` 函数中创建球体。现在我们可在显示回调函数中通过如下方式来绘制一个半径为 1.0 的球体，其中经线和纬线都被 12 等分：

```
gluSphere(mySphere, 1.0, 12, 12);
```

4.7.2 GLUT 对象

GLUT 增加了更多的三维对象。这些函数都会自动生成法线，所有的三维对象(除茶壶外)都不会生成纹理坐标。绘制时，绘制风格并没有依附在被绘对象上，每种类型的对象都对应两个函数，其中一个函数可生成线框图，另一个则生成实体(由填充多边形组成)。除了我们已经见过的立方体，GLUT 还提供了球体、圆锥、圆环面、一些规则多面体以及犹他壶。圆锥和圆环面的线框模型如图 4.9 所示，规则多面体的线框模型如图 4.10 所示，茶壶的线框模型如图 4.11 所示。

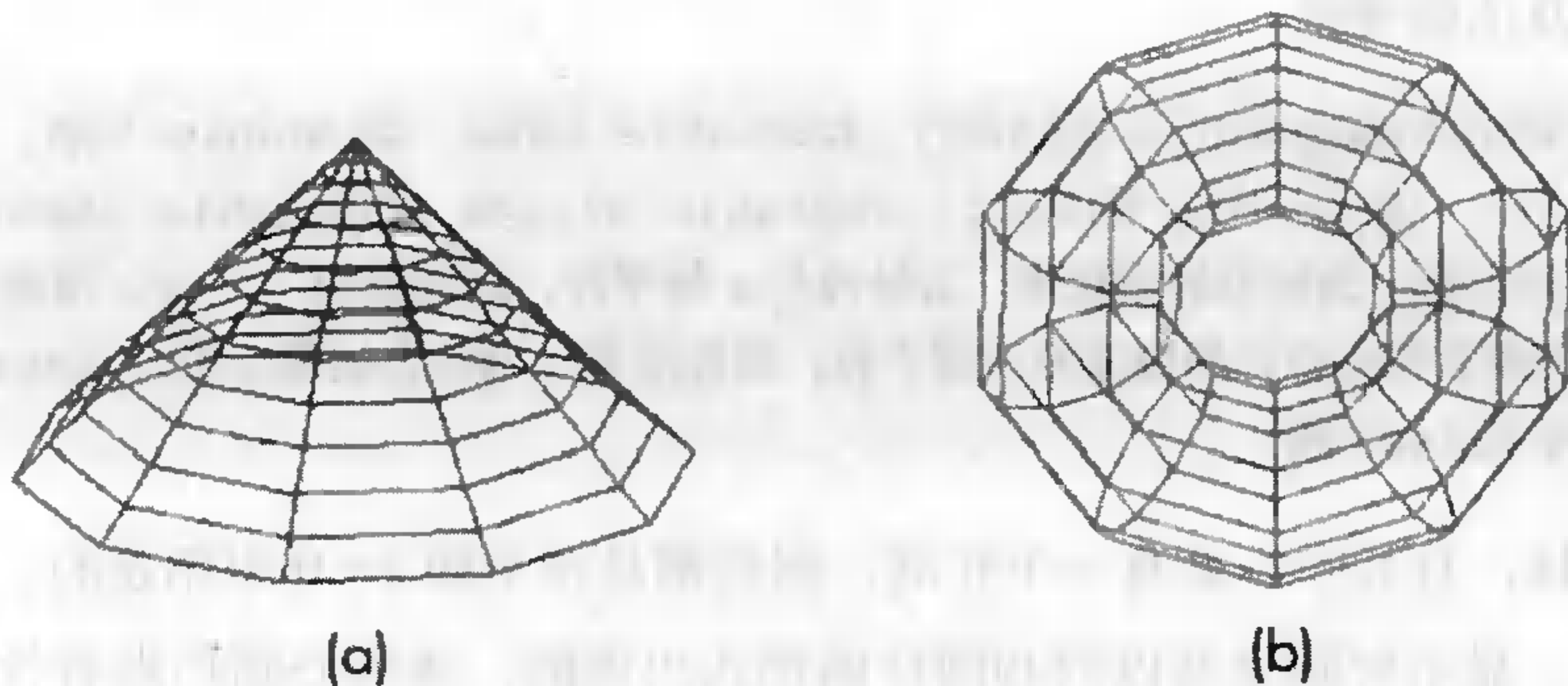


图 4.9 GLUT: (a)圆锥; (b)圆环面

【101】

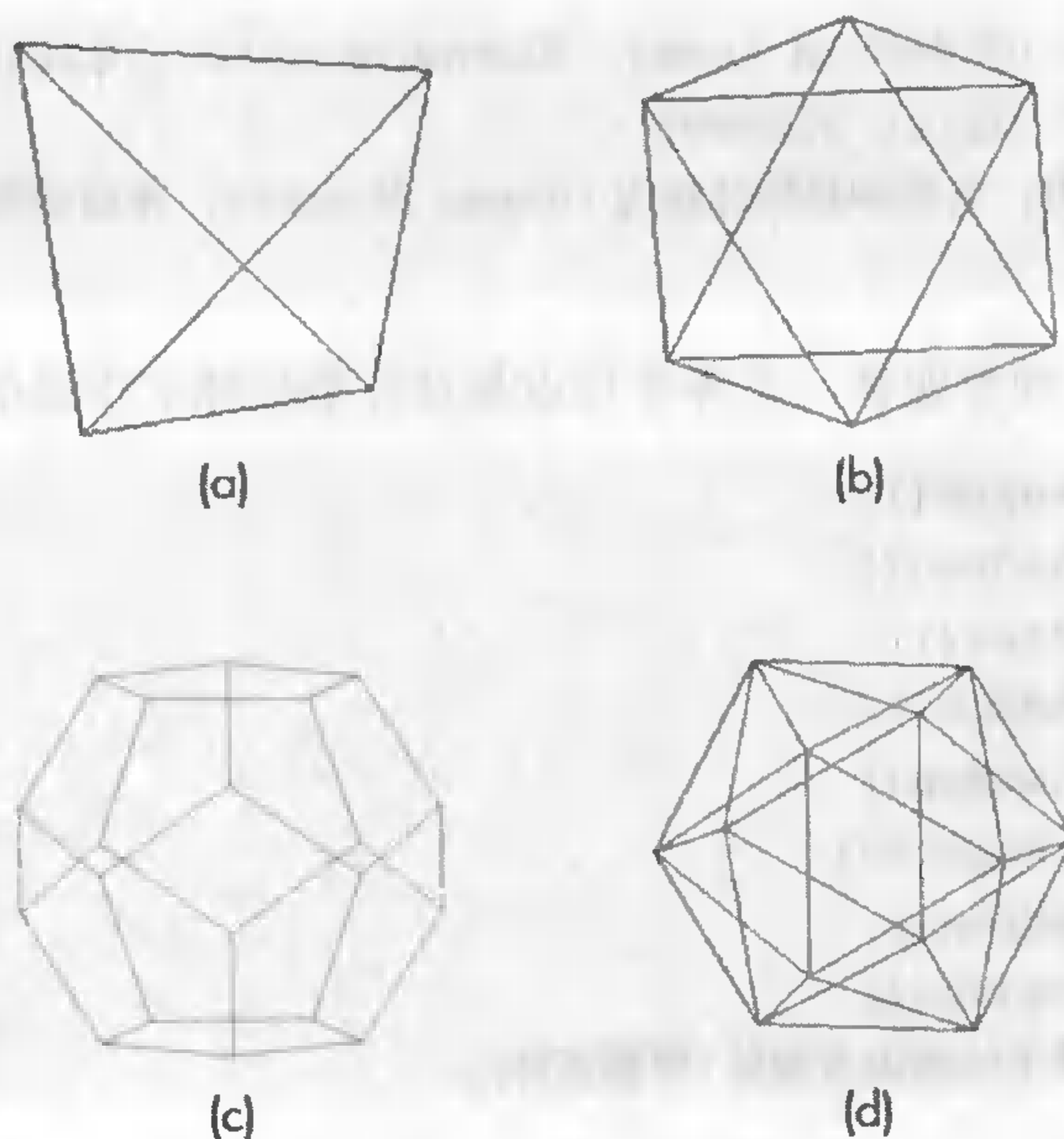


图 4.10 GLUT 理想实体: (a) 四面体; (b) 八面体; (c) 十二面体; (d) 二十面体 【102】

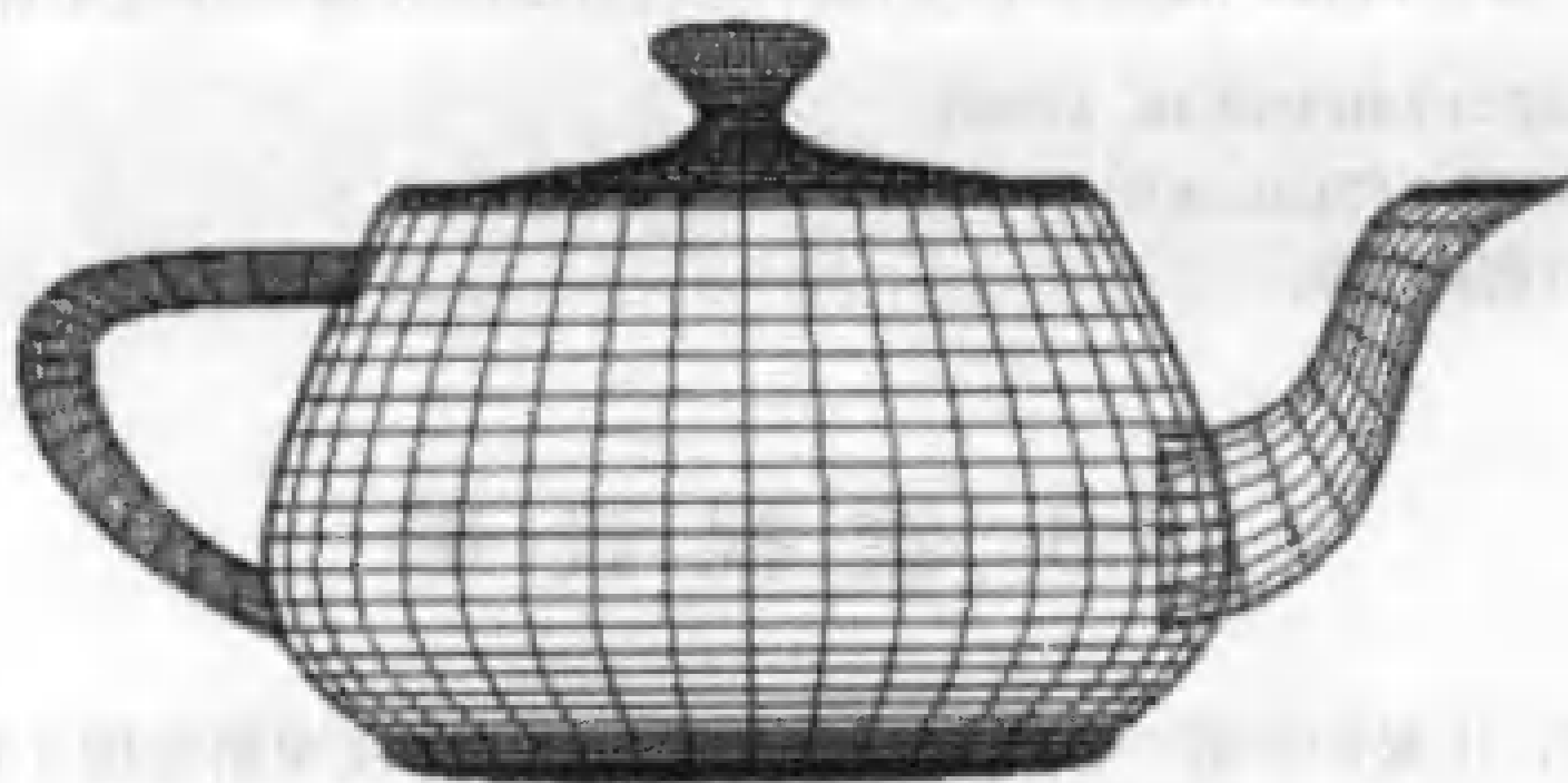


图 4.11 GLUT 茶壶

```
void glutWireSphere(GLdouble radius, GLint slices, GLint stacks)
```

```
void glutSolidSphere(GLdouble radius, GLint slices, GLint stacks)
```

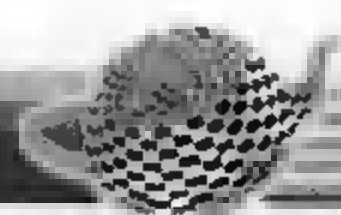
用由 stack 条经线和 slices 条纬线构成的若干多边形逼近表示一个球体。该球体的中心位于原点, 半径为 radius。

```
void glutWireCone(GLdouble base, GLdouble height, GLint slices,
                  GLint stacks)
```

```
void glutSolidCone(GLdouble base, GLdouble height, GLint slices,
                  GLint stacks)
```

用由 stacks 条经线和 slices 条纬线构成的多边形组来逼近表示一个圆锥。该圆锥的底面为 $z = 0$, 高度为 height。

```
void glutWireTorus(GLdouble inner, GLdouble outer, GLint sides,
                  GLint slices)
```

```
void glutSolidTorus (GLdouble inner, GLdouble outer, GLint sides,  
                    GLint slices)
```

定义了一个轴线与 z 轴平行, 内径和外径分别为 `inner` 和 `outer`, 径向切面的分割数为 `sides`, 圆环的环线数为 `slices`。

规则的多面体对象, 即理想实体, 其各个顶点都是在单位球上定义的。

```
void glutWireTetrahedron()  
void glutSolidTetrahedron()  
void glutWireOctahedron()  
void glutSolidOctahedron()  
void glutWireDodecahedron()  
void glutSolidDodecahedron()  
void glutWireIcosahedron()  
void glutSolidIcosahedron()
```

生成所有顶点都位于单位球上的规则多面体(理想实体)。

犹他壶是用 OpenGL 表面生成的(参见第 9 章)。多年以来, 该茶壶一直被用来测试各种绘制算法。它由 192 个顶点构成。该茶壶在生成时既包括法线又包含纹理坐标。

```
void glutWireTeapot(GLdouble size)  
void glutSolidTeapot(GLdouble size)  
生成给定尺寸(size)的犹他壶。
```

4.8 透 视 投 影

具备一定基础之后, 让我们来看一些更一般的观察方式。透视观察实现了图 4.1 所示的合成摄像机模型。观察范围为 4 个汇聚于投影中心的平面所限定, 形成了一个无限深度的观察金字塔。但是, 与正交观察一样, 我们会为其增设一个前(近)裁剪面和一个后(远)裁剪面。这样透视投影下的视域体就如图 4.12 所示。注意, 投影平面可处在摄像机前方的任何位置上。

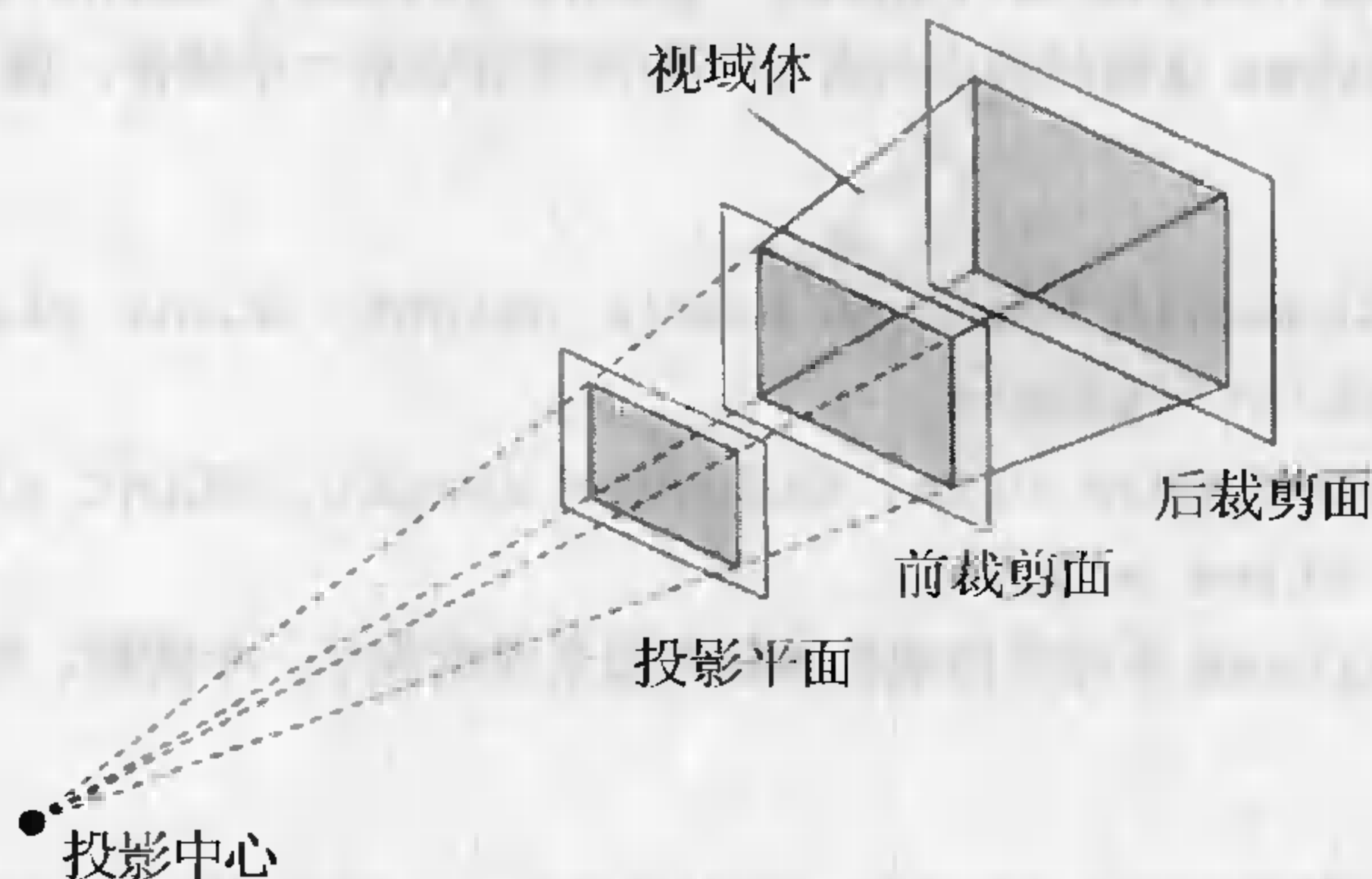


图 4.12 透视投影中的视域体

【103~104】

OpenGL 提供了函数 `glFrustum()` 用于创建所需的透视投影矩阵。该函数的参数与 `glOrtho()` 完全相同。

```
void glFrustum(GLdouble left, GLdouble right, GLdouble bottom,
               GLdouble top, GLdouble near, GLdouble far)
```

为透视投影定义了一个矩阵。前裁剪面和后裁剪面分别由 `near` 和 `far` 确定, 这些值是相对于投影中心度量的, 必须为正值, 且应满足 $far > near$ 。前裁剪面的左下角和右上角在摄像机坐标系中的坐标分别为 `(left, bottom, near)` 和 `(right, top, near)`。

与正交投影不同(摄像机可位于裁剪体内部), 在透视投影中, 视域体必须位于摄像机前方。所有的参数都是相对摄像机坐标系中定义的, 所以 `near` 和 `far` 都必须为正, 且 `far` 的值应大于 `near`。

提示 切记用于观察的参数都是在摄像机坐标系中定义的。参数 `near` 和 `far` 分别是摄像机到裁剪平面的距离。对于透视投影, 应有 $far > near > 0$ 。 【104】

由 `glFrustum()` 计算所得的矩阵将作用于当前矩阵。必须首先确保矩阵模式已被正确地设置, 通常会从一个单位矩阵开始, 所以下面的代码具有一定的代表性:

```
glMatrixMode(GL_PROJECTION);
glLoadIdentity();
glFrustum(left, right, bottom, top, near, far);
```

与正交投影相同, 投影矩阵定义了裁剪体以及投影的类型, 但是尚未对摄像机的方位进行定义。OpenGL 中的摄像机位于原点, 并指向 z 轴的负方向。通过函数 `glFrustum()` 修改观察条件等价于改变摄像机的透镜, 而非其位置。为了改变位置或朝向, 我们必须借助模型视图矩阵相对于摄像机移动场景中的对象, 或者通过 OpenGL 变换, 或者使用函数 `gluLookAt()`。

虽然由 `glFrustum()` 所形成的裁剪体具有很强的通用性, 但利用该函数所提供的接口要想获得所期望的视图并非易事。问题在于当我们改变近裁剪面和远裁剪面之间的距离时, 视域体的各边相对于其中心的角度变化可能非常显著。当用户为使所期望的对象位于该视域体内而修改这些距离参数时, 位于某些边上的对象则可能由于这些角度的变化而从视野中消失。

一种更自然的接口更接近于我们使用真实摄像机的方式。在希望通过一个固定摄像机看到更多的对象时, 我们会选择对透镜进行调整以获得广角视图。函数 `gluPerspective()` 就提供了这样一个接口。

```
void gluPerspective(GLdouble fov, GLdouble aspect, GLdouble near,
                   GLdouble far)
```




依据 y 方向的视角 (fov), 近裁剪面和远裁剪面的纵横比以及摄像机到两个裁剪面的距离 (near 和 far) 指定一个投影矩阵。

其中, 近裁剪面和远裁剪面具有相同的纵横比。但是, 如果纵横比不为 1, 则视域体的顶角和边角便不同。函数 `gluPerspective()` 使用的是视域体顶面和底面的夹角。由于该裁剪体可通过为 GLU 库中的函数 `glFrustum()`、`gluPerspective()` 正确选择参数而构造出来。对用户来说, 通过为摄像机定位、确定近裁剪面和远裁剪面, 然后用一个较宽的角度来用 `gluPerspective()` 设置观察条件是最简便的方式。一旦我们对其他参数的设置都满意, 缩小视角的范围就比较容易了。

使用透视投影的一个潜在问题是深度精度的损失, 在显示设备中该问题可能非常明显。该问题的根源在于深度缓存中位深度的限制以及由透视投影引起的非线性比例变换。当近裁剪面非常靠近投影中心时(当我们像观察非常靠近摄像机的物体时), 情况可能会更糟。

【105~106】

提示 近裁剪面与摄像机距离过近时, 将为透视投影中的深度计算带来数值误差。

提示 `gluPerspective()` 与 `glFrustum()` 相比, 缺乏一些通用性。

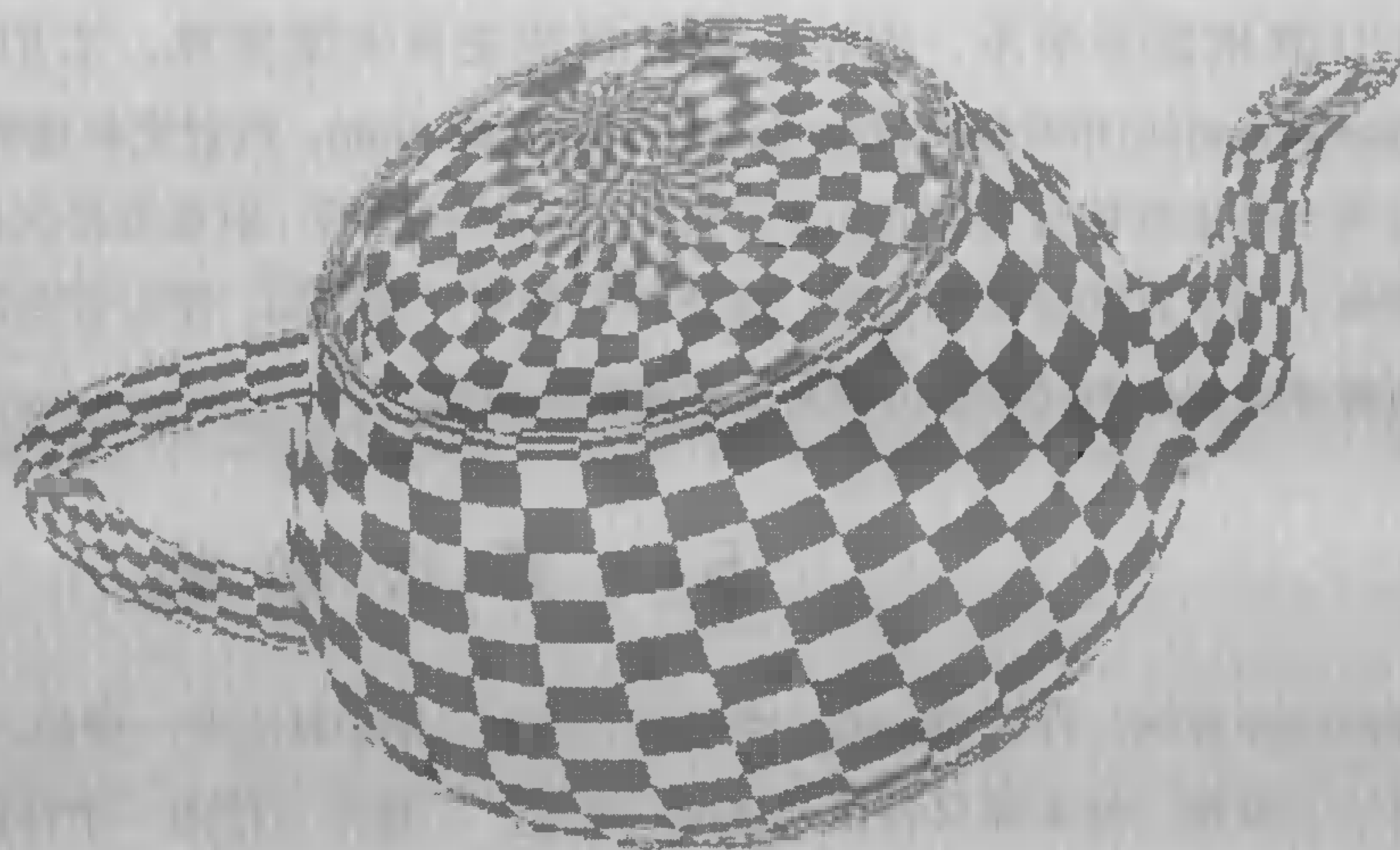
4.9 编程练习

1. 依据构建多边形的思想用多边形来逼近表示球体。其中多边形的各边可由经线和纬线确定。除了极点需要用三角形扇面表示外, 其他地方都用四边形条带来表示。
2. 编写一个程序, 实现交互式地移动摄像机。显示一个或多个对象, 并确保摄像机在移动时总是朝向场景中的同一点。
3. 高度场是一个二维数组, 其中的每个元素都是相应点的海拔高度。编写一个程序在三维空间中以多边形网格的形式将这些数据显示出来。
4. 编写一个交互式程序, 允许用户输入位于平面 $y = 0$ 上的某多边形的各顶点。程序应能允许用户将该多边形垂直向上进行拉伸, 使之成为三维物体, 从而就定义了该物体的各条侧边。拉伸后的顶面与原始多边形的大小完全相同, 其高度与各侧边的长度一致。

第5章

几何变换

变换是操纵几何对象、为场景制作动画效果及获取理想视图的关键。本章将介绍 OpenGL 几何变换的基础知识。我们采取的方式并非推导出各种常用的变换，如平移、旋转和缩放，而是只关注如何使用 OpenGL 所提供的变换函数。我们将演示如何使用变换来建立各种各样的现象的模型。



5.1 线性变换

在计算机图形学中，我们只与两类基本几何实体打交道：点(或顶点，表示空间中的位置)和方向(或向量)。尽管这两种类型的指定方式非常类似，它们实际上表达的是不同的概念。例如，在三维空间中，可以说点(1, 0, 0)，也可说方向(1, 0, 0)。这个点描述的是坐标系空间中固定的一个位置。而这个向量表达的是 x 轴正方向，不含位置信息。这两种实体都可进行变换。可移动点的位置，也可将向量旋转以表达一个不同的方向。

变换将顶点和向量映射到另一些顶点和向量。有无穷多种方法可实现点与点之间、向量与向量之间的映射。但在计算机图形学中，我们只对所有的变换的一种重要性质感兴趣：它们是否能够保持直线和线段不变。其理由具有物理意义和实际意义。

在现实世界中，我们对物体实施的许多变换都具有线性保持性。例如，我们来考虑旋转和平移。如果研究一个立方体，将其绕着自身的一条边旋转，得到的仍然是一个立方体。同样，如果我们拾取了该立方体，并将其移至某些地方，它仍然是一个立方体。如果对该立方体实施一系列平移和旋转，则其基本的几何性质(如尺寸和角度)不会发生任何变化。正是由于这些原因，旋转和平移被称为刚体变换(rigid-body transformation)。【107~108】

到目前为止，我们所使用的所有图元都是通过顶点定义的。大多数情况下，我们都是用顶点来定义线段(而线段本身就是一种图元)或定义多边形的各条边。顶点和向量都将经过 OpenGL 流水线。顶点被装配成组以描述对象(例如多边形)，而且 OpenGL 将确定哪些对象位于裁剪体内。不妨考虑一下一种变换将直线段映射为曲线所带来的结果。如果限定只进行线性保持变换，则我们只需要对线段的端点(即两个顶点)实施变换。接下来就可让光栅化器来决定经过变换的直线段的内部点的位置。

在计算机图形学中，有两类线性保持变换非常重要，它们分别是仿射变换(affine transformation)和射影变换(projective transformation)。仿射变换包括平移、旋转及比例变换。这种变换能够保持直线间的平行性，并且是可逆的。射影变换包括我们已经讨论过的透视变换。由于这些变换都是将三维实体投影到二维空间，所以都是不可逆的。顶点和向量的仿射变换和射影变换都可借助矩阵运算来实现。

5.2 齐次坐标

我们已经看到，许多 OpenGL 函数在二维和三维中是完全一样的。实际上，在 OpenGL 中，对于二维和三维实体只存在一种惟一表示。二维是三维的一个特例，只是所有的二维图元

都位于平面 $z=0$ 中。所以，在 OpenGL 中，二维点 (x, y) 与三维点 $(x, y, 0)$ 实际上是相同的。本质上，OpenGL 是在四维空间中工作的，而三维只是一个特例。虽然无需了解有关该系统——称为**齐次坐标系**(homogeneous coordinate)的所有数学知识，但是如果能够了解一些该系统的基本知识，将有助于更清楚地理解许多 OpenGL 函数的功能。通常，一个三维点的内部表示为 $(x, y, z, 1)$ ，而一个二维点的内部表示为 $(x, y, 0, 1)$ 。在四维空间中创建一个更具一般性的点是可以做到的，按照 (x, y, z, w) 的形式直接定义或通过变换来定义都是可以的。这样的点最终将作为三维点 $(x/w, y/w, z/w)$ 被显示，前提是 w 不为 0。

方向(即向量，如法向量)的表示形式为 $(x, y, z, 0)$ ，等价于无穷远的点。

有了上述表示方法，我们用于建模、观察及投影的所有变换都可由维数为 4×4 的矩阵直接作用于齐次坐标系中的点和向量而得到。当设置好某种 OpenGL 变换(例如旋转或平移)时，我们实际上是在设置一个 OpenGL 内部的 4×4 矩阵。也可通过直接指定所需的 16 个元素来对变换矩阵进行设置。

【108~109】

5.3 模型-视图变换与投影变换

图 5.1 是 OpenGL 流水线起始部分的一个简化图。每个顶点都将经过由当前模型-视图矩阵和投影矩阵(这两个矩阵是 OpenGL 状态的一部分)定义的两重变换。一开始，这两个矩阵都被设置为 4×4 的单位矩阵。

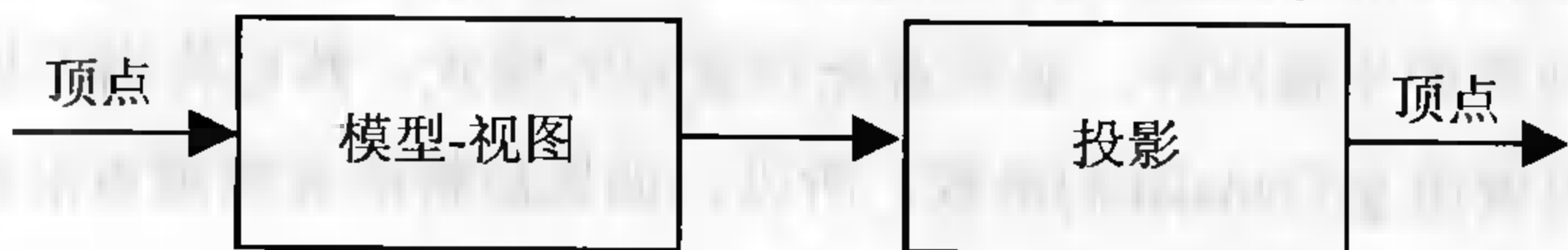


图 5.1 OpenGL 流水线中的变换矩阵

虽然这些矩阵是由相同的 OpenGL 函数进行设置的，实际上二者的用途截然不同。模型-视图矩阵用于相对于摄像机为物体定位，而投影矩阵指定了投影及裁剪体，并将顶点映射至一个归一化的坐标系中。

5.4 平 移

在图 5.2 中，我们看到了一个摄像机及一个对象。假定在对象坐标系中，对象位于原点处。由于在 OpenGL 中，默认情况下摄像机也位于原点，所以我们希望将摄像机与对象分离开，这可以通过移动对象或移动摄像机来实现。一个较简单的解决方案是将对象或摄像机沿着 z 轴移动。平移操作实际上就是为对象的每个顶点增加一个位移。所以，如果 $(x, y, z, 1)$ 是位

于 (x, y, z) 处顶点的齐次坐标,通过一次平移变换,该顶点就会变为 $(x + d_x, y + d_y, z + d_z, 1)$ 。我们可通过使用函数 `glTranslate*()` 将模型-视图矩阵设为一个平移矩阵而将该变换应用于所有的顶点。

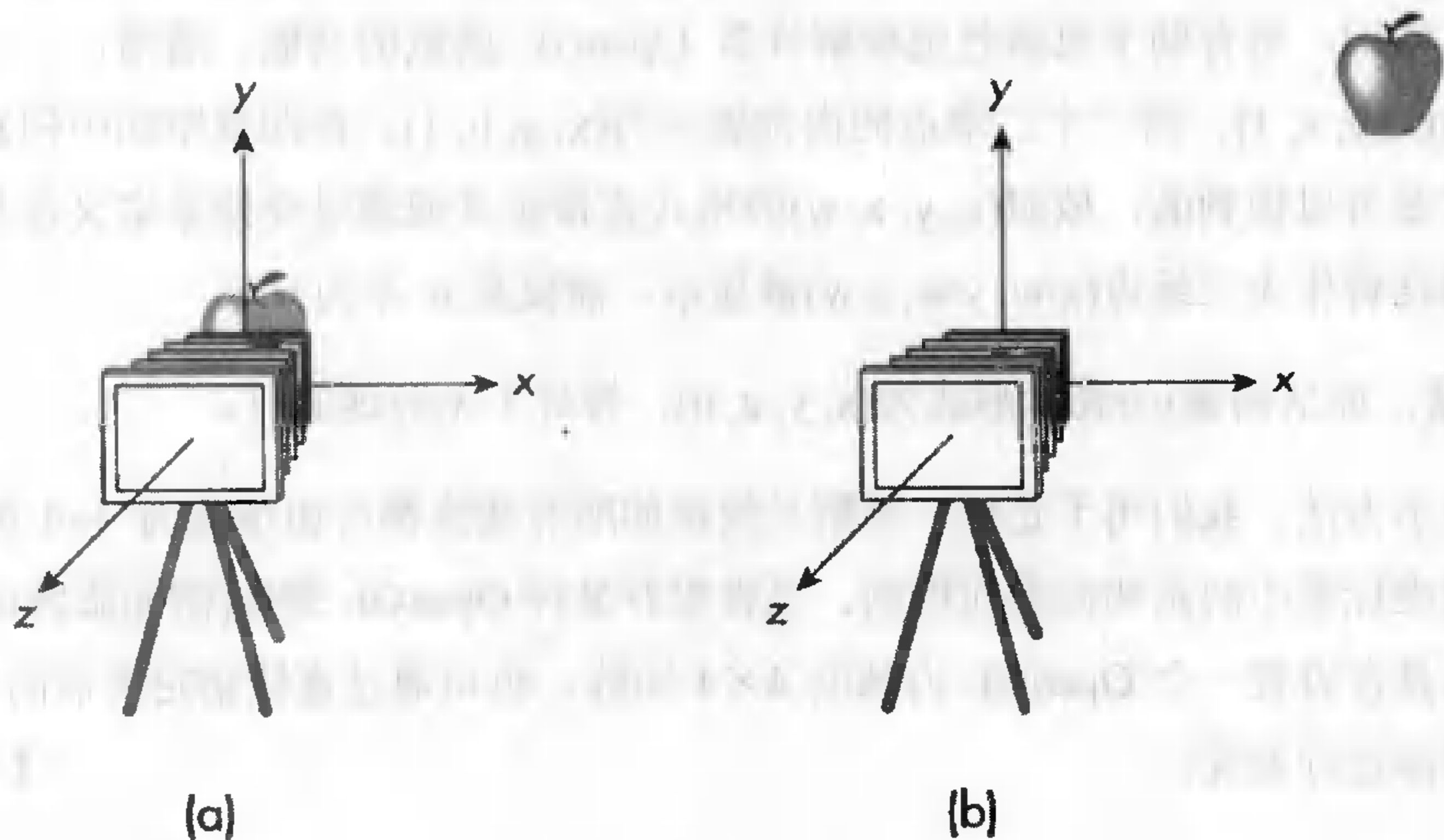


图 5.2 摄像机与对象: (a)摄像机与对象都位于原点; (b)将对象从原点移开

```
void glTranslate<fd>(type dx, type dy, type dz)
```

通过将以参数 `dx`, `dy` 和 `dz` 为参数的平移矩阵与当前做右乘来改变当前矩阵。`type` 可为 `GLfloat` 或 `GLdouble` 类型。【109~110】

平移被应用于当前矩阵(由当前矩阵模式所决定,即 `GL_PROJECTION` 或 `GL_MODELVIEW`)。如果想得到一个纯粹的平移矩阵,必须首先设置矩阵模式,然后将当前矩阵初始化为一个单位矩阵,最后再调用 `glTranslate()` 函数。所以,如果想将所有的顶点沿着 z 轴负方向平移一个单位,可这样做:

```
glMatrixMode(GL_MODELVIEW);
glLoadIdentity();
glTranslatef(0.0, 0.0, -1.0);
```

注意,对 `glTranslate*()` 函数的参数的一种解释是它们表示相对于原点处的一个位置固定的摄像机所移动的距离。由此,正值的 d_x 表示对象相对摄像机向右发生了移动,正值的 d_y 表示对象相对摄像机向上发生了移动,正值的 d_z 表示对象在摄像机之后距离摄像机的距离。这些距离是在如图 5.3 所示的右手坐标系中定义的。

另外还需要注意,这些距离与用户绘制对象的距离是一致的,即它们是在对象坐标系中度量的。

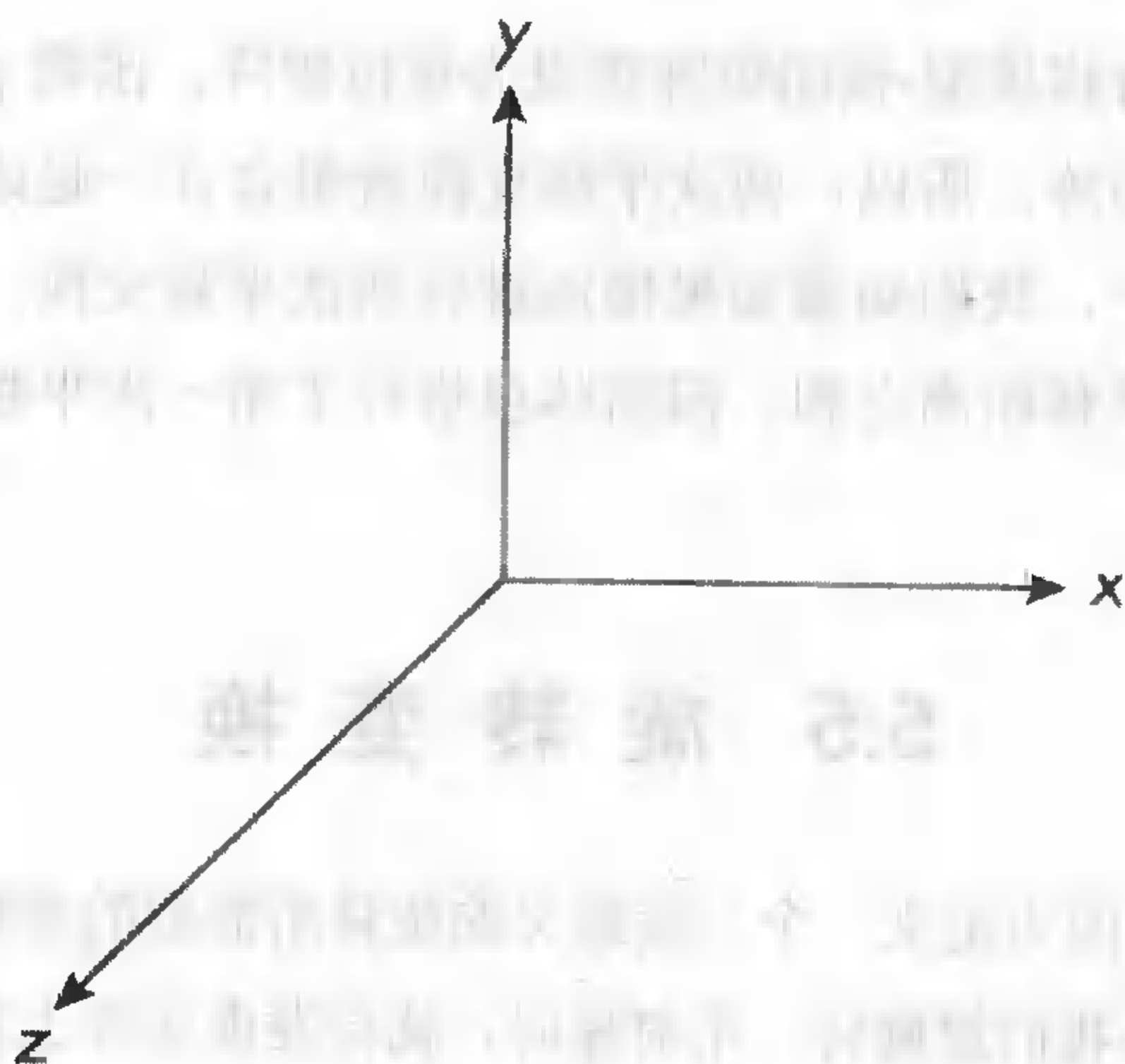


图 5.3 右手坐标系

提示 平移的距离是在右手坐标系中定义的。 z 方向的负距离可将对象从摄像机后方移至摄像机前方。 【110~111】

5.4.1 变换的级联

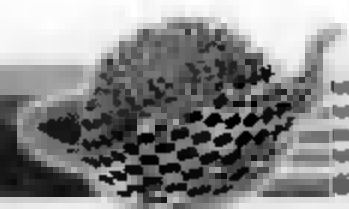
现在假定我们希望创建一个由两个对象构成的场景，它们都位于摄像机的前方。可这样做：

```
glMatrixMode(GL_MODELVIEW);
glLoadIdentity();
glTranslatef(0.0, 0.0, -2.0);
glutWireTetrahedron();
glLoadIdentity();
glTranslatef(0.0, 0.0, -3.0);
glutWireCube();
```

或者用如下代码：

```
glMatrixMode(GL_MODELVIEW);
glLoadIdentity();
glTranslatef(0.0, 0.0, -2.0);
glutWireTetrahedron();
glTranslatef(0.0, 0.0, -3.0);
glutWireCube();
```

这两个程序片段会产生不同的输出结果。究竟是什么原因？在第一个片段中，我们绘制了一个立方体和一个四面体，且二者的中心都位于原点处。第 1 个对象(四面体)从摄像机(即原点)移开 2 个单位，第 2 个对象(立方体)从原点移开 3 个单位。在绘制立方体前，我们将模型-视图矩阵重新设为单位矩阵。立方体和四面体的位置是彼此完全独立进行指定的。在



第2个片段中,我们没有将模型-视图矩阵重设为单位矩阵。函数 `glTranslate*()` 形成了一个应用于当前矩阵的平移矩阵。所以,两次平移变换就组合在一起或级联,从而形成了一个复合变换。在这种情况下,我们知道如果接连进行两次平移变换,等价于一次平移变换,只是平移的距离为两次平移距离之和。四面体仅进行了第一次平移变换,而立方体相当于经历了两次平移变换。

5.5 旋转变换

旋转定义起来颇有难度,因为定义一个一般意义的旋转所需要的参数比定义一个一般意义的平移所需的参数更多。当我们想旋转一个对象时,就会发现实际上我们可以绕着任意轴对该对象进行旋转。您可能还会注意到,有一个点不会受到转动的影响,而该点不一定在对象上。我们将该点称为旋转不动点(fixed point)。除了指定旋转轴,我们还必须指定绕着该轴的旋转角。图5.4展示了一般旋转所需的参数。两个点 p_1 和 p_2 , 确定了一条线段,该线段给出了旋转轴的方向。点 p_0 为旋转不动点,通常对应于旋转物体的中心。【111~112】

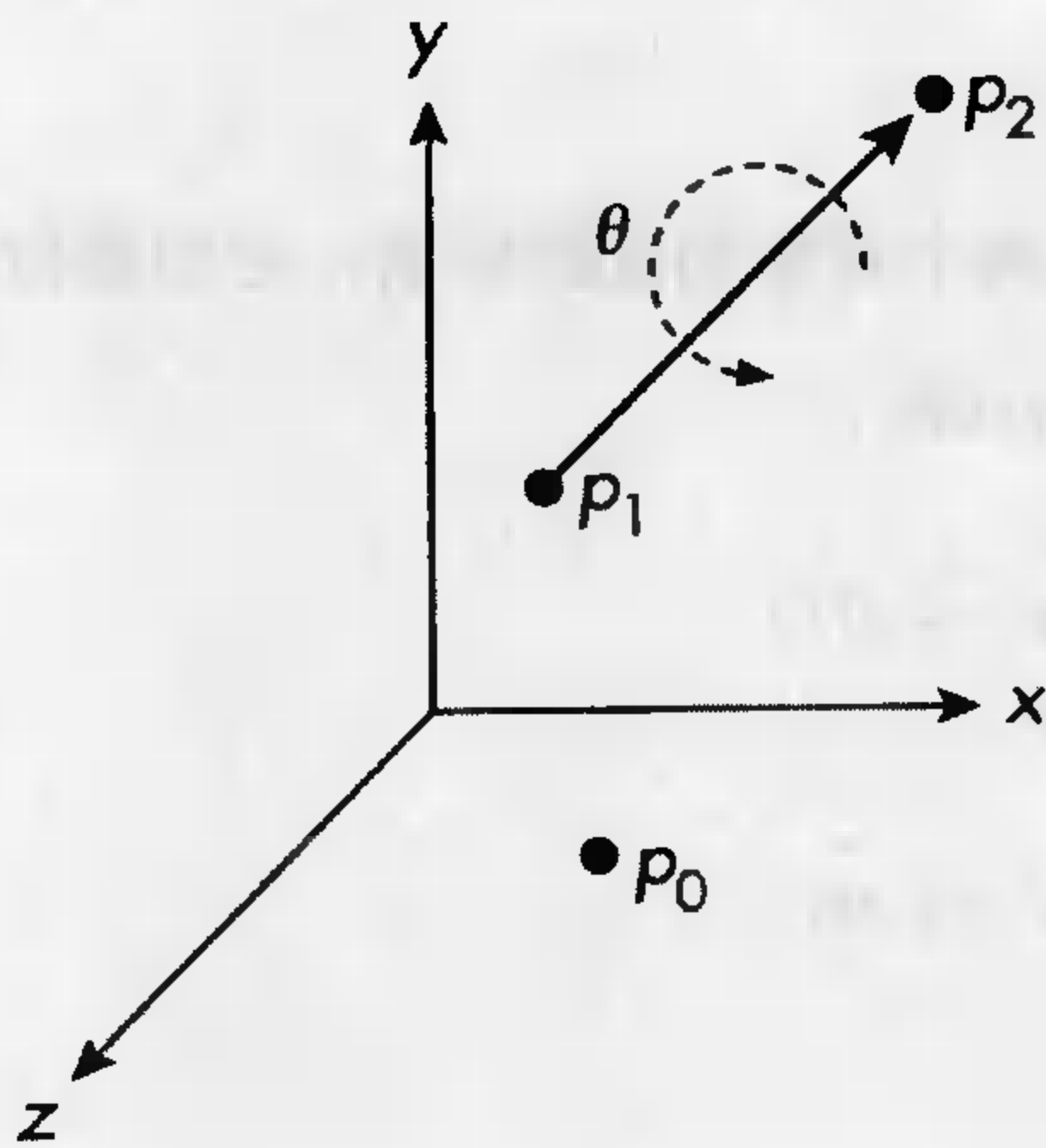


图 5.4 定义一个一般旋转

OpenGL 中的旋转函数 `glRotate*()` 假定旋转不动点位于坐标原点。稍后我们将看到如何处理一个一般的旋转不动点。给定旋转轴的方向(用三维向量来表示)和旋转角,OpenGL 就会构造一个旋转矩阵。旋转角是这样规定的:从旋转轴的正端往原点看,按逆时针旋转的角度为正。所以,在右手坐标系中,绕着 z 轴旋转 90° ,将使 x 的正半轴旋转到原先 y 轴正半轴的位置上。

提示 从向量的末端往始端看时,绕着该向量逆时针方向的旋转定义为正方向。

与平移变换一样,旋转也是用 `float` 类型或 `double` 类型的参数来指定的。

【112】


```
void glRotate<fd>(type angle, type dx, type dy, type dz)
```

形成一个旋转轴为(dx, dy, dz)的旋转矩阵, 旋转不动点为坐标原点。type 可为 GLfloat 或 GLdouble。

级联：绕着任意固定点的旋转

我们可用旋转和平移来实现绕着一个不动点而非原点的旋转。其主要思路是我们可通过平移将不动点平移至原点处, 然后使用函数 `glRotate*()` 实现所期望的旋转。最后, 我们还需要通过一次平移将不动点变换到初始位置。所以, 如果我们希望绕着方向(d_x, d_y, d_z)旋转, 而使旋转不动点为(x_f, y_f, z_f), 可以这样做:

```
glMatrixMode(GL_MODELVIEW);
glLoadIdentity();
glTranslatef(xf, yf, zf);
glRotatef(angle, dx, dy, dz);
glTranslatef(-xf, -yf, -zf);
```

请注意上述变换的次序。在 OpenGL 中, 变换应用的次序与其在程序中出现的次序相反。换言之, 最后指定的变换将最先得得到应用。这种不太符合直觉的结果是由于每次在 OpenGL 中定义的变换都通过自右乘(postmultiplication)作用于当前矩阵的。所以, 在我们的例子中, 首先从一个单位矩阵开始, 然后通过实施一次平移变换将不动点平移至原点。然后我们再对当前矩阵应用一次旋转变换。最后一次变换是第一次平移变换的逆变换, 通过这次变换, 不动点的位置又被还原。由于顶点和模型-视图矩阵是通过右乘进行运算的, 所以从数学上看最后的矩阵将首先得到应用。当然, 所有顶点只会进行一次模型-视图变换; 该变换是我们指定的所有变换的组合。

提示 记住, 在绘制任何几何对象之前, 都是最后指定的变换首先得到应用。

务必牢记, 模型-视图矩阵是 OpenGL 状态的一部分。所以, 除非我们将其修改, 否则所有在程序中后来出现的图元都会绕着同一个不动点旋转。位于那些没有改变当前矩阵的显示列表中的任意顶点都会受到相同的模型-视图矩阵的影响。反之, 如果在显示列表中将当前矩阵改变, 则在显示列表执行之后这种变化将会发挥作用。我们可在显示列表的一开始将当前矩阵入栈, 而在显示列表的末尾将原矩阵出栈, 就可不受状态改变带来的影响。【113~114】

提示 由于变换会影响 OpenGL 的状态, 所以对于显示列表内部对矩阵所做的修改要特别留意。

5.6 比例变换

由 OpenGL 提供的第 3 种标准变换是比例变换。当我们对一个物体进行比例变换时, 我们使得该物体变得更大或更小。但是, 我们可以不沿着所有的方向做均匀的比例变换。不妨

考虑对如图 5.5 所示的立方体实施比例变换。该变换使得原立方体在一个方向上变长，而在另一个方向上变短。在第一种情形中，比例因子大于 1，而在第二种情形中，比例因子小于 1。我们也注意到，在比例变换中也存在一个不动点。在图 5.5 中，不动点为原点。函数 `glScalef()` 允许我们沿着三个坐标轴方向分别指定不同的比例因子。虽然不动点位于原点处，但我们借助平移变换可将其移动到任意所期望的位置。

```
void glScale<fd>(type sx, type sy, type sz)
```

依据比例因子 `sx`、`sy` 和 `sz` 创建一个比例变换矩阵，其不动点位于原点处。`type` 可取 `GLfloat` 或 `GLdouble`。

比例因子可取负值，此时除了比例变换外，还伴随有镜像变换。例如，`glScalef(1.0, -2.0, 0.5)` 将保持 x 坐标不变，而使对象关于 y 轴做镜像变换的同时沿 y 轴方向进行拉伸，并沿着 z 轴方向将对象压缩。

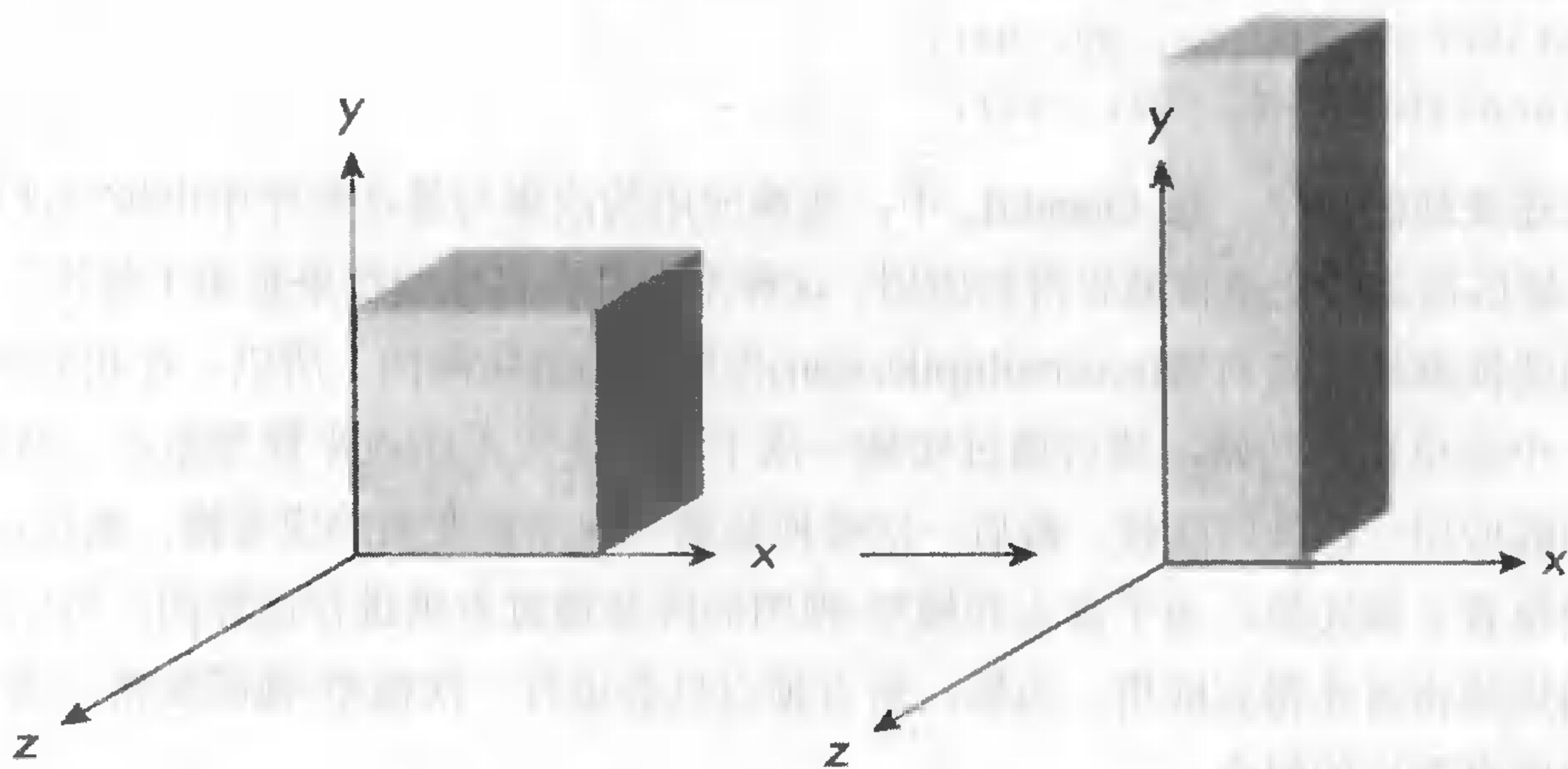


图 5.5 由非均匀比例因子引起的比例变换

【114】

5.7 一个旋转的立方体

现在我们将第 4 章中建好的立方体模型与第 2 章中旋转矩形的思想结合起来。这次，除了研究对象从二维变成了三维外，我们将使用变换来实现旋转。此外，我们还要将程序改造为完全交互式的。

首先，我们为立方体的各个面指定一些颜色：

```
GLfloat colors[][3] = { {1.0, 0.0, 0.0}, {0.0, 1.0, 0.0},  
                        {0.0, 0.0, 1.0}, {1.0, 1.0, 0.0},  
                        {0.0, 1.0, 1.0}, {1.0, 0.0, 1.0} }
```

然后我们可将立方体的绘制代码修改为：


```

void colorcube()
{
    glColor3fv(color[0]);
    polygon(0, 3, 2, 1);
    glColor3fv(color[1]);
    polygon(2, 3, 7, 6);
    glColor3fv(color[2]);
    polygon(3, 0, 4, 6);
    glColor3fv(color[3]);
    polygon(1, 2, 6, 5);
    glColor3fv(color[4]);
    polygon(4, 5, 6, 7);
    glColor3fv(color[5]);
    polygon(5, 4, 0, 1);
}

```

注意，我们必须将颜色函数从先前版本的 `polygon()` 函数中移除。

我们将使用空闲回调函数为旋转角提供一个固定的增量。我们将使用鼠标回调函数来选择将围绕哪个轴进行旋转。显示回调函数将使物体绕着指定的旋转轴进行旋转。在主函数中，我们定义如下回调函数：

```

glutDisplayFunc(display);
glutIdleFunc(spinCube);
glutMouseFunc(mouse);

```

鼠标回调函数实现起来比较容易。注意，关于每个轴的旋转角以及轴的编号都是全局变量。

```

int axis;
float theta[3];
void mouse(int btn, int state, int x, int y)
{
    if(btn == GLUT_LEFT_BUTTON && state == GLUT_DOWN)
        axis = 0;
    if(btn == GLUT_MIDDLE_BUTTON && state == GLUT_DOWN)
        axis = 1;
    if(btn == GLUT_RIGHT_BUTTON && state == GLUT_DOWN)
        axis = 2;
}

```

【115~116】

空闲函数在每次被调用时都将指定的转角增加 2 度，并对旋转角以 360 取模。

```

void spinCube()
{
    theta[axis] += 2.0;
}

```



```

        if(theta[axis] > 360.0, theta[axis] -= 360.0;
        glutPostRedisplay();
    }

```

最后，显示回调函数在绘制立方体之前，将其分别绕着 x , y , z 轴进行旋转。

```

void display()
{
    glClear(GL_COLOR_BUFFER_BIT|GL_DEPTH_BUFFER_BIT);
    glLoadIdentity();
    glRotatef(theta[0], 1.0, 0.0, 0.0);
    glRotatef(theta[1], 0.0, 1.0, 0.0);
    glRotatef(theta[2], 0.0, 0.0, 1.0);
    colorcube();
    glutSwapBuffers();
}

```

5.8 直接设置矩阵

我们可通过适当选取旋转、平移以及比例变换的次序而构造任意的仿射变换。通常，我们在用户程序中进行这种构造。但是，有时我们希望直接构造一个模型-视图矩阵或投影矩阵，或用我们在程序中的矩阵来修改当前投影矩阵或模型-视图矩阵。

在 OpenGL 中，矩阵都是按列优先的顺序存放在一个包含 16 个元素的一维数组中，而非 4×4 的二维数组。OpenGL 提供了两个可用来对矩阵进行操作的函数：`glLoadMatrix*()` 和 `glMultMatrix*()`。第一个函数将一个用户自定义数组加载为当前矩阵。第二个函数用一个用户自定义的数组右乘当前矩阵。

【116~117】

```
void glLoadMatrix<fd>(type * m)
```

将 `type` 类型(可取 `GLfloat` 或 `GLdouble`)的数组 `m` 加载为当前矩阵。

```
void glMultMatrix<fd>(type * m)
```

用矩阵 `m` 右乘当前矩阵。

自行设置矩阵的一个重要用途是为基本变换增加剪切变换。图 5.6 是一个对二维矩阵沿 x 方向做剪切变换的例子。这种效果看起来好像是由顶部和底部分别作用的一对相反方向的力所造成的。这种变换也属于仿射变换，因为它具有线性保持性。我们也可看出，对于该变换也存在一个不动点，即图中的原点。

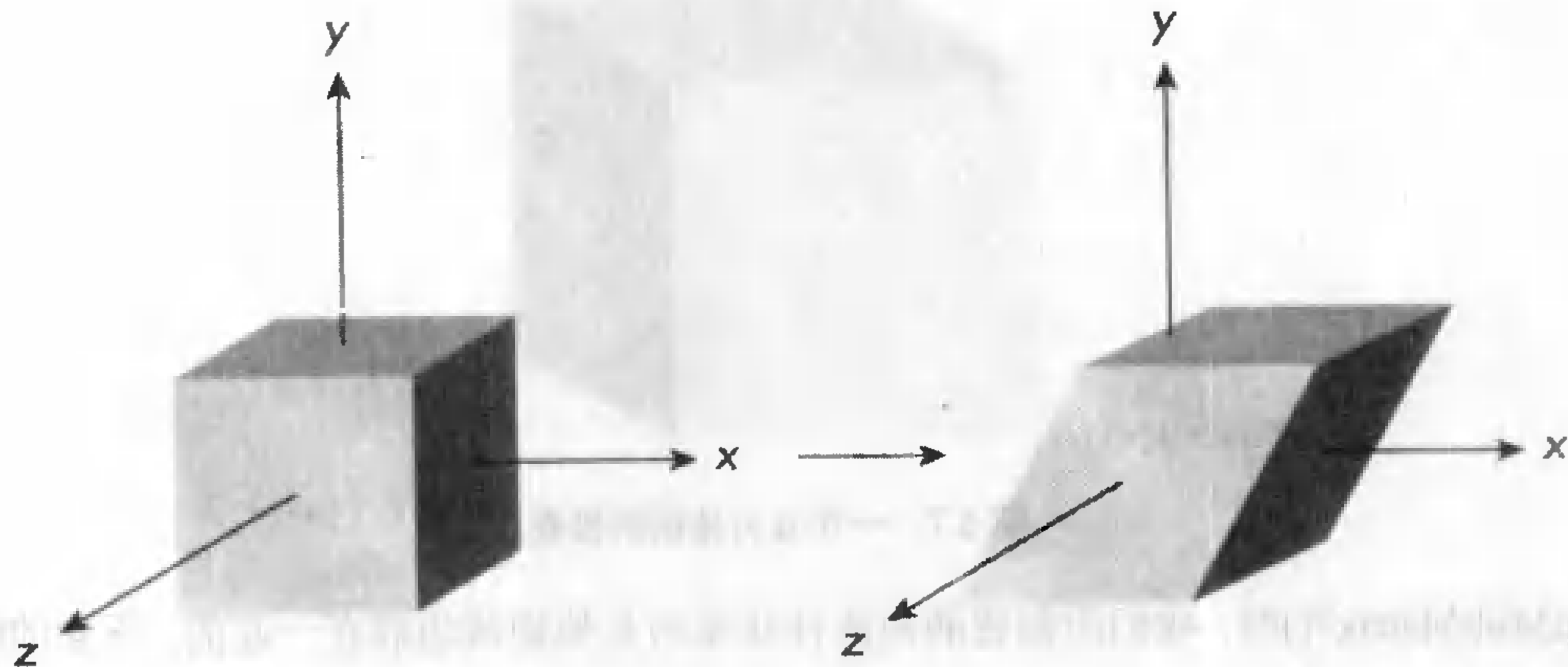


图 5.6 发生剪切的立方体

实现剪切变换的矩阵如下：

$$M = \begin{bmatrix} 1 & \cot\theta & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

这里略去了对 M 的推导过程。我们可在程序中构造该矩阵：

```
GLfloat m[16];
for(i = 0; i < 16; i++)
    m[i] = 0.0;
m[0] = m[5] = m[10] = m[15] = 1.0;
m[4] = 1.0/tan(DEG_TO_RAD*theta);
```

```
glLoadMatrixf(m);
```

【117】

我们可在剪切变换矩阵的对角线上方增加或替代一些正切项来实现任意三个方向的剪切变换。剪切不仅为我们创建对象增加了灵活性，而且可以赋予我们更加机动的观察方式。例如，图 5.7 中的立方体的投影被称为斜投影(oblique projection)。它是一种平行投影，但无法直接得到。因为在 OpenGL 中惟一的平行投影矩阵只是由 `glOrtho()` 所形成的矩阵中的一种形式。但是，我们可以借助类似于 M 的剪切矩阵来修改由 `glOrtho()` 所形成的矩阵，以创建如图 5.7 所示的视图。

```
glMatrixMode(GL_PROJECTION);
glLoadIdentity();
glOrtho(left, right, bottom, top, near, far);
glMultMatrixf(m);
```

【117】

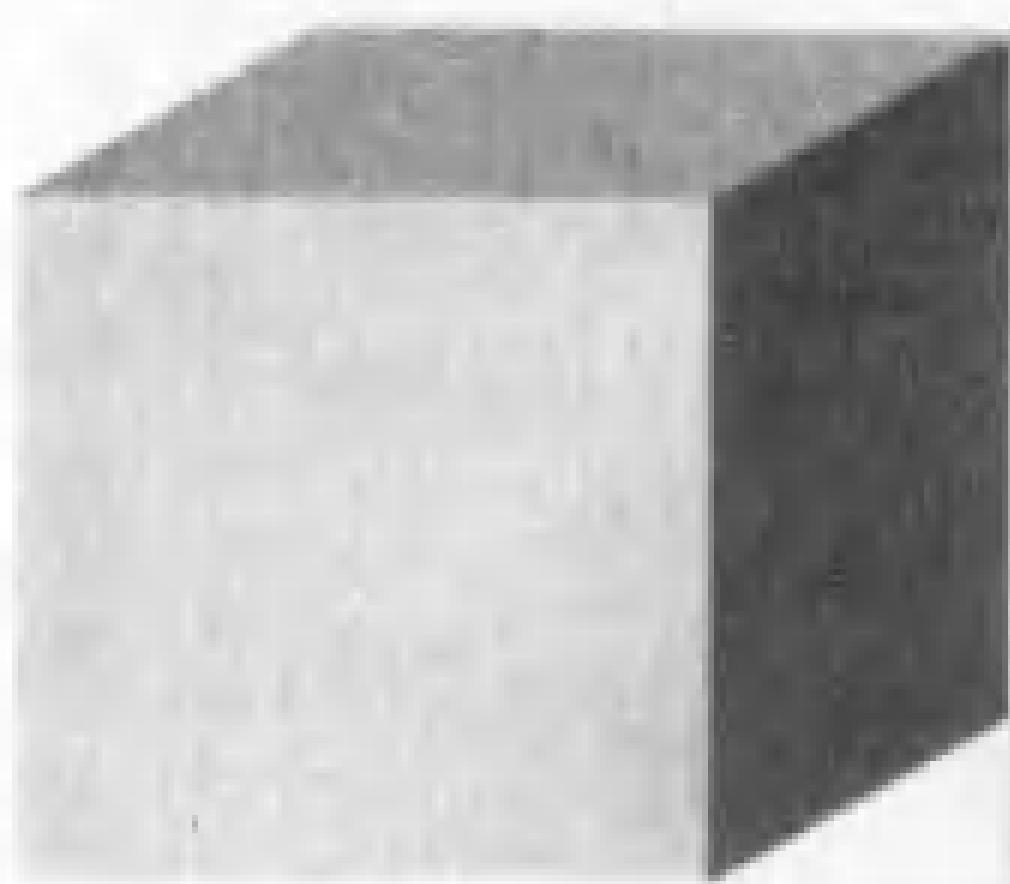


图 5.7 一个立方体的斜投影

使用 `glMultMatrix*()` 时, 我们所创建的矩阵往往是与其他矩阵级联在一起的。下面的例子依据点光源 (x_l, y_l, z_l) 生成了一个立方体的阴影, 该阴影被投射到平面 $z = 0$ 上。该矩阵可通过三个步骤来创建。首先, 我们将光源移到坐标原点, 然后再进行投影。最后我们将光源复位。所需的投影矩阵为:

$$M = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & -\frac{1}{y_l} & 0 & 0 \end{bmatrix}$$

创建该矩阵的代码为:

```
GLfloat m[16];
for(i = 0; i < 15; i++)
    m[i] = 0.0;
m[0] = m[5] = m[10] = 1.0;
m[7] = -1.0/y1;
```

【118】

这次我们需要将该立方体绘制两次。第一次绘制时, 我们依据模型-视图矩阵绘制该立方体。然后我们将当前颜色修改为阴影对应的颜色, 并通过两个平移矩阵和阴影投影矩阵来修改当前模型-视图矩阵。我们可将这些变换应用于当前矩阵, 因为为了指定立方体的方位, 无论实施何种变换, 这些变换都将同时应用于阴影多边形。

```
glMatrixMode(GL_MODELVIEW);
/*set up model-view matrix and attributes for cube*/
cube();

/*save matrix and color*/

glPushMatrix();
glPushAttrib(GL_CURRENT_BIT);
```



```
glTranslate(x1, y1, z1);
glMultMatrix(m);
glTranslate(-x1, -y1, -z1);

/*draw shadow in black*/

glColor3f(0.0, 0.0, 0.0);
cube();

/*restore matrix and color*/

glPopMatrix();
glPopAttrib();
```

请注意上述代码中保存(压栈)当前颜色和当前矩阵的必要性。因为这使得我们能够继续对其他对象的绘制或当立方体位置变化后，对其进行重绘。由该段程序生成的一幅图像如图 5.8 所示。

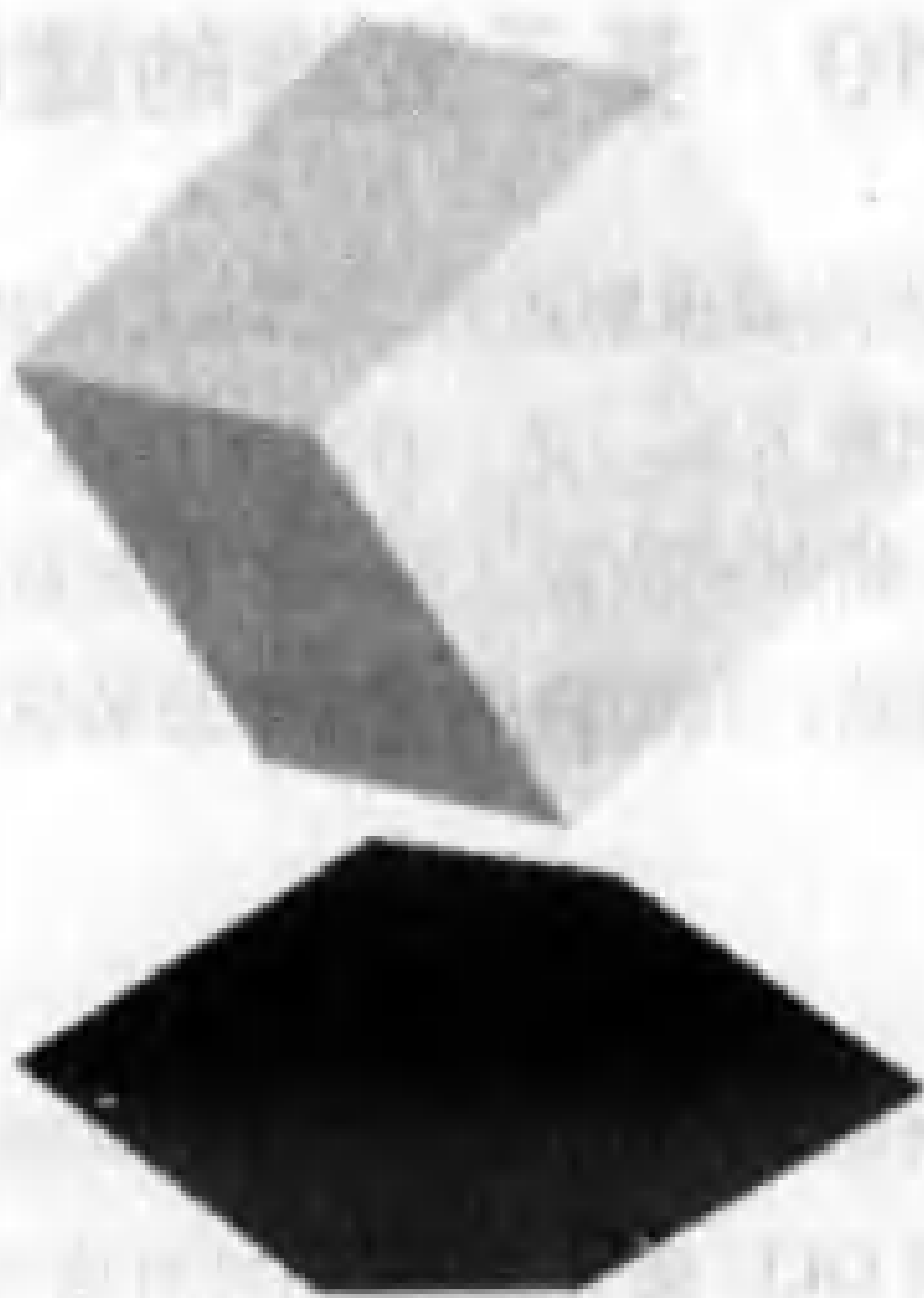
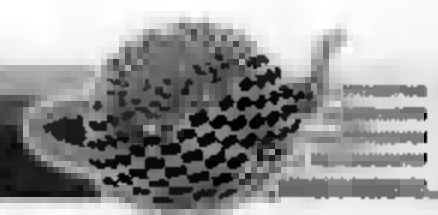


图 5.8 立方体及其阴影

【119】

5.9 变换与坐标系

我们已经看到，OpenGL 使用了多种坐标系。坐标系每改变一次都相当于进行了一次变换。所以，我们可用两种等价的观点来看待模型-视图矩阵的功能。我们可将其视为相对于摄像机移动物体的一种变换。或者我们也可将其视为这样一种运算：依据对象在世界坐标系中的描述给出对象在摄像机坐标系中的描述。这两种观点都是正确的。从应用程序开发者的角度看，第一种观点可能更实用。而第二种观点更有助于理解该矩阵的内部机理。



这两种观点对于解释为何观察函数 `glOrtho()`、`glFrustum()`和 `glPerspective()`中的参数都是相对摄像机来度量的非常有帮助。这些变换几乎总是用来形成投影矩阵的。该矩阵所作用的顶点已经依据模型-视图矩阵进行了变换，所以已经处于摄像机坐标系中。由此，在这个阶段应用的任何函数的参数都应在摄像机坐标系中。

如果对投影变换进行同样的推理，我们会将其视作这样一种变换：将对象在摄像机坐标系中的描述变换为在裁剪坐标系(`clip coordinate`)中的描述。裁剪坐标以齐次坐标的形式保留了顶点的信息，并将顶点映射到一个归一化的立体中。一旦完成除法(即 w 分量不为 1.0 的情形)，顶点便处于三维坐标系中，该坐标系称为归一化设备坐标系(`normalized device coordinate`)。虽然投影的过程是将顶点从三维空间映射到二维观察表面中，但由于 OpenGL 总是尽可能久地保持着三维信息，所以如消隐、光照、纹理映射等运算都可在流水线中尽可能靠后的时候来完成。最后，可见实体通过视口变换被映射到窗口坐标系中，该坐标系的单位为像素。

5.10 基于变换的建模

虽然变换可被应用于模型-视图矩阵或投影矩阵。实际上，变换的主要用途是通过模型-视图矩阵为对象建模。我们来研究两种方案。第一种方案是用一些标准对象来构建场景。每个对象必须可以独立地调整尺寸、朝向和位置。第二种方案是构建一些彼此互相连接的对象，这些对象必须作为整体而发生移动。我们将用变换来封装这些复杂的关系。

5.10.1 实例化

假定我们现有一组通用对象，例如立方体和球体，我们可能已有创建这些对象的代码，或者也可能这些对象是由 GLUT 或 GLU 提供的。我们可对这些已有对象应用仿射变换来创建其他类型的对象。通常我们将这些基本对象称为符号(symbol)，每个在程序中出现的对象都被称为是该符号的一个实例。

【120~121】

首先从立方体着手，通过比例变换我们可以创建任意的直平行六面体。如果我们还允许剪切变换，我们还可由立方体创建出平行六面体。我们可通过对一个球体进行非均匀比例变换而得到一个椭圆球体。然后我们就可对这些对象应用旋转矩阵和平移矩阵分别为其定向和定位。能够将对象以所期望的尺寸、方向和位置组成模型的变换矩阵称为实例变换矩阵。

通常，我们用于构建模型的对象都能以一种方便的形式进行尺寸、位置和方向的调整。例如，球体通常定义为中心在原点的一个单位球。立方体的边长为 1.0 或 2.0，其各边与格坐标轴平行。GLU 圆柱体以 z 轴为对称轴，其底面为 $z = 0$ 。以这些对象为基础，我们几乎总

是希望将对象缩放到理想的尺寸，然后将其旋转，最终将其平移到所期望的位置上。所以，一个典型的建模过程如下：

```
glMatrixMode(GL_MODELVIEW);
glLoadIdentity();
glTranslatef(x, y, z);
glRotatef(theta, dx, dy, dz);
glScalef(sx, sy, sz);
/* code for objects */
```

请再次注意变换的次序。

5.10.2 层次模型

在许多应用程序中，模型的各部分之间都具有依赖关系。如果我们对模型的某一部分应用一个变换，则该部分产生的变化将导致其他部分也发生移动。在大多数这样的应用程序中(包括图形动画和机器人)，模型都是层次性的。这类模型的各部件可按树型数据结构进行组织，该结构由结点和结点之间的连接组成。除了顶层结点或根结点外，每个结点都有一个父结点；除终端结点或叶结点外，所有结点都有一个或多个子结点。图 5.9 展示了一个简单的机械臂，其层次表示如图 5.10 所示。图 5.11 和图 5.12 展示了一幅我们想要生成动画的简单图形及其层次表示。在这些层次中，一个对象的位置和方向将受到其父结点对象的位置和方向的影响(其父结点对象的父结点)。

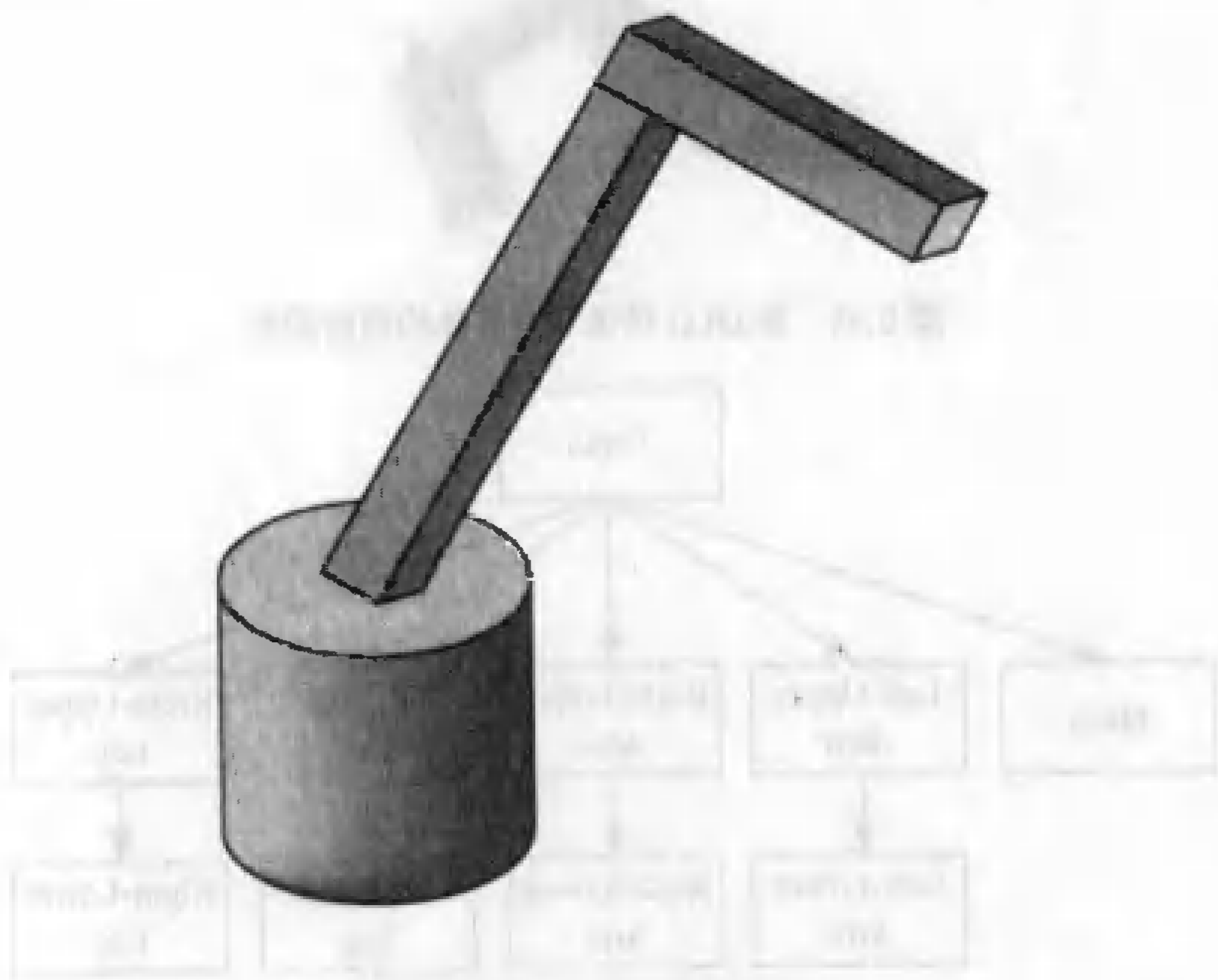


图 5.9 一个简单的机械臂

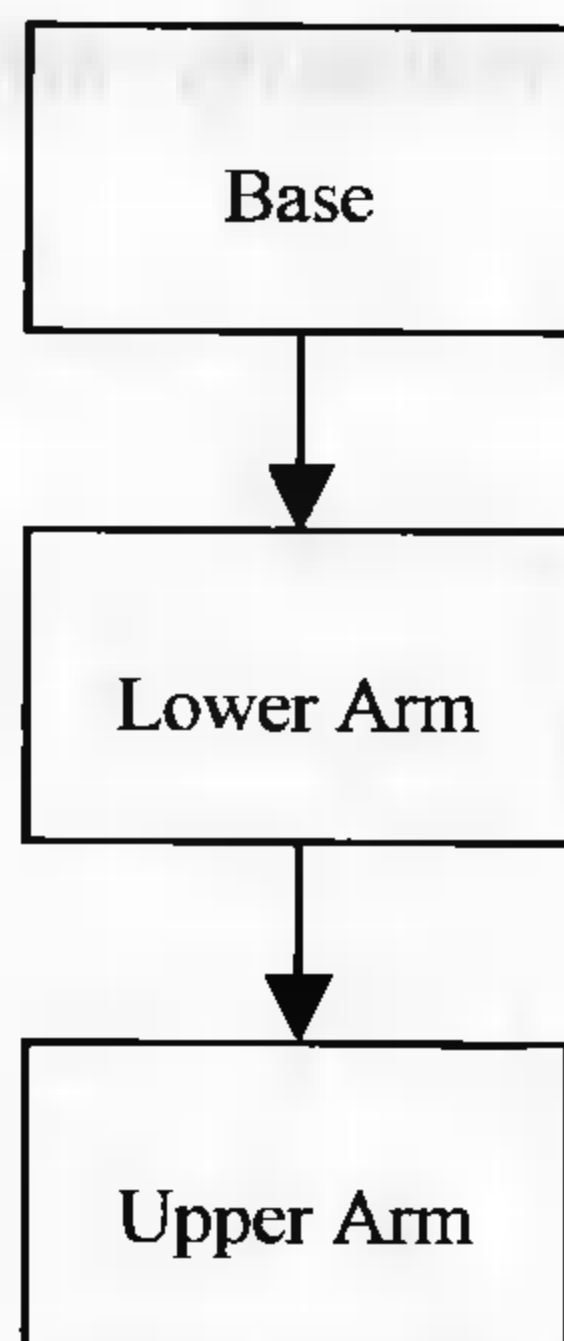


图 5.10 机械臂的层次表示



图 5.11 由 GLU 球体和圆柱体构造的图形

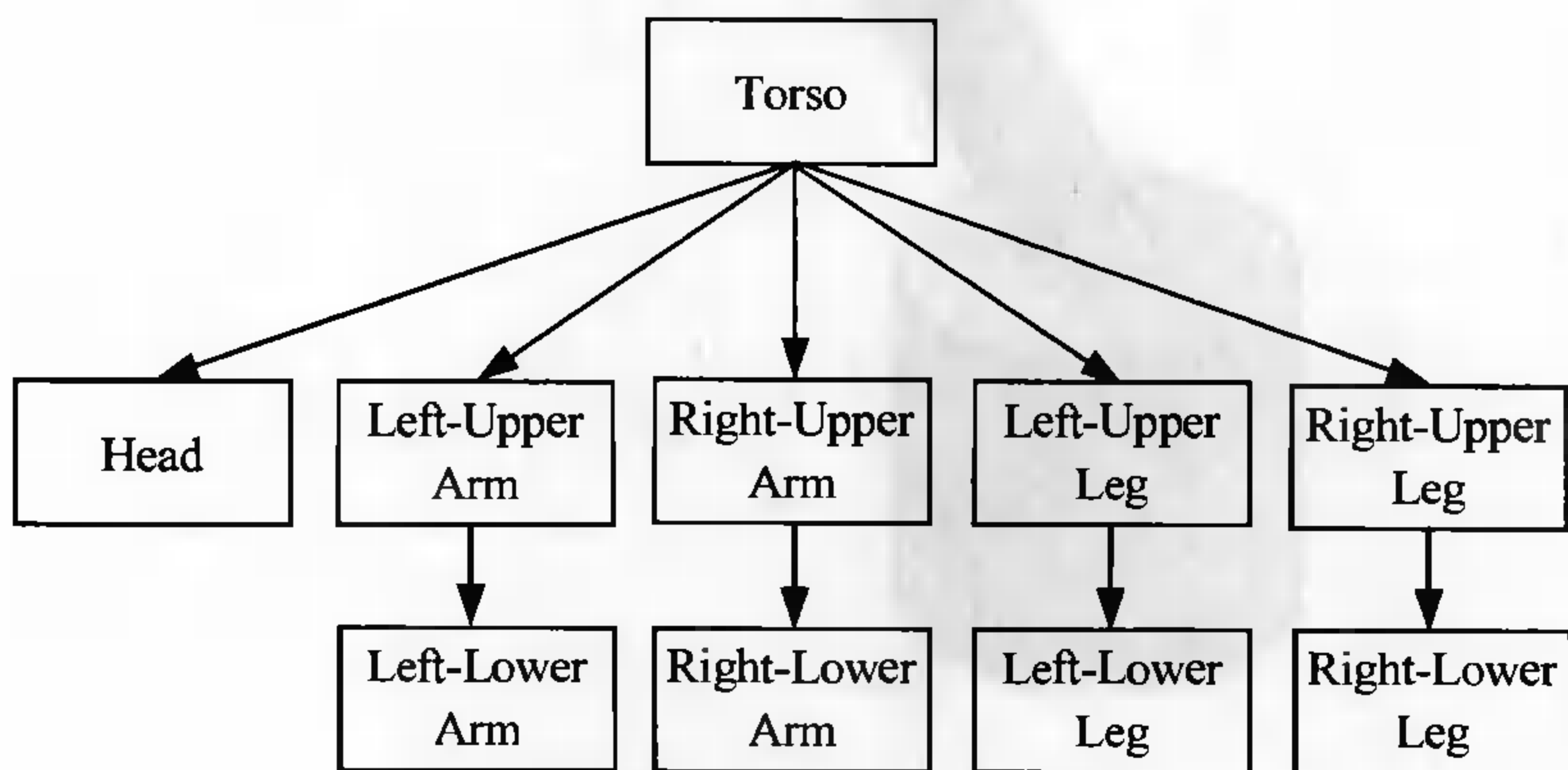


图 5.12 该图形的层次表示

这些例子中都有通过旋转关节相互连接的部件，从而可将动画和运动简化为一组关节角的计算，当这些关节角中的一个或多个发生变化时，将场景重新进行绘制即可。

依据 OpenGL 的观点，我们更倾向于用变换来表示这些模型。我们刚刚提到的实例变换存在一些问题，因为这种变换是将各个符号独立地放置在场景中。然而，如果我们还记得 OpenGL 变换是直接作用于当前矩阵的，就能想到每个变换实际上表示一次相对变化，即从一种比例、位置和方向变为另一种比例、位置和方向。【121~122】

现在我们来考虑一个简单的机器人模型。它由三部分组成：base, lower arm 和 upper arm。每一个部件都是在其自身的坐标系中用标准的 OpenGL 对象(如圆柱体或经过比例变换的立方体等)进行描述的，如图 5.13 所示。

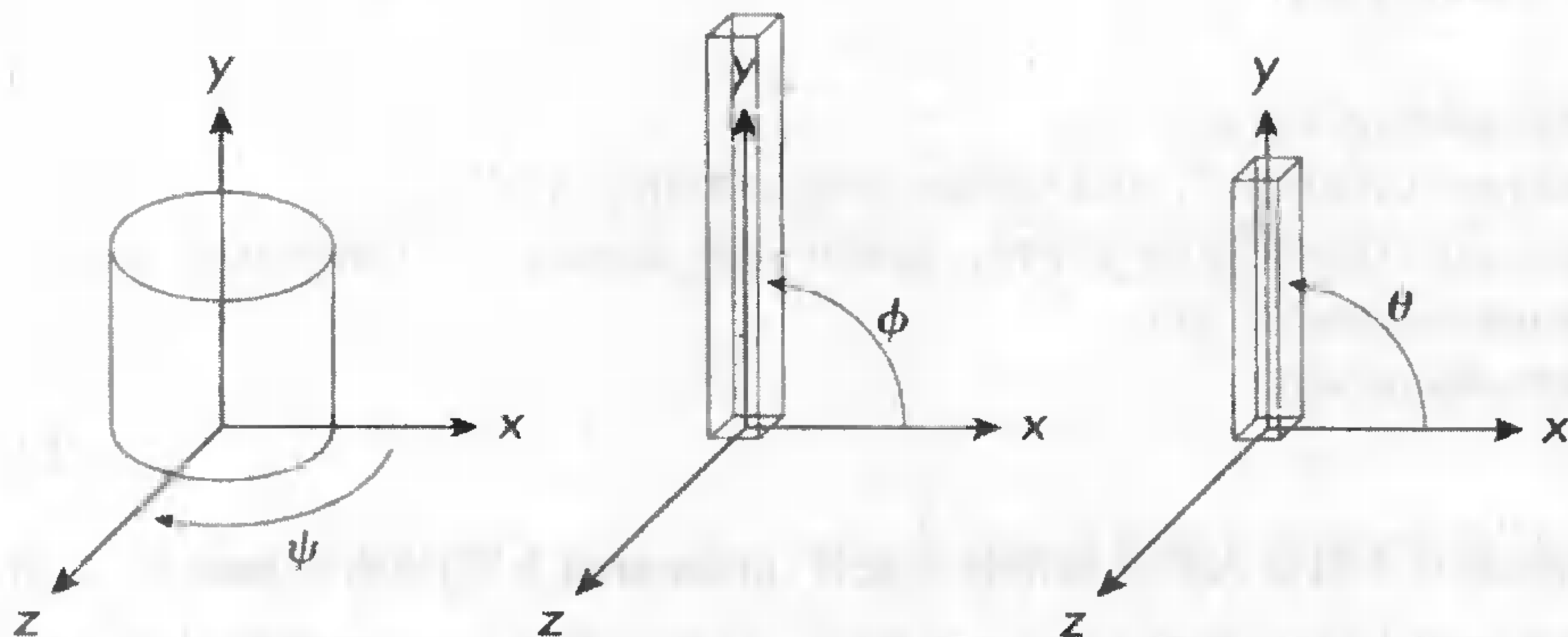


图 5.13 机器人的组成部件

假定我们用圆柱表示 base(基础)，而用比例缩放过的立方体表示 upper arm(上臂)和 lower arm(下臂)；接下来我们就可通过三个函数——base(), lower_arm()和 upper_arm()——来定义这些部件。注意，二次曲面对象必须在 main()或 init()函数中进行定义，而描述各种长度的参数可在程序中任意位置来定义。通过使用函数 glPushMatrix()和 glPopMatrix()，我们在用当前模型-视图矩阵来为整个图形定位的同时还可将其保留，供绘制其他对象时使用。【122】

```
GLUquadricObj *p; /*pointer to quadric object*/

void base()
{
    glPushMatrix();

    /*rotate cylinder to align with y axis*/
    glRotatef(-90.0, 1.0, 0.0, 0.0);

    /*cylinder aligned with z axis*/
    /*render with five slices for base and five along length*/
}
```



```

    gluCylinder(p, BASE_RADIUS, BASE_RADIUS, BASE_HEIGHT, 5, 5);
    glPopMatrix();
}

void upper_arm()
{
    glPushMatrix();
    glTranslatef(0.0, 0.5*UPPER_ARM_HEIGHT, 0.0);
    glScalef(UPPER_ARM_WIDTH, UPPER_ARM_HEIGHT, UPPER_ARM_WIDTH);
    glutWireCube(1.0);
    glPopMatrix();
}

void lower_arm()
{
    glPushMatrix();
    glTranslatef(0.0, 0.5*LOWER_ARM_HEIGHT, 0.0);
    glScalef(LOWER_ARM_WIDTH, LOWER_ARM_HEIGHT, LOWER_ARM_WIDTH);
    glutWireCube(1.0);
    glPopMatrix();
}

```

【123~124】

base(基础)可独立于机器人的其他部件而旋转。lower arm(下臂)依附在 base 之上,可相对 base 转动。upper arm(上臂)可相对 lower arm 转动,但 base 和 lower arm 的转动都回对 upper arm 产生影响。由于 lower arm 位于 base 的顶部,所以它必须向上平移。upper arm 应向上平移的高度等于 base 的高度与 lower arm 的高度之和。下面的回调函数体现了上述这些关系。这部分代码是增量式的。因为这些变换描述的都是每个部件组合到其父结点对象时的变化。这些代码表明,当一个部件发生变化或移动时,大多数矩阵都不必重新计算。 【124】

```

void display(void)
{
    glClear(GL_COLOR_BUFFER_BIT);
    glMatrixMode(GL_MODELVIEW);
    glLoadIdentity();
    glColor3f(1.0, 0.0, 0.0);
    glRotatef(theta[0], 0.0, 1.0, 0.0);
    base();
    glTranslatef(0.0, BASE_HEIGHT, 0.0);
    glRotatef(theta[1], 0.0, 0.0, 1.0);
    lower_arm();
    glTranslatef(0.0, LOWER_ARM_HEIGHT, 0.0);
    glRotatef(theta[2], 0.0, 0.0, 1.0);
}

```



```

upper_arm();
glutSwapBuffers();
}

```

我们可用多种方式为该程序增加动画效果。一种方法是通过依附在某个鼠标按键(假定为三键鼠标)上的菜单来选择哪个角需要改变,而用另外两个按键增加或减小该角度。作为替代方案,我们也可用键盘来完成上述选择。

在我们的第一个例子中,在执行回调函数的过程中并不需要保存任何关于模型-视图矩阵的信息,因为这些变换都是增量式的。现在我们来回顾一下图 5.10 中的树型结构,可以看出它是一种非常简单的情形,即每个结点至多只有一个孩子结点。图 5.11 中的模型与之相比就显得比较复杂。它由与躯干相连的若干部件组成。每条胳膊和腿都由两个部件构成。每条胳膊和腿的位置和方向都与躯干的位置和方向有关,而与其他胳膊或腿的方位无关。假定我们能够独立地构建每个部件,并将相关信息放入一组函数中,如 `head()`, `torso()` 以及 `left_upper_arm()`。每个部件都可通过一次平移以及一次或多次转动而相对其父结点对象进行定位,这主要取决于该部件与其父结点对象的连接方式。

显示回调函数必须对图 5.12 所示的树结构进行遍历(traverse)。即我们必须访问每个结点,并使用正确的模型-视图矩阵绘制每个结点所代表的部件。一个标准的先序遍历从该树的左子树开始依序遍历每个结点。当遍历完左子树之后,再回到右子树,并递归地重复上述过程。下面我们来分析一下这种遍历方法对于层次模型为什么是适用的。

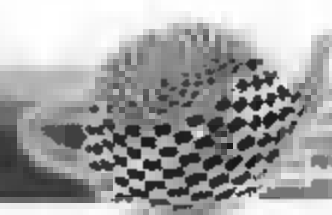
首先我们来绘制躯干。可用一个角度来描述其方位,利用该角度我们可以 y 轴为转轴,将躯干进行旋转。接下来轮到对头部进行绘制。然而,我们注意到要想到达胳膊和腿,必须先回到躯干。我们绘制头部时所使用的任何矩阵都不能用来绘制胳膊和腿。我们并不打算重新计算原先应用于躯干的矩阵,所以我们需要用 `glPushMatrix()` 将其压入矩阵堆栈中。所以当到达头部所对应的结点时,我们对当前模型-视图矩阵进行修改,以绘制出头部(这里使用了两个关节角)。返回躯干结点后,需要借助 `glPopMatrix()` 恢复原先的模型-视图矩阵。绘制完左臂后必须返回躯干结点,所以当借助出栈操作后,我们必须立即执行 `glPushMatrix()` 以保留同一模型-视图矩阵的一个副本。虽然下面的代码看起来有些复杂,规则其实非常简单。每次访问到具有一个未被访问的右子结点的左结点时,我们执行入栈操作,而每次返回该结点时,执行出栈操作。注意,我们必须在显示回调函数的最后也执行一次出栈操作,以保证入栈和出栈的次数相等。

【125~126】

```

void display()
{
    glClear(GL_COLOR_BUFFER_BIT|GL_DEPTH_BUFFER_BIT);
    glMatrixMode(GL_MODELVIEW);

```

```
glLoadIdentity();
glColor3f(1.0, 0.0, 0.0);

glRotatef(theta[0], 0.0, 1.0, 0.0);
torso();
glPushMatrix();

glTranslatef(0.0, HEADX, 0.0);
glRotatef(theta[1], 1.0, 0.0, 0.0);
glRotatef(theta[2], 0.0, 1.0, 0.0);
glTranslatef(0.0, HEADY, 0.0);
head();

glPopMatrix();
glPushMatrix();
glTranslatef(LUAX, LUAY, 0.0);
glRotatef(theta[3], 1.0, 0.0, 0.0);
left_upper_arm();

glTranslatef(0.0, LLAY, 0.0);
glRotatef(theta[4], 1.0, 0.0, 0.0);
left_lower_arm();

glPopMatrix();
glPushMatrix();
glTranslatef(RUAX, RUAY, 0.0);
glRotatef(theta[5], 1.0, 0.0, 0.0);
right_upper_arm();

glTranslatef(0.0, RLAY, 0.0);
glRotatef(theta[6], 1.0, 0.0, 0.0);
right_lower_arm();

glPopMatrix();
glPushMatrix();
glTranslatef(LULX, LULY, 0.0);
glRotatef(theta[7], 1.0, 0.0, 0.0);
left_upper_leg();

glTranslatef(0.0, LLLY, 0.0);
glRotatef(theta[8], 1.0, 0.0, 0.0);
left_lower_leg();

glPopMatrix();
glPushMatrix();
glTranslatef(RULX, RULY, 0.0);
glRotatef(theta[9], 1.0, 0.0, 0.0);
right_upper_leg();
```



```

glTranslatef(0.0, RLLY, 0.0);
glRotatef(theta[10], 1.0, 0.0, 0.0);
right_lower_leg();

glPopMatrix();
glutSwapBuffers();
}

```

【126~127】

我们可将该程序进一步完善,并像前面的机器人例程那样增加动画效果。例如,我们可用菜单来选择11个角度中哪个要发生变化,并用鼠标或键盘的两个按键来增加或减小该角度。

我们可通过定义一个节点数据结构(如左孩子-右兄弟结构)将上述代码推广:

```

typedef struct treenode
{
    GLfloat m[16];
    void(*f)();
    struct treenode *sibling;
    struct treenode *child;
}treenode;

```

该结构使我们可将应用于每个结点的父结点的矩阵及该结点的绘制函数保存起来。例如,我们可通过用 OpenGL 计算前面用到的旋转矩阵并将其保存在结点中,并连同其他信息一起来定义躯干。

【127】

```

treenode torso_node;
glLoadIdentity();
glRotatef(theta[0], 0.0, 1.0, 0.0);
glGetFloatv(GL_MODELVIEW_MATRIX, torso_node.m);
torso_node.f = torso;
torso_node.sibling = NULL;
torso_node.child = &head_node;

```

一旦我们定义好了所有的结点(比如在 init()函数中),就可在显示回调函数中对该数据结构进行遍历:

```

void traverse(treenode * root)
{
    if(root == NULL) return;
    glPushMatrix();
    glMulMatrix(root->m);
    root->f();
    if(root->child != NULL) traverse(root->child);
    glPopMatrix();
    if(root->sibling != NULL) traverse(root->sibling);
}

```



```

}

void display()
{
    glClear(COLOR_BUFFER_BIT|GL_DEPTH_BUFFER_BIT);
    glMatrixMode(GL_MODELVIEW);
    glLoadIdentity();
    traverse(&torso_node);
    glutSwapBuffers();
}

```

我们也可通过使用动态创建的结点来创建动态结构。例如，我们可定义：

```
typedef treeNode * tree_ptr;
```

然后定义：

```
tree_ptr torso_ptr;
torso_ptr = malloc(sizeof(treeNode));
```

树结点的定义方式也几乎完全相同，依然是借助指针，与以前同样的方式递归地进行遍历：

```
traverse(torso_ptr);
```

使用这种方法的一个优点就是我们可在应用程序中创建或删除结点，这样就为操作模型带来了极大的灵活性。

【128~129】

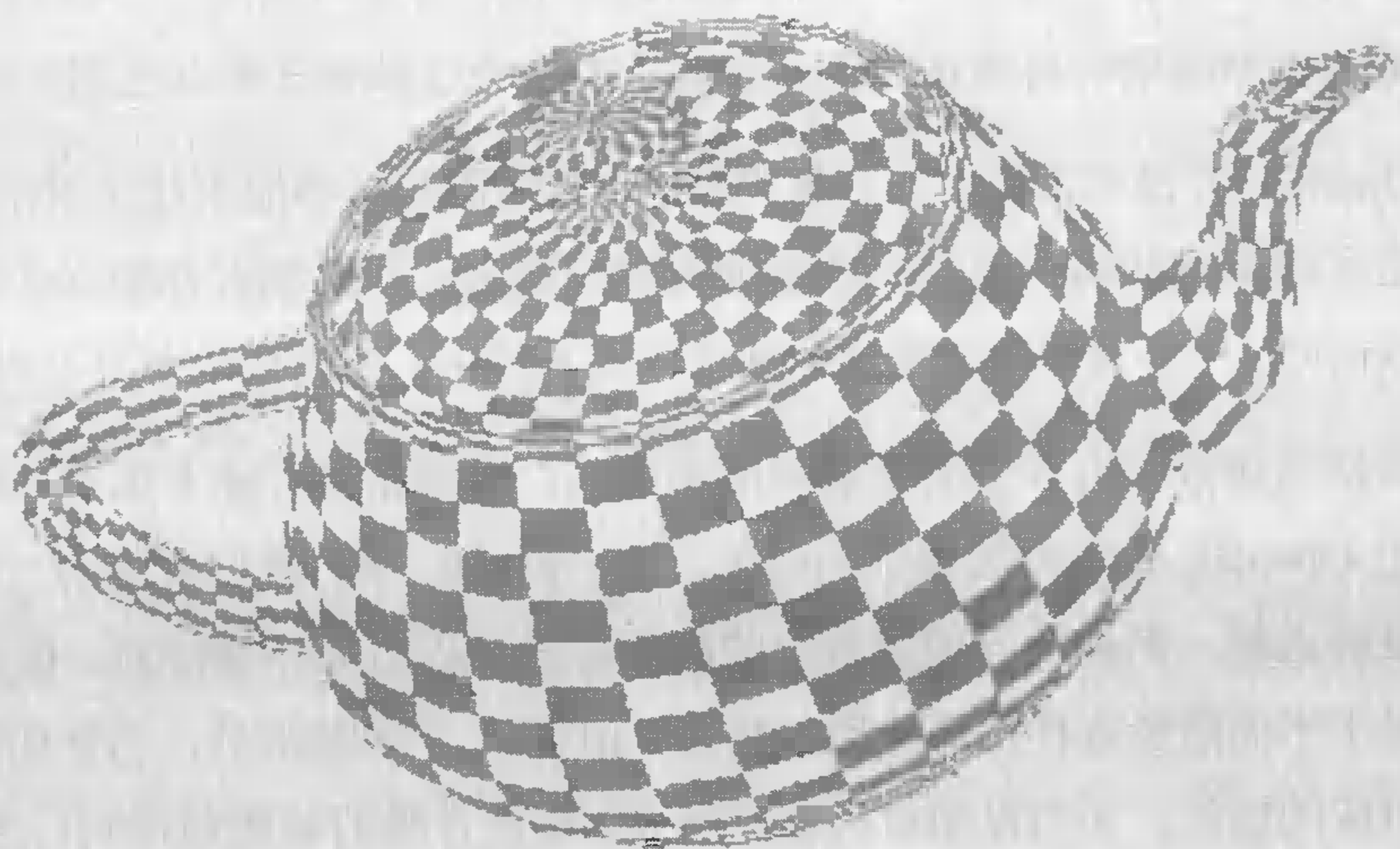
5.11 编程练习

1. 重新编写将矩形旋转的程序，使其可通过鼠标来控制矩形旋转的启动和停止。
2. 编写一个交互式程序，使其可以通过鼠标实现对于一个立方体的旋转、平移及比例缩放。
3. 编写一个带有菜单的程序，菜单项为一些 GLUT 和 GLU 对象。用户可通过选择菜单项中的任意一种对象，并以所期望的尺寸和方向将其显示在屏幕上。
4. 用一些简单图元来编写一个程序，将您自己设计的房屋显示出来。将该程序设计成交互式的，以使用户可在房屋中漫游。

第 6 章

光照与材质

在现实世界中，我们所看到的颜色是由物体的材质和光照交互作用决定的。OpenGL 通过一个综合了漫反射、镜面反射、环境光以及发射光等多种因素的光照模型来模拟该过程。这个模型允许我们定义各种各样的光源及材质属性。我们还将分析 RGBA 颜色模式中的第 4 个分量——alpha 值，并介绍如何用它来显示半透明的表面。



6.1 光照与材质之间的交互

到目前为止，我们在场景中看到的各种颜色值都是通过简单地对 OpenGL 的颜色状态进行设置而得到的，这种方式允许颜色与一个或多个顶点建立关联。当我们定义一个可视对象时，如果没有对当前颜色进行修改，则在显示器中我们只能看到这种单一的颜色。这种情形从物理角度看是缺乏真实感的。我们在现实世界中看到的各种颜色都是来自一个或多个光源的光线与物体的材质交互作用的结果。情况更糟的是，如果我们对一个三维对象只赋予一种固定颜色值，我们将无法从图像中体会出任何立体感。在二维照片或绘画中，物体之所以会呈现出三维效果，是因为我们看到了所绘制对象中的一些细微的变化，或者说明暗的变化。这些明暗效果是由光照与物体的材质交互作用决定的。如果我们想显示一个红色的球体，却在 OpenGL 中只为其赋予单一的颜色值，则这个球体看起来只会是一个均匀着色的二维的圆。我们需要为 OpenGL 添加的功能正是对物体明暗效果处理的能力，只有这样才会增加视觉上的真实感。

【131】

在物理世界中，我们所看到的明暗效果是光照与材质之间通过大量复杂的交互而实现的。由光源发出的光投射到物体上，其中一部分被物体表面所吸收，另一些则被反射。物体呈现出的颜色取决于到达物体的光线的颜色以及表面所反射的光的频率。由表面反射的光线可能范围很大，也可能很窄，这主要取决于物体表面的光滑度。反射光可能还会到达其他表面，同样又会被部分反射、部分吸收。当场景中存在高反射率的物体时，绝大多数光线在最终被吸收之前都将经历许多次这样的反射。

所以，为了使绘制算法生成符合物理规律的明暗效果，我们必须进行全局的光照计算。该计算中将利用场景中所有物体和光源的信息来为场景中的每一点计算明暗度。这种方法除了自身巨大的计算量外，还需要借助完整的对象数据库。一些绘制策略，如光线跟踪，能够对类型有限的材质进行全局计算，但这些绘制算法难以达到实时绘制的要求。

OpenGL 并没有采用全局计算模型。它采用的是一种流水线体系结构，每个图元都被独立地送入该流水线内，而且那些可能为可见的图元在绘制时都独立于所有其他的图元。由于在 OpenGL 中，顶点是彼此独立地送入流水线中，所以明暗计算是以顶点为单位逐个进行的。

虽然当场景中有大量高反射率的物体时，绘制结果可能不那么理想。但在大多数情况下，用 OpenGL 都能得到较好的结果。在 OpenGL 中，我们可借助一些技巧来给出全局计算的良好近似。例如，一般场景中的阴影都需要由全局计算得到。但是，从上一章我们看到，对于平面阴影这种特殊情形，我们可由第 2 次绘制得到。尽管如此，对于由非高反射物体组成的场景，采用以顶点为单位进行明暗度计算的确能够得到相当不错的结果。

6.2 Phong 模型

人们已经建立了许多用来模拟光线与物体材质之间的交互的物理模型。令人遗憾的是，这些模型对于实时图形学来说都显得过于复杂。Phong 提出了一种近似光照模型，该模型易于计算而且在计算机图形学中十分有用。OpenGL 在明暗计算中采用的是一种改进的 Phong 模型，该模型已在商业图形卡中得到了硬件支持。

假定有一条从三维空间中的一个点光源发出的光线投射到某一表面。考虑表面上的某一点 P 。Phong 模型是以图 6.1 所示的 4 个向量为基础的。从 P 到光源的方向由向量 L 给出。视点位于由 P 点引出的方向 V 上。注意，视点或光源可在距离表面无穷远处。反射光离开表面的角度既与入射光的角度有关，也与入射点处表面的方向有关。表面中某一点处的局部朝向由法向量 N 给出，该向量在那一点与表面垂直。如果表面是高反射的，就会呈现镜面反射的效果，大部分光都将沿着全反射的方向 R 离开表面。在 OpenGL 中，方向 V 可由观察条件得出。 L 可通过光源函数来指定。局部法向量 N 应由用户程序给出。向量 R 可由 N 和 L 计算得到。

【132~133】

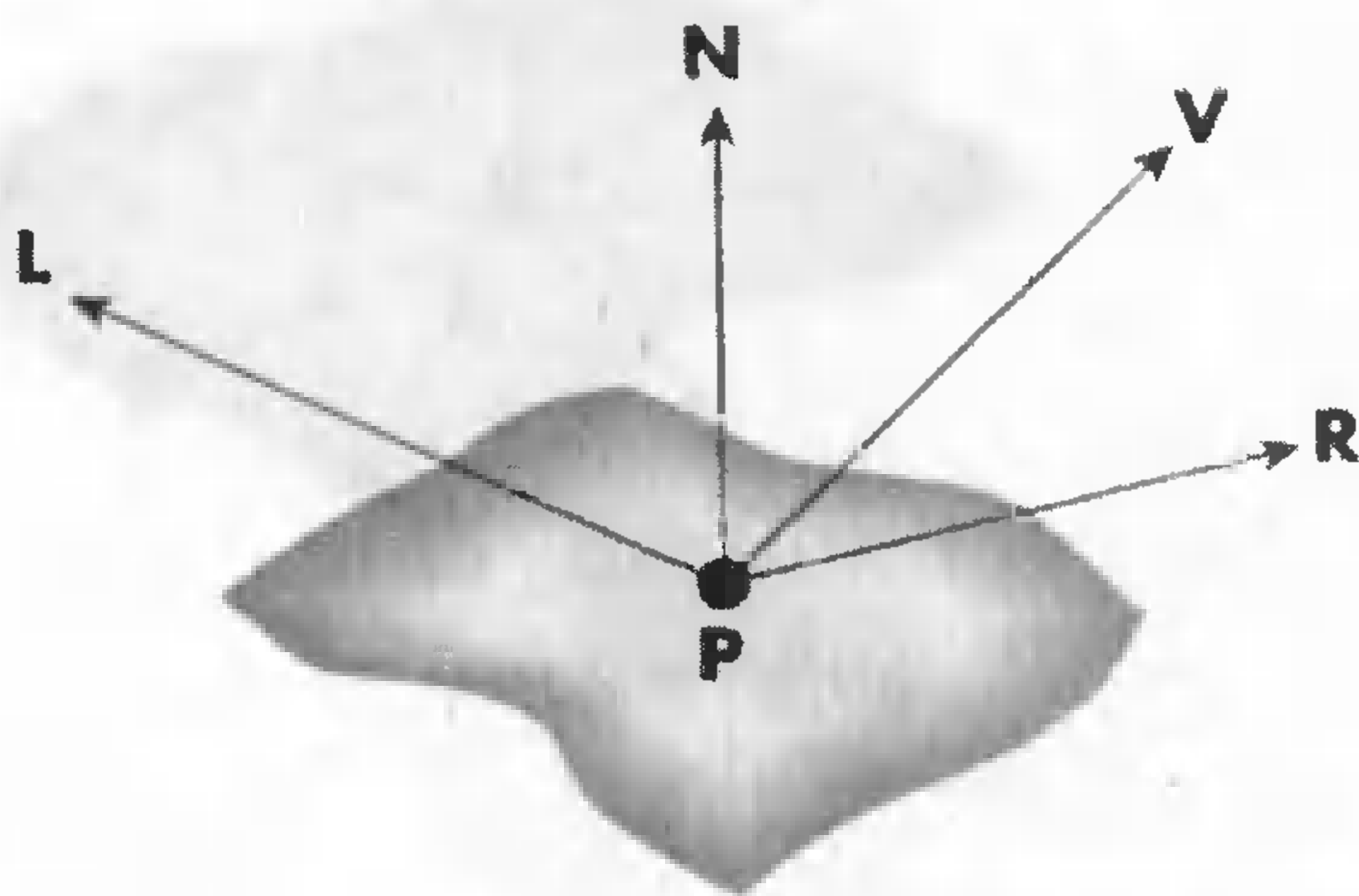


图 6.1 Phong 模型中使用的 4 个向量

Phong 模型认为一个点的明暗度由 4 种类型的贡献决定：漫反射(diffuse reflection)、镜面反射(specular reflection)、环境反射(ambient reflection)以及来自材质的发射光(emissive light)。Phong 模型使用的颜色模式为 RGB，且该模型在使用时是独立作用于每个顶点的。由于该光照模型具有叠加性，来自多个光源的光线可以叠加在一起。所以，对于每个顶点，Phong 模型将对每个点光源计算一次，则该顶点的明暗度将由来自所有光源的贡献总和决定。光栅化器利用这些顶点的明暗度来获取每个片段的颜色。

6.2.1 漫反射

到达表面的入射光部分被材质所吸收，部分被反射。在漫反射条件下，反射光均匀地散射到各个方向。由此，对于不同角度的观察者来说，漫反射表面看起来都是一样的。所以，在计算漫反射对某一顶点的明暗度贡献时，不需要考虑 V 。但是，这种贡献确实依赖于光源与表面法线之间的夹角(见图 6.2)。当光源与法线方向一致时，漫反射的贡献将达到最大。漫反射表面通常看起来颜色像塑料一样不够鲜明。我们用入射光的每个 RGB 颜色分量的反射比率来描述表面。一个表面之所以会呈现出某种颜色(当用白光照射时)，正是因为它更多地吸收了某些频率的光。在 Phong 模型中，每个 RGB 分量都可具有不同的属性。我们也可将光线由于距离(即光源与表面之间的距离)而产生的衰减因素考虑进来，而对明暗度计算做出一些改进。

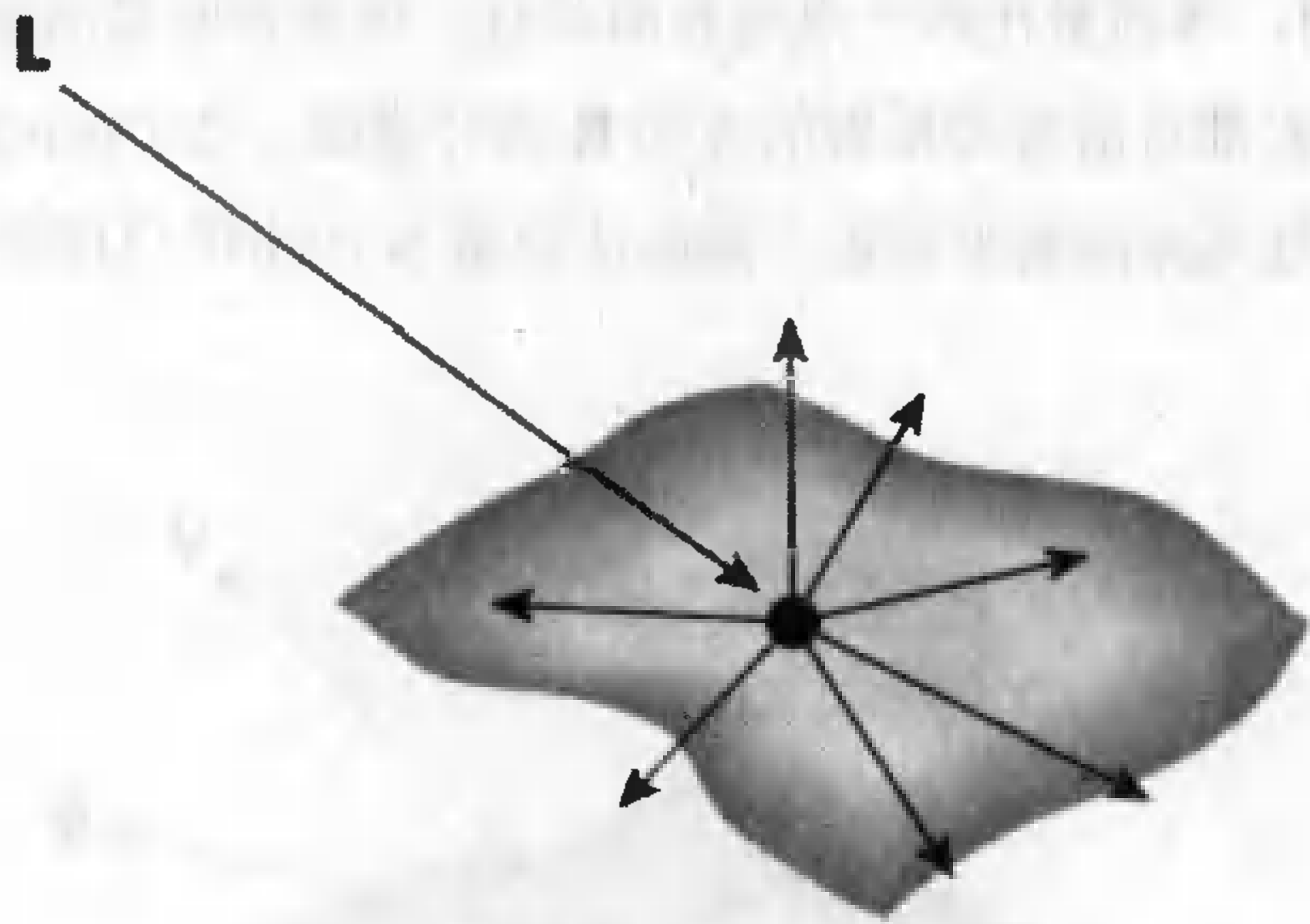


图 6.2 漫反射

【134】

6.2.2 镜面反射

镜面反射表面对于入射光同样也是部分吸收、部分反射。但是，镜面反射表面更加光滑，反射光都集中在全反射方向 R 附近。由于反射光非常集中，所以观察者所看到的还要受到 R 与 V 之间夹角的影响(参见图 6.3)。

表面越光滑，反射光就越集中。在 Phong 模型中用辉度系数(shininess coefficient)来描述反射光的集中程度。我们还必须指定入射光被反射的比率。镜面反射表面包括抛光材质(例如金属)，并负责形成图像中常见的高光区域。但是，处于效率的考虑，OpenGL 对 Phong 模型还做了一次修正，即在计算高光区域时用 $L + V$ 来替代 N 。

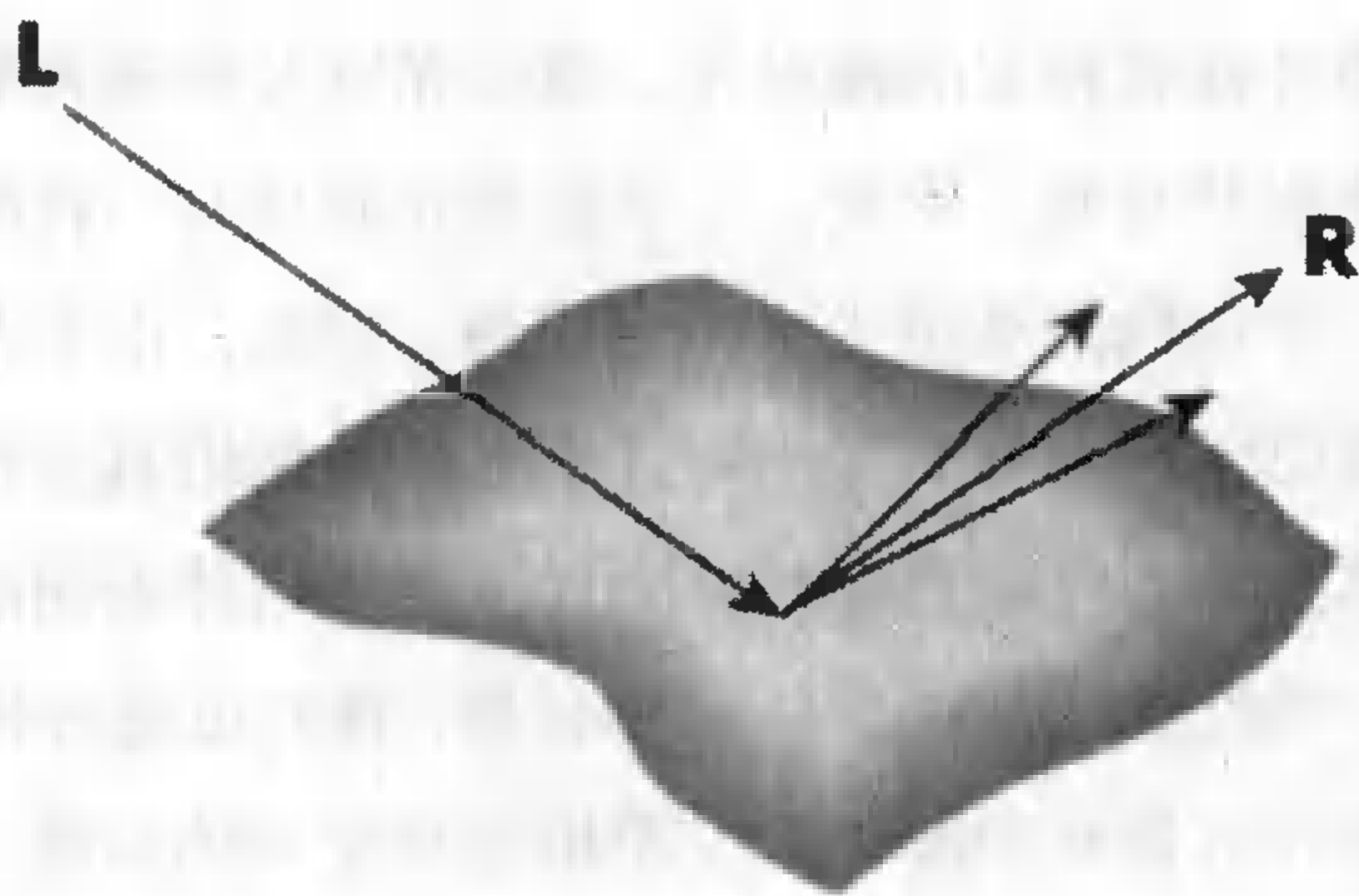


图 6.3 镜面反射

6.2.3 环境反射

Phong 模型假定光源为理想点光源。但是,现实世界中的光源往往都有一定的面积。此外,全局现象(如多重反射)使得场景中每个点都会接收来自许多方向的入射光。处理这种光源的简单方法就是为场景中的每个点增添恒定数量的光照。我们称这种光为环境光,这非常类似于当一个房间中有多个光源和均匀散射灯时我们所看到的效果。环境光到达物体表面后同样也是部分被吸收部分被反射,但观察者所接收的光线与上述 4 个向量没有任何关系,而只依赖于入射光的亮度及其反射的比率。

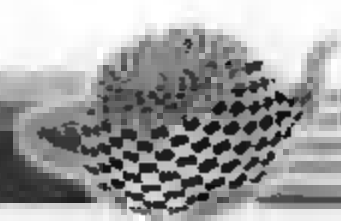
6.2.4 发射光

在现实世界中,光源都是有一定面积的,所以在场景中是可见的。此外,一个表面除了能够将入射光反射外,可能还会发出一定的光。我们可为 Phong 模型添加一个发射项,该发射项不会对入射光造成任何影响,但非常有助于为可视光源或发光体建模。来自表面的发射项不被用来做明暗计算。所以,无论光源或材质如何,一个纯粹的发射表面都将呈现一致的视觉效果,而且发射光不受视点位置的影响。

6.3 OpenGL 中的光照

一个真实的光源都具有一定的面积,而且由其发出的每种频率的光都具有一定的光强。此外,在不同方向上光源发出的光可能存在差别。要在一个以追求实时光照计算为目标的系统中为这样的光源建模实在是一件过于困难的事。所以,OpenGL 只提供了有限种类的光源,并为用户提供了控制调节这些光源的借口,而将逼近真实效果的任务留给了用户。由于 OpenGL 使用的颜色模式为 RGB,所以光照计算需要在每个颜色分量上分别进行。

在 OpenGL 中,我们可使用点光源、聚光灯以及环境光。光源与物体的间距可为有限也可为



无穷远。并且每个光源都可具有独立的漫射光、镜面光以及环境光等属性。为每种光源指定如此众多的参数乍看起来有些奇怪。毕竟，一个真实光源只有一种颜色，况且不能同时用一个蓝色的漫射光光源和一个白色的环境光光源来描述。但是，由于在 OpenGL 中，我们无法实现全局光照，我们只能用这种人为添加的工具来更好地逼近真实情形。例如，假定在一间红色的房间内有一个白色光源。由该光源发出而直接到达物体表面的光是白色的，但从红色墙面反射的光中将携带一些红色或粉色的环境光分量。我们可通过使用漫射光分量和镜面光分量以及一个红色的环境光分量在 OpenGL 中模拟这种全局的贡献。 【135~136】

材质的建模需要以补偿方式来进行。对于每个表面，我们必须为其赋予独立的环境反射、漫反射以及镜面反射分量(或使用默认值)。这些参数表示了每种类型的入射光被反射的比率。我们也可为表面添加发射光分量。每个参数都有独立的 RGB 颜色值。

为了实现明暗效果，必须启用光照计算，而且每种光源也必须被启用。对于单个光源，我们可以这样做：

```
glEnable(GL_LIGHTING);  
glEnable(GL_LIGHT0);
```

启用光照意味着请求 OpenGL 进行明暗计算。此时所有的颜色都将由光源和材质属性来决定，而非 glColor*()。每个光源也必须分别打开或关闭。

提示 一旦光照被启用，由 glColor*()指定的颜色值将不再使用。

光照和材质属性都是 OpenGL 状态的一部分。法向量也同样是。所以，用户通常必须通过函数 glNormal*()来提供法向量。一些函数，例如 glutSolidTeapot()将自动计算出所有的法向量。通常需要为每个顶点或每个多边形进行计算。法向流经 OpenGL 流水线时，将与模型-视图矩阵的逆转置相乘。

```
void glNormal3<bsidf>(type dx, type dy, type dz)
```

```
void glNormal3<bsidf>v(type * v)
```

将当前法向量定义为 type 类型的向量(dx, dy, dz)或是一个指向 type 类型的数组 v 的指针。

6.4 光源的指定

指定光源的主要函数是 glLight*()。该函数的两种形式分别对应标量类型参数和向量(或数组)类型参数。为了设置所有可能的选项，通常需要对这些函数进行多次调用。 【136】

```
void glLight<if>(GLenum light, GLenum param, type value)
```

```
void glLight<if>v(GLenum light, GLenum param, type *value)
```

为 OpenGL 光源 light 设置标量类型或向量类型的参数，即将参数 param 设为 value。

对一个点光源来说，我们可将其位置(GL_POSITION)设为 (x, y, z) ，也可以设置其漫射光(GL_DIFFUSE)、镜面光(GL_SPECULAR)以及环境光(GL_AMBIENT)RGBA 分量。这些颜色属性的颜色模式为 RGBA。由于我们目前只考虑不透明表面，所以第 4 个分量设为 1.0。

光源 0 的默认设置与所有其他光源存在一些差异。默认情况下，对于所有其他光源都没有环境光分量。对于光源 0，漫射光和镜面光分量的默认值为白色(1.0, 1.0, 1.0, 1.0)。所有其他光源的对应的默认值为黑色，即(0.0, 0.0, 0.0, 1.0)。利用这些默认值可以在无需指定任何参数的情况下构造一个简单的光源。位置参数的默认值为(0.0, 0.0, 1.0, 0.0)。该值是在摄像机坐标系中度量的，所以它位于摄像机的后方。注意，该位置是以齐次坐标的形式给出的， w 分量为 0 表明该光源位于无穷远处。因为 $w=0$ 意味着它表示一个方向而非一个点。我们将这种类型的光源为方向光(directional light)，它对应于从一个指定方向发出的平行光。我们可通过指定方向和如图 6.4 所示的角度来定义一个聚光灯。

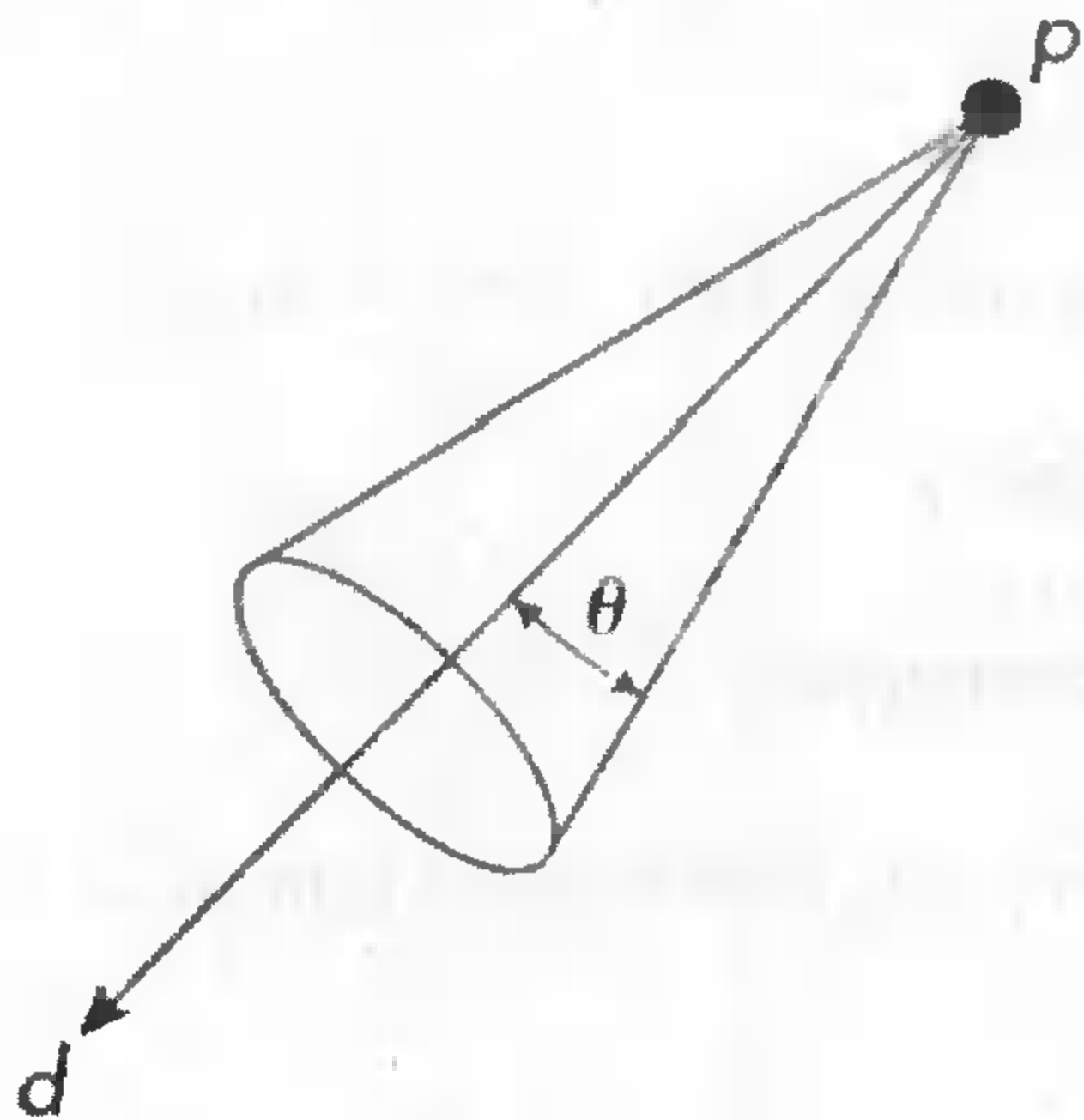
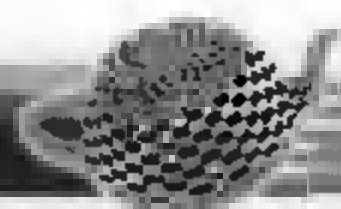


图 6.4 聚光灯的指定

用于指定方向的参数名为 GL_SPOT_DIRECTION，其默认值为摄像机坐标系中的 z 轴负方向。注意，该方向是由三元组 (dx, dy, dz) 给出的。描述聚光灯还需要一个角度参数(GL_SPOT_CUTOFF)，该角度大小为聚光灯投射的光锥的圆锥角的一半。该角度的默认值为 180 度，此时聚光灯就变成了一个点光源。可通过为 GL_SPOT_EXPONENT 赋予一个非零值而使得聚光灯内的光线数量随着离开中心距离的增长而呈指数衰减。【137】

由物理点光源发出的光通常是按照与光源到表面距离的平方的倒数成正比的规律衰减的。该衰减应被应用到镜面光和漫射光分量上。但是，由于理想点光源只是对真实的分布式光源的一个近似，用纯粹的二次衰减将会导致图像的对比度过强。我们可通过采用一种更一般的衰减形式 $1/(a + bd + cd^2)$ 来获得更加柔和、逼真的效果。其中 d 是表面点到光源的距离， a, b, c 分别为常量(GL_CONSTANT_ATTENUATION)、一次项(GL_LINEAR_ATTENUATION)和二次项(GL_QUADRATIC_ATTENUATION)系数。默认情况下是不考虑衰减的，即 $a=1$,



$b = c = 0$ 。

光源与变换

近距光源的位置以及远距光源的方向都是以齐次坐标的形式给出的，这些坐标都将像其他几何实体一样被送入 OpenGL 流水线。所以，这些参数一经定义，也会受到模型-视图矩阵的影响。由此，依据光源的位置或方向的定义在代码中位置，我们可将光源指定在一个固定位置，而摄像机或物体移动；或者光源移动，而物体和摄像机保持静止；或者摄像机、物体、光源三者彼此独立地运动。

现在来分析一个程序中的代码片断，该程序具有初始化函数 `init()`、空闲回调函数 `idle()` 和一个显示回调函数 `display()`。如果我们希望光源静止，可在 `init()` 函数中指定它的位置，并不再对其修改。

```
void init()
{
    GLfloat light_pos[] = {1.0, 2.0, 3.0, 1.0};
    /* rest of init() */
    glEnable(GL_LIGHTING);
    glEnable(GL_LIGHT0);
    glMatrixMode(GL_MODELVIEW);
    glLoadIdentity();
    glLightfv(GL_LIGHT0, GL_POSITION, light_pos);
}
```

摄像机和物体的位置以后都可通过模型-视图矩阵进行修改，但是光源的位置只在 `init()` 中有定义，所以它永远不会发生改变。

现在假定我们有一个空闲回调函数，它所完成的工作只是实现角度的自增，因为我们希望光源绕着物体旋转。

```
GLfloat dx = 1.0; dy = 1.0; dz = 1.0;
GLfloat light_pos[] = {1.0, 2.0, 3.0, 1.0};
GLfloat theta = 0.0;
```

```
void idle()
{
    angle += 2.0;
    if(angle > 360.0) angle -= 360.0;
}
```

```
void display()
{
    ...
```



```

glClear(COLOR_BUFFER_BIT|GL_DEPTH_BUFFER_BIT);
glMatrixMode(GL_MODELVIEW);
glLoadIdentity();
glLookAt(1.0, 1.0, 1.0, 0.0, 0.0, 0.0, 0.0, 1.0, 0.0);
glPushMatrix();
    glRotatef(angle, dx, dy, dz);
    glLightfv(GL_LIGHT0, GL_POSITION, light_pos);
glPopMatrix();

/*draw objects here*/

glutSwapBuffers();
}

```

这段代码中，我们在增加旋转之前先将模型-视图矩阵压入矩阵堆栈中。在完成旋转之后，我们为光源设定了新的位置，这个位置会被旋转和原来的模型-视图矩阵修改。所以，在绘制物体之前，我们将模型-视图矩阵中弹出，以使其不受旋转的影响。

如果将代码中的入栈操作和出栈操作移去，则光源和物体都将发生旋转。通过在代码的不同位置修改模型-视图矩阵，并插入入栈和出栈操作，我们就能实现物体与光源以不同方式的移动。

假定在进行模型-视图变换之前，我们将光源放置在原点处。此时模型-视图矩阵为一单位矩阵。然后我们将光源的位置固定在摄像机坐标系的原点。接下来如果以如下方式调用函数 `gluLookAt()`

```
gluLookAt(eyex, eyey, eyez, 0.0, 0.0, 0.0, upx, upy, upz);
```

达到的效果与保持视点(摄像机坐标系原点)不变，而将物体移至 $(-eyex, -eyey, -eyez)$ 完全相同。但是由于我们并未改变光源的位置(它已经位于摄像机坐标系的原点)，我们实际上是将光源与视点绑定在了一起。当然，每次将光源的位置重新设定在视点所在处也可取得相同的效果。 **【139~140】**

在下面的代码中，我们定义了一个具有亮红色的漫射光分量，亮白色的镜面光分量以及少量的灰色环境光分量的光源。该光源位于无穷远处，并且从摄像机后方的左上角发出光线：

```

GLfloat position0[] = {1.0, 1.0, 1.0, 0.0};
GLfloat diffuse0[] = {1.0, 0.0, 0.0, 1.0};
GLfloat specular0[] = {1.0, 1.0, 1.0, 1.0};
GLfloat ambient0[] = {0.1, 0.1, 0.1, 1.0};

glEnable(GL_LIGHTING);
glEnable(GL_LIGHT0);

```



```
glMatrixMode(GL_MODELVIEW);
glLoadIdentity();
glLightfv(GL_LIGHT0, GL_POSITION, position0);
glLightfv(GL_LIGHT0, GL_DIFFUSE, diffuse0);
glLightfv(GL_LIGHT0, GL_SPECULAR, specular0);
glLightfv(GL_LIGHT0, GL_AMBIENT, ambient0);
```

我们还可可在上述代码中添加少量的 `glLight*()` 函数调用, 而将该光源改变为聚光灯, 并为其增加衰减因子。

6.5 材质的指定

材质属性应与光照属性相匹配。每种材质对每种类型的光都具有不同的反射性质。由于在 OpenGL 的状态中, 每个光源都具有若干光照属性, 而材质只有一组属性。用于设置这组属性的函数为 `glMaterial*()`。

```
void glMaterial<if>(GLenum face, GLenum name, type value)
void glMaterial<if>v(GLenum face, GLenum name, type *value)
```

为材质的某一面 `face` (`GL_FRONT`, `GL_BACK`, `GL_FRONT_AND_BACK`) 设置标量或向量参数。参数 `name` 的类型为 `type`。

我们已经看到, 多边形同时具有正面和背面。这些面的属性可以相同, 也可不同。对于每个面, 我们可为其设置漫反射(`GL_DIFFUSE`)、镜面反射(`GL_SPECULAR`)以及环境反射(`GL_AMBIENT`)属性。通常, 环境反射属性和漫反射属性是相同的, 所以可使用 `GL_AMBIENT_AND_DIFFUSE` 将二者一起设置。这些参数都是 RGBA 颜色值。其中第 4 个分量表示透明度, 该分量可用来模拟半透明材质。对于不透明表面, 该分量为 1.0。默认的表面参数对应于一个灰色的表面, 该表面具有少量的环境反射率(0.2, 0.2, 0.2, 1.0)、较高的漫反射率(0.8, 0.8, 0.8, 1.0), 而且没有镜面反射率(0.0, 0.0, 0.0, 1.0)。每个表面都可以向外辐射(`GL_EMISSION`)。这一项不受光照计算的影响, 所以无论光源如何, 表面看起来都是一样的。最后, 还有一个辉度系数(`GL_SHININESS`), 该参数的值越大, 材质的光泽度就越高, 这是由于此时镜面高光集中在全反射方向附近的一个小区域内。【140~141】

我们可为每个属性单独进行一次函数调用。如果我们经常需要修改材质属性, 例如当场景中有许多物体或某个物体的每个面都具有不同属性时, 我们可将设置这些参数的函数调用都整合在一个函数中。另外一个策略就是设计一种包含材质属性的结构:

```
typedef struct materialStruct{
    GLfloat ambient [4];
    GLfloat diffuse[4];
    GLfloat specular[4];
```



```
    GLfloat shininess;
}materialStruct;
```

现在来设置我们的材质:

```
materialStruct brassMaterials = {
    {0.33, 0.22, 0.03, 1.0},
    {0.78, 0.57, 0.11, 1.0},
    {0.99, 0.91, 0.81, 1.0},
    27.8
}
materialStruct redPlasticMaterials = {
    {0.3, 0.0, 0.0, 1.0},
    {0.6, 0.0, 0.0, 1.0},
    {0.8, 0.6, 0.6, 1.0},
    32.0
}
materialStruct whiteShinyMaterials = {
    {1.0, 1.0, 1.0, 1.0},
    {1.0, 1.0, 1.0, 1.0},
    {1.0, 1.0, 1.0, 1.0},
    100.0
};
```

并编写一个将上述材质赋予物体表面的函数:

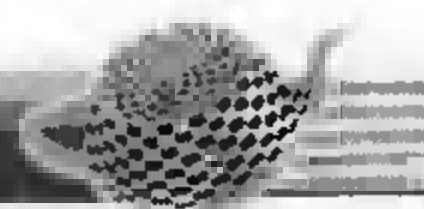
```
void materials(materialStruct *materials)
{
    /*define material properties for front face of all polygons*/
    glMaterialfv(GL_FRONT_AND_BACK, GL_AMBIENT,
        materials -> ambient);
    glMaterialfv(GL_FRONT_AND_BACK, GL_DIFFUSE,
        materials -> diffuse);
    glMaterialfv(GL_FRONT_AND_BACK, GL_SPECULAR,
        materials -> specular);
    glMaterialfv(GL_FRONT_AND_BACK, GL_SHININESS,
        materials -> shininess);
}
```

【141~142】

现在, 当我们希望为某种材质设置属性时, 以设置红色塑料材质为例, 只需进行一次函数调用:

```
materials(&redPlasticMaterials);
```

如果我们想在程序运行时修改光源的属性, 也可采取完全相同的策略。



6.6 旋转立方体的明暗计算

现在我们来对第 5 章的旋转立方体程序进行修改,以演示 OpenGL 的光照和材质。如果我们仍旧想用鼠标来确定旋转轴的方向,则光照和材质属性的修改可用键盘来实现。我们可将黄铜材质、红色塑料材质以及白色光亮材质的定义连同描述材质的数据结构的定义放在程序开头的位置。我们也可用同样的方式对光源的属性进行定义。

```
typedef struct lightingStruct{
    GLfloat ambient[4];
    GLfloat diffuse[4];
    GLfloat specular[4];
}lightingStruct;

lightingStruct whiteLighting = {
    {0.0, 0.0, 0.0, 1.0},
    {1.0, 1.0, 1.0, 1.0},
    {1.0, 1.0, 1.0, 1.0}
};

lightingStruct coloredLighting = {
    {0.2, 0.0, 0.0, 1.0},
    {0.0, 1.0, 0.0, 1.0},
    {0.0, 0.0, 1.0, 1.0}
};
```

```
GLfloat light0_pos[4] = {0.90, 0.90, 2.25, 0.00};
```

【142~143】

接下来我们定义两个指针,以用来在不同的材质和光源之间进行切换:

```
materialStruct *currentMaterials;
lightingStruct *currentLighting;
```

对于明暗度计算,我们需要为每个面指定一个法线。我们将法线信息存放在一个数组中:

```
GLfloat normals[][3] = {{0.0, 0.0, -1.0}, {0.0, 1.0, 0.0},
                        {-1.0, 0.0, 0.0}, {1.0, 0.0, 0.0},
                        {0.0, 0.0, 1.0}, {0.0, -1.0, 0.0}};
```

我们可用如下重写编写的立方体函数将立方体绘制出来:

```
void colorcube()
{
    glNormal3fv(normals[0]);
    polygon(0, 3, 2, 1);
```



```

glNormal3fv(normals[1]);
polygon(2, 3, 7, 6);
glNormal3fv(normals[2]);
polygon(0, 4, 7, 3);
glNormal3fv(normals[3]);
polygon(1, 2, 6, 5);
glNormal3fv(normals[4]);
polygon(4, 5, 6, 7);
glNormal3fv(normals[5]);
polygon(0, 1, 5, 4);
}

```

请注意，在 5.7 节的例子中，所有的 `glColor3fv()` 都已被 `glNormal3fv()` 所取代。由于法线也被视作一种几何图元，所以也将受到模型-视图矩阵的影响，而且将随立方体一起旋转。

我们还必须在 `init()` 函数中定义一些初始属性，并将光照启用。

【143】

```

void init()
{
    glEnable(GL_LIGHTING);
    glEnable(GL_LIGHT0);

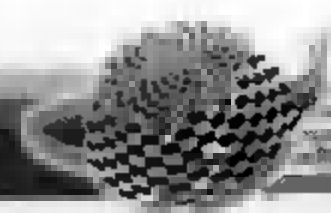
    currentMaterials = &redPlasticMaterials;

    glMaterialfv(GL_FRONT, GL_AMBIENT,
        currentMaterial->ambient);
    glMaterialfv(GL_FRONT, GL_DIFFUSE,
        currentMaterial->diffuse);
    glMaterialfv(GL_FRONT, GL_SPECULAR,
        currentMaterial->specular);
    glMaterialf(GL_FRONT, GL_SHININESS,
        currentMaterial->shininess)

    currentLighting = &whiteLighting;

    glLightfv(GL_LIGHT0, GL_AMBIENT,
        currentLighting->ambient);
    glLightfv(GL_LIGHT0, GL_DIFFUSE,
        currentLighting->diffuse);
    glLightfv(GL_LIGHT0, GL_SPECULAR,
        currentLighting->specular);
    glLightfv(GL_LIGHT0, GL_POSITION, light0_pos);
}

```

最后,我们用键盘回调函数来实现对明暗参数的修改以及何时停止旋转:

```
void key(unsigned char k, int x, int y)
{
    switch(k)
    {
        case '1':
            glutIdleFunc(NULL);
            break;
        case '2':
            glutIdleFunc(spinCube);
            break;
        case '3':
            currentMaterials = &redPlasticMaterials;
            break;
        case '4':
            currentMaterials = &colorCubeMaterials;
            break;
        case '5':
            currentMaterials = &brassMaterials;
            break;
        case '6':
            currentLighting = &whiteLighting;
            break;
        case '7':
            currentLighting = &coloredLighting;
            break;
        case 'q':
            exit(0);
            break;
    }

    glMaterialfv(GL_FRONT, GL_AMBIENT,
        currentMaterial->ambient);
    glMaterialfv(GL_FRONT, GL_DIFFUSE,
        currentMaterial->diffuse);
    glMaterialfv(GL_FRONT, GL_SPECULAR,
        currentMaterial->specular);
    glMaterialfv(GL_FRONT, GL_SHININESS,
        currentMaterial->shininess);

    glLightfv(GL_LIGHT0, GL_AMBIENT,
        currentLighting->ambient);
    glLightfv(GL_LIGHT0, GL_DIFFUSE,
```



```

        currentLighting->diffuse);
    glLightfv(GL_LIGHT0, GL_SPECULAR,
        currentLighting->specular);

    glutPostRedisplay();
}

```

【144~145】

6.7 对明暗计算的控制

明暗计算需要耗费大量的系统资源。在许多情况下, OpenGL 可将一些计算省去, 而对结果图像只产生少量影响, 有时甚至没有任何影响。函数 `glLightModel*()` 允许我们通知 OpenGL 如何进行光照计算。

【145】

```

void glLightModel<if>(GLenum param, type value)
void glLightModel<if>v(GLenum param, type *value)

```

为 `param`(`GL_LIGHT_MODEL_AMBIENT`, `GL_LIGHT_MODEL_LOCAL_VIEWER` 或 `GL_LIGHT_MODEL_TWO_SIDE`) 设置光照模型属性。

可能您还记得, 由于物体的正面和背面的法线是方向相反的, 而且可以具有不同的材质属性。所以, 为背面进行明暗度计算需要一些额外的工作。在许多情况下, 我们所使用的模型并不需要将背面显示出来。例如, 前面我们看到, 对于凸物体, 我们无法看到任何一个背面。OpenGL 能够利用这种情况, 不对背面进行任何的光照计算。如果确实需要两面的光照计算(例如, 在能够看到物体内部时), 可以通过如下函数调用来请求 OpenGL 进行正确的计算:

```
glLightModeli(GL_LIGHT_MODEL_TWO_SIDE, GL_TRUE);
```

提示 如果光照模型没有被设置成两面的, 则所有对背面明暗的计算都将是错误的。

OpenGL 能够针对某些情形对明暗计算做出进一步的简化。例如, 如果视点距离物体较远, 则当我们移动物体时, 从物体上任意点指向视点的向量(用于计算镜面反射)几乎没有什么变化。所以, 如果我们通知 OpenGL 视点与场景的距离无穷远:

```
glLightModeli(GL_LIGHT_MODEL_LOCAL_VIEWER, GL_TRUE);
```

则这部分的光照计算就可被简化。

当我们讨论环境光时, 它其实属于每个光源的一部分。依据在于物理光源都会产生环境光, 因为它发出的大多数光都被场景中的各个表面所反射。所以, 如果所有的光源都被禁用, 环境光将不复存在。但是, 即便所有的光源都被关闭, 我们仍然希望少量的环境光存在。这时, 我们可通过设置一个全局环境光光源来达到这种效果:


```
glLightModelfv(GL_LIGHT_MODEL_AMBIENT, global_ambient);
```

其中, 数组 `global_ambient` 包含了 RGBA 颜色值。

6.8 平滑着色

光照计算是以顶点为单位逐个进行的。OpenGL 通过使用修改后 Phong 模型借助当前状态来计算顶点的颜色值。如果明暗模型(通过函数 `glShadeModel()` 设置)被设为平面着色模式 (`GL_FLAT`), 则只对多边形的最后一个顶点进行颜色计算, 这样每个多边形将只呈现一种纯色。这种情形对于由平面多边形构成的物体(例如立方体)是适宜的。在这种情况下, 即使该物体各个面的材质属性完全相同, 每个多边形也会呈现出不同的明暗效果, 因为每个多边形相对于光源和视点都具有不同的朝向。【146~147】

但是, 如果遇到用多边形对曲面对象(如 GLU 二次曲面对象)进行近似表示的情形, 为每个多边形赋予一种恒定的明暗度只会强化一种感觉: 我们是在用多边形对曲面对象近似表示。然而, OpenGL 可依据顶点的颜色值对多边形内部的颜色进行插值。所以, 如果我们将 `glShadeModel()` 中的参数设为 `GL_SMOOTH`, 则 OpenGL 将对每个顶点进行光照计算, 并利用顶点的颜色值对多边形内部进行插值。图 6.5 展示了在用多边形近似表示球体时, 平滑着色和平面着色之间的差别。注意, `GL_SMOOTH` 为默认值。

使用较大的多边形时, 平滑着色可能会令人失望。假定我们用一个很大的矩形来表示房屋内的一面墙, 并想让墙面呈现出明暗的变化。所以, 我们选择了平滑着色。但是, 结果却可能是墙面的中心比四个墙角暗很多。这种现象正是由依据四个角对整个墙面进行插值造成的。对这类问题通常的解决方案是将该矩形离散化为若干较小的多边形。

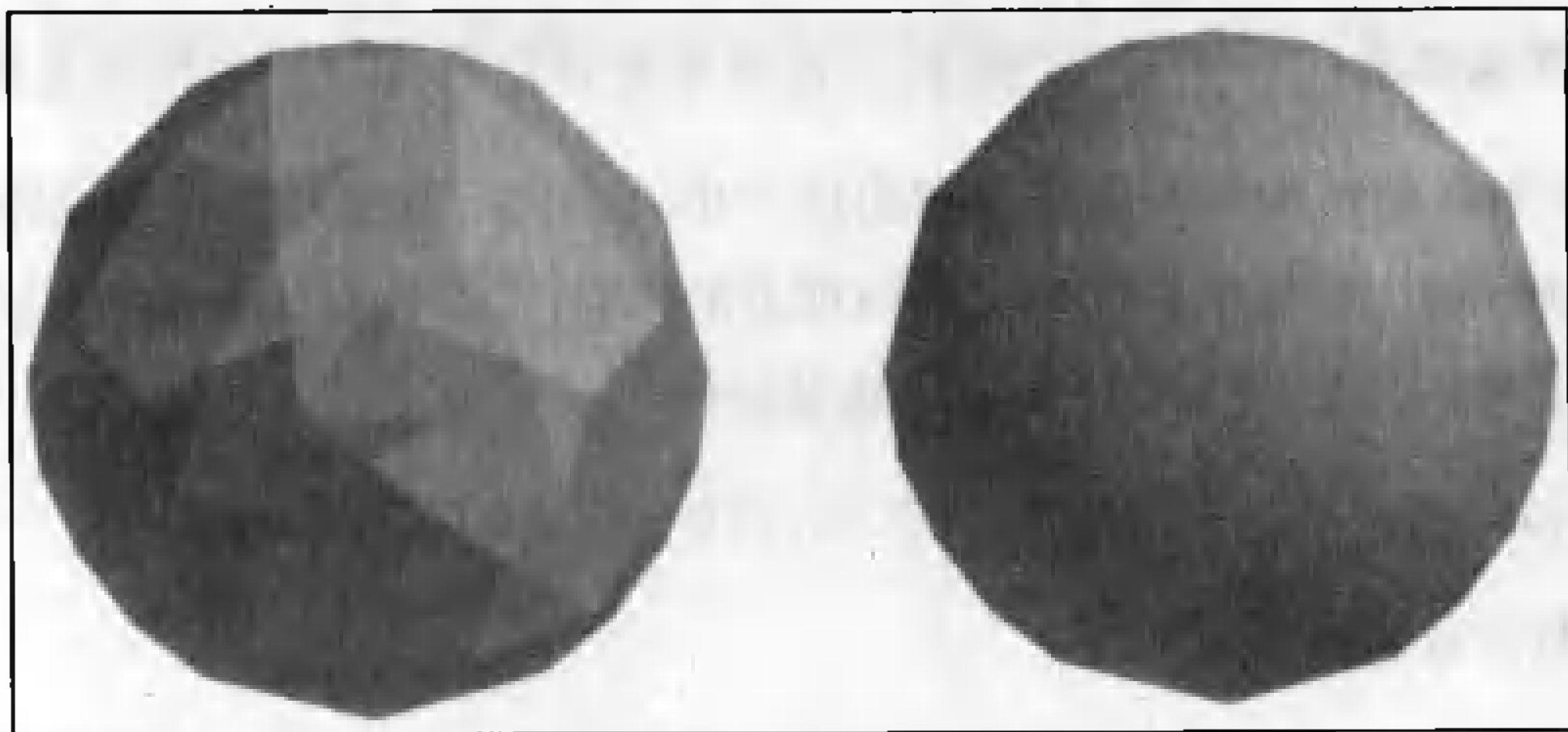


图 6.5 用平面着色和平滑着色的多边形近似表示一个球体

提示 如果您使用较大的多边形, 则明暗计算的效果可能难以令人满意, 这种结果是由依据顶点的明暗对一个较大的区域进行插值造成的。

6.9 法线的处理

明暗计算的质量很大程度上取决于表面的法向量，它通常需要我们自己进行计算。一个平面多边形的整个表面都具有一个恒定的法向量，这可由组成多边形的前三个顶点计算得到。但是，当我们希望平滑着色时，每个多边形的法向量并不是我们所需要的。 【147~148】

平滑着色有时被称为 Gouraud 着色。这其实有点用词不当。Gouraud 原来的想法是每个顶点的法线方向应由所有共享该顶点的多边形的法线决定。但由于 OpenGL 独立地对每个多边形进行处理，所以 OpenGL 缺少依据周围多边形对顶点法向量进行计算的信息。因此，应用程序必须知道哪些多边形共享某一顶点，并正确地计算每个顶点的法线。对于图 6.5 中的平滑图像，为了获得每个顶点的精确法向量，我们可以利用这样一个事实：我们正在近似表示一个球体。而球体上任意一点的法线方向总沿着由球心指向该点的方向。

光照计算要求法向量为单位向量：即法向量的三个分量的平方和为 1。通常，在程序中实现法向量的规范化更为高效。当然，我们也可以通过如下函数调用来启用自动向量规范化：

```
glEnable(GL_NORMALIZE);
```

在进行显示处理时，该函数将对每个法向量都执行一次，所以使用该功能必定会带来性能上的损失。

当我们实施变换时，旋转和平移都不会影响法向量的长度(模)，但是比例变换会影响。所以，法向量有可能不再是单位向量，所以明暗计算的结果将是错误的。

提示 光照计算要求法向量为单位向量。

提示 比例变换将改变法向量的长度(模)。

6.10 透 明 度

现在我们来研究第 4 个颜色分量——RGBA 颜色中的 A。该分量称为 alpha 值。在通常的绘制中，该值一般被忽略。如果首先通过如下函数调用启用融合(blending)，则 alpha 值的意义将体现出来：

```
glEnable(GL_BLEND);
```

虽然我们能以多种方式使用 alpha 值，但它通常的用途仍是决定一种颜色或材质的透明程度。透明度为 1.0 表明材质是不透明的，它会将所有位于其后方的物体遮挡。透明度为 0.0

则表明该材质是完全透明的，故不可见。介于 0~1 之间的值则表明材质为半透明的，位于其后方的物体的一些颜色将与材质的颜色发生融合，alpha 的值决定了材质上某一点对图像中相应点颜色的贡献权重。【148~149】

alpha 值的使用为我们增加了一些额外的功能，但同时也衍生出许多选项，而且有一些地方需要引起注意。一个基本问题是当我们对两个半透明的多边形进行融合并显示时，绘制的顺序非常重要。与之相比，只绘制不透明的多边形时，用于消隐的深度缓存算法保证了无论多边形以何种顺序进行绘制，结果总是惟一的。

假定现有两个多边形需要融合。我们先不考虑光照因素，并假定每个多边形都有一个 RGB 颜色值和 A(alpha)值。同时假定颜色缓存也已被清除为由 `glClearColor()` 所设置的清除色。您可能还记得清除色是一个 RGBA 颜色，所以，如果我们已将融合启用，则在我们绘制任何多边形之前，最初的窗口将是半透明的，且位于其后的其他窗口将是可见的。现在，我们来假定窗口的初始颜色如下设置为：

```
glClearColor(0.0, 0.0, 1.0, 1.0);
```

因此它是蓝色、完全不透明的。现在假定第一个多边形由下列代码定义：

```
glColor3f(1.0, 0.0, 0.0, 0.5);  
glBegin(GL_POLYGON);  
/*vertices go here*/  
glEnd();
```

假定该多边形是可见的，其颜色必须与颜色缓存中的当前值进行融合，以表现出半透明的效果。我们不但需要确定放入颜色缓存(多边形正是在这里被绘制)中的新 RGB 值，而且要确定一个新 A 值。这个新 A 值表示多边形与背景色融合后透明度如何。OpenGL 提供了多个确定如何将颜色进行融合的常量以及 alpha 值。这个模型基于这样一种思想：颜色缓存是我们存放颜色的目标地点，而用于表示多边形颜色的状态值是源。当融合被禁用后，源颜色只是简单地取代颜色缓存中相应的颜色值。但是当融合启用后，新的目标颜色将是旧的目标颜色与图 6.6 所示的源颜色组合的结果。

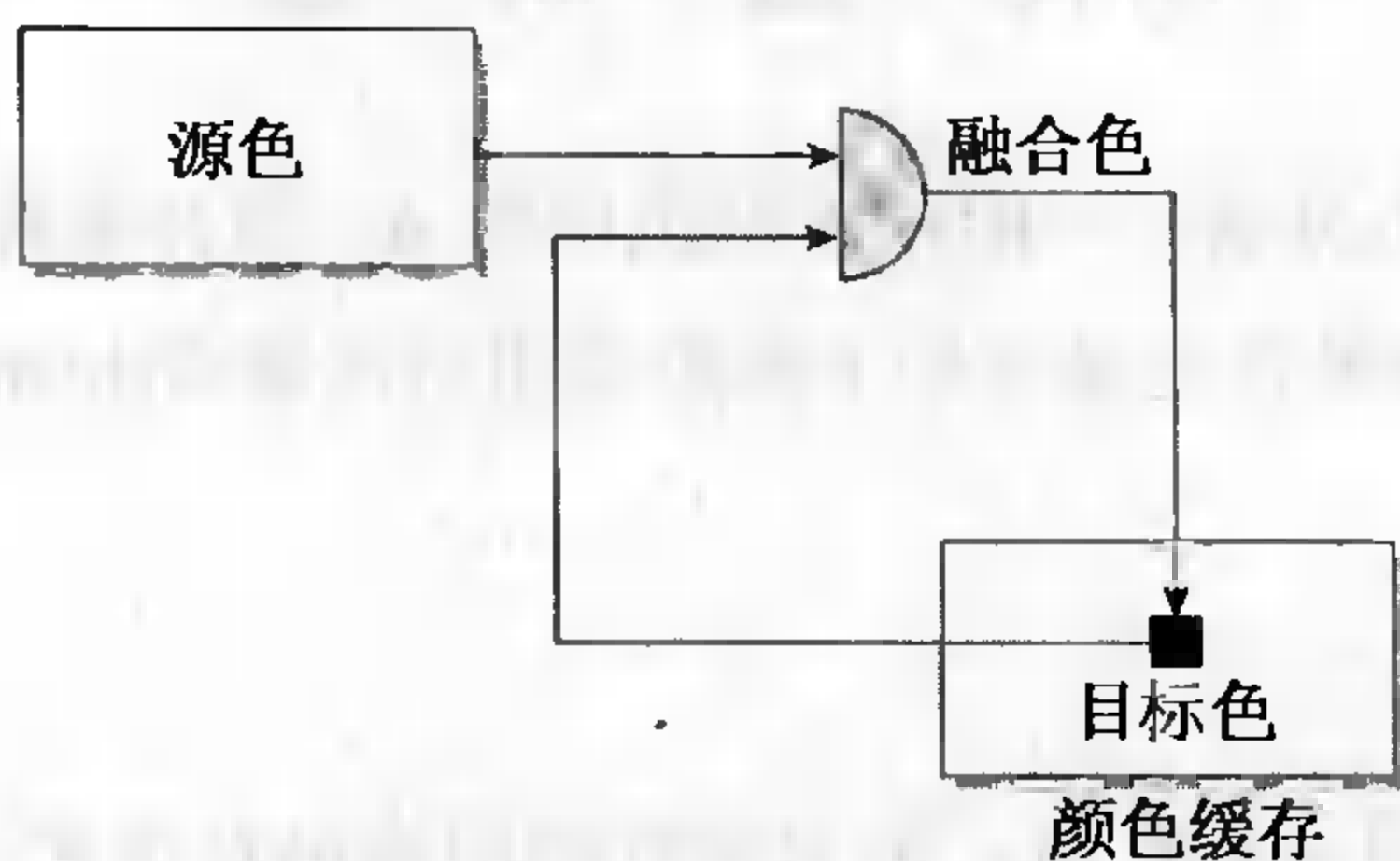


图 6.6 融合运算的写操作模型

假定被绘制的多边形上某一点在颜色缓存中的颜色值为 (R_d, G_d, B_d, A_d) 。令 (R_s, G_s, B_s, A_s) 表示多边形的颜色。通过融合，我们又形成了一种新的目标颜色，融合的方法是将源颜色与一个源融合因子(source blending factor)的乘积与目标颜色和目标融合因子(destination blending factor)的乘积叠加在一起。假定 (S_r, S_g, S_b, S_a) 和 (D_r, D_g, D_b, D_a) 分别表示源融合因子和目标融合因子。则新的目标颜色为 $(S_r R_s + D_r R_d, S_g G_s + D_g G_d, S_b B_s + D_b B_d, S_a A_s + D_a A_d)$ 。在OpenGL中，我们通过函数glBlendFunc()来设置这些因子。 【149~150】

```
void glBlendFunc(GLenum source, GLenum destination)
设定源融合因子和目标融合因子。
```

在15个可选的融合因子中，一些只能应用于源颜色，一些只能应用于目标颜色，另外一些可同时应用于源颜色和目标颜色。我们将考虑一些最重要的情形。

当我们绘制多边形的表面时，对于源融合因子和目标融合因子最常见的选择分别是GL_SRC_ALPHA和ONE_MINUS_SRC_ALPHA。现在来考虑一开始时颜色缓存被清为一种不透明色 $(R_c, G_c, B_c, 1)$ 。当我们将一个具有颜色 (R_s, G_s, B_s, A_s) 的多边形混合进来后，多边形的目标颜色为 $(A_s R_s + (1 - A_s) R_c, A_s G_s + (1 - A_s) G_c, A_s B_s + (1 - A_s) B_c, A_s^2 + (1 - A_s) A_c) = (R_d, G_d, B_d, A_d)$ 。我们可为这个新颜色建立一些重要属性。如果初始颜色的每个分量都在 $(0.0, 1.0)$ 区间内，则新颜色值的范围也在该区间内。因此，颜色值永远不会出现上溢或下溢。如果多边形是透明的 $(A_s = 0)$ ，则新颜色与背景色一致；如果多边形是不透明的，则新颜色与多边形颜色一致。这样的结果正符合真实材质的性质。现在假定我们增加第二个多边形，其颜色值为 $(R_{s'}, G_{s'}, B_{s'}, A_{s'})$ 。这两个多边形的重叠区域的颜色值为：

$$(A_{s'} R_{s'} + [1 - A_{s'}][A_s R_s + (1 - A_s) R_c], A_{s'} G_{s'} + [1 - A_{s'}][A_s G_s + (1 - A_s) G_c], \\ A_{s'} B_{s'} + [1 - A_{s'}][A_s B_s + (1 - A_s) B_c], A_{s'}^2 + [1 - A_{s'}][A_s^2 + (1 - A_s) A_c])$$

虽然这个表达式相当繁琐，但我们可验证RGB和A的新值仍在区间 $(0.0, 1.0)$ 内。但是，多边形的绘制顺序将非常重要。如果我们改变两个多边形的绘制顺序，最终得到的颜色值将略有不同。这种情形不但符合物理规律，而且由于我们通常无法保证多边形被送入OpenGL流水线的顺序，所以当多边形的绘制顺序发生变化时，便会得到不同的图像。 【150~151】

提示 为了获得一致的半透明表面的明暗效果，我们通常必须对多边形进行排序。

只要将源因子(源融合因子的简称，下同)和目标因子(目标融合因子的简称，下同)分别设为GL_SRC_ALPHA和GL_ONE，我们就能保证绘制的结果与多边形的绘制顺序无关。仍然考虑前面提到的例子。第一次将多边形与背景色进行融合时，我们将得到颜色值 $(A_s G_s + R_c, A_s G_s + G_c, A_s B_s + B_c, A_s^2) = (R_d, G_d, B_d, A_d)$ 。这些颜色值可能会出现上溢，但是我们可通过将清除色设为黑色而避免这样的事情发生，此时融合结果将变为 $(A_s R_s, A_s G_s, A_s B_s, A_s^2)$ 。当我们

对第二个多边形进行融合时,我们将得到颜色值($A_s R_s + A_s' R_s', A_s G_s + A_s' G_s', A_s B_s + A_s' B_s', A_s^2 + A_s'^2$)。这里存在的问题是,如果A比较小,结果图像可能看起来会比较暗淡。这是因为颜色分量只是被缩放而未进行融合运算。如果A的值较大,则颜色值可能发生上溢。

在真实感图形学中,最困难的情形莫过于当一些多边形为不透明,而另外一些为半透明。虽然我们可能倾向于接受对半透明多边形融合时顺序相关的事实,我们必须正确地处理不透明的多边形。每个不透明多边形必须将位于其后的物体遮挡。

我们可用深度缓存来跟踪某个多边形是否位于目前为止所有已绘制多边形的前方。但对于半透明的多边形,就如何改变深度缓存中的值,我们必须采取不同的手段。当一个半透明的多边形流经绘制流水线时,如果它位于当前所有多边形的前方时,它将得到绘制。但是,这样的多边形不应该对深度缓存中的内容造成影响,因为它没有遮挡任何位于其后方的物体。OpenGL 提供了一个函数——`glDepthMask()`,可用来将深度缓存设为只读的(`GL_FALSE`)或可写的(`GL_TRUE`)。

```
void glDepthMask(GLboolean flag)
```

将深度缓存设为只读的(`GL_FALSE`)或可写的(`GL_TRUE`)。默认值为 `GL_TRUE`。

假定我们再次来考虑立方体的那个例子,但是这次立方体有多个面都是半透明的。我们仍可像以前那样定义各个顶点,但是对于各面我们这次使用 `RGBA` 颜色值,这样可使前三种颜色为半透明。 【151】

```
GLfloat vertices[][3] = {
    {-1.0, -1.0, 1.0}, {-1.0, 1.0, 1.0},
    { 1.0, 1.0, 1.0}, {1.0, -1.0, 1.0},
    {-1.0, -1.0, -1.0}, {-1.0, 1.0, -1.0},
    { 1.0, 1.0, -1.0}, {1.0, -1.0, -1.0}};
```

```
GLfloat colors[][4] = {
    {1.0, 0.0, 0.0, 0.5}, {0.0, 1.0, 1.0, 0.5},
    {1.0, 1.0, 0.0, 0.5}, {0.0, 1.0, 0.0, 0.5},
    {0.0, 0.0, 1.0, 1.0}, {1.0, 0.0, 1.0, 1.0}};
```

我们可像从前那样使用函数 `polygon()`和 `cube()`来绘制该立方体,请务必确保在绘制之前先检查每个多边形是否为不透明的。

```
void polygon(int a, int b, int c, int d)
{
    glColor4fv(color[a]);
    if(color[a][4] != 1.0)
        glDepthMask(GL_FALSE);
    else
```



```

    glDepthMask(GL_TRUE);

    glBegin(GL_POLYGON);
        glVertex3fv(vertices[a]);
        glVertex3fv(vertices[b]);
        glVertex3fv(vertices[c]);
        glVertex3fv(vertices[d]);
    glEnd();
}
void cube()
{
    polygon(0, 3, 2, 1);
    polygon(2, 3, 7, 6);
    polygon(3, 0, 4, 7);
    polygon(1, 2, 6, 5);
    polygon(4, 5, 6, 7);
    polygon(5, 4, 0, 1);
}

```

请务必在 `main()` 函数或 `init()` 中启用融合，并对融合因子进行设置：

```

glEnable(GL_BLEND);
glBlendFactor(GL_SRC_ALPHA, GL_ONE_MINUS_SRC_ALPHA);

```

【152】

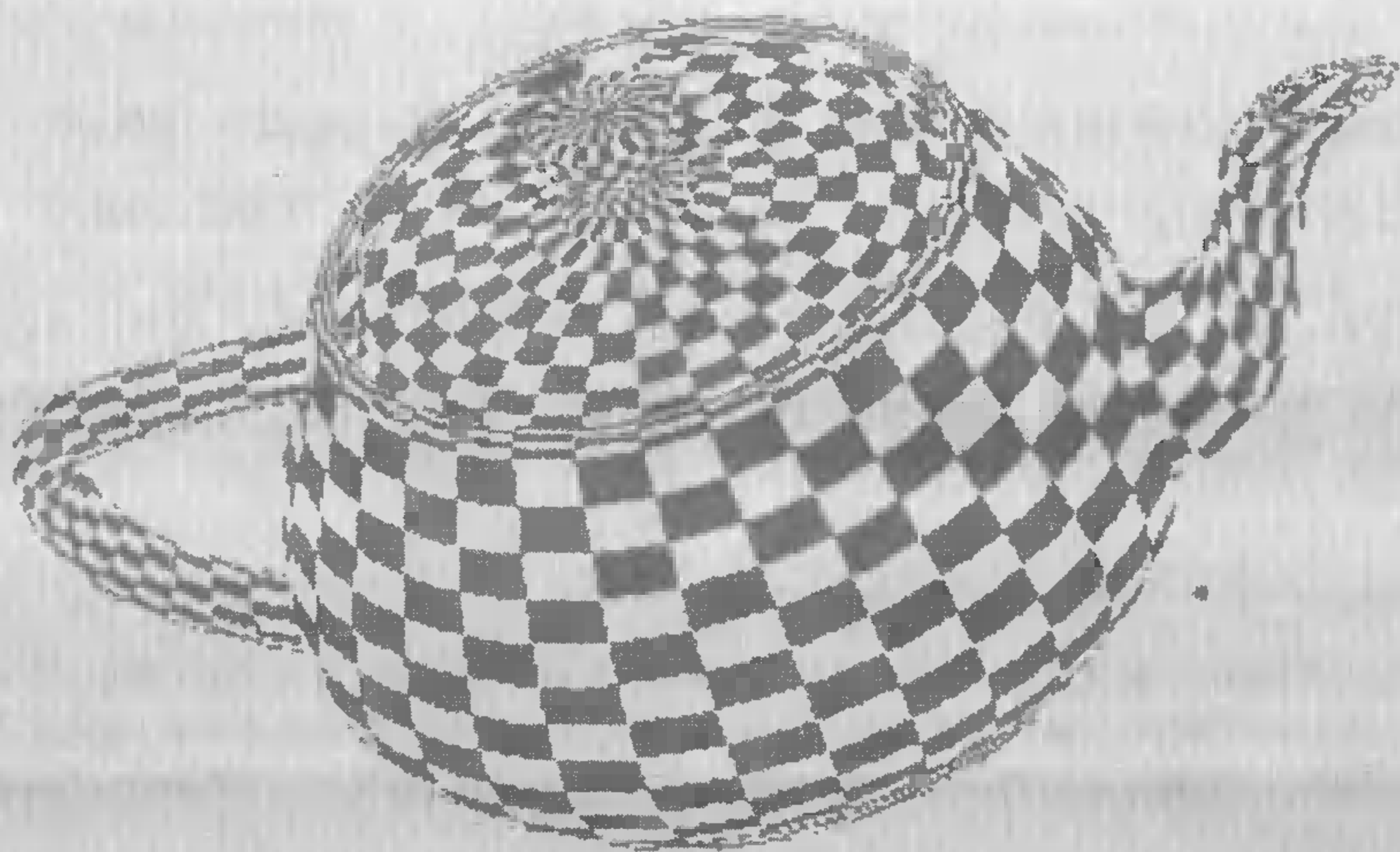
6.11 编程练习

1. 将明暗计算应用于我们在第2章介绍的离散三角形。研究离散程度与明暗的平滑性之间的关系。
2. 编写一个能显示一个或多个曲面对象(如球体、圆盘、圆锥等)的程序，并交互式地为其进行明暗计算。用一个滑动条来调整光照和材质参数，或通过菜单选择某些固定配置方案。

第7章

离散图元

到目前为止，我们对几何实体已经有了一定的了解，这些实体流经 OpenGL 的几何流水线，并经过了模型-视图变换和投影变换。OpenGL 也允许直接对位或一组位(或像素)进行操作，这些元素流经一个并行流水线(parallel pipeline)，并最终到达帧缓存中。我们既可从帧缓存中进行位的读操作，也可进行写操作。当然，也可以利用 OpenGL 在像素流水线中提供的各种特性。



7.1 像素和位图

在图形流水线的末端是帧缓存，图像正是在这里形成。帧缓存本质上是一个缓存集合。对于屏幕坐标系中的每个 x 和 y 坐标，都存在一组相应的位，我们可将这组位视作像元或像素。

①如果 OpenGL 的具体实现还支持其他缓存，例如累积缓存、模板缓存及附加颜色缓存，则每个像素的位数将会更多。我们可将缓存的格局想象成图 7.1 所示的形式。缓存由多个位面组成，每个位面都具有与屏幕完全相同的分辨率。各组位面分别对应于不同的 OpenGL 缓存。屏幕坐标系中任意一点上的所有位构成了一个像素。

【155】

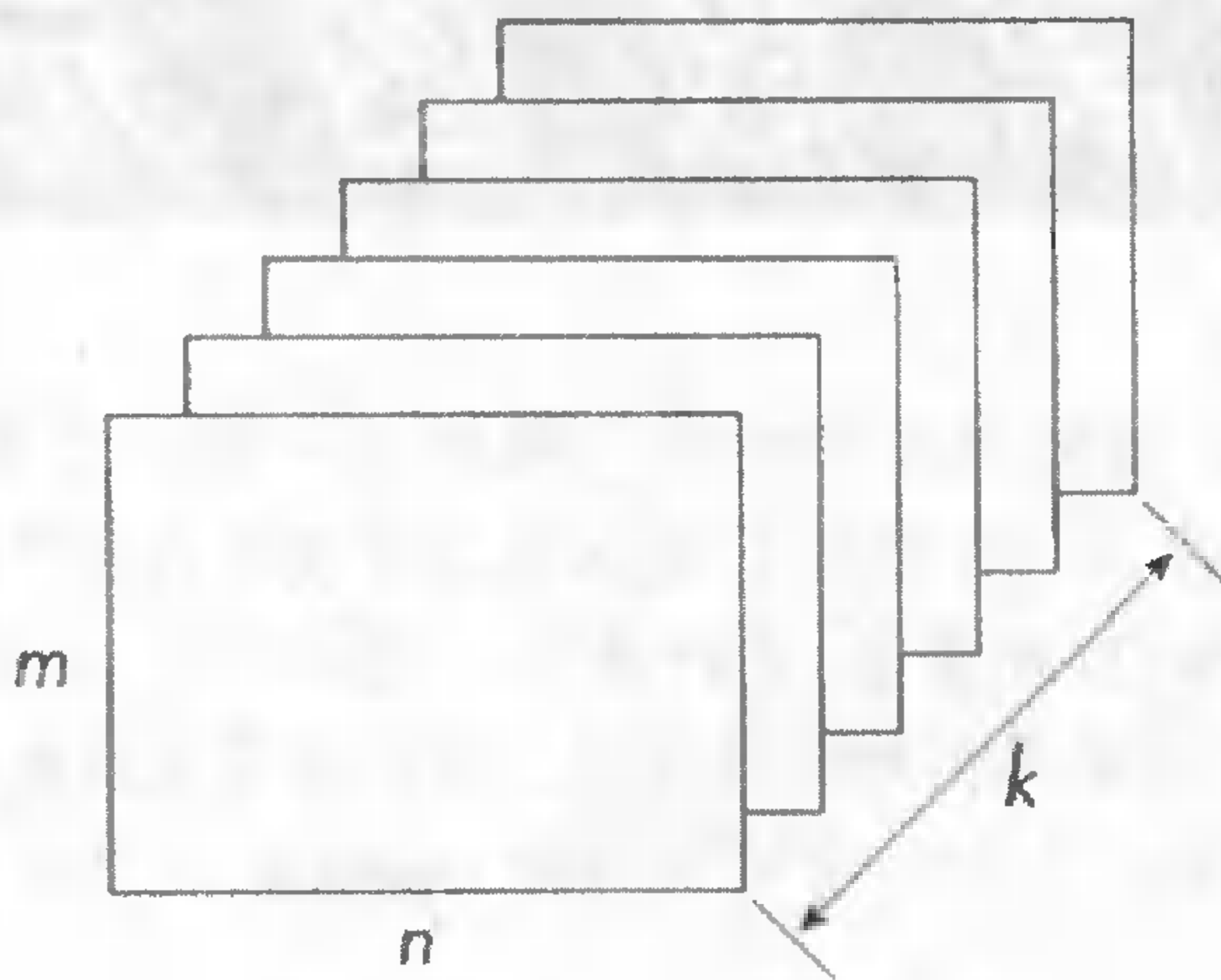


图 7.1 分辨率为 $n \times m$ 的帧缓存，它有 k 个平行的位面

现在换一个视角，如果希望应用程序能够直接访问这些缓存，我们就应该具备读/写这些像素矩阵的能力。但是，我们通常只能对像素的某些特定位组进行操作，即那些对应于 OpenGL 缓存中的某个缓存中的位。所以，我们用术语“像素”来表示帧缓存中处于位置 (x, y) 处的一组位，通常是整数个字节。有了这个定义，依据我们希望对其执行写操作的目标缓存，就可写入深度像素或 RGBA 像素。OpenGL 允许通过一种称为“位块传输操作”的运算来对像素的矩形块进行操作，而完成该操作只需要一次函数调用。

此外，我们还希望能够对由位构成的矩形数组(称之为“位图”)进行操作。虽然位图可被视作一种特殊类型的像素矩形块，但实际上它们的使用方式与一般的矩形块有很大差异，所

① 大多数图形学教材中的术语“帧缓存”都是指形成 RGB 或 RGBA 图像的那个缓存。像素则是一组位，这组位为屏幕坐标系中任意一点赋予了 RGB 或 RGBA 值。在 OpenGL 中，这些位被认为是存在于“颜色缓存”中，而帧缓存被视作所有缓存的集合，即帧缓存包含了深度缓存和若干颜色缓存。OpenGL 中对这些术语的使用方式能够较准确地反映图形系统的实际实现细节。

以需要专门提供一些对位图和像素进行操作的函数。在 OpenGL 中, 像素图(pixelmap)和位图(bitmap)是两种基本的非几何图元。

图 7.2 展示了图形系统体系结构的一个简化版本。与其他程序一样, 这里的应用程序也依靠标准处理器来运行。当应用程序执行一个修改了图形子系统状态的函数或一个生成几何图元(如顶点)的函数时, 图形子系统便开始发挥作用。几何图元流经几何流水线, 该流水线负责实施各种变换(如取景变换、模型变换和投影变换等)、光照及裁剪。最终, 那些可见的图元将被光栅化, 于是就产生了存在于帧缓存中像素的片段集合。非几何图元将流经另外一条并行流水线。在片段处理阶段, 这两条流水线将进行融合。

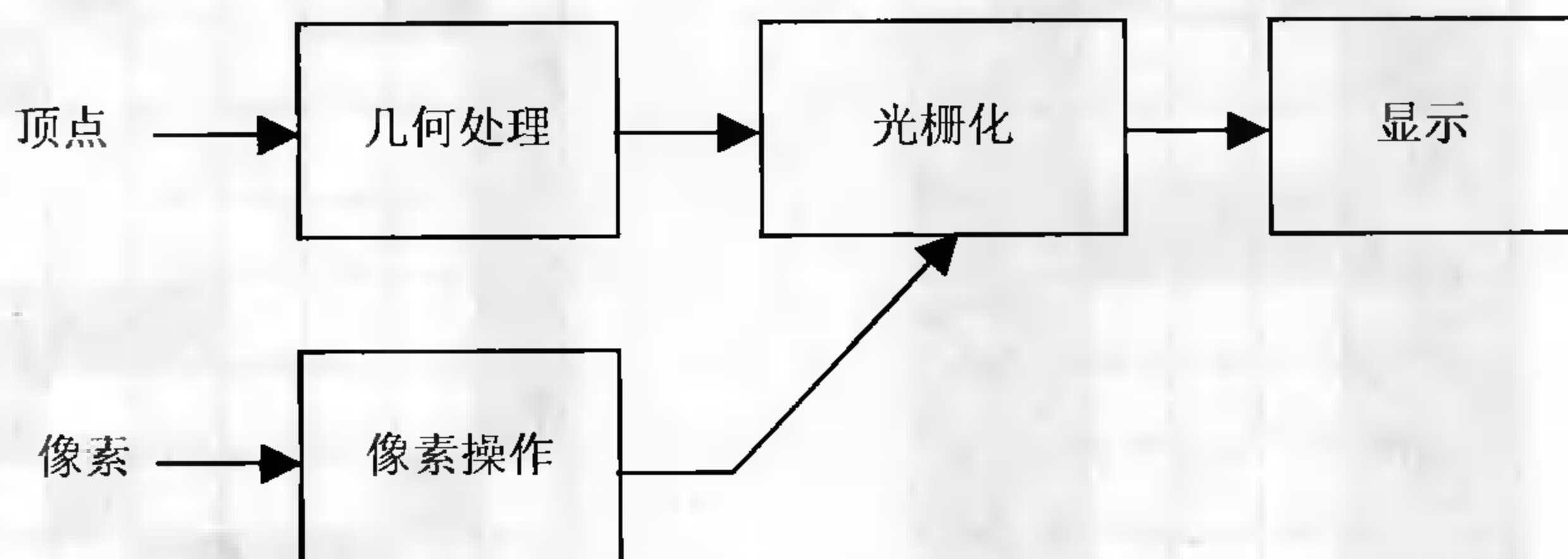


图 7.2 带有并行流水线的简化版 OpenGL 体系结构

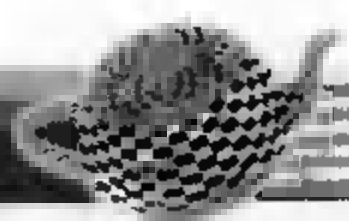
在概念层次上看, 对位图和像素的操作非常简单。可在程序中对其进行定义, 或从文件中对其进行读取。然后, 将其转移到帧缓存中期望的位置。像素的读取包括确定帧缓存中的正确位置及从该位置进行值的提取。不幸的是, 在现实世界中, 事情往往并非如此简单。我们必须考虑应用程序、图像文件及帧缓存内部所使用的像素格式之间的差异。而且还必须考虑由存储体系(即存储离散数据的方式)之间差异而带来的复杂性。【156~157】

像素的类型也是一个必须考虑的因素。从应用程序的角度看, 一个像素有可能代表一个 RGB 颜色值, 也可能表示一个 RGBA 值, 还有可能表示一个灰度值或深度值。在应用程序端, 这些值可以多种形式进行表示。例如, 颜色可用字节型、整型或浮点型的数来表示。

此外, OpenGL 还具有一系列可在流水线内部执行的操作, 其中每种操作都有许多选择。例如, 可在当像素流经流水线时对其进行修改。也可以不同的方式对帧缓存进行写操作。幸运的是, 许多参数都可以使用其默认值, 这样就不会在开发中引入过多的复杂性。

7.2 位 图

位图是一个由位构成的矩形数组。它们最常见的用途是显示光栅文本, 这种文本是由一些小矩形位块定义的字符组成。图 7.3 展示了一些位图化的字体。每个字体都用一个 7×9



的矩形数组来表示。在第 2 章中曾了解到,依据我们需要的是固定宽度的字体还是比例字体,矩形数组的尺寸可以彼此不同。此外,当想要显示由字符构成的字符串时,一个字符的周围可能还会有一些额外的位以帮助正确地分隔各个字符。

位图也可用于交互式程序中的一些实体,如光标或十字光标。位图还可用作模板以确定某一位置上是否应执行某些运算。例如,可用一幅分辨率与屏幕窗口完全相同的位图来决定颜色缓存中相应位置上的像素是否被显示。【157】

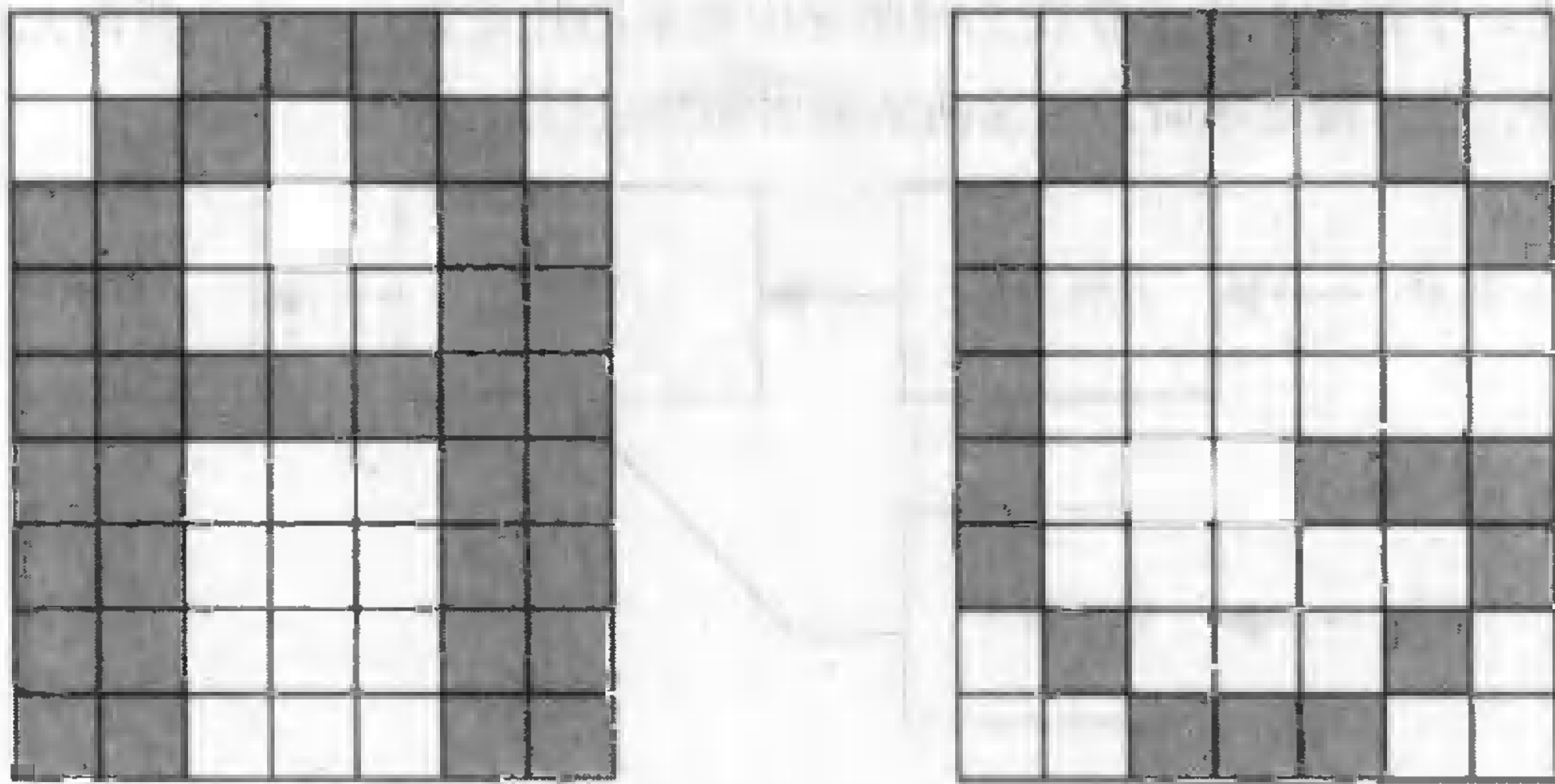


图 7.3 位图化的字符

7.2.1 位图的显示

位图最简单的一个用途就是用于显示一个二维的图案。假定在我们的程序中已经定义好一个位图。为了便于 OpenGL 来使用它,我们将位存储在一个一维数组中。例如,下面的代码生成了 4096 位以描述一个 8×8 的棋盘。即,如果将这 4096 位理解为一个 64×64 的数组,就得到了 0 和 1 交替出现的 8×8 的位块。

```
GLubyte wb[2] = 0x00, 0xff;
GLubyte check[512];
int i, j;
for(i = 0; i < 64; i++)
for(j = 0; j < 8; j++)
    check[i*8+j] = wb[(i/8+j)%2];
```

借助函数 `glBitmap()` 来显示这些位时,是将它们作为二维位图来显示的,在该函数中,可以设定位图的行数和列数。如果选择将这些位显示成一个矩形图(64 行 $\times$ 64 列),就可得到如图 7.4 所示的结果。

可通过如下调用形式将该位图显示出来:


```
glBitmap(64, 64, 0.0, 0.0, 0.0, 0.0, check);
```

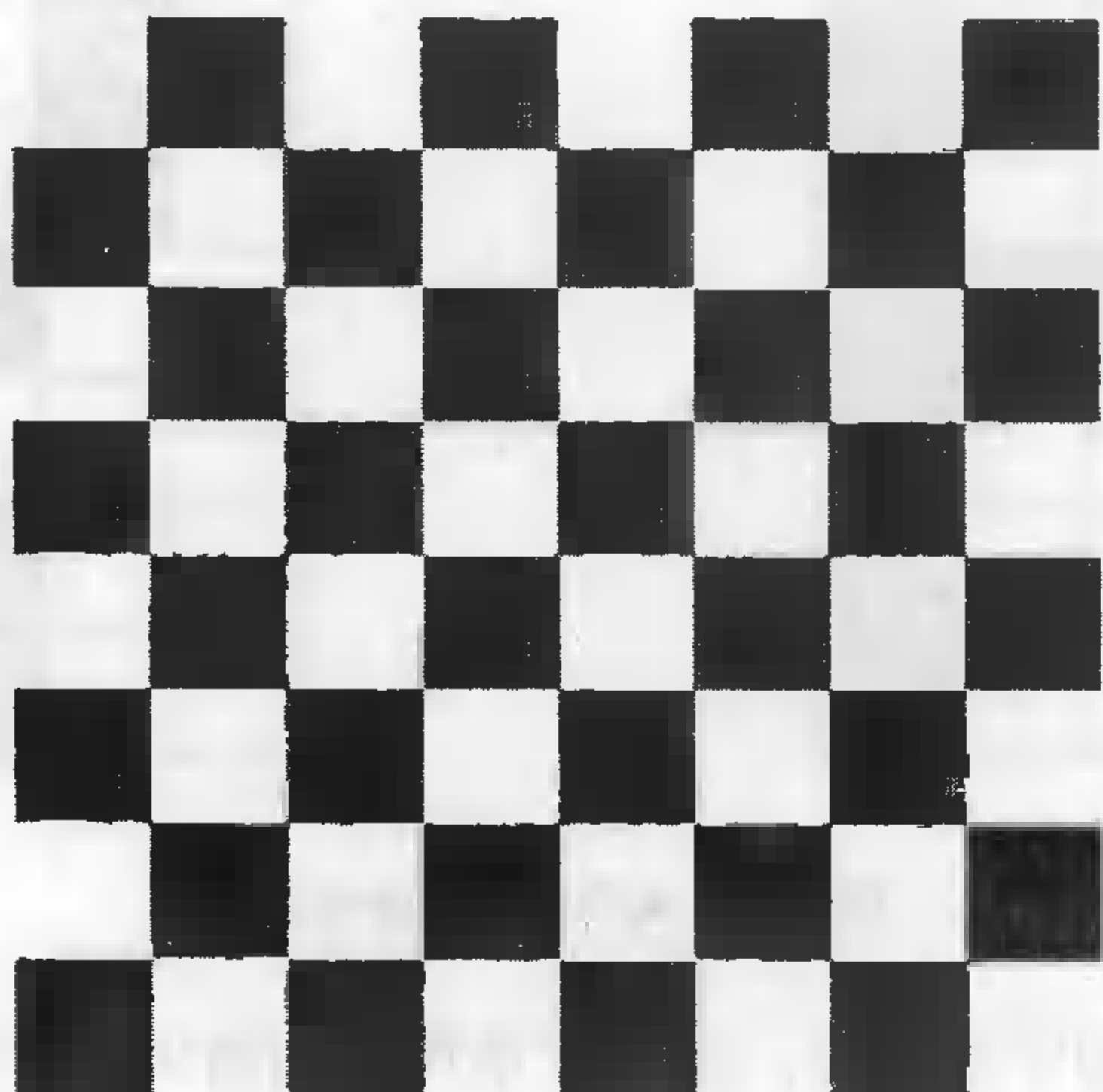


图 7.4 显示为 8×8 棋盘图案的 64×64 位图

该位图中的每一位都对应于图像中的一个像素。我们自然要问，到底对应哪个像素？针对该问题，可利用在第 2 章介绍过的函数 `glRasterPos*()` 来设定当前光栅位置。该函数的第 3 个和第 4 个参数(即 `x0` 和 `y0`)表示相对于当前光栅位置的偏移量，该偏移量可用来确定位图的左下角应位于颜色缓存中的位置。当位图绘制完毕，光栅位置将以第 5 和第 6 个参数(即 `xi` 和 `yi`)作为 `x` 和 `y` 坐标的增量而增加。在我们的例子中，图案只是简单地显示在当前光栅位置处，而且光栅位置保持不变。

增加这些参数的主要原因是为了创建文本。位图字符都是以矩形图案的形式存储的。一些字体对所有的字符使用尺寸相同的矩形块，而另外一些字体针对不同的字符采用不同尺寸的矩形块。但是，即便所有的字符都存储在相同尺寸的矩形块中(例如作为 5×7 的字符集)，仍然存在一些潜在的问题。例如，大写字母 A 可能会以图 7.5 的形式被显示出来。此时用于确定字符起始输出位置的文本基线(baseline)位于左下角。注意，在该字符的起始位置，第 1 列为空，这部分空间可用于将该字符与前面的字符分隔开来。所以，对于字符 A，只需要将 `x0` 和 `y0` 都设为 0.0 即可。现在来考虑图 7.5 所示的小写字母 g。它也正好处在一个 5×7 的矩形块中，但是该字符具有下伸部分(descender)，这表示该字符的一部分应位于基线以下。所以，为了正确地显示该字符，需要将 `y0` 设为负值。而 `y0` 的确切值则要取决于观察条件，因为观察条件将影响光栅的位置。在下一节中，将继续讨论该问题。请注意，我们有可能使用另一个更大的字符集，该字符集中不同字符的基线可能完全不同。但是，这样的字符集需要字符对应的矩形块中留出更多的空间，于是必将占据更多的存储空间。如果使用 `x0` 和 `y0`，将会更加有效。

【158~159】

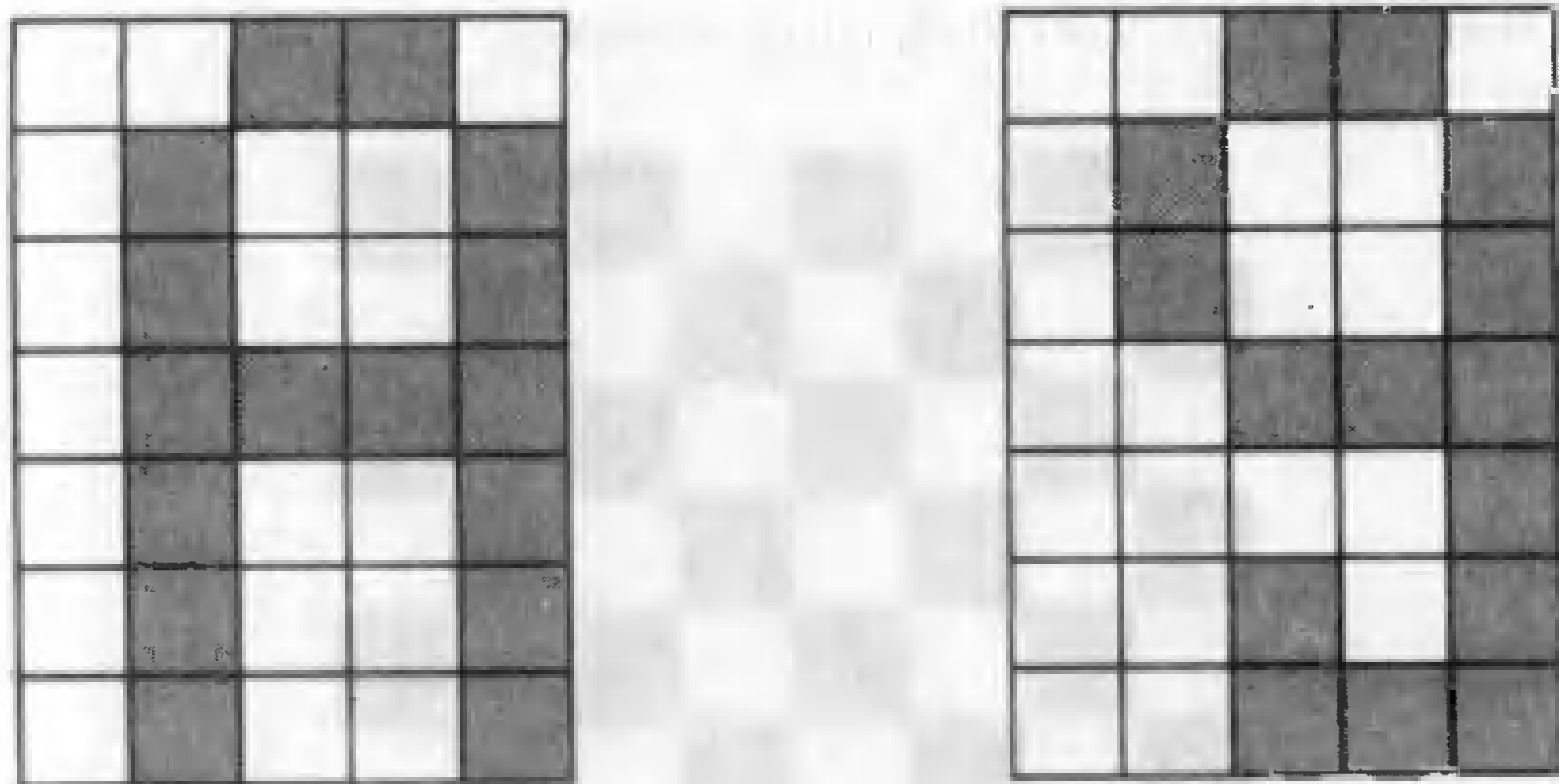


图 7.5 两个 5×7 的字符

希望显示由若干字符构成的字符串时，必须在字符之间留出一定的空间。所以需要参数 xi 和 yi 。它们的作用在于每次调用完函数 `glBitmap()` 之后，为当前光栅位置提供相应的增量。如果对每个字符正确执行函数 `glBitmap()`，则光栅位置总会被自动设置在下一个字符的起始位置。

【159】

对于位图出现在屏幕的位置及方式的控制，还有一些微妙的地方。首先，我们来解决字符出现位置这个问题。

```
void glBitmap(GLsizei c, GLsizei r, GLfloat xb, GLfloat yb,
              GLfloat xi, GLfloat yi, GLubyte * bits)
```

依据数组 `bits` 绘制一幅宽为 r ，高为 c 的位图。位图的起始位置到当前光栅位置的偏移量为 $x0$ 和 $y0$ 。当位图显示完毕后，当前光栅位置则以 xi 和 yi 作为坐标增量而增加。

7.2.2 位图和几何图元的融合

在第 2 章中提到，光栅位置可用整数或浮点数指定为一维、二维甚至四维的形式。光栅位置也流经几何流水线。所以，光栅位置在最终变为屏幕坐标系中的一个位置之前也经历了模型视图变换和投影变换。如果经过变换的光栅位置在视口之外，则位图将不被显示。注意，决定光栅位置的几何流水线和位图流经的像素流水线在光栅化时合二为一，这将影响一幅位图是否会被显示及显示的位置。如果仅仅想显示像素和位图，可借助一个窗口和一个与屏幕相匹配的视口。例如，如果 w 和 h 分别为由重绘回调函数所设置的视口宽度和高度，则可做如下函数调用：

```
gluOrtho2D(0.0, (GLfloat)w, 0.0, (GLfloat)h);
```

此时，如果模型视图矩阵为单位阵，则对光栅位置按照如下形式进行设置：

```
glRasterPos2i(x, y);
```


这样便设置了窗口坐标系中的一个位置，该位置与视口中的位置完全相同，它决定了所生成的位图的起始位置。

如果希望在同一个程序中既使用光栅图元又使用几何图元，光栅位置的设置就比较复杂，因为对场景中多边形的观察条件可能使得如何确定光栅位置的增量并不十分明朗，而光栅的位置将直接关系到字符的显示位置是否正确。对于该问题的一种解决方案是使用两组观察条件，一组用于几何图元，而另外一组用于位图。在应用程序内部，可依据需要重新定义必要的变换或借助矩阵堆栈来保存和恢复变换。 【160】

7.2.3 颜色与模板

在我们的例子中，将 1 位视作黑色，而将 0 位视作白色。实际上，位图就是一个模板。在位图中所有值为 1 的位置上，都会看到一种由当前光栅颜色决定的颜色。当前光栅颜色也是 OpenGL 状态的一部分。而在位图中值为 0 的位置上，位图将不会影响帧缓存中相应位置上的像素，这样当位图放在颜色缓存上方时，就可以看到 0 位下方是什么颜色。假定将颜色缓存清为红色，并将当前绘制颜色设为绿色。如果按照如下的方式进行清屏并将该位图绘制出来：

```
glClearColor(1.0, 0.0, 0.0, 1.0);
glColor3f(0.0, 1.0, 0.0);
glClear(GL_COLOR_BUFFER_BIT);
glBitmap(64, 64, 0.0, 0.0, 0.0, 0.0, check);
```

将看到一个红色和绿色相间的棋盘图案。

提示 位图实际是一个模板，只有值为 1 的位会影响帧缓存中的颜色。

在前面的代码中，设置光栅颜色的方式与在使用几何图元的程序中的方式完全相同，只是这里影响的是位图的颜色值。OpenGL 同时保存了当前绘制颜色和当前光栅颜色，两者都是 OpenGL 状态的一部分。当前颜色都是通过函数 glColor*()来设置的。但是，当前光栅颜色可通过函数 glRasterPos*()来锁定。所以，在如下代码执行后：

```
glColor3f(1.0, 0.0, 0.0);
glBegin(GL_POLYGON);
:
glEnd();
glRasterPos2i(xr, yr);
glColor3f(0.0, 1.0, 0.0);
glBitmap((64, 64, 0.0, 0.0, 0.0, 0.0, check);
glBegin(GL_POLYGON);
:
glEnd();
```


第一个多边形将被绘制为红色，而第二个被绘制为绿色。棋盘图案则被绘制为红色，这是因为光栅颜色实际上是最后一次调用 `glRasterPos2i()` 函数时系统的绘制颜色。

提示 光栅颜色是由 `glRasterPos*()` 函数锁定的。

【161】

7.3 绘制模式

到目前为止，在所有的例子中，由位图中的位构成集合都变成了颜色值为当前光栅颜色的像素。这实际上是基于两点假设的。第一条假设是，绘制位图的缓存与绘制几何图元的颜色缓存是同一个。除非显式指定绘制缓存，默认情况就是如此。第二条假设是，当用当前颜色绘制像素时，该颜色值将影响帧缓存的内容。同样，默认情况下也是如此，但对于位和像素如何被绘制到帧缓存中我们还有更多的控制手段。

OpenGL 使用了一种类似于图 7.6 所示的绘制模型。

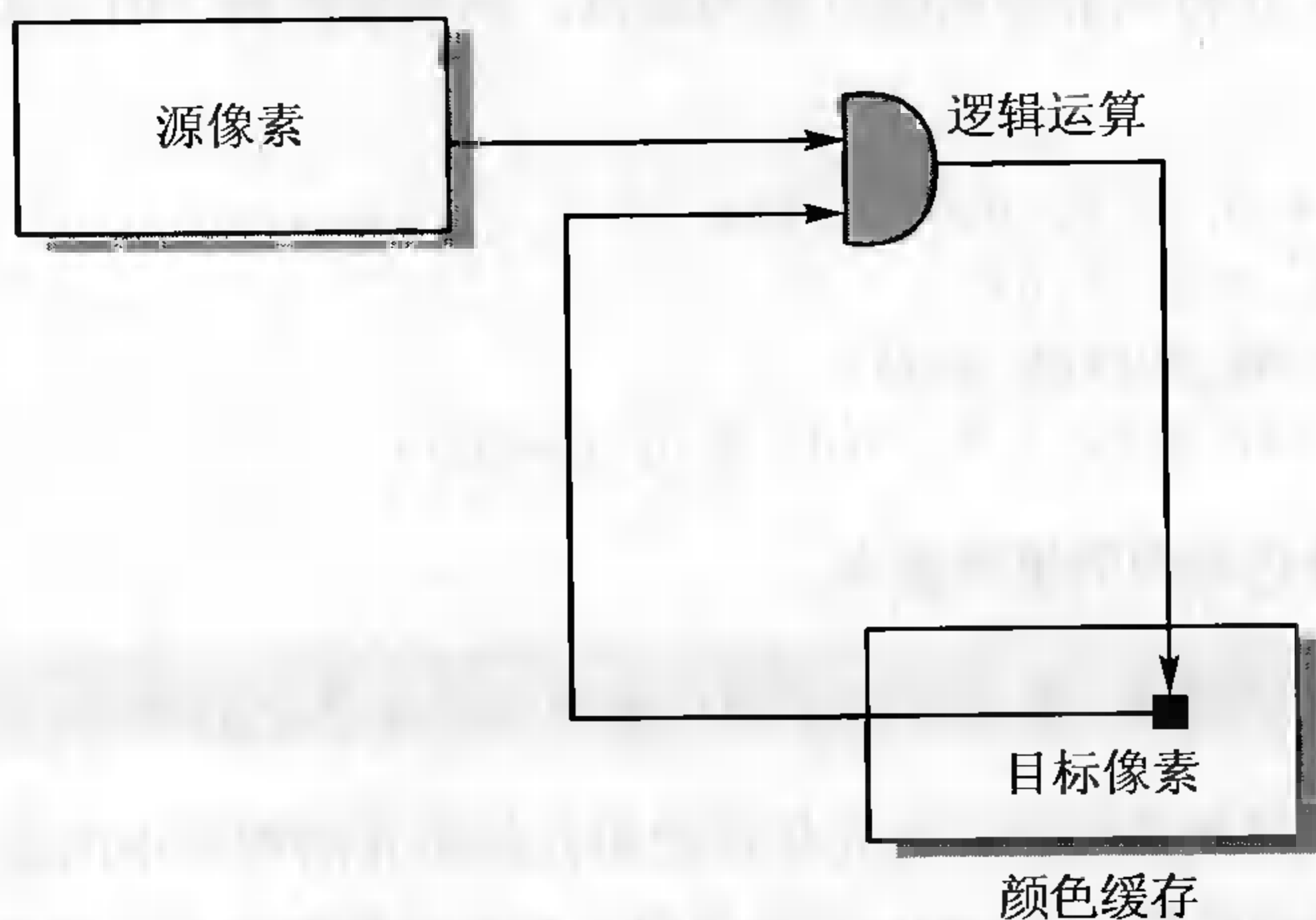


图 7.6 位图的写操作模型

该模型本质上与第 6 章用于融合的模型完全相同。当希望对像素或位图进行写操作时，首先需要观察要进行绘制的目标地点的已有内容。绘制的结果与该内容有关。不妨考虑位图中的一个单个位。在绘制位图时，位图中的每一位都对应于屏幕中的一个像素。我们称该位为“源位”(source bit)。如果该位为 1，则 OpenGL 将用当前光栅颜色来影响目标像素。可将该颜色视作源颜色(source color)，它也就是与源位对应的源像素(source pixel)所具有的颜色。帧缓存中对应的像素称为“目标像素”(destination pixel)。在最简单的情况下，对于帧缓存中与位图中值为 1 的位对应的每一个位置，相应源颜色将取代帧缓存中目标像素的颜色。但是，如果我们首先观察目标像素的颜色，就可令目标像素的新值为源颜色和目标

颜色按照某种规则运算的结果。在 OpenGL 中, 该函数是在源颜色和目标颜色之间逐位进行运算的。可借助函数 `glLogicOp()` 来为将要进行运算的一对位选择 16 种可能的逻辑运算中的任意一种。【162】

```
void glLogicOp(GLenum op)
```

如果逻辑运算 (`GL_COLOR_LOGIC_OP`) 被启用, 该函数可用来将要整合到帧缓存中的源像素和目标像素之间采用 16 种逻辑运算中的哪一种。参数值可以是 `GL_COPY` (默认值)、`GL_OR` 或 `GL_XOR`。

必须通过如下函数调用来启用逻辑运算:

```
glEnable(GL_COLOR_LOGIC_OP);
```

当 `glLogicOp` 的参数取默认值 (`GL_COPY`) 时, 结果将与不启用逻辑运算时的结果相同: 源像素仅仅是简单地取代目标像素的值。如果参数取其他值, 则会很有趣。例如, 我们总可以用源像素和目标像素的逻辑或 (逐位进行, 参数取 `GL_OR`) 或者将源像素逐位取反 (参数取 `GL_COPY_INVERTED`) 来取代目标像素的值。其中许多运算可能会由于逻辑运算的逐位进行而在显示器上生成一些意想不到的颜色。如果令一个红色像素和一个黄色像素进行逻辑或运算, 则运算的结果与黄色或红色的关系并不十分明朗。

提示 逻辑运算是在源颜色和目标颜色之间逐位进行的。所以, 对于那些位深度超过 1 的像素, 运算的结果可能视觉上看起来非常奇怪。

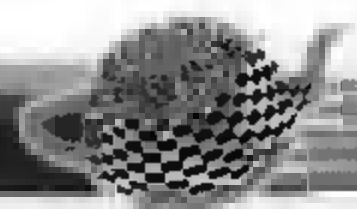
在所有的运算中, 最有趣同时也是最有用的无疑是异或运算 (`GL_XOR`)。该运算的大部分应用都是基于这样一种性质, 即如果该运算连续被应用两次, 则将返回最初的状态。所以, 如果用符号 $\oplus$ 来表示异或运算 XOR, 用 x 和 y 来表示参与运算的位, 则有:

$$x = x \oplus y \oplus y$$

所以, 如果 x 是帧缓存中某一位的值, 而且使用异或模式将位 y 绘制到与 x 在的位置上, 将得到 $x \oplus y$, 第二次绘制 y 时, 将再次得到 x 。

假定希望从屏幕中移除某个对象。可以先将颜色缓存清空, 然后再将所有的对象重新绘制。这种策略需要返回头来寻找所有定义了这些对象的代码或显示列表。如果使用异或运算, 只需将同一对象在同一地点绘制两次即可。第二次绘制时会使第一次绘制无效。下面是一个简单的鼠标回调函数, 该函数在每次单击鼠标左键时先将棋盘图案重新绘制在一个旧位置上, 然后将该图案再次显示在鼠标位置处。【163】

```
void mouse(int btn, int state, int x, int y)
{
    if(btn == GLUT_LEFT_BUTTON && state == GLUT_DOWN)
    {
        glBitmap(64, 64, 0.0, 0.0, 0.0, 0.0, check);
    }
}
```

```
        glRasterPos2i(x, hh-y);
        glBitmap(64, 64, 0.0, 0.0, 0.0, 0.0, check);
        glFlush();
    }
}
```

其中, `ww` 是视口和窗口的高度(该值保存在全局变量中), 与在重绘回调函数中进行的设置相同。

```
void reshape(int w, int h)
{
    glViewport(0, 0, w, h);
    glMatrixMode(GL_PROJECTION);
    glLoadIdentity();
    gluOrtho2D(0.0, (GLfloat)w, 0.0, (GLfloat)h);
    hh = h;
    glMatrixMode(GL_MODELVIEW);
    glLoadIdentity();
}
```

在 `main()` 函数中所需设置的模式如下:

```
glEnable(GL_COLOR_LOGIC_OP);
glLogicOp(GL_XOR);
```

在这段例程中, 该位图起初被绘制在窗口的左上角。

```
void display()
{
    glClear(GL_COLOR_BUFFER_BIT);
    glColor3f(1.0, 0.0, 0.0);
    glRasterPos2f(0.0, 0.0);
    glBitmap(64, 64, 0.0, 0.0, 0.0, 0.0, check);
    glFlush();
}
```

所以, 每次当鼠标左键单击时, 棋盘图案都将首先绘制到原来的旧位置上, 因为此时光栅位置尚未发生变化。该运算移除了棋盘图案的先前实例。接下来, 光栅位置被更新到与鼠标位置匹配的位置上, 所以棋盘图案将被绘制到新位置上。【164】

实现这一简单思想的应用程序包括在屏幕中鼠标光标的移动, 橡皮线和橡皮矩形及弹出式菜单的实现。它们都包括对象的绘制(可能绘制到其他对象之上), 然后返回屏幕, 恢复到最初的状态。

7.4 像素的读/写

现在来研究如何从帧缓存中读/写多位像素的数组。从概念层次来看，这个过程非常简单。首先来考虑写操作。图像数据即可在应用程序中定义也可是读取自一个文件。我们希望将这些数据移动到帧缓存中，这可借助函数 `glDrawPixels()` 来完成。读操作则是一个逆过程。要将像素值从帧缓存读取到应用程序中需要借助于函数 `glReadPixel()`。但是，还必须考虑另外一些细节问题。这些细节问题大多源自像素在帧缓存中与在系统内存中表示方式之间的差异。而不同的计算机体系结构对于数字的不同表示方式则让这些问题变得更为突出。此外，无论像素数据最终如何保存在帧缓存中，我们仍然希望在程序中用多种方式来表示像素数据，包括整型、字节型和浮点型。

可想象一下图 7.7 所示的过程。像素通过流水线从应用程序到达帧缓存，在该流水线中，像素在系统内存中的原始格式经历了一个解包过程。像素在通过像素变换运算(pixel transfer operation)从应用程序到达帧缓存的过程中，像素值可被修改。例如，我们可能希望在帧缓存的允许范围内对像素值进行缩放。还可以使用像素映射(pixel-mapping)运算，该运算使得我们可以借助于查找表将像素值映射为其他值。还可以用映射运算来实现两种不同的颜色系统间的颜色取反运算，或通过颜色均衡来调整显示器的属性。然后就可对像素实施比例运算，以便当这些像素被放入帧缓存中时完成对这些像素的复制。最后还需要进行一系列的测试，例如深度测试(用 z-缓存算法来实现)，该测试会在每个像素被写入帧缓存之前进行。

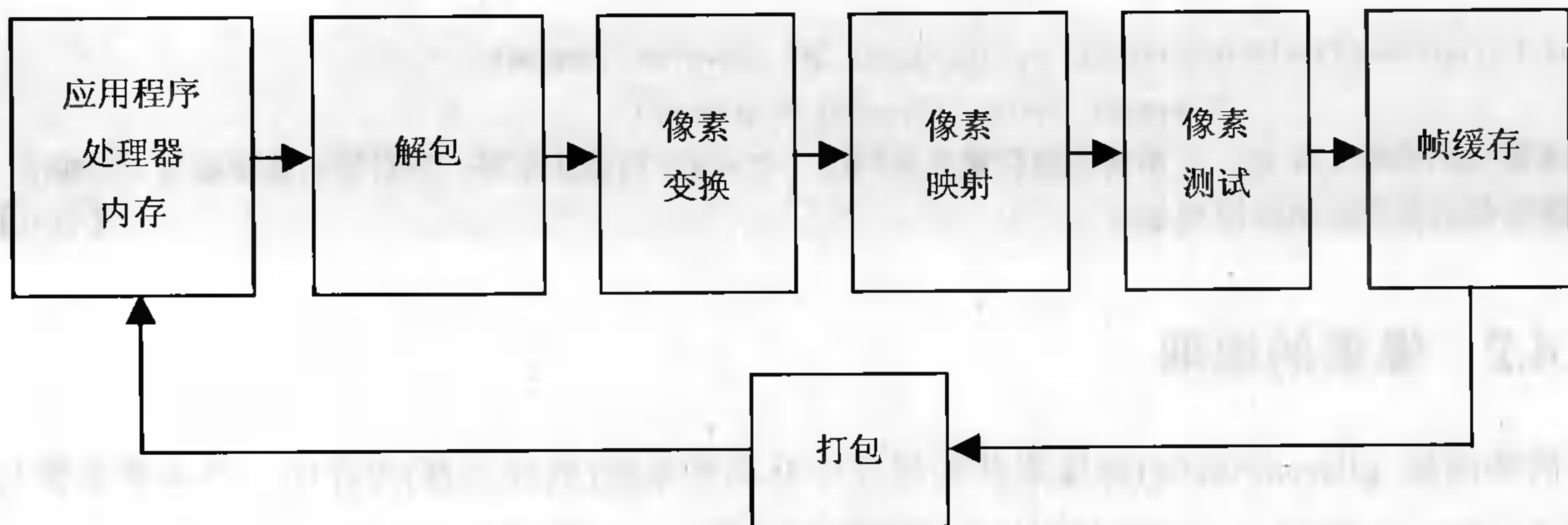
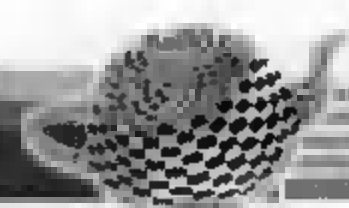


图 7.7 像素流水线

【165~166】

从帧缓存中读取像素时也将经历一个类似的过程。只是需要对格式的转换(打包, packing)给予特别关注，因为打包决定着帧缓存中的信息以何种形式保存在处理器内存中。

还可以选择从哪个缓存中进行读和写操作。既可向前台颜色缓存或后台颜色缓存写入 RGB 或 RGBA 像素，也可对深度值进行读写操作，还可对一个缓存进行读操作，而将内容写入



另一个缓存,甚至是正在进行读取操作的缓存。这种体系结构表明,不但可以用 OpenGL 来显示离散数据(如图片和文本),还可以用 OpenGL 来处理图像。

7.4.1 像素的写操作

执行像素写操作的基本函数是 `glDrawPixels()`, 该函数将在当前光栅位置处绘制一个像素矩形。我们必须指定所要绘制的像素的格式。例如,如果在程序中有一幅形式如下的 RGB 图像:

```
Glubyte image[WIDTH][HEIGHT][3];
```

则用 `GL_RGB` 来指定格式,用 `GL_UNSIGNED_BYTE` 来指定类型,并用如下形式来完成绘制:

```
glDrawPixels(WIDTH, HEIGHT, GL_RGB, GL_UNSIGNED_BYTE, image);
```

这种格式将像素数据指定为三元组的形式。如果这些像素数据依分量存储在三个不同的数组中,则可使用参数 `GL_RED`、`GL_GREEN` 和 `GL_BLUE`, 然后就可分别访问这三个数组了。其他的格式包括 `GL_RGBA` 和 `GL_DEPTH_COMPONENT`(准备写入深度缓存)。类型参数大多数情况下都取标准类型(如 `GL_INT`, `GL_FLOAT`), 也可取一些用于压缩图像数据打包类型。例如,类型 `GL_UNSIGNED_BYTE_3_3_2` 对应于这种情形:用一个字节来表示一个 RGB 值,其中 3 位用于表示 R 分量的值,3 位用于表示 G 分量的值,2 位用于表示 B 分量的值。

```
void glDrawPixels(GLsizei w, GLsizei h, GLenum format,  
                  GLenum type, GLvoid * array)
```

该函数依据数组 `array`, 在当前光栅位置处绘制出一个 $w \times h$ 的像素矩形。像素格式被指定为 `format`, 像素数据的类型被指定为 `type`。 【166】

7.4.2 像素的读取

可借助函数 `glReadPixels()` 将像素从帧缓存中移动到系统(或处理器)内存中。该函数参数与 `glDrawPixels()` 相同,二者的使用场合截然相反。所以,如下函数调用:

```
glReadPixels(0, 0, columns, rows, GL_RGB,  
             GL_UNSIGNED_BYTE, image);
```

将从帧缓存的左下角开始读取 $rows \times columns$ 个 RGB 像素。这些值将以每个像素三个无符号字节型的形式返回,而与像素在帧缓存中的表示形式无关。

```
void glReadPixels(GLint x, GLint y, GLsizei w, GLsizei h, GLenum format,  
                  GLenum type, GLvoid * array)
```


该函数从帧缓存中的位置(x, y)开始将一个 $w \times h$ 的像素矩形块读进数组 `array` 中。像素的格式被指定为 `format`, 写入数组的数据类型为 `type`。

像素读取操作的返回值将受限于帧缓存的分辨率。写入像素时, 通常只需指定颜色值, 而不必关心在帧缓存中分配了多少位用于存储和显示该颜色。OpenGL 的具体实现将尽其所能取得最好的分辨率。许多系统都使用一种称为“颜色抖动(dithering)”的方法, 该方法在颜色放进帧缓存时, 将颜色值的低阶位作出一些改动, 以平滑颜色之间的过渡, 否则在那些仅为每种颜色分量分配了很少位的显示器上, 这种颜色之间过渡的不均匀性就会非常明显。但是, 当从一个颜色缓存中进行读取时, 返回值将取决于硬件中为每种颜色实际分配的位数。所以, 很可能虽然在程序中用 0~255 之间的值来表示不同的红色, 但由于帧缓存只为红色分量分配了 4 位, 而使读取的返回值只能区分 16 种不同的红色。如果显示器经过了抖动处理, 而且将同样的值写到一个更大的区域中, 在进行读取操作时就可区分出一些值之间的细微差别。这都要归功于抖动。此外, 在同一个系统中, 用于单缓存和双缓存的显示器位数可能存在差异。

提示 进行像素读取操作时, 请务必确保已经知道显示器的分辨率, 而且已将一些可能的抖动考虑进来。

提示 在 OpenGL 的具体实现中, 像素的存储方式可能与它们在处理器内存中的存储形式不同。

所以, 在读取像素时, 应该首先检查分辨率的位数。例如, 可使用如下查询函数来确定位数。如果显示器经过了抖动处理, 则应通过如下调用将抖动关闭:

```
glGetIntegerv(GL_RED_BITS, &nbits);
glDisable(GL_DITHER);
```

这样, 对于写入的任何一个给定颜色, 总能得到相同的量化值。

【167~168】

7.4.3 像素的复制

有时, 我们希望将像素从帧缓存中的一部分转移到另一部分。一种方案是在调用 `glDrawPixels()` 函数之后接着调用 `glReadPixels()` 函数, 这样虽然可行, 但为了实现帧缓存到帧缓存之间的操作, 不得不对系统内存进行频繁地操作。一种更好的方案是借助于函数 `glCopyPixels()`。该函数允许完成上述操作而无需借助于系统内存。与 `glReadPixels()` 函数一样, 我们也需要设置开始复制的起始位置, 同时用当前光栅位置确定复制的目标位置。如果处理的是 RGB 或 RGBA 颜色, 则缓存参数应设为 `GL_COLOR`。


```
void glCopyPixels(GLint x, GLint y, GLsizei w, GLsizei h, GLenum buffer)
```

该函数从帧缓存 `buffer` 中的 `(x, y)` 开始, 将一个 `w×h` 的像素块复制到当前光栅位置处。

7.5 缓存的选择

在对 RGBA 值进行读/写操作时, 实际上是在与帧缓存中的一个颜色缓存打交道。在单缓存模式下, 用于读/写的默认缓存为前台颜色缓存(`GL_FRONT`)。在双缓存模式下, 用于读/写的默认缓存为后台缓存(`GL_BACK`)。几何图元通常放置在这些缓存中。

但是, 大多数 OpenGL 实现都支持额外的颜色缓存。例如, 如果实现支持双缓存立体图像, 则将由 4 个不同的颜色缓存, 即 `GL_RIGHT_FRONT`、`GL_LEFT_FRONT`、`GL_RIGHT_BACK` 和 `GL_LEFT_BACK` (对于单缓存立体图像则是 `GL_RIGHT` 和 `GL_LEFT`)。可能还需要一些辅助的颜色缓存(`GL_AUX0`、...、`GL_AUXn`)来做其他用途。例如, 在第 3 章中讨论拾取操作时, 我们了解到拾取需要额外的绘制。通常, 从这些额外缓存选择一个来保存额外绘制路径的结果会带来许多便利。

可以通过 `glReadBuffer()` 函数选择从哪个缓存进行读取操作, 通过函数 `glDrawBuffer()` 来指定对哪个缓存进行写操作。

```
void glReadBuffer(GLenum buffer)
```

选择要进行读取操作的缓存。

```
void glDrawBuffer(GLenum buffer)
```

选择要进行绘制的目标缓存。

在交互式程序中, 有时希望同时对前台缓存和后台缓存进行写操作。这可以通过将函数 `glDrawBuffer()` 的参数指定为 `GL_FRONT_AND_BACK` 来实现。【168】

附加缓存有多种使用方式。一种用途就是作为第 3 章介绍的拾取操作的一种替换机制。在这种机制中, 对每个对象分别以不同的颜色将其绘制到某个不被显示的颜色缓存中。然后就可以用鼠标回调函数返回的鼠标位置来确定从该颜色缓存中的哪个位置进行颜色的读取。不妨来考虑一个具有明暗效果的球体。在屏幕上呈现出来的这个球体的不同位置具有不同的明暗度, 而该球体却以一种单一的明暗度被绘制到其他的缓存中。这项技术需要一次额外的绘制, 但提供了一种简单的拾取策略。我们也可以在前后台缓存发生置换之前将同一场景两次绘制到后台缓存中, 第一次对不同的对象使用不同的颜色以便拾取, 第二次绘制时则可按照我们的意愿来显示所有对象。

另一个应用是在需要用多摄像机进行绘制的场合。例如, 如果场景中有一面镜子, 我们可以首先用一个位于镜面后方的摄像机将场景绘制到一个附加缓存中。然后当像往常一样绘

制场景时,再对该缓存进行读取,并将其作为纹理映射到镜面上即可(关于纹理映射将在第8章讨论)。

7.6 像素存储模式

在系统内存和帧缓存之间数据的移动有可能正确,也可能不正确,这要取决于用户的系统。问题在于 OpenGL 可能需要一些关于字节如何在处理器内存中组织的附加信息。例如,不同的系统在决定一个短整型数或一个整型数的有效字节上可能存在差异。一些机器可能要求数据应在内存中对齐于 4 个字节的边界。在某些情况下,虽然可能存在多种组织图像数据的许可方法,但这些方法的效率之间往往存在显著的差异。OpenGL 提供了函数 `glPixelStore*()` 以允许程序针对图像数据在处理器中的存储形式作出调整。

```
void glPixelStore (fd) (GLenum param, type value)
```

将像素存储模式参数 `param` 设为 `value`。

【169】

对于不同的机器,可能具有不同的情况,该参数有许多可选值。两个更常用的选项如下:

```
glPixelStorei(GL_UNPACK_SWAP_BYTES, GL_TRUE);
glPixelStorei(GL_PACK_SWAP_BYTES, GL_TRUE);
```

这些函数为短整型数或整型数将字节从最低有效位到最高有效位重新排序,而在字节内部保持各位的顺序不变。两个重要的选项如下:

```
glPixelStorei(GL_UNPACK_ALIGNMENT, 1);
glPixelStorei(GL_PACK_ALIGNMENT, 1);
```

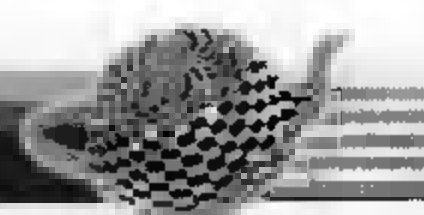
这两个函数总是使用下一个字节分别将数据打包或解包。注意,可以对从帧缓存读取像素(打包)和从处理器内存向帧缓存写入像素(解包)独立地进行控制。

提示 如果图像显示时出现了一些意想不到的结果,请检查像素存储模式的设置。

幸运的是,如果只是在一种单一的体系结构下进行处理,则一般情况下不必关心这些选项,完全可以使用这些参数的默认值。如果能够像前面例子中一样尽可能地使用无符号字节型数据,就可以避免许多问题的出现。

7.7 PPM 图像的显示

在使用来自外部数据源的图像时,它们通常是以一些标准格式(如 JPEG、GIF、TIFF 或 PPM)存在的。这些格式的多样性很大程度上是由图像数据格式的多样性造成的。图像可以是 RGB 图像、索引图像、RGBA 或者灰度图。图像数据的类型可能是整型、字节型或者浮点型。



此外,为了减小图像文件的大小,图像数据还可能被压缩。OpenGL 缺少处理标准格式图像的一个原因就是现有的图像格式过于纷繁复杂。所以,我们常常不得不亲自编写代码来实现图像文件的读取及将我们在帧缓存中生成的图像以标准格式进行保存。通常,可以使用标准软件(如 Photoshop 或 GIMP 图像浏览器)来实现图像格式之间的转换,而非编写大量的图像读/写程序。

可通过编写一个读取标准格式的图像文件并用 OpenGL 将其显示的程序来说明这种基本思路。我们将以 PPM(Portable Pixel Map)格式的图像为例。这是一种无压缩格式,而且非常简单。下面的例子将涵盖无压缩格式的大部分主题。对压缩格式的处理超出了本书的范围,故不赘述。

像其他大多数图像文件一样,PPM 文件也是以文件头开始的,文件头标识了文件的类型、大小、图像的格式及一些附加的注释。紧随文件头之后的便是图像数据。PPM 文件将 RGB 图像以连续的 RGB 像素的形式进行存储,每个分量都以一个 ASCII 码形式的整数来表示。头文件的大致结构如下:

```
P3
# comment 1
# comment 2
:
# comment n
rows columns max
```

P3 表明该文件是一个 PPM 文件。紧随其后可以是任意数量的注释行,每行注释都以#开始,而且最多不能超过 70 个字符。接下来的三个值都是整型的,分别表示图像的行数、列数及数据中 RGB 的最大值。像素数据的最小值被假定为 0。这些值之后便是一系列 RGB 值,每个 RGB 值都是表示三个分量值的 ASCII 字符串。PPM 格式使用一个或多个空白字符(空格、Tab 或换行符)来分隔不同的值。【170】

下面来编写读取 PPM 文件的程序,该程序运行时将提示用户输入 PPM 文件的名称。对于文件头的读取有一些复杂,因为我们并不知道到底有多少行注释。还需要分配一段缓存以保存读取到的数据。

```
FILE * fd;
char c;
char b[70];

printf("enter file name\n");

scanf("%s", b);
```



```

fd = fopen(b, "r");

fscanf(fd, "%[^\n]", b);
if(b[0] != 'P' || b[1] != '3')
{
    printf("%s is not a PPM file!\n", argv[1]);
    exit(0);
}

```

读取第一行后, 需要检查头两个字符是否为 P3。然后, 将另起一行读取第一个字符。如果该字符为#, 则表明该行为注释行。然后将该行的其余部分读取到缓存中。如果字符不为#, 则用函数 `ungetc()` 将其放回文件流。现在就可以开始读取图像的行数、列数及颜色分量的最大值了。 【171~172】

```

fscanf(fd, "%c", &c);
while(c == '#')
{
    fscanf(fd, "%[^\n]", b);
    printf("%s\n", b);
    fscanf(fd, "%c", &c);
}
ungetc(c, fd);
fscanf(fd, "%d %d %d", &n, &m, &k);

```

我们可用行数和列数来设置窗口的尺寸:

```
glutInitWindowSize(n, m);
```

我们打算使用动态数组, 这样程序就可以处理任意维数的图像。下面分配一个容量为 $3 \times n \times m$ 个无符号字节的一维数组。

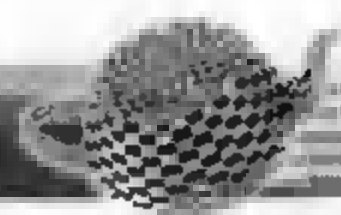
```

GLubyte * image;
nm = n * m;
image = (GLubyte *)malloc(3 * sizeof(GLuint) * nm);

```

在 OpenGL 程序中使用动态数组时, 分配一维数组而非指向数组的指针数组非常重要。如果按后一种形式创建动态数组, 我们便无法将该数组传递给 OpenGL 函数(如 `glDrawPixels()` 函数)。我们准备用一个无符号字节型的数组来保存图像数据。虽然 OpenGL 可以使用各种各样的数据类型, 当分辨率比较高时(即便显示器能够用 8 位以上来表示每个颜色分量), 我们实际很难看出差别。但是, 由于数据总是以整型给出的, 所以必须用比例因子对其进行缩放:

```
float s = 255.0 / k;
```

现在可以读取数据了。如果 s 的值为 1.0(对于 PPM 文件, 通常情况如此), 我们可以跳过比例变换这一步。

```
if(k == 255)
    for(i = 0; i < nm; i++)
    {
        fscanf(fd, "%d %d %d", &red, &green, &blue);
        image[3 * nm - 3 * i - 3] = red;
        image[3 * nm - 3 * i - 2] = green;
        image[3 * nm - 3 * i - 1] = blue;
    }
else for(i = 0; i < nm; i++)
{
    fscanf(fd, "%d %d %d", &red, &green, &blue);
    image[3 * nm - 3 * i - 3] = red * s;
    image[3 * nm - 3 * i - 2] = green * s;
    image[3 * nm - 3 * i - 1] = blue * s;
}
```

【172~173】

最后, 可通过如下回调函数将图像显示出来:

```
void display()
{
    glClear(GL_COLOR_BUFFER_BIT);
    glRasterPos2i(0, 0);
    glDrawPixels(n, m, GL_RGB, GL_UNSIGNED_BYTE, image);
    glFlush();
}
```

提示 OpenGL 用一维数组来保存二维信息。在程序中需要用到动态数组时, 用一维数组来保存图像数据通常更为安全。

注意, 在本例中没有用到几何函数。通过选择与图像尺寸匹配的窗口尺寸, 当显示图像时, 窗口便被整幅图像所填充, 如图 7.8 所示。但是, 在重新调整窗口尺寸时便会出现一些问题, 如果将窗口尺寸变小, 我们将只会看到图像的一部分; 当窗口尺寸变大时, 窗口中又会出现一些空白, 如图 7.9 所示。如果借助函数 `glPixelZoom()`, 则可以避免这一问题。关于该函数, 将在 7.10 节中进行讨论。这种情形与几何实体的显示有本质上的区别。显示一个具有多边形的场景时, 可在窗口尺寸发生变化时在重绘回调函数中对投影矩阵进行修改, 以使图像填充整个视口。目前的情况是由于像素流经的是自己的流水线, 丝毫不受投影矩阵和模型视图矩阵的影响造成的。

【173~174】

如果用这些图像定义纹理映射(第 8 章将讨论该主题), 就可对图像进行缩放以使窗口尺寸变

化时，图像尺寸也能进行相应的变化。

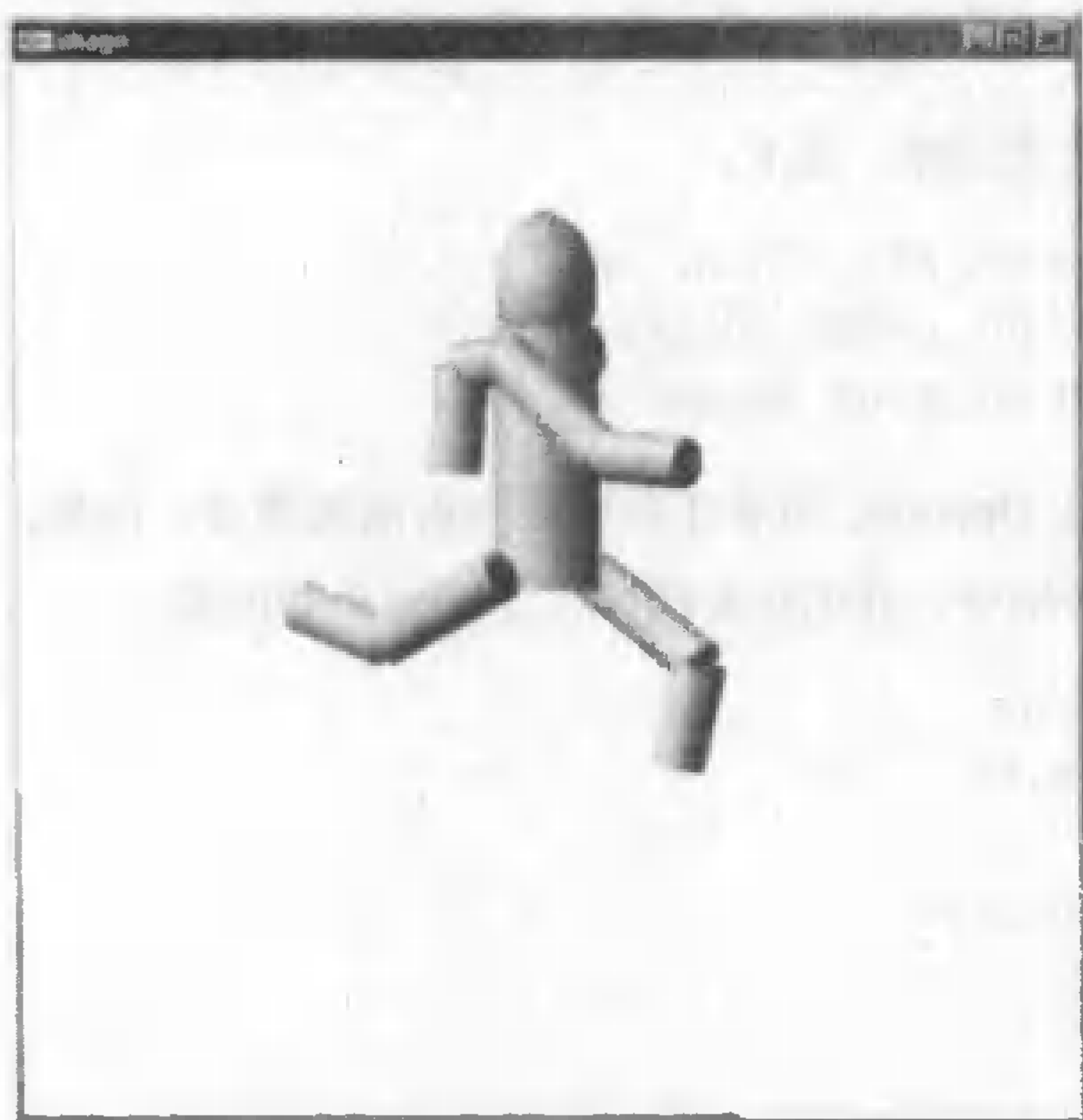


图 7.8 窗口尺寸与图像尺寸匹配时显示出的 PPM 图像



图 7.9 窗口尺寸发生变化后显示出的 PPM 图像

提示 由于图像矩形和位图都存在于帧缓存中，这样它们的屏幕尺寸便不受重绘回调函数的影响。

对于上述程序可尝试的一种修改是将图像数组用整型而非字节型来表示。

```
GLuint * image;
nm = n * m;
image = (GLuint *)malloc(12 * sizeof(GLuint) * nm);
```


然后就可使用参数为整数的 `glDrawPixels()`:

```
glDrawPixels(n, m, GL_RGB, GL_UNSIGNED_INT, image);
```

接下来可以使用像素变换运算, 如下:

```
glPixelTransferf(GL_RED_SCALE, s);
glPixelTransferf(GL_GREEN_SCALE, s);
glPixelTransferf(GL_BLUE_SCALE, s);
```

这样, 比例运算就可由 OpenGL 而非在应用程序内部来完成。但是, 必须确保像素存储模式已被正确设置。在本例中, 我们还打印输出了文件头的内容。

```
#include <stdio.h>
#include <stdlib.h>

#include <GL/glut.h>
int n;
int m;

GLuint * image;

void display()
{
    glClear(GL_COLOR_BUFFER_BIT);
    glRasterPos2i(0, 0);
    glDrawPixels(n, m, GL_RGB, GL_UNSIGNED_INT, image);
    glFlush();
}

void myreshape(int w, int h)
{
    glMatrixMode(GL_PROJECTION);
    glLoadIdentity();
    gluOrtho2D(0.0, (GLfloat)n, 0.0, (GLfloat)m);
    glMatrixMode(GL_MODELVIEW);
    glLoadIdentity();
    glViewport(0, 0, w, h);
}

int main(int argc, char ** argv)
{
    FILE * fd;
    int k, nm;
    char c;
```



```

int i;
char b[71];
float s;
unsigned int red, green, blue;

printf("enter file name/n");
scanf("%s", b);
fd = fopen(b, "r");
fscanf(fd, "%[^\\n] ", b);
if(b[0] != 'P' || b[1] != '3')
{
    printf("%s is not a PPM file!\\n", b);
    exit(0);
}
printf(("s is a PPM file!\\n", b);
fscanf(fd, "%c", &c);
while(c == '#')
{
    fscanf(fd, "%[^\\n]", b);
    printf("%s\\n", b);
    fscanf(fd, "%c", &c);
}
ungetc(c, fd);
fscanf(fd, "%d %d %d", &n, &m, &k);

printf("%d rows %d columns max value = %d\\n", n, m, k);

nm = n * m;

image = malloc(3 * sizeof(Gluint) * nm);

s = 255./k;

for(i = 0; i < nm; i++)
{
    fscanf(fd, "%u %u %u", &red, &green, &blue);
    image[3 * nm - 3 * i - 3] = red;
    image[3 * nm - 3 * i - 2] = green;
    image[3 * nm - 3 * i - 1] = blue;
}
printf("read image\\n")
glutInit(&argc, argv);
glutInitDisplayMode(GLUT_SINGLE|GLUT_RGB);

```



```

glutInitWindowSize(n, m);
glutInitWindowPosition(0, 0);
glutCreateWindow("image");
glutReshapeFunc(myreshape);
glutDisplayFunc(display);
glPixelTransferf(GL_RED_SCALE, s);
glPixelTransferf(GL_GREEN_SCALE, s);
glPixelTransferf(GL_BLUE_SCALE, s);
glClearColor(1.0, 1.0, 1.0, 1.0);
glutMainLoop();
}

```

【174~176】

7.8 灰度图像的使用

在许多应用中，我们需要处理只包含灰度明暗的图像。这些图像通常是指灰度图。灰度图像可以存储在元素为标量的标量数组而非元素为 RGB 三元组的数组中。这样的灰度图像通常获取自现实世界中，且灰度值与红色、绿色和蓝色分量都相关，这些分量值可用黑白摄像机来度量。或者，灰度值也可理解为我们在黑白电视机中看到的值。 【176~177】

灰度值与标准 RGB 分量之间的关系可用如下基于感知的方程来定义：

$$L = .30R + .59G + .11B$$

由上式可以推知，我们在单色显示器上所看到的内容主要是受绿色分量支配的。该方程同时由显示器的属性(如 CRT 的磷光质涂层的长度)和我们的视觉系统的属性所决定。我们的视觉系统对蓝色有着较低的敏感度，这是由于蓝色位于可见光谱的末端，而对绿色却非常敏感。所以，为了正确地处理灰度，当需要在灰度和 RGB 之间进行转换时，可用正确的比例因子对像素值进行缩放。

7.9 像素映射

RGB 颜色表示法的一个缺陷是它没有考虑各种显示技术中所使用的 RGB 系统之间的差异，这些显示技术包括 CRT、电影胶片 and 投影设备。颜色缓存中的同一个 RGB 像素在两种不同的输出设备上的显示效果可能会存在差异，而这种差别可能是由胶片染料和 CRT 磷光质涂层之间的差异造成的。在 OpenGL 中，可以通过像素映射(pixel mapping)来对这些问题做出补偿。该特性允许每个像素依据一张用户定义的表来进行修改。我们可以各种模式来运用这一特性。可为红色分量、绿色分量、蓝色分量及 alpha 分量分别使用 4 张独立的表。这样，当蓝色值准备写入帧缓存中时，与蓝色分量对应的表就会对每种蓝色进行修改。如果希望

使用该颜色映射，首先应调用以下函数：

```
glPixelTransferi(GL_MAP_COLOR, GL_TRUE);
```

接下来用函数 `glPixelMap*v()` 建立映射关系。

```
void glPixelTransfer<if>(GLenum name, type value)
```

该函数将像素变换模式参数 `name` 设为 `value`。该函数可用来启用颜色映射 (`GL_MAP_COLOR`)、比例变换及为缩放提供偏置。

```
void glPixelMap<ui us f>v(GLenum map, GLint size, type * array)
```

该函数为具有类型 (`unsigned int`、`unsigned short` 或 `float`) 的像素建立由 `map` 指定的像素映射表。该映射表的长度为 `size`，表的内容保存在数组 `array` 中。映射表 `map` 的值包括 `GL_PIXEL_MAP_I_TO_R` (或 `G、B`) 和 `GL_PIXEL_MAP_R_TO_R` (或 `G_TO_G、B_TO_B`)。前者用来建立颜色索引与颜色的映射关系，而后者用来重新调整 RGB 颜色的比例变换。`size` 必须为 2 的整数次幂。【177】

一般说来，尺寸参数由为像素的每个分量分配的内存大小所决定。所以，如果每个 RGB 分量都用一个字节来表示，而且希望建立蓝色分量与蓝色分量之间的映射，则可能看到以下函数调用形式：

```
float bluemap[256]
glPixelMapfv(GL_PIXEL_MAP_B_TO_B, 256, bluemap);
```

这里，位于映射关系输入端和输出端的蓝色值通常位于区间 (0, 1) 内。建立红色和绿色分量的映射关系的代码也与上述代码非常相似。

使用像素映射的另一个常见例子是伪彩色 (`pseudocolor`)。即假定有一幅灰度图，而我们希望用彩色来表现不同的灰度值。这项技术与索引颜色的使用非常类似，但是，这种处理只针对应用程序内部有意义的灰度值进行。一种典型的用法如下所示：

```
for(i = 0; i < 256, i++)
{
    redmap[i] = i/255.0;
    if(i < 128) greenmap[i] = i/128.0
    else greenmap[i] = 1.0 - (i - 128.0)/127.0;
    blue_map[i] = 1.0-i/255.0;
}
glPixelMapfv(GL_PIXEL_MAP_I_TO_R, 256, redmap);
glPixelMapfv(GL_PIXEL_MAP_I_TO_R, 256, greenmap);
glPixelMapfv(GL_PIXEL_MAP_I_TO_R, 256, bluemap);
```

在上述代码中，黑色被映射为蓝色，白色被映射为红色，而位于中间的灰色被映射为具有不同明暗度的绿色。

也可通过 OpenGL 变换来改变 RGB 和 RGBA 颜色，具体做法是将矩阵模式设为 `GL_COLOR`，

并在形成模型视图矩阵和投影矩阵之后形成该变换矩阵。

【178】

7.10 像素的缩放

在操作像素时，大多数几何变换，如旋转和平移，都不再有效，因为像素是离散实体，而且必须位于帧缓存中的固定位置。如果希望将一个像素块进行旋转，我们只能在显示器上近似表达旋转后的外观。但是，沿着窗口坐标轴方向的比例变换是有意义的。例如，假定现有一个光栅字符集，我们希望将这些字符放大两倍，则可以复制(replicate)这些像素。即，可将字体中的每个像素绘制4次，如图7.10所示。

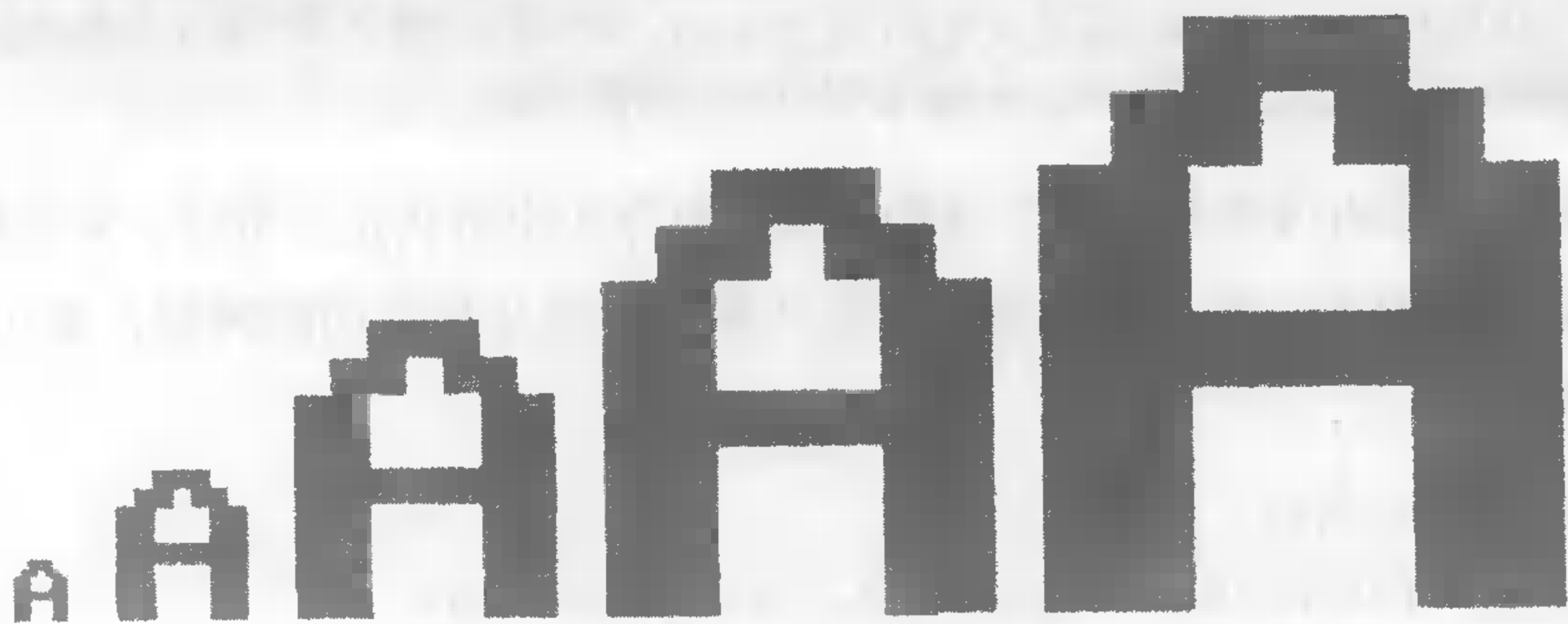


图 7.10 复制各像素以形成更大的字符

为了达到上述目的，OpenGL 提供了函数 `glPixelZoom()`。可利用该函数来实现像素的复制，以将图像放大。也可用它来减小图像的尺寸。如果比例因子为负，则图像将相对于显示器中对应的方向发生翻转。

```
void glPixelZoom(GLfloat sx, GLfloat sy)
```

该函数在像素被绘制时依据比例因子 `sx` 和 `sy` 将像素放大或缩小。如果比例因子为负，则除了要进行比例缩放外，图像还将沿着相应的方向翻转。像素的缩放不适用于位图。

注意，虽然这里允许采用小数作为因子，但当这些像素绘制出来后，用户可能会注意到图像中的一些缺陷。回顾一下前面关于 PPM 文件头的那个例子，当时为了说明 OpenGL 显示像素的原理，我们在读取文件时将图像作了翻转和镜像变换。其实完全可以像下面这样按顺序读取这些像素：

```
for(i = 0; i < nm, i++)
{
    fscanf(fd, "%d %d %d", &red, &green, &blue);
    image[3 * i] = red;
    image[3 * i + 1] = green;
    image[3 * i + 2] = blue;
}
```


然后对像素进行缩放:

```
glPixelZoom(-1.0, -1.0);
```

在某些应用程序中,可能会有我们实际需要的图像尺寸与已有图像的尺寸不一致的情形。例如,在第8章中处理纹理时,所有纹理图的大小都应为2的整数次幂。当然可以用C代码和OpenGL函数联合创建出这些图像,但是OpenGL实用库为我们提供了一个更为方便的辅助函数。

【179】

```
void gluScaleImage(GLenum format, GLint win, GLint hin,
                  GLenum typein, void * imagein, GLint wout,
                  GLint hout, GLenum typeout, void * imageout)
```

该函数将接收一幅 win×hout 的图像 imagein, 其格式为 format, 类型为 type, 并生成一幅大小为 wout×hout、格式为 format 且类型为 typeout 的图像 imageout。

7.11 OpenGL 中的图像处理

利用 OpenGL 提供的如此丰富的功能,可以开始编写许多与数字图像处理相关的运算了,这些运算包括直方图的计算、滤波(或卷积)及调色板的操作。虽然这些运算都可以在用户程序中来实现,但我们更愿意借助 OpenGL 的处理能力,毕竟它们是在硬件中实现的。其中的一条理由就是我们希望利用许多商业显卡内置的对图像处理功能的硬件支持。另外一条理由是我们希望避免图像在处理器内存和显卡内存中的频繁转移。

图像处理系统的操作对象通常为像素流,所实现的运算如下。

- 颜色查找表——用于将输入像素值(颜色值或灰度值)映射为其他值。
- 卷积或滤波——用某个像素领域中像素值的线性组合来取代该像素的值。
- 颜色矩阵运算——将 RGBA 颜色值与一个 4×4 的矩阵相乘,从而创建另外一种颜色。
- 直方图计算——计算各个像素值在一幅图像中的分布情况。

在 OpenGL 中进行图像处理的一种方法是创建一个使用了前面所介绍的像素函数的函数集。该工作的大部分内容已完成,它们构成了 OpenGL 的图像处理功能子集,关于该部分内容的详细介绍可以在《OpenGL 编程指南》一书中找到。这些函数是对 OpenGL 的一种扩展,所以并非得到所有的 OpenGL 实现的支持。但是,由于它们如此有用,实际上已有许多系统对其提供了支持。这些函数都基于图 7.11 所示的图像处理流水线。

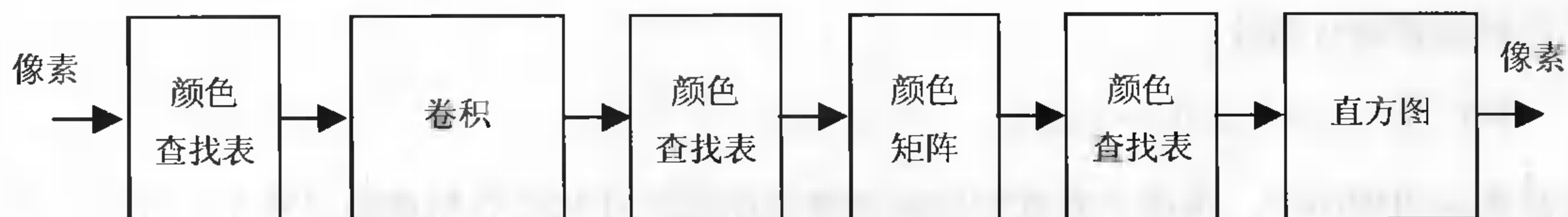


图 7.11 图像处理流水线

图像处理的另一个应用是纹理映射(将在第 8 章介绍)的使用。纹理映射整合了像素和几何流水线, 并通过首先将图像转为纹理图而允许直接对像素进行各种变换。【180~181】

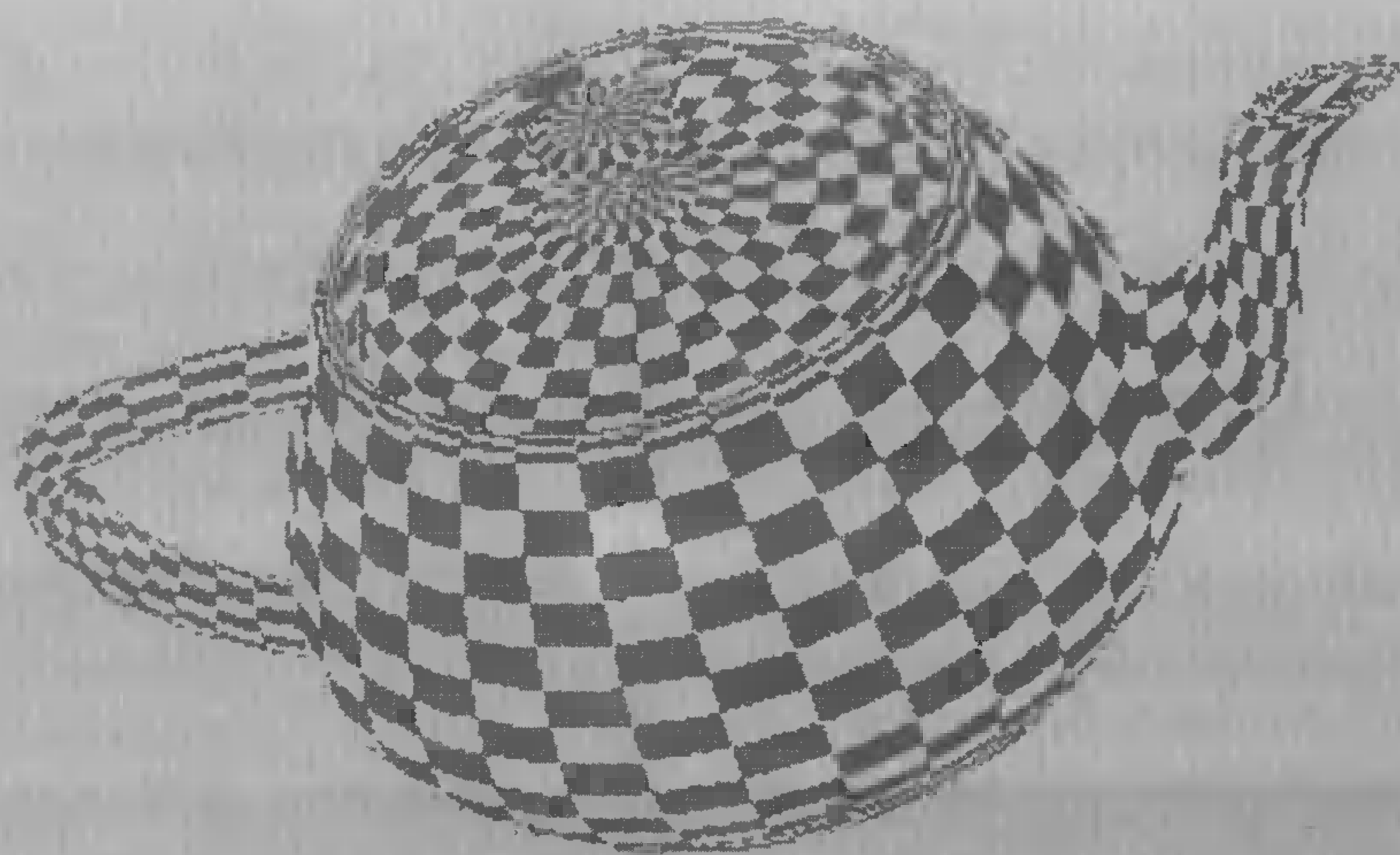
7.12 编程练习

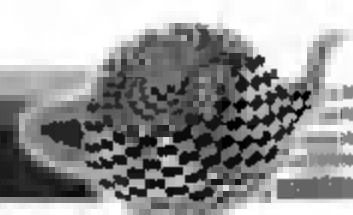
1. 如果读者在第 3 章最后的练习中编写了一个绘制程序, 接下来请使用逻辑运算为该程序增加橡皮条功能。例如, 如果希望绘制一条直线段, 可以通过单击鼠标左键来确定直线段的一个端点, 只要左键一直处于按下状态, 程序便从左键最初按下的位置到鼠标光标目前所在的位置画出一条新线段。释放鼠标左键后, 鼠标的最终位置决定了直线的第二个端点的位置。对于圆和矩形, 也使用相似的策略。
2. 一幅图像的直方图实际上就是表示图像中每个灰度或某个颜色分量在图像中出现的次数的一个表格。编写一个程序来计算一幅具有 256 级灰度的灰度图像的直方图。利用该直方图构造一个查找表, 以便在该表应用于同一幅图像时, 所生成的图像具有相同的直方图。
3. 可以借助查找表来修改灰度值或颜色分量的分布, 以达到增强许多图像显示效果的目的。编写一段程序, 实现允许用户交互式地修改调色板中的值。一种可行的方法是允许用户为用一个 3~4 段的分段线性曲线来表示输入和输出值之间的映射关系, 然后再用像素映射来逼近这些曲线。

第 8 章

纹理映射

纹理映射(texture mapping)通过将像素与几何对象结合,为图像营造出一种非常复杂的视觉效果,避免了建立大量几何模型所需的开销。要实现纹理映射,需要同时使用几何流水线和像素流水线。虽然将纹理映射应用在对象上的原理非常简单,要想达到较好的效果仍然需要对各种参数进行精心设置。我们先从一个简单的例子开始,然后在后面几节中陆续丰富其功能。





8.1 什么是纹理映射

到目前为止,我们已经了解了两种在屏幕上显示对象的方法。我们既可以建立该对象的几何模型(通常使用三维多边形来建模),然后将其送入几何流水线;也可用像素块来表示对象。每种方法都有自身的局限性。像素可以表现局部细节,但却不具备三维属性。虽然在硬件中实现了几何流水线的系统可以达到每秒处理上百万计的多边形,但在模拟许多自然现象(如火焰、草地、水波或云彩)时,这些系统处理多边形的速度仍然难以满足要求。

然而我们所要做的就是用一种称为纹理映射的方法将每种方法最突出的特性结合起来。假定我们现有一个 $n \times m$ 的像素数组。我们并不将其看作有离散元素构成的数组,而是将其视作一个连续数组。该数组中的任意一点通过变量 s 和 t 来定义。所以,如图 8.1 所示,对于每个坐标对 (s, t) , 都有一个与之对应的像素值,该值通常是灰度值或 RGB 或 RGBA 值。这个连续数组就是一个二维纹理。在提到原始元素时,我们称其为纹理元(texel)而非像素。【183】

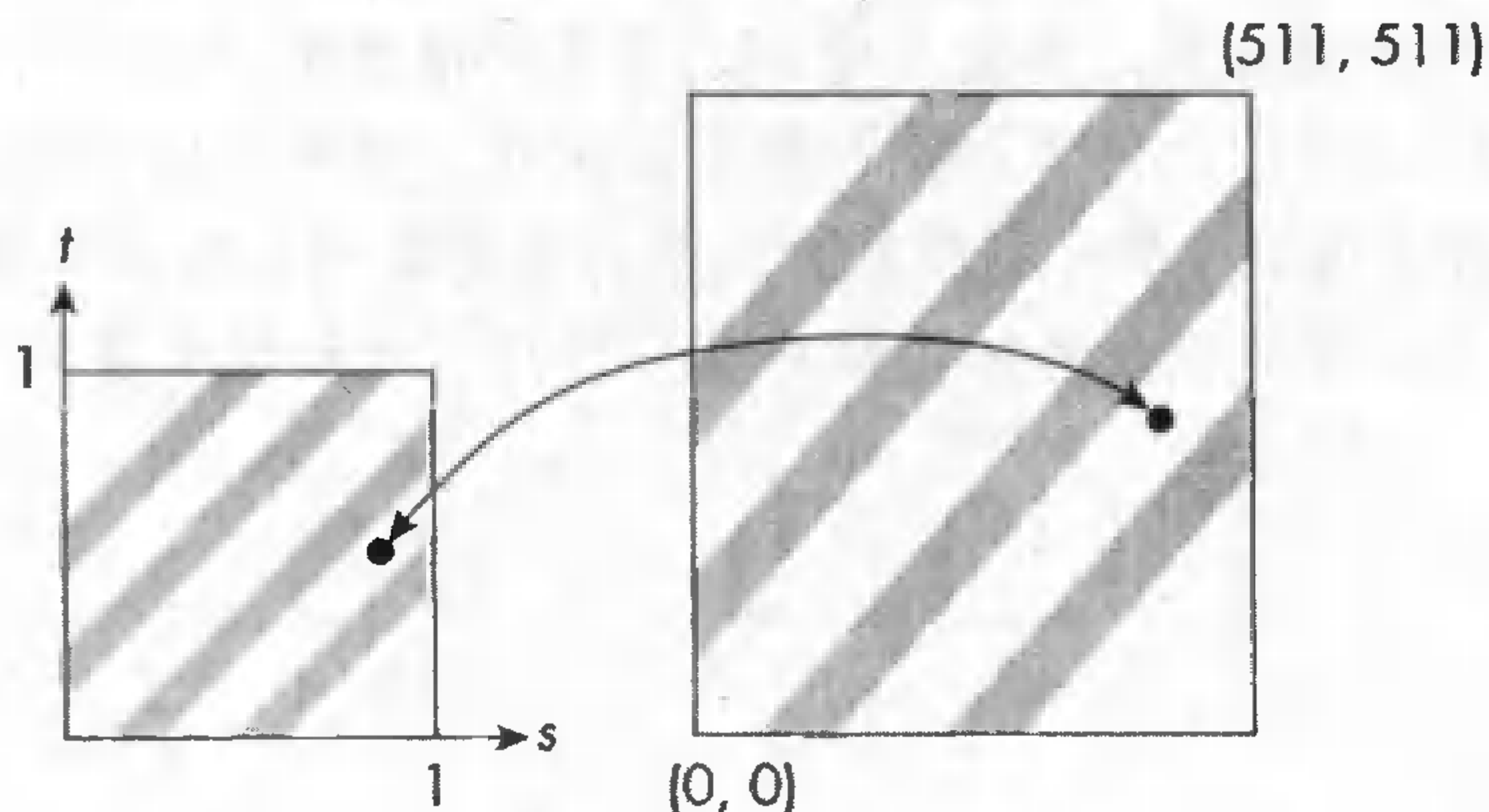


图 8.1 作为 (s, t) 空间中连续图像的纹理图和作为离散图像的纹理图

现在我们来考虑三维空间中的一个几何对象。其表面上的每一点都对应于三维世界坐标系中的一个坐标 (x, y, z) 。如果我们能够通过一对函数将对象坐标系中的每一点 (x, y, z) :

$$\begin{cases} s = f(x, y, z) \\ t = g(x, y, z) \end{cases}$$

都与纹理坐标系中的一点 (s, t) 建立关联,我们就可用纹理图像中的颜色或灰度值来确定对象表面上可见点的颜色。该过程如图 8.2 所示,它反映了纹理映射的本质。

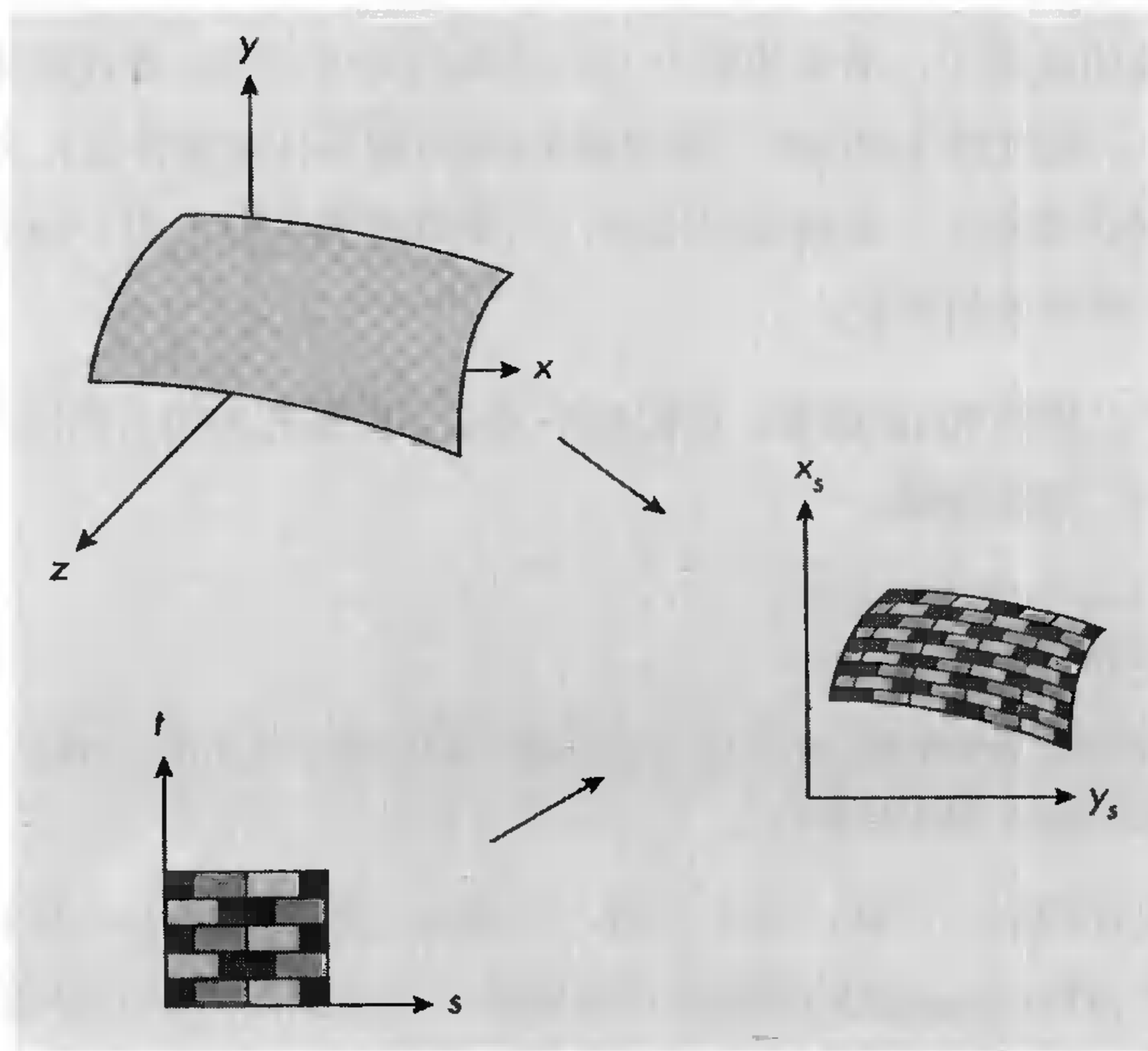


图 8.2 将纹理与表面结合起来, 并对窗口坐标系中的像素着色

至此, 纹理映射看起来还是比较抽象, 也没有将它与 OpenGL 如何处理像素和几何对象联系起来。主要的困难在于像素(纹理元)和几何图元通过的是不同的流水线, 而且在需要纹理值的时候, 我们却处在片段处理环节。所以, 对于如何匹配表面上的点与纹理元并不清楚。OpenGL 对于该问题的解决方式是强制应用程序为每个顶点都定义纹理坐标。更具体地讲, 纹理坐标像颜色和法线一样, 也是 OpenGL 状态的一部分, 而且当某个顶点定义后, 它使用的是当前纹理坐标。所以, 我们从不显式地定义函数 f 和 g 。我们的解决方案是通过为顶点定义纹理坐标来在程序中提供一些采样值。OpenGL 借助插值来确定每个片段在光栅化期间所需的纹理坐标。

8.2 纹理图的创建

纹理图(texture map)的使用有三个基本步骤。首先, 必须明确将要用于纹理映射的图像。该图像可以是第 7 章介绍的读入的图像, 也可是在应用程序内部定义的图像。纹理图还可以是从 OpenGL 的某一颜色缓存中复制得到的图像。其次, 需要指定一些确定纹理应用方式的参数。最后, 必须在程序中为每个顶点定义纹理坐标。【184~185】

但是, 与其他 OpenGL 功能一样, 有许多选项既会对处理的效率造成影响, 也会对图像的外观造成影响。下面我们将研究一个简单的例子, 然后再来讨论这些选项。

OpenGL 同时支持一维、二维和三维纹理映射。其中二维纹理映射最为常见。这里我们将图

像映射到几何对象的表面上,如多边形上。但是我们也可以用一维纹理来为线段或曲线创建一种颜色模式。三维纹理使用的是三维纹理元(有时称为体素或体元)。然后 OpenGL 将该纹理体元的值映射到表面上。从概念层次讲,三维纹理图类似于用一块具有特定纹理的材质来雕刻出一个三维对象的形貌。

假定程序中有一个二维的 RGB 图像。我们暂不关心该图像的来源。我们可以通过如下形式将该图像设为一个二维纹理图:

```
GLubyte myimage[64][64][3];
glEnable(GL_TEXTURE_2D);

glTexImage2D(GL_TEXTURE_2D, 0, GL_RGB, 64, 64, 0, GL_RGB,
GL_UNSIGNED_BYTE, myimage);
```

【185~186】

首先,必须启用纹理映射。注意,对于一维、二维和三维纹理映射,我们必须分别启用。其次,我们用函数 `glTexImage2D()` 来指定纹理图像。该函数的大部分参数的含义都很清楚。尽管如此,我们还是按顺序来研究一下这些参数。

- **GL_TEXTURE_2D** 该参数通知 OpenGL 我们指定的是一个二维纹理。该参数只有另外一个可选项,即 `GL_PROXY_TEXTURE_2D`,该选项在我们想确认是否还有可用资源来支持我们希望使用的纹理时非常有用。
- **0** 该参数负责设置纹理图的细节层次。OpenGL 支持一种称为多级渐进纹理的机制(稍后我们将进行讨论),该机制使得我们可以利用一系列具有不同分辨率的纹理图像来提高分辨率。**0** 代表最高的细节层次,当我们不希望使用多级渐进纹理时,都可将该参数设为 **0**。
- **GL_RGB** 该参数给出了图像数据的内部格式。常量 `GL_RGB` 把图像数据视为来自某个 RGB 图像。其他值允许我们将图像数据指定为灰度形式(`GL_LUMINANCE`)、RGBA 形式(`GL_RGBA`)或存储图像数据的其他格式(这些格式中,每个分量不是用 8 位来表示)。
- **64** 纹理中的行数。行数和列数必须为 2 的整数次幂。^①
- **64** 纹理的列数。
- **0** 边界宽度。一幅纹理图可以在四周具有一个像素宽的边界,以便使用纹理滤波来创建平滑的纹理图。

① 虽然 2 的整数次幂是 OpenGL 规范中的一部分,比较新的显卡目前都无此限制。

- **GL_RGB**: 所要使用的纹理元的类型。对于像素数据, 该参数和它之后的参数具有相同的选项, 这已在第7章讨论过。
- **GL_UNSIGNED_BYTE**: 所要使用的纹理元的格式。
- **myimage**: 指向图像数据构成的数组的指针。 【186】

```
void glTexImage2D(GLenum target, GLint level, GLint iformat,
                  GLsizei width, GLsizei height, GLint border,
                  GLenum format, GLenum type, GLvoid * texels)
```

该函数依据类型为 **type**, 格式为 **format** 的数组 **texels** 创建一个二维的大小为 **height**×**width** 的纹理。纹理的边界被指定为 **b** 个纹理元宽。图像数据的格式为 **iformat**。level 用于多级渐进纹理。

虽然该函数中有许多参数, 但这里大部分参数的用法都与图像处理函数中参数的用法类似。纹理映射从应用程序中提取图像数据, 然后将这些数据送入像素流水线, 就像绘制像素时所做的一样。所以, 在提取图像数据并将其放入纹理内存之前, OpenGL 必须知道这些数据在处理器内存中的格式。因此, 我们可能必须通过函数 **glPixelStore*()** 来设置一些选项。但在像素流水线的末端有一些不同。首先, 纹理元进入的是纹理内存而非帧缓存。依赖于具体实现, 纹理内存可能是用于帧缓存的同一段内存中的一部分, 或者它也可能是一段专门的内存。但与帧缓存一样, OpenGL 可以用自己的格式来存储纹理元, 于是对于较大的纹理, 将纹理元从应用程序中转移到纹理内存中可能会耗费大量的时间。此外, 纹理内存的可用资源可能会十分有限。比较新的显卡中的纹理内存一般都有许多 MB。8.9 节将介绍 OpenGL 通过纹理对象可以支持多种纹理。

提示 纹理图像的维数必须为 2 的整数次幂。

提示 要将纹理图像从处理器内存中转移到纹理内存中, 可能会花费大量的时间。

我们可以类似的方式借助函数 **glTexImage1D()** 和 **glTexImage3D()** 来分别创建一维和三维纹理图。每种纹理必须分别启用 (**GL_TEXTURE_1D** 或 **GL_TEXTURE_3D**)。 【187】

```
void glTexImage1D(GLenum target, GLint level, GLint iformat,
                  GLsizei width, GLint border, GLenum format,
                  GLenum type, GLvoid * texels)
void glTexImage3D(GLenum target, GLint level, GLint iformat,
                  GLsizei width, GLsizei height, GLsizei depth,
                  GLint border, GLenum format, GLenum type,
                  GLvoid * texels)
```

这两个函数分别用来创建一维纹理图和三维纹理图。

8.3 纹理坐标

几何对象上的点与纹理元之间的映射是通过函数 `glTexCoord*()` 来完成的。纹理坐标也是 OpenGL 状态的一部分。像顶点一样，纹理坐标的内部表示形式也是四维的，通常我们用字母 (s, t, r, q) 来表示纹理坐标。如果我们指定的维数小于 4，则默认情况下有 $t=r=0, q=1$ 。

```
void glTexCoord<1234><sifd>(type scoord, type tcoord, ...)
```

```
void glTexCoord<1234><sifd>v(type * vcoord)
```

这些函数可将当前纹理坐标设为 `scoord`, `tcoord`……或通过数组 `vcoord` 来指定当前纹理坐标。

在二维情况下，我们假定在 `glTexImage2D()` 中指定的纹理元数组在一个连续矩形中，该矩形中任意一点的纹理坐标都位于区间 $(0, 1)$ 内。所以，即使该数组的行数和列数不同，其左下角的纹理坐标仍然为 $(0, 0)$ 而右上角的纹理坐标仍然为 $(1, 1)$ ，如图 8.3 所示。

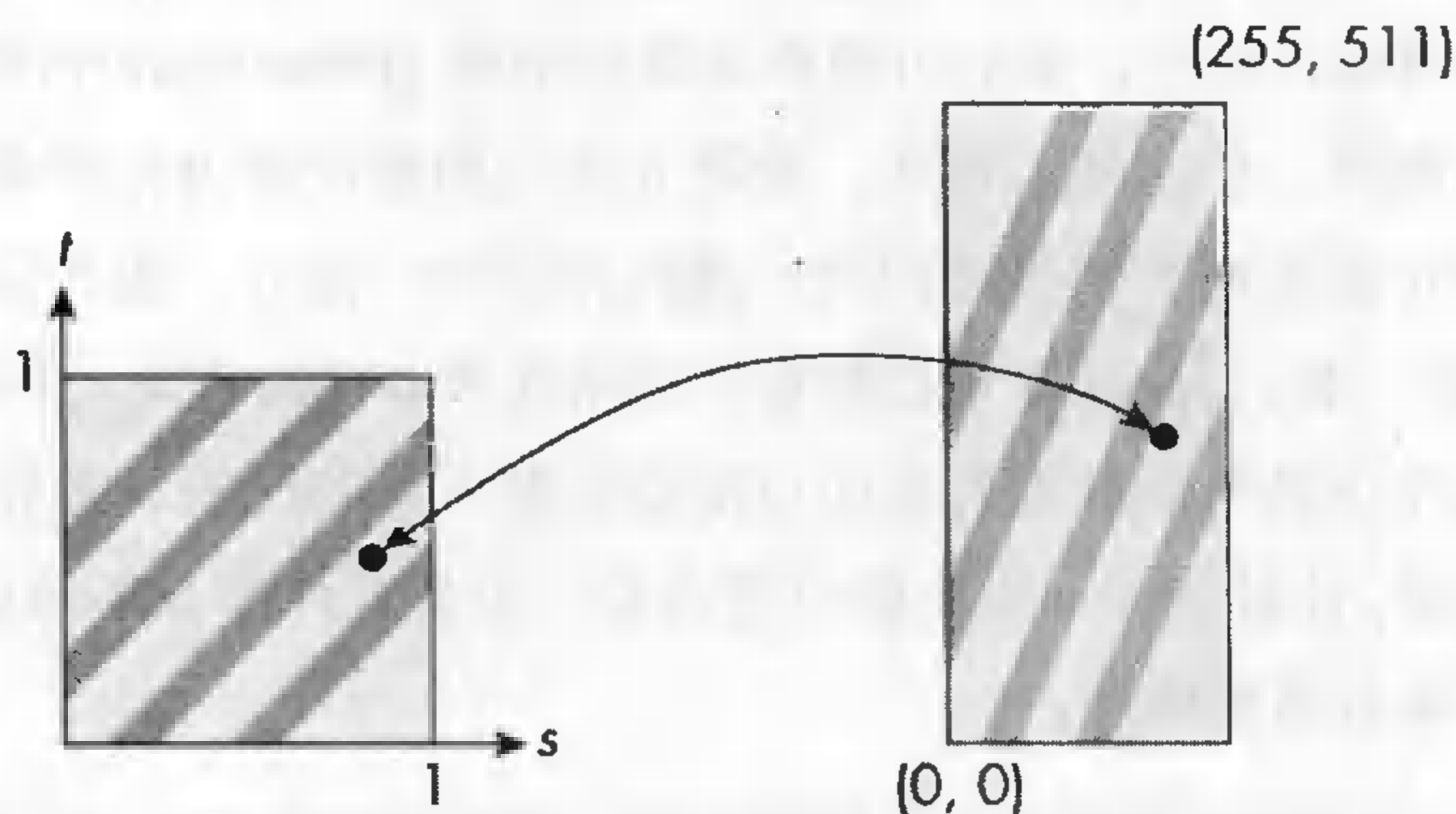


图 8.3 一幅 256×512 图像的纹理坐标与纹理数组

现在假定我们要将整个纹理映射到一个四边形中。在指定每个顶点之前，我们先为其指定纹理坐标：

```
glBegin(GL_QUADS);
    glTexCoord2f(0.0, 0.0);
    glVertex3fv(vertex[0]);
    glTexCoord2f(0.0, 1.0);
    glVertex3fv(vertex[1]);
    glTexCoord2f(1.0, 1.0);
    glVertex3fv(vertex[2]);
    glTexCoord2f(1.0, 0.0);
    glVertex3fv(vertex[3]);
glEnd();
```

【188】

我们已经有效地为每个顶点指定了一个纹理坐标。在绘制期间，OpenGL 利用对纹理元数组

中的值进行插值以获得内部点的纹理值。该方法类似于 OpenGL 构造平滑的颜色和明暗的方式。但是, 由于纹理元都是离散的, 我们可能还是能看出一些人工处理的痕迹。OpenGL 提供了一些参数可用来帮助我们对插值、纹理采样以及纹理平滑进行控制。下一节将讨论这些参数。

图 8.4 展示了映射到四边形中的棋盘纹理的两个例子。棋盘图案对于说明各种选项非常有用, 从中我们可以清晰地了解 OpenGL 到底是如何进行纹理映射的。如果该四边形正对着摄像机(或观察方向), 我们将得到左边的图像。如果移动该四边形的一角, 则该四边形将不再是矩形, 在保持纹理坐标不变的条件下, 我们将得到右边的图像。在这两种情形中, 纹理图都将被拉伸以与四边形匹配, 而内部值则需要借助插值从纹理图中得到。但是, 从图 8.4 可见, 内部插值将导致纹理的畸变。我们看到 OpenGL 将该四边形绘制成了两个三角形, 所以当纹理映射到图 8.4 中右边的多边形中时, 纹理图案发生了显著的畸变。当然, 我们可以为该四边形的各顶点选择不同的纹理坐标而避免这样显著的畸变。但是, 对于如何将纹理坐标与顶点匹配实际上还是不清楚。

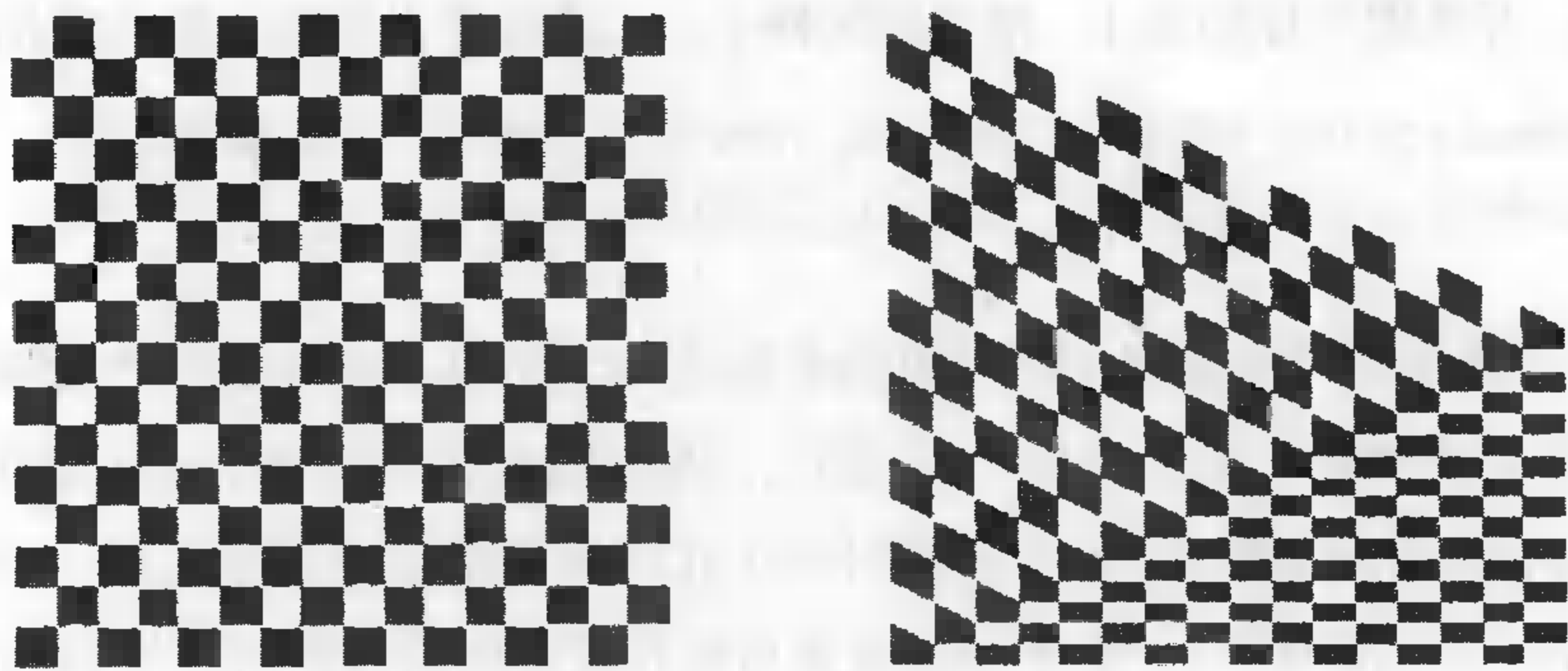


图 8.4 将棋盘纹理映射到一个四边形中

【189~190】

纹理坐标的内部存储形式是四维的, 这与顶点的存储方式一致。为了实现与顶点关系的对等, 也有一个 4×4 的纹理矩阵可用来与纹理坐标相乘而实现纹理坐标的各种变换。该矩阵的初始值为单位阵。我们可通过设置矩阵模式:

```
glMatrixMode(GL_TEXTURE);
```

以及使用与我们运用到模型视图矩阵和投影矩阵的同样的技术来修改该矩阵。于是, 我们就可用纹理矩阵通过纹理坐标的变换来实现给定纹理的旋转、平移以及比例变换。

8.4 纹 理 参 数

虽然函数 `glTexCoord*()` 和 `glTexImage*()` 是与纹理映射有关的核心函数, 但我们仍然必须对一些附加的必要参数进行设置。此外, 还有一些可选参数有助于我们对如何将纹理映射到

对象表面更好地进行控制。这些参数都是通过函数 `glTexParameter*()` 设置的。

```
void glTexParameter<if>(GLenum target, GLenum name, type value)
void glTexParameter<if>v(GLenum target, GLenum name, type * value)
这些函数为类型为 target(GL_TEXTURE_1D、GL_TEXTURE_2D 或 GL_TEXTURE_3D) 的纹理将参数
name 设为 value。
```

这些必要的参数决定了当 s 、 t 、 r 或 q 的值超出区间 $(0, 1)$ 时应做何处理以及如何运用采样和滤波。

虽然在纹理图中我们假定纹理坐标的各个分量都位于区间 $(0, 1)$ 中，但实际上在函数 `glTexCoord*()` 中完全可以指定任意值。我们可用参数 `GL_TEXTURE_WRAP_*` 来通知 OpenGL 如果这些值超出假设的范围应如何处理。遇到这种情况时，有两种处理方式。其一，重复该值(`GL_REPEAT`)；即，如果该值为正，则直接使用其小数部分，若该值为负，则取与其相加和为正的最小整数。我们可为每个纹理坐标分别作出决策。其二，我们可以对纹理坐标进行箝位处理(`GL_CLAMP`)，强制其位于 $(0, 1)$ 区间内；即，将如果纹理坐标值为负，则强制其取 0，若纹理坐标超过 1，则强制其取 1。二维纹理坐标的一种典型用法如下：

```
glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_S, GL_REPEAT);
glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_T, GL_REPEAT);
```

纹理图是在片段处理阶段映射到几何对象的表面上的。所以，纹理图并不是真的映射到几何对象的表面上，而使映射到一个片元(片段)上，而片段是表面的一个小区域在窗口坐标系中的投影。从另外一个角度看，每个像素对应于几何表面上的一个小区域，同时也对应于纹理坐标系中的一个小区域，后者有时被称为与该片段对应的像素的原像(preimage)。依据纹理坐标值、表面尺寸以及观察条件的不同，每个纹理元可能会覆盖多个像素(放大)，每个像素也可能会覆盖多个纹理元(缩小)，如图 8.5 所示。 【190~191】

这种情况更为复杂，因为像素和纹理元不一定正好是对齐的，而且当我们需要处理透视或曲面时，纹理元的原像并不是矩形的。对于该问题，最简单的解决方案就是借助点采样(point sampling)。每个像素的中心都被(通过插值)映射到纹理坐标系中的一点上，然后 OpenGL 使用该点处的值。虽然该方法既简单，效率又高，但是它可能造成令人无法接受的视觉效果，如走样误差。如果用采样点邻域中的一组纹理元的均值来作为该采样点处的纹理值，则会取得比较平滑的效果。这里的平均采用的是线性滤波的形式。OpenGL 用一个 2×2 的由采样点邻域中的纹理元构成的数组作为滤波器。我们需要为放大和缩小分别指定这些选项。这些参数分别是 `GL_TEXTURE_MAG_FILTER` 和 `GL_TEXTURE_MIN_FILTER`，其对应值可从 `GL_NEAREST` 和 `GL_LINEAR` 中选择。如果我们实现的速度尽可能的快，则应在初始化步骤中按如下方式对这些参数进行设置：


```
glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MAG_FILTER, GL_NEAREST);
glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MIN_FILTER, GL_NEAREST);
```

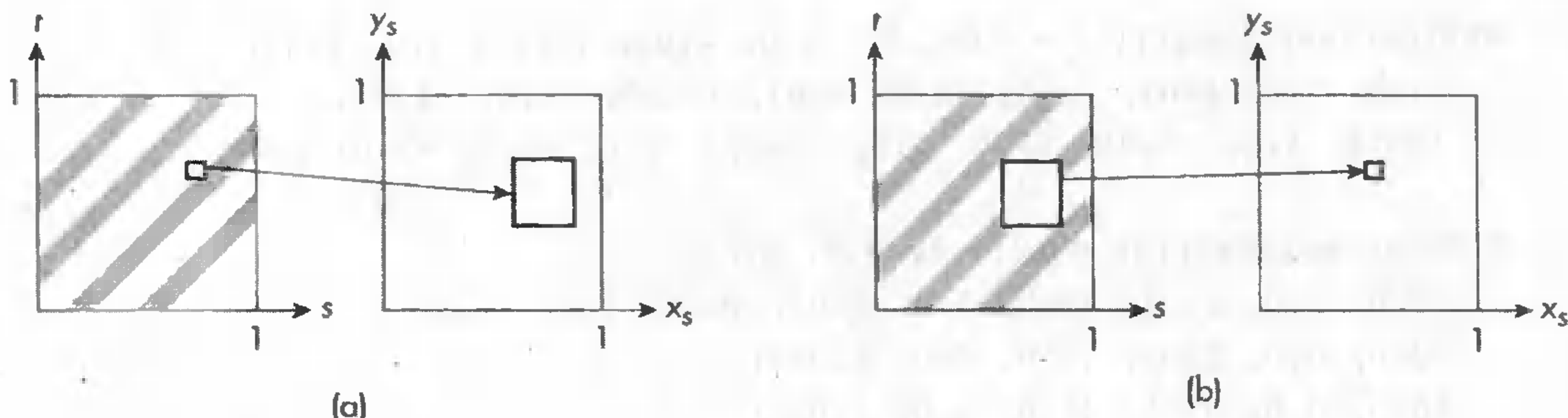


图 8.5 (a)放大; (b)缩小

纹理映射为编程人员协调处理效率和纹理滤波之间的关系提供了许多选择。透视观察是一个较难处理的问题。实现纹理插值的最快的方法并不能处理由于透视变换造成的形状畸变问题。大部分时候,您可能并不会注意到这个问题,但是当纹理较为规则时,如条状纹理或棋盘格纹理,则该问题就会比较突出。一些 OpenGL 实现能够依据透视的条件对绘制器进行调整。函数 `glHint()` 可用于请求 OpenGL 实现生成一幅具有较好视觉效果图像,即便绘制器的绘制速度较慢,也没有关系。^①

```
glHint(GL_PERSPECTIVE_CORRECTION_HINT, GL_NICEST);
```

【191~192】

该函数的第 2 个参数除了可取 `GL_NICEST` 外,还可取 `GL_FASTEST`。较新的显卡一般都支持透视校正纹理映射,而丝毫不会带来性能上的降低。

```
void glHint(GLenum option, GLenum hint);
```

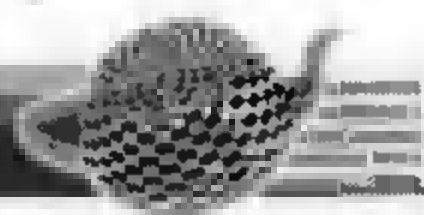
该函数要求将 `hint` 应用于 `option`。

可为其他绘制选项,如直线和多边形的反走样(`GL_LINE_SMOOTH_HINT`, `GL_POLYGON_SMOOTH_HINT`)请求线索(`hint`,即告知 OpenGL 您最关心的是显示效果还是运行效率)。

8.5 一个带有纹理的旋转立方体

下面我们来介绍一个简单的示例程序,该程序将一个棋盘格纹理映射到一个旋转立方体。旋转立方体在前几章已经介绍过。注意,如果我们只使用标准的默认参数设置,实现纹理映射只需要极少的工作。

^① 现代显卡在硬件中提供了许多选项,所以采用纹理滤波时透视校正可能是自动进行的,而且这样不会造成性能上的损失。



```
#include <stdlib.h>
#include <GL/glut.h>

GLfloat vertices[][3] = {{-1.0, -1.0, 1.0}, {-1.0, 1.0, 1.0},
    {1.0, 1.0, 1.0}, {1.0, -1.0, 1.0}, {-1.0, -1.0, -1.0},
    {-1.0, 1.0, -1.0}, {1.0, 1.0, -1.0}, {1.0, -1.0, -1.0}};

GLfloat colors[][3] = {{{1.0, 0.0, 0.0},
    {0.0, 1.0, 1.0}, {1.0, 1.0, 0.0}, {0.0, 1.0, 0.0},
    {0.0, 0.0, 1.0}, {1.0, 0.0, 1.0}};
    {0.0, 0.0, 0.0}, {1.0, 1.0, 1.0}};

void polygon(int a, int b, int c, int d)
{
    /* draw a polygon via list of vertices */

    glBegin(GL_POLYGON);
        glColor3fv(colors[a]);
        glTexCoord2f(0.0, 0.0);
        glVertex3fv(vertices[a]);
        glColor3fv(colors[b]);
        glTexCoord2f(0.0, 1.0);
        glVertex3fv(vertices[b]);
        glColor3fv(colors[c]);
        glTexCoord2f(1.0, 1.0);
        glVertex3fv(vertices[c]);
        glColor3fv(colors[d]);
        glTexCoord2f(1.0, 0.0);
        glVertex3fv(vertices[d]);
    glEnd();
}

void colorcube()
{
    /* map vertices to faces */
    polygon(0, 3, 2, 1);
    polygon(2, 3, 7, 6);
    polygon(3, 0, 4, 7);
    polygon(1, 2, 6, 5);
    polygon(4, 5, 6, 7);
    polygon(5, 4, 0, 1);
}
```



```

}

static GLfloat theta[] = {0.0, 0.0, 0.0};
static GLint axis = 2;

void display(void)
{
    glClear(GL_COLOR_BUFFER_BIT|GL_DEPTH_BUFFER_BIT);
    glLoadIdentity();
    glRotatef(theta[0], 1.0, 0.0, 0.0);
    glRotatef(theta[1], 0.0, 1.0, 0.0);
    glRotatef(theta[2], 0.0, 0.0, 1.0);

    colorcube();

    glutSwapBuffers();
}

void spinCube()
{
    theta[axis] += 2.0;
    if(theta[axis] > 360.0) theta[axis] -= 360.0;
    glutPostRedisplay();
}

void mouse(int btn, int state, int x, int y)
{
    if(btn == GLUT_LEFT_BUTTON & state == GLUT_DOWN)
        axis = 0;
    if(btn == GLUT_MIDDLE_BUTTON & state == GLUT_DOWN)
        axis = 1;
    if(btn == GLUT_RIGHT_BUTTON & state == GLUT_DOWN)
        axis = 2;
}

void myReshape(int w, int h)
{
    glViewport(0, 0, w, h);
    glMatrixMode(GL_PROJECTION);
    glLoadIdentity();
    if(w <= h)
        glOrtho(-2.0, 2.0, -2.0 * (GLfloat)h / (GLfloat)w,
                2.0 * (GLfloat)h / (GLfloat)w, -10.0, 10.0);
    else

```



```

        glOrtho(-2.0 * (GLfloat)w / (GLfloat)h,
                2.0 * (GLfloat)w / (GLfloat)h, -2.0, 2.0,
                -10.0, 10.0);

    glMatrixMode(GL_MODELVIEW);
}

int main(int argc, char **argv)
{
    GLubyte image[64][64][3];
    int i, j, r, c;
    for(i = 0; i < 64; i++)
    {
        for(j = 0; j < 64; j++)
        {
            c = (((i&0x8) == 0) ^ ((j&0x8) == 0)) * 255;
            image[i][j][0] = (GLubyte)c;
            image[i][j][1] = (GLubyte)c;
            image[i][j][2] = (GLubyte)c;
        }
    }

    glutInit(&argc, argv);
    glutInitDisplayMode(GLUT_DOUBLE | GLUT_RGB |
        GLUT_DEPTH);
    glutInitWindowSize(500, 500);
    glutCreateWindow("colorcube");

    glutReshapeFunc(myReshape);
    glutDisplayFunc(display);
    glutIdleFunc(spinCube);
    glutMouseFunc(mouse);
    glEnable(GL_DEPTH_TEST);
    glEnable(GL_TEXTURE_2D);

    glTexImage2D(GL_TEXTURE_2D, 0, GL_RGB, 64, 64, 0, GL_RGB,
        GL_UNSIGNED_BYTE, image);

    glTexParameterf(GL_TEXTURE_2D, GL_TEXTURE_WRAP_S, GL_CLAMP);
    glTexParameterf(GL_TEXTURE_2D, GL_TEXTURE_WRAP_T, GL_CLAMP);
    glTexParameterf(GL_TEXTURE_2D, GL_TEXTURE_MAG_FILTER,
        GL_NEAREST);
    glTexParameterf(GL_TEXTURE_2D, GL_TEXTURE_MIN_FILTER,
        GL_NEAREST);

```



```
glutMainLoop();
}
```

【192~195】

8.6 将纹理映射到表面

到目前为止,对于在片段处理中纹理是如何被应用的,以及纹理映射如何与几何对象的明暗进行交互,尚未交待清楚。如果运行一下上述程序,您将注意到每点的颜色值实际上是用顶点颜色进行插值得到的颜色和纹理矩阵中颜色混合的结果。但是,这并不是惟一的选择。您也可以尝试通过重新定义这些颜色,以使其具有透明度分量,而将纹理与融合运算结合起来。

```
GLfloat colors[][4] = {{1.0, 0.0, 0.0, 0.5},
    {1.0, 1.0, 0.0, 0.5}, {0.0, 1.0, 0.0, 0.5},
    {0.0, 0.0, 1.0, 0.5}, {1.0, 0.0, 1.0, 0.5},
    {0.0, 1.0, 1.0, 0.5}};
```

接下来将在多边形函数中使用函数 `glColor4f()`, 注意启用融合, 但不启用深度缓存。

```
glEnable(GL_BLEND);
glBlendFunc(GL_SRC_ALPHA, GL_ONE_MINUS_SRC_ALPHA);
```

【195~196】

然后我们将看到纹理和多边形颜色之间的一种更有趣的混合效果。正如第7章所述,融合的结果与操作顺序有关。

融合运算的默认模式称为调制(modulation)。在该模式下,纹理的颜色与每个面片计算得到颜色相乘。此外还可通过函数 `glTexEnv*()` 设置一些其他的方法。最重要的模式是替代(replacement)模式,在该模式下,只靠纹理的颜色来决定我们在帧缓存中看到的颜色。这些模式都是通过函数 `glTexEnv*()` 来设置的。

```
void glTexEnv<if>(GLenum target, GLenum param, type value)
void glTexEnv<if>v(GLenum target, GLenum param, type * value)
该组函数将纹理参数 param 设为 value; target 必须取 GL_TEXTURE_ENV.
```

默认的调制模式等价于执行以下函数:

```
glTexEnvf(GL_TEXTURE_ENV, GL_TEXTURE_ENV_MODE, GL_MODULATE);
```

如果我们使用了参数 `GL_REPLACE` 而非 `GL_MODULATE`, 我们将只使用纹理颜色。此外还有一些其他选项,包括 `GL_BLEND` 和 `GL_DECAL`, 这些参数确定了在使用 alpha 通道或指定纹理环境颜色(代码如下所示)时如何进行融合。

```
glTexEnvfv(GL_TEXTURE_ENV, GL_TEX_ENV_COLOR, color);
```


该函数调用后会实现对象的明暗与纹理颜色的融合。

8.7 边界与尺寸调整

使用线性滤波时,可能要面对的一个最大的困难是,由于在纹理边界上可能缺少一个或多个纹理元,纹理的边界应如何处理。对于该问题的一种解决方案是借助函数 `glTexImage*()` 将纹理图的每个边向外扩展一个纹理元。于是,纹理图的尺寸一定是 $(2^m + 2b) \times (2^n + 2b)$, 其中 b 取 0 或 1。

我们也可通过如下方式指定自动使用的一种边界颜色:

```
glTexParameter3fv(GL_TEXTURE_2D, GL_TEXTURE_BORDER_COLOR, color);
```

OpenGL 要求纹理的尺寸(小于任何边界)为 2 的整数次幂。正如我们在第 7 章指出的,我们可用 GLU 函数 `gluScaleImage()` 对一幅图像进行转换,使其尺寸满足纹理图的要求。我们也可借助函数 `glCopyTexImage2D()` 从帧缓存中的图像获取纹理图。【196】

```
void glCopyTexImage2D(GLenum target, GLint level, GLint iformat, GLint x,
                     GLint y, GLsizei w, GLsizei h, GLint border)
```

该函数从当前绘制缓存中复制一幅 $w \times h$ 的图像。这些参数与 `glCopyPixels()` 函数中的参数相同, `target` 取 `GL_TEXTURE_2D`。除非使用多级渐进纹理,否则参数 `level` 一般都取 0。

也可以使用函数 `glTexSubImage2D()` 将像素复制到一个已有纹理的一个子区域中。

```
void glTexSubImage2D(GLenum target, GLint level, GLint xoffset,
                    GLint yoffset, GLsizei w, GLsizei h, GLenum format,
                    GLenum type, GLvoid * texels)
```

该函数将一个由 `type` 和 `format` 描述的纹理元数组复制到起始点位于 $(xoffset, yoffset)$ 的纹理内存中。

函数 `glCopyTexSubImage2D()` 将取代帧缓存中的纹理的一个子区域。

```
void glCopyTexSubImage2D(GLenum target, GLint level, GLint xoffset,
                        GLint yoffset, GLint x, GLint y, GLsizei w,
                        GLsizei h)
```

该函数从 (x, y) 开始将一个 $w \times h$ 的像素数组复制到纹理内存中。其余参数与函数 `glTexSubImage2D()` 的参数完全一致。

8.8 多级渐进纹理

使用纹理图的困难之一在于当一个较大的纹理数组对应于图像中的一个像素时,只采用滤波这种方法并不能带来良好的近似效果。例如在透视观察条件下,当一个小物体距离摄像

机较远时就会出现这种情形。此外, 无论是否使用点采样或滤波, 我们从纹理图中得到的值终究只是理想纹理值的一个近似。我们更希望纹理值是由一个较大的纹理区域的纹理值进行平均的结果。实现这一目标的一种可能的方式是通过对于若干组纹理元进行平均而创建一些较小的纹理图。例如, 我们可以依据一幅 64×64 的纹理图将每四个纹理元构成的组进行平均, 而创建一幅 32×32 的纹理图。但是, 一旦我们创建好这些新的纹理数组, 接踵而至的一个新问题就是我们到底应该使用哪个纹理图。OpenGL 有一个称为多级渐进纹理映射的选项, 该选项能够实现选取过程的自动化。【197~198】

假定我们要创建一系列图像, 每次都通过将四个像素合为一个像素而将图像的高度和宽度减半。如果原始图像的尺寸为 64×64 , 则我们可以创建出尺寸为 32×32 、 16×16 、 8×8 、 4×4 、 2×2 以及 1×1 的一系列图像。^①我们称第一幅图像为第 0 级, 第二幅图像为第 1 级, 以此类推。现在我们就可以针对每幅图像调用函数 `glTexImage2D()`, 每次调用都使用各自不同的级别和的维数, 如下面的代码所示:

```
GLubyte image0[64][64][3];
GLubyte image1[32][32][3];
:
GLubyte image5[1][1][3];
glTexImage2D(GL_TEXTURE_2D, 0, GL_RGB, 64, 64, GL_RGB,
             GL_UNSIGNED_BYTE, image0);
glTexImage2D(GL_TEXTURE_2D, 1, GL_RGB, 32, 32, GL_RGB,
             GL_UNSIGNED_BYTE, image1);
:
glTexImage2D(GL_TEXTURE_2D, 5, GL_RGB, 1, 1, GL_RGB,
             GL_UNSIGNED_BYTE, image5);
```

要使用这些图像, 我们还需要设置缩小过滤器:

```
glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MIN_FILTER,
                GL_NEAREST_MIPMAP_NEAREST);
```

像前面这样设置参数只能达到最低的质量。这样设置后, OpenGL 将使用纹理元尺寸与像素尺寸相匹配的最接近的图像, 同时 OpenGL 将使用最接近的纹理元。有时我们可以以牺牲效率为代价使用最接近的图像和线性滤波(`GL_LINEAR_MIPMAP_NEAREST`), 也可选择(`GL_NEAREST_MIPMAP_LINEAR`)两个与被映射纹理的像素的尺寸最接近的两级渐进纹理, 并依据 `GL_NEAREST` 准则从每个渐进纹理图中生成一个纹理值。最终的纹理值为这两

^① 如果原始图像的长和宽不相等, 我们依然是每次创建新图像时对每维尺寸减半, 较小的一维尺寸减小到 1 便保持不变, 而另一维继续减半, 直到也为 1 为止。



个值的加权平均。或者同时使用两种方法(GL_LINEAR_MIPMAP_LINEAR)。虽然通常使用多级渐进纹理时都要用到一组由同一幅图像生成的尺寸递减的图像序列,但没有理由一定要这样做。当一个表面投影到屏幕中的一个很小区域中时,就可以使用比较简单的纹理,而当摄像机逐渐接近物体时,可以用不同级别的图像逐渐展示出物体表面的丰富细节。

如果我们想依据一幅原始图像生成一个多级渐进纹理集,可以借助 OpenGL 的 `gluBuild2DMipmaps()` 函数来生成这些图像。该函数会将这些图像加载到纹理内存中,所以我们无需调用函数 `glTexImage2D()`。由于多级渐进纹理图像已经在纹理内存中,所以在程序中便不再需要额外的存储空间。

【198】

```
int gluBuild2DMipmaps(GLenum target, GLint iformat, GLint w, GLint h,  
                      GLenum format, GLenum type, void * texels)
```

该函数创建并加载一个多级渐进纹理图像集。该函数的参数与 `glTexImage2D()` 的参数相同。

8.9 纹理坐标的自动生成

虽然我们可以通过设置许多参数来使纹理映射效率更高,二维纹理映射的本质其实再简单不过了:我们只是将一幅图像映射到一个对象的表面上。一旦您熟悉了这些参数和选项,最大的困难只是在于如何确定顶点的纹理坐标。这既涉及理论问题,也涉及实践问题。有两个类比对于理解该问题很有帮助,请务必铭记于心:世界地图和墙纸映射。

创建世界地图时面临的一个问题是如何将位于球面上的东西用平面表现出来。如果不对一些形状进行扭曲,我们不可能完成这样的映射。在标准的墨卡托(Mercator)投影中,我们将大部分形状扭曲集中在极点附近。但是,如果观察一下地图集,您会发现不同类型的地图处理变形的方式是不同的。二维纹理映射问题实际上是这个地图问题的逆问题。给定一幅矩形地图,我们如何使其环绕在曲面上?肯定其中要有一些畸变发生。我们分配纹理坐标的方式决定了畸变发生位置以及误差的分布。

如果试图将墙纸贴在一个曲面上,也有畸变的问题。但是,即便在一个平的表面上,当我们想要使用另一幅图像时,也存在着两幅图像的边界如何衔接的问题。我们曾经看到,在纹理映射中,使用边界会带来一些帮助,但是还有另外一个与将要被映射墙纸或纹理图的区域的尺寸相关的问题。在图形学中,我们接触的表面一般都是通过若干组多边形来定义的。虽然可能有一种简单的方法用于为任意多边形分配纹理坐标,但每个多边形的尺寸可能都不同。不妨想象一下如果整个棋盘格纹理都被映射到每个多边形上,无论其尺寸如何,图像将会发生什么样的畸变。

我们再返回来考虑一下为一个球体生成纹理图的问题。描述球面点的一种方式是通过如下

参数方程:

$$x(u, v) = r \cos u \sin v$$

$$y(u, v) = r \cos u \cos v$$

$$z(u, v) = r \sin v$$

由于 u 的取值范围为 $0 \sim 360$ 度, 而 v 的取值范围为 $0 \sim 180$ 度, 依据上述参数方程, 我们便可生成一个中心位于原点的半径为 r 的球体。我们也可依据上述方程生成多边形, 只是需要 u 和 v 取一组等间隔的值。GLU 和 GLUT 二次曲面对象就是依据该原理生成的。 u 和 v 取一组固定值时便可生成经线和纬线。如果我们想用单幅纹理图覆盖整个球面, 则可以使用如下纹理坐标:

$$s = u/360.0$$

$$t = v/180.0$$

【199~200】

这样指定纹理坐标是可行的, 但是在极点处将会产生一些畸变。如果用其他方法来生成这个球体, 将看到不同的畸变模式。

所有二次曲面对象都可通过类似于球面中用到的两参数的函数来描述。由于一个 OpenGL 二次曲面对象是用多边形逼近表示的, 而所有的顶点都位于二次曲面的实际表面上, 所以 OpenGL 可以计算出每个顶点的正确的纹理坐标。我们用 GLU 函数 `gluQuadricTexture()` 来启用纹理坐标的自动生成功能。

```
void gluQuadricTexture(GLUquadricObj * obj, GLboolean mode)
```

该函数可依据 mode 的值为二次曲面对象 obj 开启 (GL_TRUE) 或关闭 (GL_FALSE) 纹理坐标自动生成功能。

图 8.6(a) 展示了一个二次锥体, 该锥体是通过将 GLU 圆柱(顶面半径设为 0)以及用边缘来表现的多边形构成的。实现代码如下:

```
GLUquadricObj * cone;
cone = gluNewQuadric();
glEnable(GL_CULL_FACE);
gluQuadricDrawStyle(cone, GLU_FILL);
glPolygonMode(GL_FRONT_AND_BACK, GL_LINE);
gluCylinder(cone, 1.0, 0.0, 2.0, 20, 20);
```

如果我们用前面的方法自行定义棋盘纹理, 则可以通过下列代码将该棋盘纹理映射到圆锥表面:

```
GLUquadricObj * cone;
cone = gluNewQuadric();
```

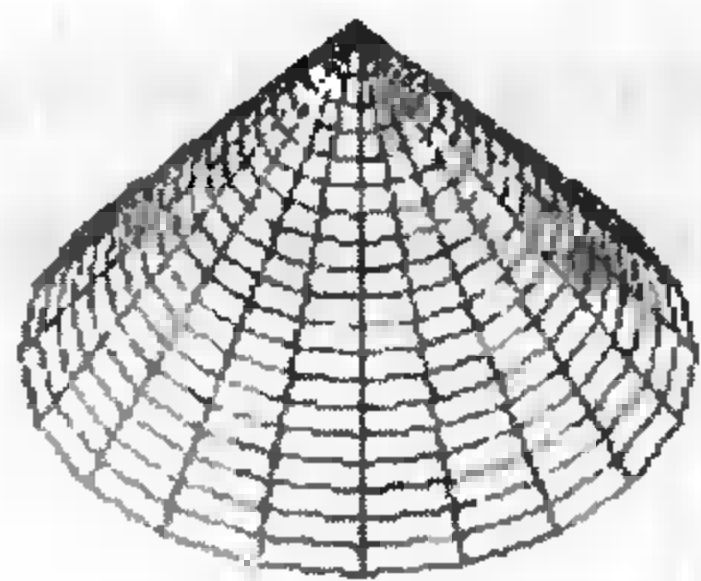


```

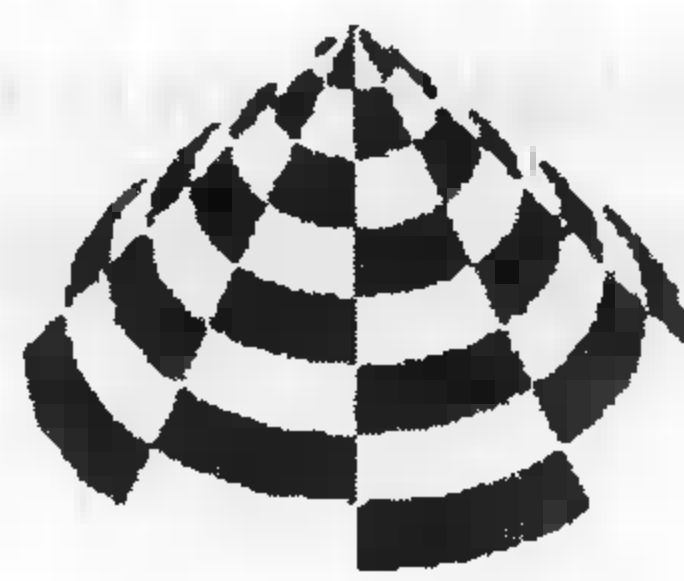
glEnable(GL_CULL_FACE);

glPolygonMode(GL_FRONT_AND_BACK, GL_FILL);
glEnable(GL_TEXTURE_2D);
gluQuadricDrawStyle(cone, GL_FILL);
gluQuadricTexture(cone, GL_TRUE);
glTexEnvf(GL_TEXTURE_ENV, GL_TEXTURE_ENV_MODE, GL_MODULATE);
gluCylinder(cone, 1.0, 0.0, 2.0, 20, 20);

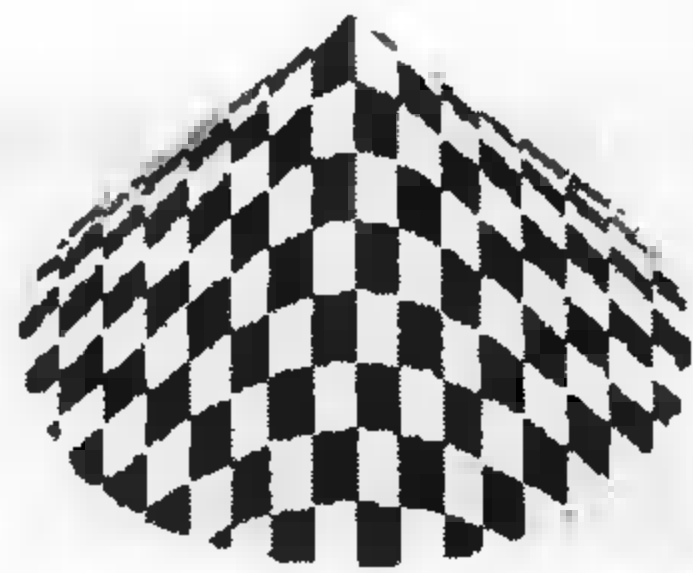
```



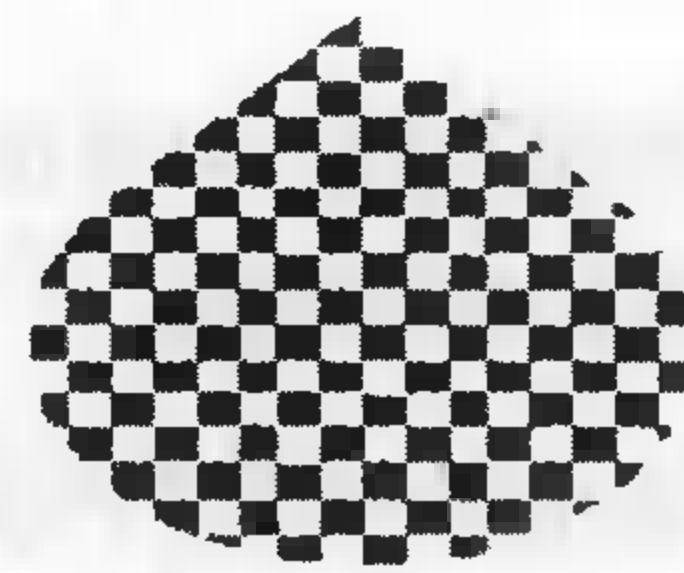
(a) 具有多边形各边的圆柱体



(b) 用 gluQuadricTexture 来显示



(c) 用 glTexGen 及参数 GL_OBJECT_LINEAR 来显示



(d) 用 glTexGen 及参数 GL_EYE_LINEAR 来显示

图 8.6 对用 GLU 圆柱体构造的圆锥的纹理自动生成

【200~201】

该段代码的运行结果如图 8.6(b)所示。注意，当我们沿着圆锥母线向圆锥顶点移动时，会发现交替出现的白色区域和黑色区域越来越小，这是由于越靠近顶点，用于对圆锥逼近表示的多边形也越来越小。虽然我们对程序所呈现的结果大体满意，但在其他程序中，在需要将纹理映射到对象表面时，我们仍然希望纹理保持其尺寸不变。

OpenGL 为纹理坐标的自动生成提供了另外一个选项。通常，理想的纹理坐标可用 (x, y, z) 空间中的点来表达。OpenGL 允许我们生成由点到对象坐标系或观察坐标系中某个平面的距离来度量的纹理坐标。不要忘记顶点坐标的内部表示是四维的齐次坐标。所以，每个顶点的内部形式通常为 (x, y, z, w) ，其中 w 为1。 $ax + by + cz + dw$ 的值与从点 (x, y, z, w) 到由 (a, b, c, d) 确定的平面的距离成正比。例如，考虑如下平面：

$$x + 2y + 3z + 4 = 0$$

该方程的系数为 $(1, 2, 3, 4)$ 。对于以齐次坐标表示的任意点 (x_0, y_0, z_0, w_0) ， $x_0 + 2y_0 + 3z_0 + 4w_0$ 的值都与该点到上述平面的距离成比例。OpenGL 允许我们通过该公式为任意纹理坐标 $(s, t, q$ 或 $r)$ 生成一个纹理值。这样一种映射等价于通过如下形式的简单线性方程来确定纹理坐标：

$$s = a_s x + b_s y + c_s z + d_s w$$

$$t = a_t x + b_t y + c_t z + d_t w$$

【201~202】

假定在平面 $z=0$ 上有一个正方形, 假定其两个对角分别位于 $(-1, -1, 0)$ 和 $(1, 1, 0)$, 我们想将一个二维纹理映射到该正方形中。以下方程:

$$s = x/2 + 1/2$$

$$t = y/2 + 1/2$$

表示该映射。这两个方程相当于由系数 $(1/2, 0, 0, 1/2)$ 和 $(0, 1/2, 0, 1/2)$ 所确定的平面。我们必须为每个纹理坐标独立地启用该功能:

```
glEnable(GL_TEXTURE_GEN_S);
```

```
glEnable(GL_TEXTURE_GEN_T);
```

我们用函数 `glTexGen*()` 来设置必要的参数并表示这些平面。

```
void glTexGen (ifd) (GLenum texcoord, GLenum param, type value);
```

```
void glTexGen (ifd) v(GLenum texcoord, GLenum param, type * plane)
```

这组函数为纹理坐标 `texcoord` 设置纹理自动生成功能。参数 `param` 可指定为模式

`GL_TEXTURE_GEN_MODE` 或由数组 `plane` 确定的平面位于哪个坐标系 (`GL_OBJECT_LINEAR`,

`GL_EYE_LINEAR`) 中。如果模式为 `GL_TEXTURE_GEN_MODE`, 则 `value` 取 `GL_OBJECT_LINEAR` 或 `GL_EYE_LINEAR`。

所以, 对于上例, 我们应该进行如下四次函数调用:

```
GLfloat planes[] = {0.5, 0.0, 0.0, 0.5};
```

```
GLfloat planes[] = {0.0, 0.5, 0.0, 0.5};
```

```
glTexGeni(GL_S, GL_TEXTURE_GEN_MODE, GL_OBJECT_LINEAR);
```

```
glTexGeni(GL_T, GL_TEXTURE_GEN_MODE, GL_OBJECT_LINEAR);
```

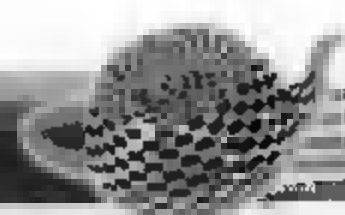
```
glTexGenfv(GL_S, GL_OBJECT_LINEAR, planes);
```

```
glTexGenfv(GL_T, GL_OBJECT_LINEAR, planes);
```

我们仍然需要用所有其他纹理函数来定义所需的纹理参数, 但我们不打算使用函数 `glTexCoord*()`。所以, 我们通过上述 6 行代码, 再加上以下代码:

```
gluCylinder(cone, 1.0, 0.0, 2.0, 20, 20);
```

就可生成如图 8.6(c) 所示的映射了纹理的圆柱体。如果我们使用 `GL_OBJECT_LINEAR` 模式, 则纹理图被固定在表面上, 当我们移动该曲面时, 纹理也随之移动。所以, 纹理看起来就像是直接绘制到表面上一样。如果使用 `GL_EYE_LINEAR` 模式, 纹理坐标则是基于观察坐标系中的顶点位置, 所以在移动物体时, 赋给顶点的纹理坐标将随之发生变化。于是, 当我们移动物体时, 纹理坐标也随之变动。图 8.6(d) 是在与图 8.6(c) 同样的模式下生成的, 只是前者使用的是 `GL_EYE_LINEAR`, 而非 `GL_OBJECT_LINEAR`。 【202~203】



8.10 纹理对象

在默认运算模式下, 当前纹理是 OpenGL 状态的一部分。在执行 `glTexImage*()` 时, 当前纹理就从系统内存中转移到 OpenGL 的纹理内存。如果只有一幅单独的纹理图, 上述移动过程只需进行一次。但是, 如果在程序中有多个纹理, 则会遇到一些问题。假定每个对象都有各自相应的纹理, 则就像材质属性一样, 每次我们绘制一个不同的对象时, 都必须重新加载相应的纹理。当纹理维数较大时, 性能将受到显著影响。

OpenGL 提供了一种替代方案——纹理对象(texture object)。该方案的想法非常简单。首先我们依据纹理和相应的参数值创建一个对象。然后用多个对象来填充纹理内存, 并令 OpenGL 使用一个由对象的标识符所确定的纹理。只要内存足够容纳这些纹理对象, 就可以避免纹理图的重复加载。如果内存不足以容纳我们所需的所有纹理, 我们可按优先级对纹理对象进行排序以减少从处理器到纹理内存的数据移动量。

提示 在大多数系统中, 纹理内存中的可用资源非常有限, 而我们希望纹理重新加载(从处理器内存到纹理内存)的次数尽可能地少。

与显示列表一样, 纹理标识符也应为整数。我们可借助函数 `glGenTextures()` 来搜索一些未被使用的整数。

```
void glGenTextures(GLsizei n, GLuint * name)
```

该函数返回 `name`, 第一个整型参数标识所要生成的纹理对象标识符的数量。

函数 `glIsTexture()` 允许我们检查一个给定 ID 是否已被占用。

```
GLboolean glIsTexture(GLuint name)
```

如果 `name` 已经作为一个纹理 ID 存在, 函数返回 `GL_TRUE`; 否则返回 `GL_FALSE`。

【203】

还有一个函数 `glBindTexture()` 负责在纹理对象之间进行切换, 并形成新的纹理对象。

```
void glBindTexture(GLenum target, GLuint name)
```

该函数将 `name` 与类型为 `target` (可取 `GL_TEXTURE_1D`, `GL_TEXTURE_2D` 或 `GL_TEXTURE_3D`) 的纹理绑定起来。

如果调用 `glBindTexture()` 时使用了参数 `name`, 而且 `name` 之前未被使用过, 则后续对各种纹理函数的调用便定义了 ID 为 `name` 的纹理对象。如果在调用 `glBindTexture()` 之前, `name` 就已经存在, 则其标识的纹理对象将成为当前纹理, 并被应用于对象表面, 直到下一次调用 `glBindTexture()` 为止。

如果调用 `glBindTexture()` 时将 `name` 设为 0, 则进行常规纹理调用, 当前纹理成为 OpenGL 状态的一部分, 纹理参数的当前值也将生效。

借助函数 `glDeleteTextures()`, 我们可以删除纹理对象, 并将这些纹理对象所占用的资源释放掉。

```
void glDeleteTextures(GLsizei n, GLuint * namearray)
```

该函数负责从数组 `namearray` 中删除 `n` 个纹理对象。数组 `namearray` 中保存的是纹理对象的名称。

如果纹理内存的空间不足以容纳所有的纹理, 我们可设置每个纹理对象的优先级。

```
glTexParameterf(GL_TEXTURE_2D, GL_TEXTURE_PRIORITY, priority);
```

其中, 优先级的取值范围为(0.0, 1.0), 0.0 表示优先级最低。当 OpenGL 需要为一个纹理开辟空间时, 优先级最低的纹理将从纹理内存中被移除。

8.11 用于图像操作的纹理图

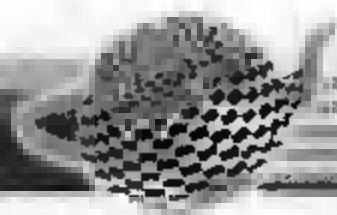
通常情况下, 操纵纹理图比像素简单一些。由于纹理是被映射到几何对象, 因此我们就可对几何对象进行操作, 而 OpenGL 会将该纹理拉伸以与对象的投影匹配。例如, 假定我们想重绘一幅图像。在有两幅或两幅以上用不同摄像机或同一摄像机以多个视角拍摄的图像时, 会引发一些问题。我们不得不将其中一幅图像进行拉伸以与另外一幅图像匹配。解决该问题的传统方法是在两幅图像之间寻找对应点(匹配点), 然后通过寻求数学函数来建立这些对应点之间的映射关系。一个更简单的解决方案是令 OpenGL 结合纹理图来完成该工作。我们可以生成这两幅图像的纹理图, 并将每幅纹理图映射到一个二次曲面上, 然后在交互式地操纵二次曲面对象的交点, 直到获得满意的拉伸效果。【204~205】

能说明使用纹理图的优点的另外一个例子是我们想要为其映射纹理的几何对象要受到 OpenGL 几何流水线中各种变换的制约。所以, 当我们将一幅图像作为纹理图显示, 并重新调整窗口尺寸时, 我们可借助重绘回调函数使得无论窗口尺寸如何变化, 图像始终能完整地显示出来。下面为了使用纹理图, 我们对第 7 章的 PPM 显示程序进行了改写。

```
#include <stdio.h>
#include <stdlib.h>
#include <GL/glut.h>

int n;
int m;

GLuint * image;
```

```
void display()
{
    glClear(GL_COLOR_BUFFER_BIT);
    glBegin(GL_QUADS);
        glTexCoord2f(0.0, 0.0);
        glVertex2i(0, 0);
        glTexCoord2f(0.0, 1.0);
        glVertex2i(0, m-1);
        glTexCoord2f(1.0, 1.0);
        glVertex2i(n-1, m-1);
        glTexCoord2f(1.0, 0.0);
        glVertex2i(n-1, 0);
    glEnd();
    glFlush();
}

void myreshape(int w, int h)
{
    glMatrixMode(GL_PROJECTION);
    glLoadIdentity();
    gluOrtho2D(0.0, (GLfloat)n, 0.0, (GLfloat)m);
    glMatrixMode(GL_MODELVIEW);
    glLoadIdentity();
    glViewport(0, 0, h, w);
}

int main (int argc, char **argv)
{
    FILE * fd;
    int k, nm;
    char c;
    int i;
    char b[100];
    float s;
    unsigned int red, green, blue;

    printf("enter file name\n");
    scanf("%s", b);
    fd = fopen(b, "r");
    fscanf(fd, "%[^\n]", b);
    if(b[0] != 'p' || b[1] != '3')
    {
        printf("%s is not a PPM file!\n", b);
        exit(0);
    }
}
```



```

}
fscanf(fd, "%c", &c);
while(c == '#')
{
    fscanf(fd, "%[^\n]", b);
    fscanf(fd, "%c", &c);
}
ungetc(c, fd);
fscanf(fd, "%u %u %u", &n, &m, &k);
nm = n * m;
image = malloc(3 * sizeof(GLuint) * nm);
s = 255./k;

for(i = 0; i < nm; i++)
{
    fscanf(fd, "%d %d %d", &red, &green, &blue);
    image[3 * nm - 3 * i - 3] = red;
    image[3 * nm - 3 * i - 2] = green;
    image[3 * nm - 3 * i - 1] = blue;
}

glutInit(&argc, argv);
glutInitDisplayMode(GLUT_SINGLE | GLUT_RGB);
glutInitWindowSize(n, m);
glutInitWindowPosition(0, 0);
glutCreateWindow("image");

glutReshapeFunc(myreshape);
glutDisplayFunc(display);
glPixelTransferf(GL_RED_SCALE, s);
glPixelTransferf(GL_GREEN_SCALE, s);
glPixelTransferf(GL_BLUE_SCALE, s);
glPixelStorei(GL_UNPACK_SWAP_BYTES, GL_TRUE);
glPixelStorei(GL_UNPACK_ALIGNMENT, 1);
glEnable(GL_TEXTURE_2D);
glTexImage2D(GL_TEXTURE_2D, 0, GL_RGB, n, m, 0, GL_RGB,
             GL_UNSIGNED_INT, image);
glTexParameterf(GL_TEXTURE_2D, GL_TEXTURE_WRAP_S,
                GL_CLAMP);
glTexParameterf(GL_TEXTURE_2D, GL_TEXTURE_WRAP_T,
                GL_CLAMP);
glTexParameterf(GL_TEXTURE_2D, GL_TEXTURE_MAG_FILTER,
                GL_NEAREST);
glTexParameterf(GL_TEXTURE_2D, GL_TEXTURE_MIN_FILTER,

```




```
        GL_NEAREST);  
glClearColor(0.0, 0.0, 0.0, 1.0);  
glColor3f(1.0, 1.0, 1.0);  
glutMainLoop();  
}
```

注意, 在调整窗口尺寸时, 窗口中图像的相对尺寸保持不变。

【205~207】

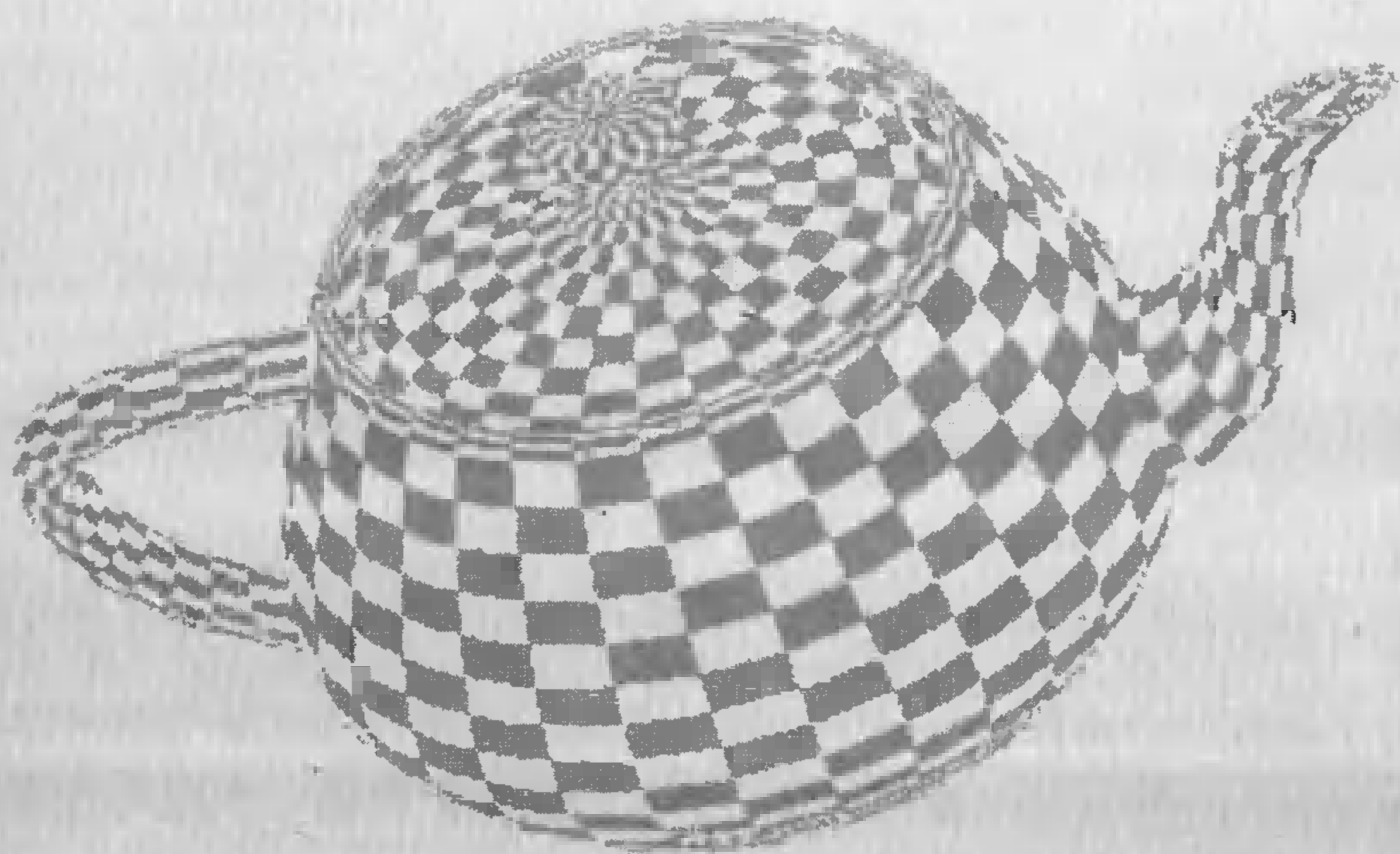
8.12 编程练习

1. 将纹理自动生成应用于一个旋转的二次曲面对象或犹他壶。编写的程序应能允许您在对象线性映射或视点线性映射之间进行切换。
2. 编写一个能显示房间内部场景的程序, 其中每面墙壁的表面都是依据一幅数字图像生成的纹理图。请通过实验研究如何移动光源以对该图像产生影响。

第 9 章

曲线与曲面

到目前为止，我们所接触到的 OpenGL 几何图元，如直线和多边形都是平面的。即使用到了 OpenGL 二次曲面对象，这些表面也是用平面多边形来绘制的。OpenGL 通过求值器提供了若干曲线和曲面，该机制使得我们能够创建贝塞尔曲线和曲面。这种机制具有很大的灵活性，因为我们可以借助它生成任意阶的多项式曲线，并可将其类型的多角曲线(polygonal curve)和曲面转化为贝塞尔曲线和曲面。



9.1 参数曲线

在许多图形程序中，使用参数多项式曲线(parametric polynomial curve)是最佳选择。借助这种曲线描述法，我们可为二维、三维或四维空间曲线的每个空间变量用独立变量(或独立参数) u 建立一个独立的方程：

$$x = x(u) = a_{x0} + a_{x1}u + a_{x2}u^2 + \dots + a_{xn}u^n$$

$$y = y(u) = a_{y0} + a_{y1}u + a_{y2}u^2 + \dots + a_{yn}u^n$$

$$z = z(u) = a_{z0} + a_{z1}u + a_{z2}u^2 + \dots + a_{zn}u^n$$

$$w = w(u) = a_{w0} + a_{w1}u + a_{w2}u^2 + \dots + a_{wn}u^n$$

其中 n 为该多项式的阶数。为描述一个二维曲线，我们只需要使用 x 和 y 坐标就够了；对于三维曲线，则需要 x 、 y 、 z 三个坐标；对于四维曲线，除了 x 、 y 、 z 外，我们还需要 w 。如果我们仅关心二维和三维曲线，则其余坐标分量将取默认值，即 $z = 0, w = 1$ 。 【209】

每当给定一个 u 值，依据参数曲线就可确定一个位于齐次坐标系中的点 $(x(u), y(u), z(u), w(u))$ 。通常我们通过限定 u 的取值范围(如 $u_{\min} \leq u \leq u_{\max}$)来定义一个曲线段，即一段有限长度的曲线。

图9.1展示了将描述每个空间变量轨迹的曲线方程结合后所生成的三维曲线 $p(u) = (x(u), y(u), z(u))$ 。

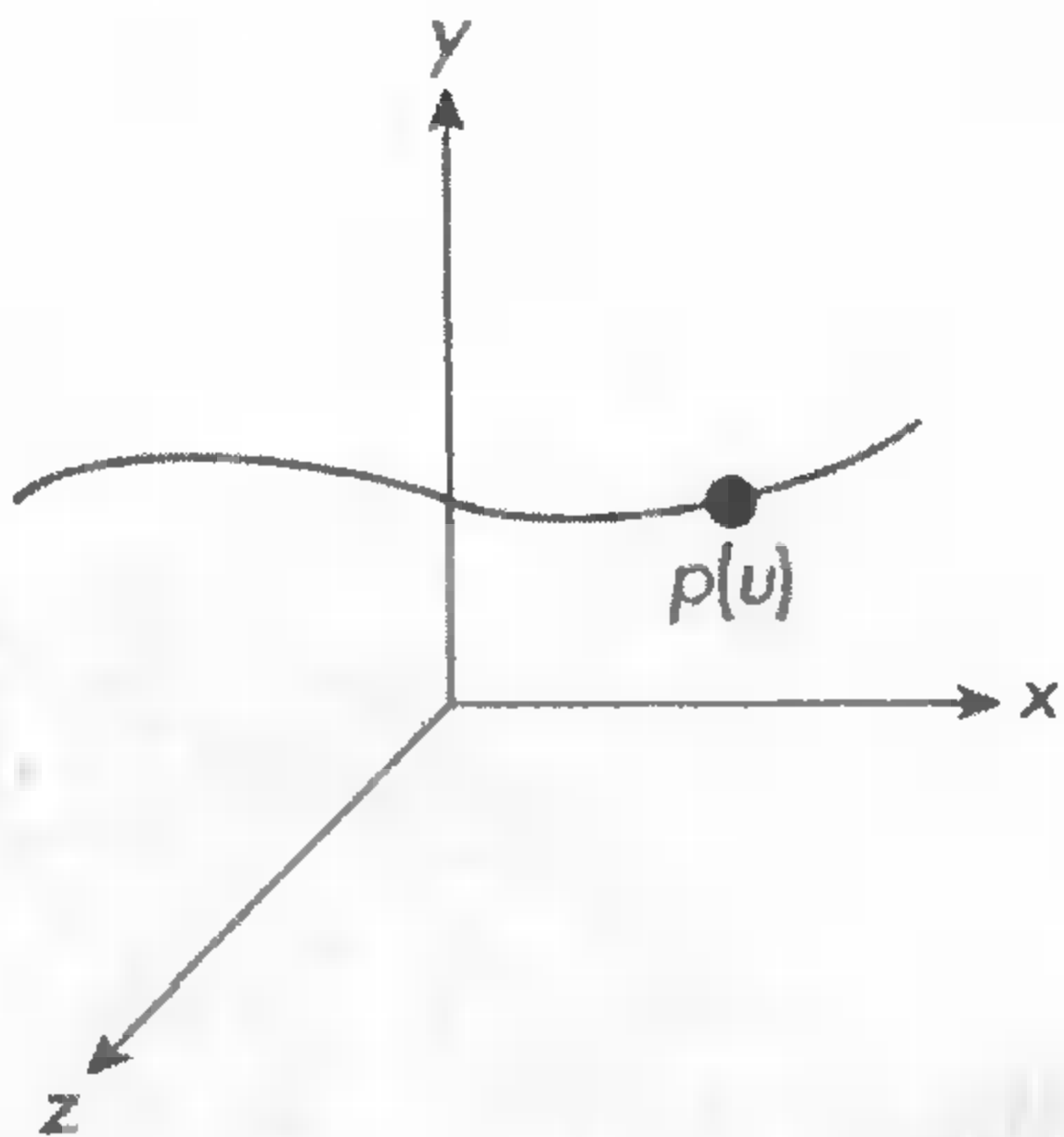


图 9.1 参数曲线

现在，我们只考虑那些用 x 、 y 、 z 坐标值就可完整描述的曲线，我们令第4个分量 w 恒为 1。如果我们要求取一个阶数为 n 的多项式曲线，我们必须在参数方程中指定 $3(n+1)$ 个系数。为了求取这些系数，我们可以提供 $3(n+1)$ 个该曲线满足的独立约束条件。我们所运用的约束条件的类型决定了曲线的特定类型。例如，我们可以提供曲线必须经过的一组三维点。

这样的曲线称为对这组点进行插值, 如图 9.2 所示。

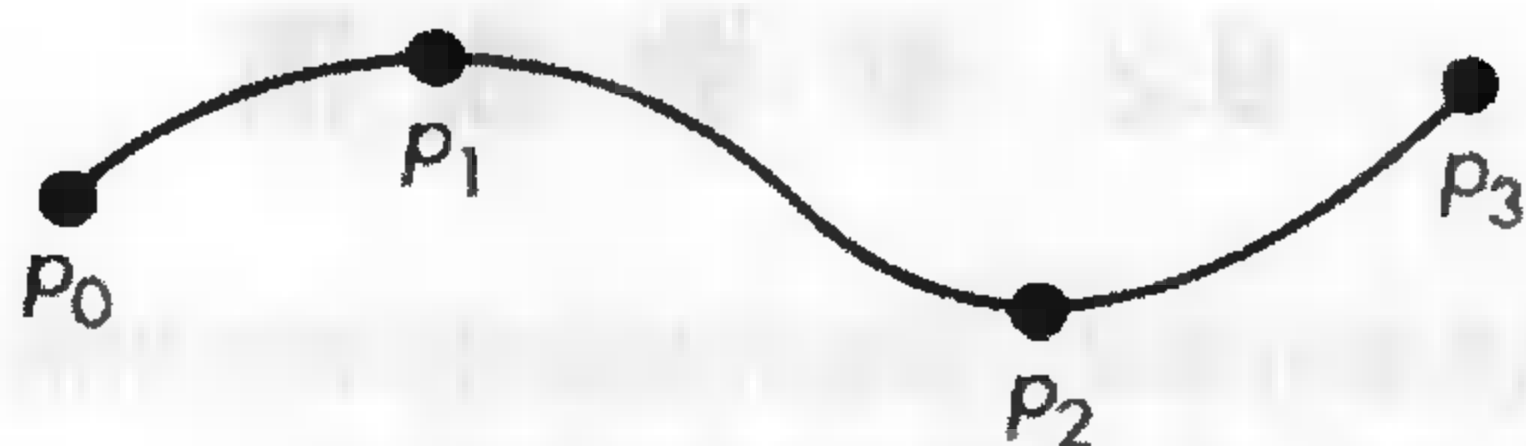


图 9.2 通过点 p_0, p_1, p_2, p_3 进行曲线插值

下面我们来考虑如下形式的三次多项式:

$$q(u) = a_0 + a_1u + a_2u^2 + a_3u^3$$

其中 u 的取值一般都在区间 $[0, 1]$ 内变化。通过简单的比例和平移变换, 我们就可对 u 的取值范围进行修改。三次曲线是到目前为止最常使用的曲线, 因为这种曲线灵活性和效率兼具。上式中, q 可以是 x, y, z 或 w 。所以, 我们需要 4 个独立条件来确定每个曲线对应的一组系数。为了进行插值, 我们必须提供该曲线一定经过的 4 个点。但是, 在计算机图形学中, 插值曲线由于平滑性不够好, 故并非最常用的类型。【210~211】

如果我们使用的是三次多项式曲线, 则仅使用 4 个条件还不足以在一个较大的空间范围内对该曲线进行控制。针对该问题, 采用高阶多项式曲线并不是一个很好的选择, 因为其计算复杂度高且难于控制, 我们采取的策略是定义许多较短的曲线段, 每个曲线段都通过我们强加给整条曲线的约束条件中的一个子集来定义。例如, 图 9.3 中的两个曲线段, 每条曲线段在对应于 u 在 $[0, 1]$ 之间取值时的子空间中都是合法的。每条曲线段都由局部数据所确定。在计算机图形学中, 这种类型的对曲线形状的局部控制方法非常重要。但是, 当处理多个曲线段时, 我们必须对接合点(即相邻曲线段汇合的地方)给予足够的关注。仅仅保证两条线段的汇合将会降低曲线设计中的灵活性。对于图 9.3 所示的例子, 我们看到如果我们用 P_0, P_1, P_2, P_3 作为第一条曲线段的数据, 在第二条曲线段中, 我们必须将 P_3 作为数据的起始点以保证整条曲线在接合点处的连续性。

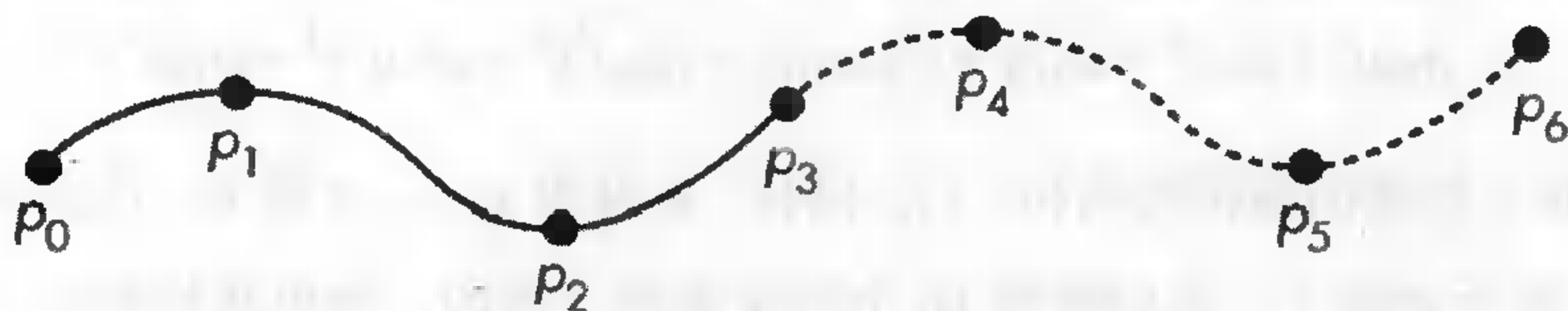
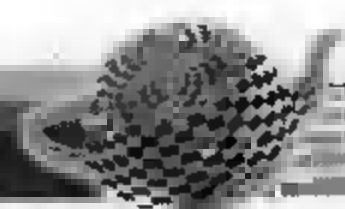


图 9.3 以接合点 P_3 为公共顶点的两条插值曲线段

一旦确定了曲线应满足的一组约束条件, 我们就有了三个联立方程组, 每个方程组都由对应于 x, y 和 z 的四个系数的 4 个方程组成。这些约束条件一般是相对空间中所谓的“控制点”而言的, 这些控制点通常由用户通过交互方式提供。



9.2 参数曲面

曲线只需用单变量 u 的函数就可定义，而参数曲面则是需要用两个变量 u 和 v 的函数来定义。曲面上每点的坐标值都要通过一个独立的函数来确定：

$$x = x(u, v)$$

$$y = y(u, v)$$

$$z = z(u, v)$$

$$w = w(u, v)$$

所以， u 和 v 的每个取值都确定了三维或四维空间中的一点，如图 9.4 所示。其中， $p(u, v) = (x(u, v), y(u, v), z(u, v))$ 。【211】

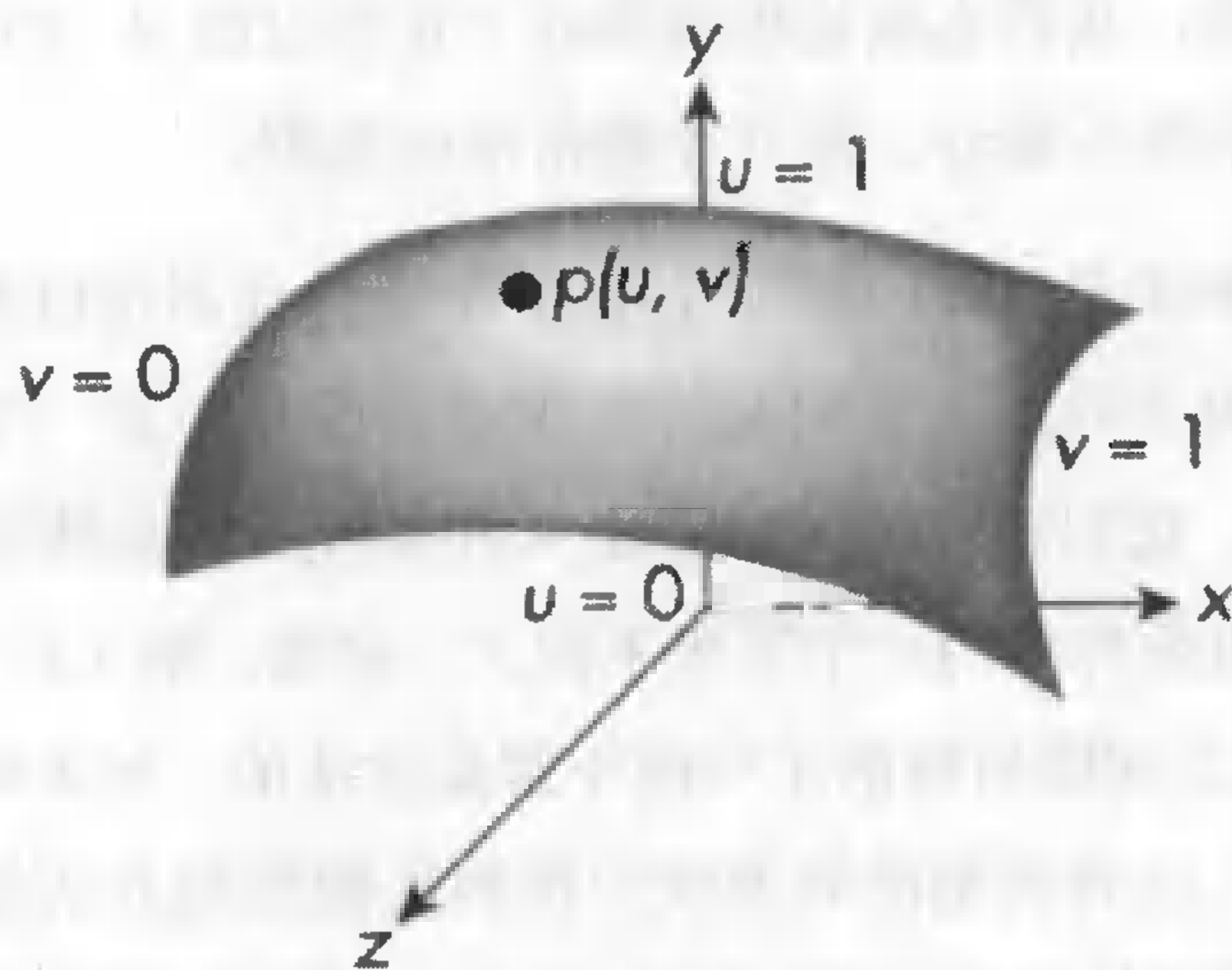


图 9.4 参数曲面片

在计算机图形学中，这些函数通常都为双三次多项式，形式如下：

$$q(u, v) = a_{00} + a_{10}u + a_{01}v + a_{11}uv + a_{20}u^2 + a_{02}v^2 + a_{21}u^2v + a_{12}uv^2 + a_{22}u^2v^2 + \\ a_{30}u^3 + a_{03}v^3 + a_{31}u^3v + a_{13}uv^3 + a_{32}u^3v^2 + a_{23}u^2v^3 + a_{33}u^3v^3$$

在上式中， u 和 v 的取值范围均为 $(0, 1)$ 。同样， q 可为 x ， y ， z 或 w 。这里共有 16 个系数，所以为了确定每个曲面片，我们需要 16 个约束条件。例如，如果我们想对一组控制点进行插值，每个曲面片都必须经过这 16 个点。所以，为了确定一个三维的曲面片(w 取 1)，我们必须确定 48 个系数。

9.3 贝塞尔曲线和曲面

在计算机图形学和CAD中,如果只是逼近某些控制点便可生成与插值相比更为光滑的曲线,则我们更倾向于使用前者。虽然有多种方式可用于定义“逼近”,但最常用的选择莫过于贝塞尔曲线和曲面。

图9.5展示了一个三次贝塞尔多项式 $q(u)$,其中 q 可为 x , y , z 或 w 。该曲线由4个控制点 Q_0 , Q_1 , Q_2 以及 Q_3 所确定。这条三次贝塞尔曲线只对 Q_0 和 Q_3 进行了内插,并用 Q_0 和 Q_1 以及 Q_2 和 Q_3 之间的直线段来确定曲线在末端处的切线斜率。这样的曲线非常光滑,而且该曲线始终落在有控制点构成的外接盒中,该外接盒称为凸包(convex hull),如图9.5所示。

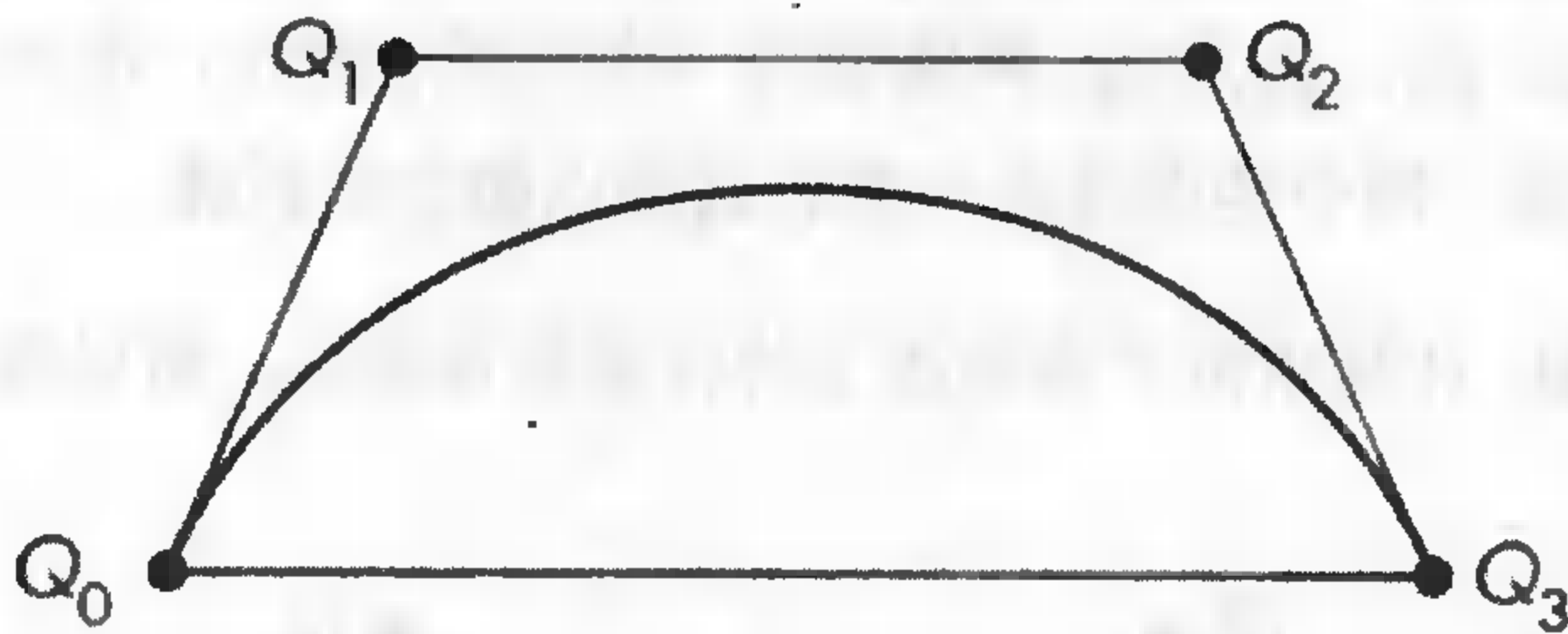


图9.5 贝塞尔曲线及其凸包

如果有一系列控制点,则第二条曲线段将由 Q_3 、 Q_4 、 Q_5 和 Q_6 确定,这就保证了曲线在接合点处的连续性。

贝塞尔曲面也是用类似的方式定义的。要定义双三次曲面,我们需要16个控制点 Q_{ij} ,其中 i 和 j 的取值范围都为 $0\sim 3$ 。曲面片依据4个角点 Q_{00} , Q_{03} , Q_{30} 以及 Q_{33} 来进行插值,并恰好在这16个控制点构成的凸包内。贝塞尔曲面实际上是贝塞尔曲线的扩展。当 u 或 v 中的一个变量取固定值时,曲面 $q(u, v)$ 便退化为以 v 或 u 为自变量的贝塞尔曲线。【212~213】

使用更高阶的曲线和曲面时,我们需要使用更多的控制点来确定每个曲线段或曲面片。例如,7阶的贝塞尔曲线便需要8个控制点。该曲线将对端点进行插值,并用其他控制点数据来逼近端点处的导数。为确定这些曲线和曲面,由 u 和 v 构成的多项式来自于一个称为Bernstein多项式的多项式族,使用这种多项式可以获得非常高的效率。

这部分内容的数学推导非常优雅,但考虑到大多数计算机图形学教科书已经涵盖了这部分内容,这里我们直接给出结果。下面列出一些贝塞尔曲线和曲面的重要性质,正是这些性质奠定了它们在计算机图形学中的重要地位。

1. 贝塞尔曲线和曲面是光滑的。

2. 贝塞尔曲线和曲面是依据控制点在局部定义的。
3. 可以高效地生成精度满足要求的任意阶的贝塞尔曲线和曲面。
4. 任何其他类型的多项式曲线和曲面都可用贝塞尔曲线和曲面表示。

最后一条性质非常重要，因为正是这条性质使得获取其他类型的曲线和曲面变得易如反掌，而且仍然可以利用 OpenGL 所提供的用于贝塞尔曲线和曲面的高效算法。在后面我们将看到一个这样的例子。

OpenGL 通过一种称为求值器的机制来实现贝塞尔曲线和曲面。这些求值器可以计算出用户指定数量的位于任意阶数的贝塞尔曲线或曲面上的点。OpenGL 则会用这些位于曲线或曲面上的点来生成标准的 OpenGL 图元，例如近似表示曲线和曲面的直线段和多边形。图 9.6 展示了一个由控制点 Q_0 , Q_1 , Q_2 和 Q_3 所确定的三次贝塞尔曲线，在该曲线上已经有 5 个点被求值器精确计算得到。图中的虚线是一条有这些点确定的折线。

由于能够借助 OpenGL 计算出位于该曲线上的任意数量的点，所以我们可以达到任意所需的精度。

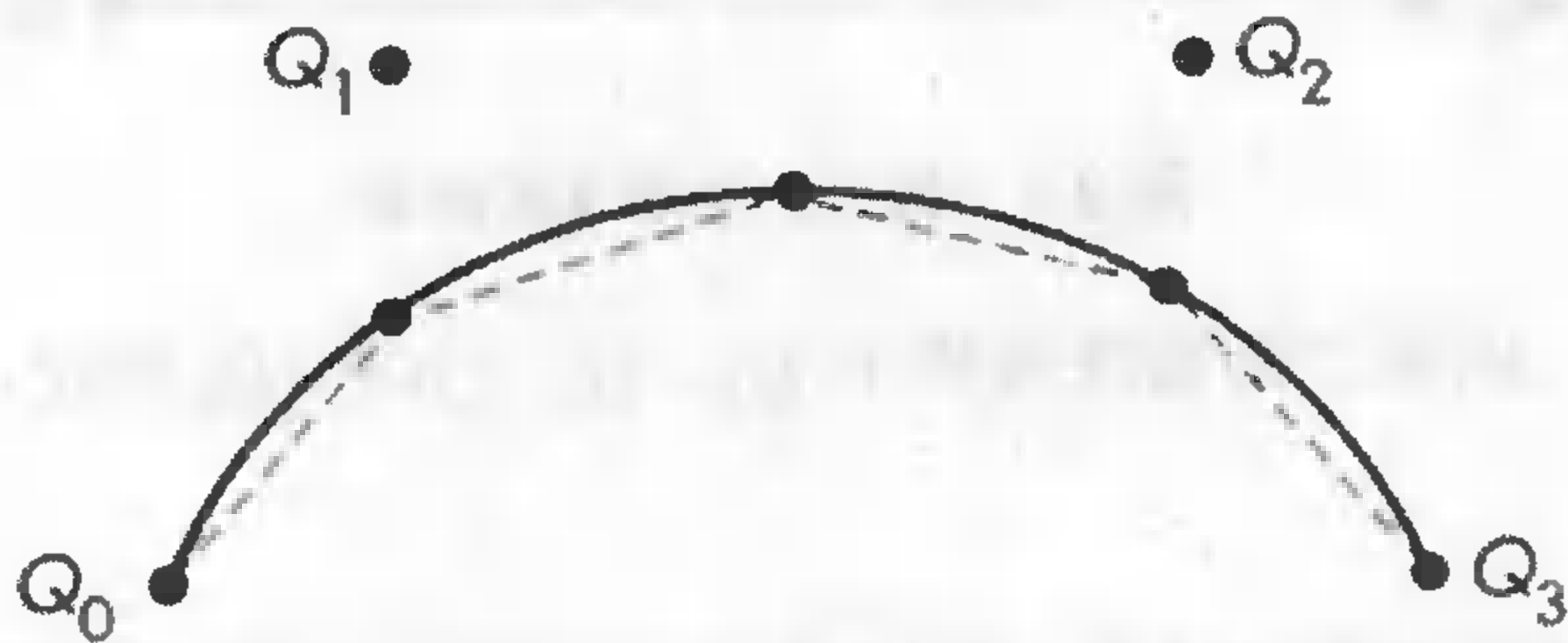


图 9.6 近似表示由 Q_0 , Q_1 , Q_2 和 Q_3 所确定的贝塞尔曲线的直线段

9.4 一维 OpenGL 求值器

对于曲线，我们可以使用一维求值器，该求值器可通过函数 `glMap1*` 来定义。

```
void glMap1 (fd) (GLenum entity, type u0, type u1, GLint stride,
                 GLint order, type * data)
```

该函数使用一个 $order-1$ 阶的 Bernstein 多项式为取值范围为 (u_0, u_1) 的实体定义一个一维求值器。在数组 `data` 中的连续数据点之间有类型为 `GLfloat` 或 `GLdouble` 的 `stride` 个变量。

我们可将该求值器用于多种实体。如果想要得到一条曲线，可将实体设为 `GL_MAP1_VERTEX_3`。每次使用求值器时，通过能够取代对 `glVertex*` 调用的函数，如 `glEvalCoord1*` 和 `glMapGrid1*`，就可计算出顶点 (x, y, z) 。我们也可生成四维的顶点 (`GL_MAP1_VERTEX_4`)、法向量 (`GL_MAP1_NORMAL`)、颜色 (`GL_MAP1_COLOR_4`) 以及

纹理坐标。将要使用求值器的每种类型的实体必须通过如下形式的函数调用来启用：

```
glEnable(GL_MAP1_VERTEX_3);
```

我们可以依据需要选择贝塞尔曲线的阶数。曲线的阶数大于多项式的阶数，它与为确定该曲线所需的点数相等。在我们的曲线中，参数 u 可在任意范围内取值，这是因为任何区间 (u_0, u_1) 都可通过如下公式变换到区间 $(0, 1)$ 内

$$u' = (u - u_0) / (u_1 - u_0)$$

一旦启用了—个或多个求值器，我们就可构造对贝塞尔曲线的逼近表示。最简单的方法就是每次都使用 `glEvalCoord1f()` 来取代对 `glVertexf()` 的调用。与使用二维、三维或四维点的函数 `glVertexf()` 不同，`glEvalCoord1f()` 将 u 的一个值传递给所有启用的求值器。这些求值器将生成点、法向量、颜色或纹理坐标，具体是哪种参数要依据所启用的求值器的类型而定。例如，假定我们希望为 u 的取值范围为 $[0, 1]$ 的顶点定义并启用一个求值器，而且我们希望用 21 个点构成的折线来近似该曲线，我们可用如下代码来实现这一任务：

```
glBegin(GL_LINE_STRIP);
    for(i = 0; i <= 20; i++) glEvalCoord1f(i/20.0);
glEnd();
```

【214~215】

```
void glEvalCoord1<fd>(type u)
void glEvalCoord1<fd>v(type * u)
```

这组函数强制依据 u 的值用所有已启用的一维求值器来生成顶点、颜色、法向量或纹理值。

如果我们已用如下方式定义了另一个范围内的求值器：

```
glMap1f(GL_MAP1_VERTEX_3, u0, u1, 3, 4, data);
```

我们就可生成 21 个在该范围内的点：

```
glBegin(GL_LINE_STRIP);
    for(i = 0; i <= 20; i++)
        glEvalCoord1f(u0 + i * (u1 - u0) / 20.0);
glEnd();
```

我们也可通过如下代码启用颜色求值器：

```
glMap1f(GL_MAP1_COLOR_4, u0, u1, 3, 4, colors);
glEnable(GL_MAP1_COLOR_4);
```

其中，数组 `colors[]` 中应该有 4 个 RGBA 颜色值。执行前面的循环时，我们不但要为每次调用 `glEvalCoord1f()` 生成一条直线段，而且要利用由该颜色数组所确定的贝塞尔曲线为每个顶点生成一种新颜色。所以，如果启用平滑着色 (`GL_SMOOTH`)，在该折线上，我们将看到有数组中的第一个颜色到最后一个颜色的平滑过渡。注意，凸包的性质决定了我们将接近第 2

种颜色和第3种颜色，而不必取这些值。

使用 `glEvalCoord1*()` 的一个好处是我们可以使用 u 的任意值，而不仅仅是像我们的例子中那样取等间隔的值。但是，如果我们想要使用 u 的21个值，我们必须对 `glEvalCoord1*()` 进行21次调用。当 u 取等间隔值时，OpenGL 提供了一种替代方法。首先，我们用函数 `glMapGrid1*()` 来定义一组等间隔的 u 值。然后我们用函数 `glEvalMesh1()` 通过一次调用将求值器应用于网格上。这样，对于从 u_0 到 u_1 的21个点，我们就可像下面这样来定义网格：

```
glMap1f(GL_MAP1_VERTEX_3, u0, u1, 3, 4, data);
glEnable(GL_MAP1_VERTEX_3);
glMapGrid1f(20, u0, u1);
```

并像下面这样绘制该折线：

```
glEvalMesh1(GL_LINE, 0, 20);
```

【215~216】

```
void glMapGrid1<fd>(GLint n, type u0, type u1)
```

该函数负责定义一个将区间 (u_0, u_1) 均分为 n 份的等间隔网格。

```
void glEvalMesh1(GLenum mode, GLint first, GLint last)
```

该函数从由函数 `glMapGrid()` 定义的 u 的值 `first` 到 `last`，以模式 `mode` 绘制所有被启用的求值器。

9.5 二维求值器

二维求值器允许我们求取二维 Bernstein 多项式的值，并生成贝塞尔曲面。该机制是对一维求值器的直接扩展。但是，对于如何显示这些表面，我们具有了更多的选择。曲面是通过两参数多项式 $p(u, v)$ 定义的，当其中一个参数取固定值时，我们就得到了以另一个参数为变量的参数曲线。所以，处理曲面的原理与处理曲线的原理有许多共通之处。二维求值器的定义通过函数 `glMap2*()` 来完成，其使用则要借助于函数 `glEvalCoord2*()`。

```
void glMap2<fd>(GLenum entity, type u0, type u1, GLint ustride,
               GLint uorder, type v0, type v1, GLint vstride,
               GLint vorder, type data)
```

该函数用关于 u 和 v 的贝塞尔多项式为 `entity` 定义了一个二维求值器。控制点保存在数组 `data` 中。

```
void glEvalCoord2<fd>(type u, type v)
```

```
void glEvalCoord2<fd>v(type * uv)
```

该函数强制所有被启用的二维求值器依据 u 和 v 的值进行计算。

对于等间隔的情形，我们可使用函数 `glMapGrid2*()` 以及 `glEvalMesh2()` 来减少函数调用的次数。

【216】

`void glMapGrid2<fd>(GLint n, type u0, type u1, GLint m, type v0, type v1)`
 该函数将定义一个将区间 $[u0, u1]$ n 等分, 而将 $[v0, v1]$ m 等分所得的二维网格。

`void glEvalMesh2(GLenum mode, GLint ufirst, GLint ulast, GLint vfirst, GLint vlast)`

该函数以模式`mode`(`GL_LINE`, `GL_POINT`, `GL_FILL`)绘制所有被启用的求值器, 其中网格域变量 u 和 v 的取值范围分别为 $[ufirst, ulast]$ 和 $[vfirst, vlast]$, 该范围通过函数`glMapGrid2*()`来确定。

9.6 一个交互式例程

下面的例程中, 允许用户从终端输入数据点, 并绘制三次贝塞尔曲线, 该曲线是由若干连续的控制点集在平面中定义的。该程序允许您执行清屏操作并可重新启动, 也允许您通过键盘或鼠标来添加一些附加点。

```
#include <GL/glut.h>
#define MAX_CPTS 25

GLfloat cpts[MAX_CPTS][3];
int ncpts = 0;

static int width = 500, height = 500;

void drawCurves()
{
    int i;
    for(i = 0; i < ncpts - 3; i += 3)
    {
        glMap1f(GL_MAP1_VERTEX_3, 0.0, 1.0, 3, 4, CPTS[i]);
        glMapGrid1f(30, 0.0, 1.0);
        glEvalMesh1(GL_LINE, 0, 30);
    }
    glFlush();
}

static void display()
{
    int i;
    glClear(GL_COLOR_BUFFER_BIT);
    glBegin(GL_POINTS);
    for(i = 0; i < ncpts; i++)
        glVertex3fv(cpts[i]);
    glEnd();
    glFlush();
}
```



```

static void mouse(int button, int state, int x, int y)
{
    float wx, wy;
    if(button != GLUT_LEFT_BUTTON || state != GLUT_DOWN)
        return;
    wx = (2.0 * x)/(float)(width - 1) - 1.0;
    wy = (2.0 * (height - y))/(float)(height) - 1.0;

    if(ncpts == MAX_CPTS) return;

    cpts[ncpts][0] = wx;
    cpts[ncpts][1] = wy;
    cpts[ncpts][2] = 0.0;
    ncpts++;

    glPointSize(5.0);
    glBegin(GL_POINTS);
    glVertex3f(wx, wy, 0.0);
    glEnd();
    glFlush();
}

void keyboard(unsigned char key, int x, int y)
{
    switch(key)
    {
        case 'q': case 'Q':
            exit();
            break;
        case 'c': case 'C':
            ncpts = 0;
            glutPostRedisplay();
            break;
        case 'e': case 'E':
            glutPostRedisplay();
            break;
        case 'b': case 'B':
            drawCurves();
            break;
    }
}

```



```

void reshape(int w, int h)
{
    width = w;
    height = h;
    glMatrixMode(GL_PROJECTION);
    glLoadIdentity();
    glOrtho(-1.0, 1.0, -1.0, 1.0, -1.0, 1.0);
    glMatrixMode(GL_MODELVIEW);
    glViewport(0, 0, w, h);
}

```

```

int main(int argc, char ** argv)
{
    glutInit(&argc, argv);
    glutInitDisplayMode(GLUT_RGB);
    glutInitWindowSize(width, height);
    glutCreateWindow("Bezier Curve");
    glutDisplayFunc(display);
    glutMouseFunc(mouse);
    glutKeyboardFunc(keyboard);
    glutReshapeFunc(reshape);
    glClearColor(1.0, 1.0, 1.0, 1.0);
    glColor3f(0.0, 0.0, 0.0);
    glPointSize(5.0);
    glEnable(GL_MAP1_VERTEX_3);
    glutMainLoop();
}

```

该程序首先用 4 个控制点来定义第一条曲线段。然后以 3 个连续控制点为一组，并将这组控制点与上次绘制的曲线段的末端控制点结合起来定义新的曲线段。这种策略保证了所生成的曲线段集合是连续的。但是，这并不能保证结合点处的光滑性。 【217~219】

9.7 其他类型的曲线

贝塞尔曲线实际上是一个由一组控制点以及我们强加给该曲线的一组特定约束条件定义的多项式。如果在控制点相同的情况下，使用了一组不同的约束条件，我们将会得到另外一种类型的曲线。例如，如果要求曲线通过所有的控制点，我们将得到一条插值曲线。如果我们希望得到这种类型的曲线，完全不必放弃 OpenGL 的已有功能，而在程序中自行编写生成其他类型曲线的代码。我们所要做的只是换一种视角看待问题。

假定现有一个三次贝塞尔多项式。如果我们从该曲线中选择 4 个点, 则这些点就定义了一个三次插值多项式, 该多项式一定与最初的多项式相同。换言之, 一个三次多项式对于不同的控制点集来说即可理解为贝塞尔曲线也可理解为插值曲线。如果我们希望得到一个三次插值多项式, 且已有 4 个用于定义该多项式的控制点, 我们就可将这四个控制点转化为另外 4 个定义了一个与之等价的贝塞尔多项式的控制点(实际上, 在这两组控制点中有两个是相同的)。这种策略的优点在于我们可对这些新控制点运用 OpenGL 求值器来高效地生成所需的曲线。对于我们希望得到的每种类型的曲线, 都有一个相应的变换矩阵, 借助该矩阵我们可将其控制点转化为所期望的贝塞尔曲线控制点。对于三次插值曲线而言, 该矩阵如下:

$$M = \begin{bmatrix} 1 & 0 & 0 & 0 \\ -\frac{5}{6} & 3 & -\frac{3}{2} & \frac{1}{3} \\ \frac{1}{3} & -\frac{3}{2} & 3 & -\frac{5}{6} \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

如果我们构造一个 $4 \times n$ 的控制点数组, 其中 n 表示维数(取 2、3 或 4), 然后将矩阵 M 与该数组右乘, 就可得到一个由新的控制点构成的 $4 \times n$ 数组。

9.7.1 B 样条

贝塞尔曲线和曲面是一个称为样条(spline)的曲线和曲面家族中的成员。每种类型的样条都通过一组略有差别的约束条件来定义, 约束条件中包括了控制点以及我们希望接合点能够满足的连续类型。最常用的样条类型是三次 B 样条, 该样条在接合点处的一阶导数和二阶导数均连续。为了达到这种程度的光滑性, 需要为每个附加控制点定义一段新的 B 样条曲线段。所以, 如果控制点 Q_0, Q_1, Q_2 和 Q_3 定义了第一条曲线段, 则 Q_1, Q_2, Q_3 和 Q_4 将定义第二条曲线段。每条曲线段仅在位于中间的两个相邻控制点之间进行绘制。 【220】

如下矩阵

$$M = \begin{bmatrix} \frac{1}{6} & \frac{2}{3} & \frac{1}{6} & 0 \\ 0 & \frac{2}{3} & \frac{1}{3} & 0 \\ 0 & \frac{1}{3} & \frac{2}{3} & 0 \\ 0 & \frac{1}{6} & \frac{2}{3} & \frac{1}{6} \end{bmatrix}$$

将一组三次 B 样条曲线的控制点转化为一组与之等价的三次贝塞尔曲线的控制点。

下面的代码涵盖了为依据同一组控制点绘制出三种类型的曲线而对前面的例子所作的主要改动。用户期望得到的曲线类型可以交互方式来指定。

图 9.7 展示了由同一组控制点得到的三种不同类型的曲线。

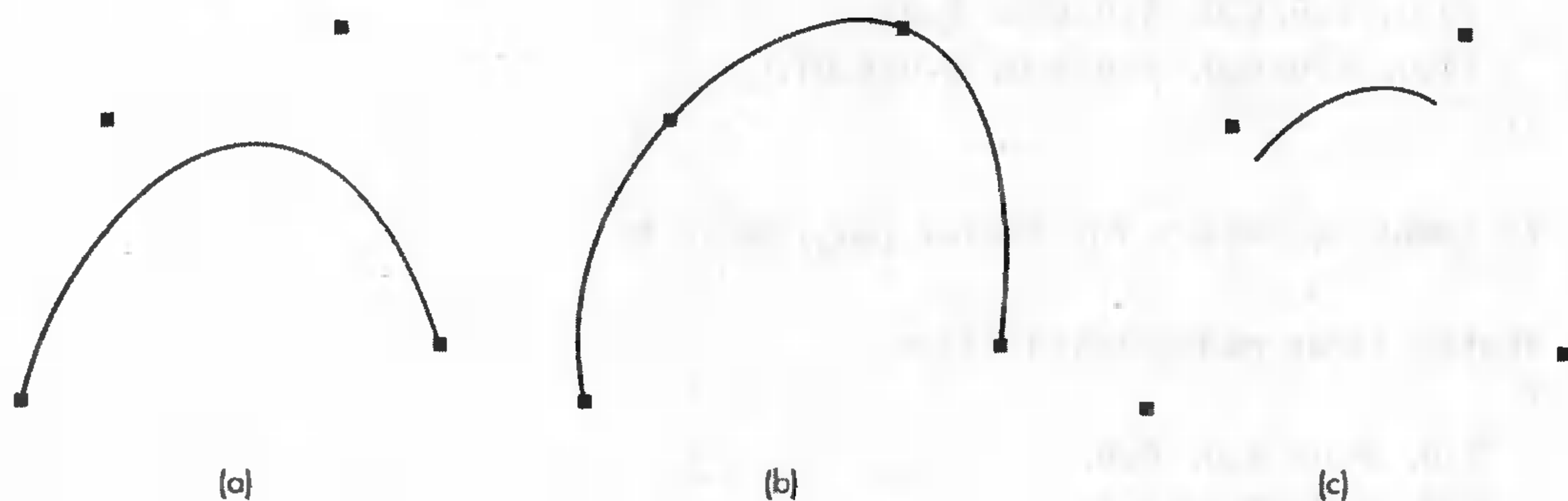


图 9.7 (a)贝塞尔曲线; (b)插值曲线; (c)由相同控制生成的 B 样条曲线

```
typedef enum
{
    BEZIER,
    INTERPOLATED,
    BSPLINE
} curveType;

void vmult(float m[4][4], float v[4][3], float r[4][3])
{
    int i, j, k;
    for(i = 0; i < 4; i++) for(j = 0; j < 3; j++)
    {
        r[i][j] = 0;
        for(k = 0; k < 4; k++) r[i][j] += m[i][k] * v[k][j];
    }
    /*interpolating to Bezier matrix */

    static float minterp[4][4] =
    {
        {1.0, 0.0, 0.0, 0.0},
        {-5.0/6.0, 3.0, -3.0/2.0, 1.0/3.0},
        {1.0/3.0, -3.0/2.0, 3.0, -5.0/6.0},
        {0.0, 0.0, 0.0, 1.0},
    },
```



```
/* B-spline to Bezier matrix */

static float mbspline[4][4] =
{
    {1.0/6.0, 4.0/6.0, 1.0/6.0, 0.0},
    {0.0, 4.0/6.0, 2.0/6.0, 0.0},
    {0.0, 2.0/6.0, 4.0/6.0, 0.0},
    {0.0, 1.0/6.0, 4.0/6.0, 1.0/6.0},
};

/* identity matrix for Bezier polynomial */

static float midentity[4][4] =
{
    1.0, 0.0, 0.0, 0.0,
    0.0, 1.0, 0.0, 0.0,
    0.0, 0.0, 1.0, 0.0,
    0.0, 0.0, 0.0, 1.0,
};

/* calculate the matrix used to transform the control points */

void computeMatrix(curveType type, float m[4][4])
{
    int i, j;

    switch (type)
    {
        case BEZIER:
            /* Identity matrix */
            m = midentity;
            break;
        case INTERPOLATED:
            m = minterp;
            break;
        case BSPLINE:
            m = mbspline;
            break;
    }
}

/* draw the indicated curves using the current control points */
```



```

static void drawCurves(curveType type)
{
    int i, j;
    int step;
    GLfloat newcpts[4][3];

    float m[4][4];

    /* set the control point computation matrix and the step size */

    computeMatrix(type, m);

    if(type == BSPLINE) step = 1;
    else step = 3;

    /*using global color matrix */

    glColor3fv(colors[type]);

    /* draw the curves */

    i = 0;
    while(i + 3 < ncpts)
    {
        /* calculate the appropriate control points */

        vmult(m, &cpts[i], newcpts);

        /* draw the curve using OpenGL evaluators */

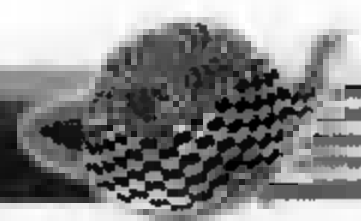
        glMap1f(GL_MAP1_VERTEX_3, 0.0, 1.0, 3, 4,
                &newcpts[0][0]);
        glMapGrid1f(30, 0.0, 1.0);
        glEvalMesh1(GL_LINE, 0, 30);

        /* advance to the next segment */

        i += step;
    }
    glFlush();
}

```

【221~224】



9.7.2 NURBS 曲线

NURBS(非均匀有理 B 样条, NonUniform Rational B-Spline)曲线和曲面来自于一个具有很大灵活性的曲线和曲面族, 并为 GLU 库所支持。NURBS 曲线是用四维参数多项式($x(u)$, $y(u)$, $z(u)$, $w(u)$)来描述的, 其中每个分量都可通过 OpenGL 求值器得到。通过将第 4 个分量 $w(u)$ 去除其余三个分量, 就可得到所需的三维点, 从而对应于 u 的每个值都有一个顶点($x(u)/w(u)$, $y(u)/w(u)$, $z(u)/w(u)$)与之对应。

9.8 犹 他 壶

犹他壶是计算机图形学中最著名的几何对象。它创建于 30 年前, 创建的初衷是为各种绘制算法提供一个标准的测试对象。这个茶壶由 32 个三次贝塞尔曲面片构成, 这些面片则是通过 306 个不同的控制点定义的。该数据集可以通过很多途径获得, 通常该数据以三维数组的形式给出, 共有 32 行, 每行有 16 个范围在[1, 306]内的整数。每个整数都是指向 306 个 (x , y , z)数据点的索引值。借助二维求值器, 可以很容易地将该茶壶显示出来。下面我们将编写一个简单的程序, 用线段的形式将茶壶绘制出来。

如果我们将数据放在两个文件 `vertices.h` 和 `patches.h` 中, 代码将会更简洁清晰。第一个文件中只定义了一个 `point` 类型的对象数组, 它给出了所有顶点的位置:

```
point vertices[306] = {{1.4, 0.0, 2.4}, ...}
```

同样, 第二个文件定义了一个三维索引数组, 它定义了构成各面片的顶点索引:

```
int indices[32][4][4] = {{1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16}, ...}
```

该数据集的编号方式要追溯到过去, 当时下标是从 1 开始的, 而非 0。所以, 第一个索引 `indices[0][0][0]` 的值为 1, 实际上是指 `vertices[0]`。其余代码非常简单明了。在 `main()` 函数中, 我们对该数据结构进行遍历, 并将所有的数据存入数组 `data[32][4][4][3]`。函数 `myinit()` 定义了一个二维网格, 该网格在每一维上都有 20 个均布于[0.0, 1.0]之间的点。这种数据指定方式使得茶壶朝向 z 轴正方向, 而茶壶的底部和顶部都与 y 轴垂直, 我们可通过旋转该茶壶而启动显示回调函数。为了创建等轴视图, 我们接下来还需要执行两种简单的旋转。 【224~225】

接下来我们讨论如何定义求值器。在这个例子中, 我们为每个面片使用了等间隔的网格。这种选择不够理想, 因为各个面片的尺寸和曲率都不同, 但是对于本例已经足够了。当用直线段和 `glEvalCoord2*()` 绘制时, 我们希望创建一个由线段构成的网格。可用如下函数来

定义一个求值器:

```
glMap2f(GL_MAP2_VERTEX_3, 0.0, 1.0, 3, 4, 0.0, 1.0, 12, 4, &data[k][0][0][0]);
```

注意, u 和 v 的跨度是不同的。在 u 中, 由于这些数据是按行存储的, 所以连续的控制点在数据数组中相隔三个浮点数。但在 v 中, 我们希望按列访问连续的控制点, 所以相邻两点之间相隔 12 个浮点数。

我们希望得到分别对应于令 u 取固定值而 v 为变量以及令 v 取固定值而 u 为变量的两种曲线的两组直线段。如果只绘制少量的折线, 但为每条折线计算出许多点, 仍然可以得到一条光滑而整齐的图像。我们可以在显示函数中通过两重循环来做到这一点; 第一重循环绘制 v 为常数时的曲线, 第二重循环绘制 u 为常数时的曲线。对于每个面片, 我们绘制 5 条以 u 为自变量的曲线和 5 条以 v 为自变量的曲线。每条曲线都通过使用 21 次求值器来近似。下面是完整的程序, 其运行结果如图 9.8 所示。

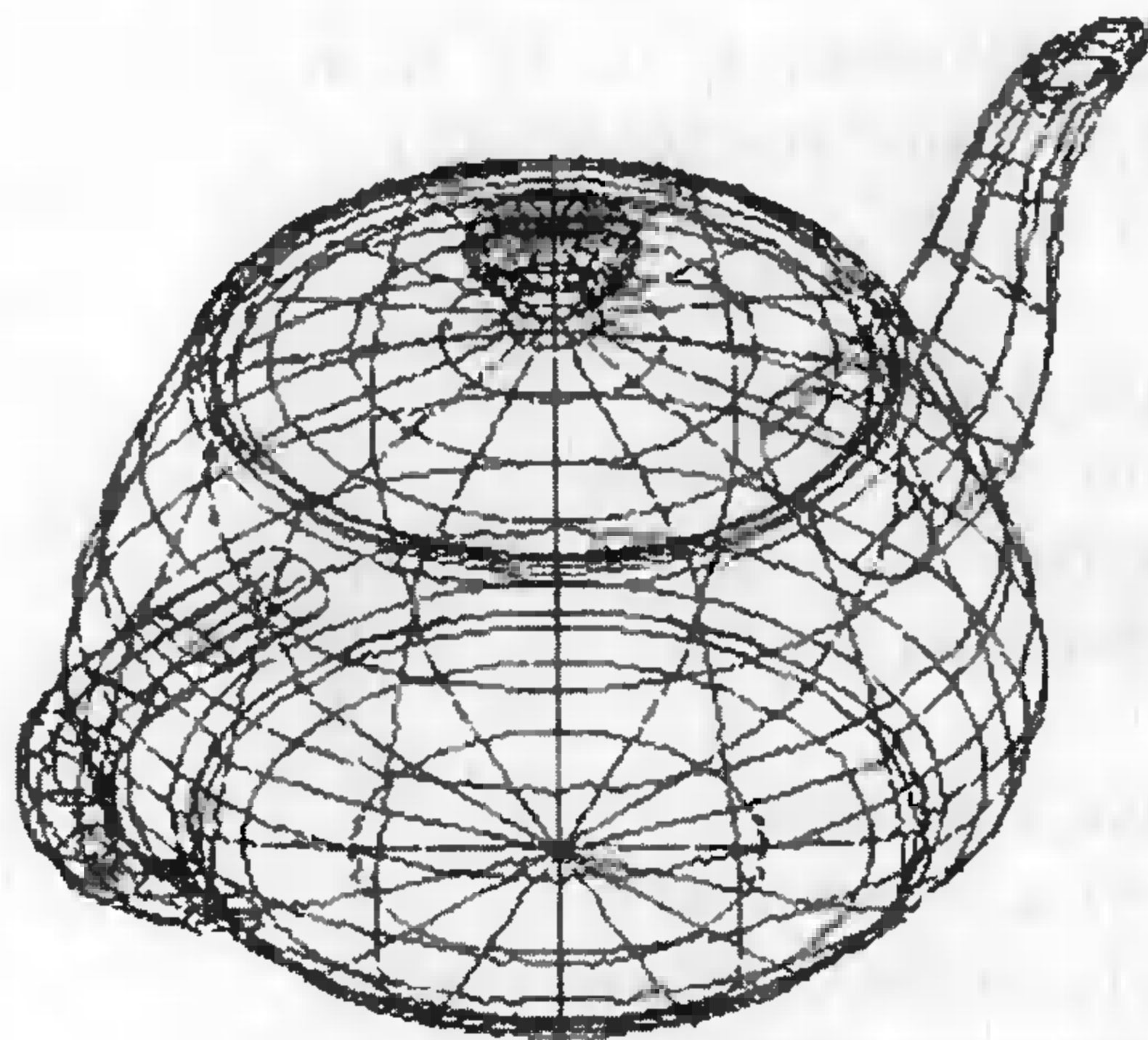


图 9.8 用 u 和 v 对应的一维网格绘制的犹他壶

```
#include <stdlib.h>
#include <GL/glut.h>

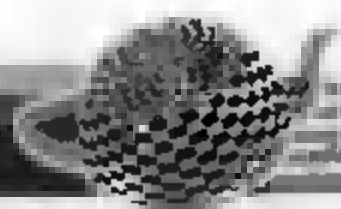
GLfloat data[32][4][4][3];

/* 306 vertices */

#include "vertices.h"

/* 32 patches */

#include "patches.h"
```

```
void display(void)
{
    int i, j, k;

    glClear(GL_COLOR_BUFFER_BIT);
    glColor3f(1.0, 1.0, 1.0);
    glLoadIdentity();
    glTranslatef(0.0, 0.0, -10.0);
    glRotatef(-35.26, 1.0, 0.0, 0.0);
    glRotatef(-45.0, 0.0, 1.0, 0.0);

    /* data aligned along z axis, rotate to align with y axis */

    glRotatef(-90.0, 1.0, 0.0, 0.0);
    for(k = 0; k < 32; k++)
    {
        glMap2f(GL_MAP2_VERTEX_3, 0, 1, 3, 4,
                0, 1, 12, 4, &data[k][0][0][0])
        for(j = 0; j <= 4; j++)
        {
            glBegin(GL_LINE_STRIP);
            for(i = 0; i <= 20; i++)
                glEvalCoord2f((GLfloat)i/20.0,
                             (GLfloat)j/4.0);
            glEnd();
            glBegin(GL_LINE_STRIP);
            for(i = 0; i <= 20; i++)
                glEvalCoord2f((GLfloat)i/4.0,
                             (GLfloat)i/20.0);
            glEnd();
        }
    }
    glFlush();
}

void myReshape(int w, int h)
{
    glViewport(0, 0, w, h);
    glMatrixMode(GL_PROJECTION);
    glLoadIdentity();
    if(w <= h)
        flOrtho(-4.0, 4.0, -4.0 * (GLfloat)h / (GLfloat)
                w, 4.0 * (GLfloat)h / (GLfloat)w, -20.0,
                20.0);
    else
```



```

        glOrtho(-4.0 * (GLfloat)w / (GLfloat)h, 4.0 *
                (GLfloat)w / (GLfloat)h, -4.0, 4.0, -20.0,
                20.0);
    glMatrixMode(GL_MODELVIEW);
}

void myinit()
{
    glEnable(GL_MAP2_VERTEX_3);
    glClearColor(1.0, 1.0, 1.0, 1.0);
}

main(int argc, char * argv[])
{
    int i, j, k, m, n;
    for(i = 0; i < 32; i++)for(j = 0; j < 4; j++)
        for(k = 0; k < 4; k++)for(n = 0; n < 3; n++)
        {
            /* put teapot data into single array for use by OpenGL */

            m = indices[i][j][k];
            for(n = 0; n < 3; n++) data[i][j][k][n] =
                vertices[m-1][n];
        }
    glutInit(&argc, argv);
    glutInitDisplayMode(GLUT_SINGLE | GLUT_RGB);
    glutInitWindowSize(500, 500);
    glutCreateWindow("teapot");
    myinit();
    glutReshapeFunc(myReshape);
    glutDisplayFunc(display);

    glutMainLoop();
}

```

【225~227】

由于我们使用的是规则网格，我们可采用另外一种方法，即通过使用函数 `glMapGrid2*()` 和 `glEvalMesh2()` 来避免所有的函数调用。于是，我们就可在程序的初始化部分用如下方式定义网格：

```
glMapGrid2f(4, 0.0, 1.0, 4, 0.0, 1.0);
```

并在显示回调函数中对该网格进行求值：


```

for (k = 0; k < 32; k++)
{
    glMap2f(GL_MAP2_VERTEX_3, 0, 1, 3, 4, 0, j, 12, 4, & data[k][0][0][0]);
    glEvalMesh2(GL_LINE, 0, d, d);
}

```

当 $d=2$ 时，所生成的图像如图 9.9 所示。

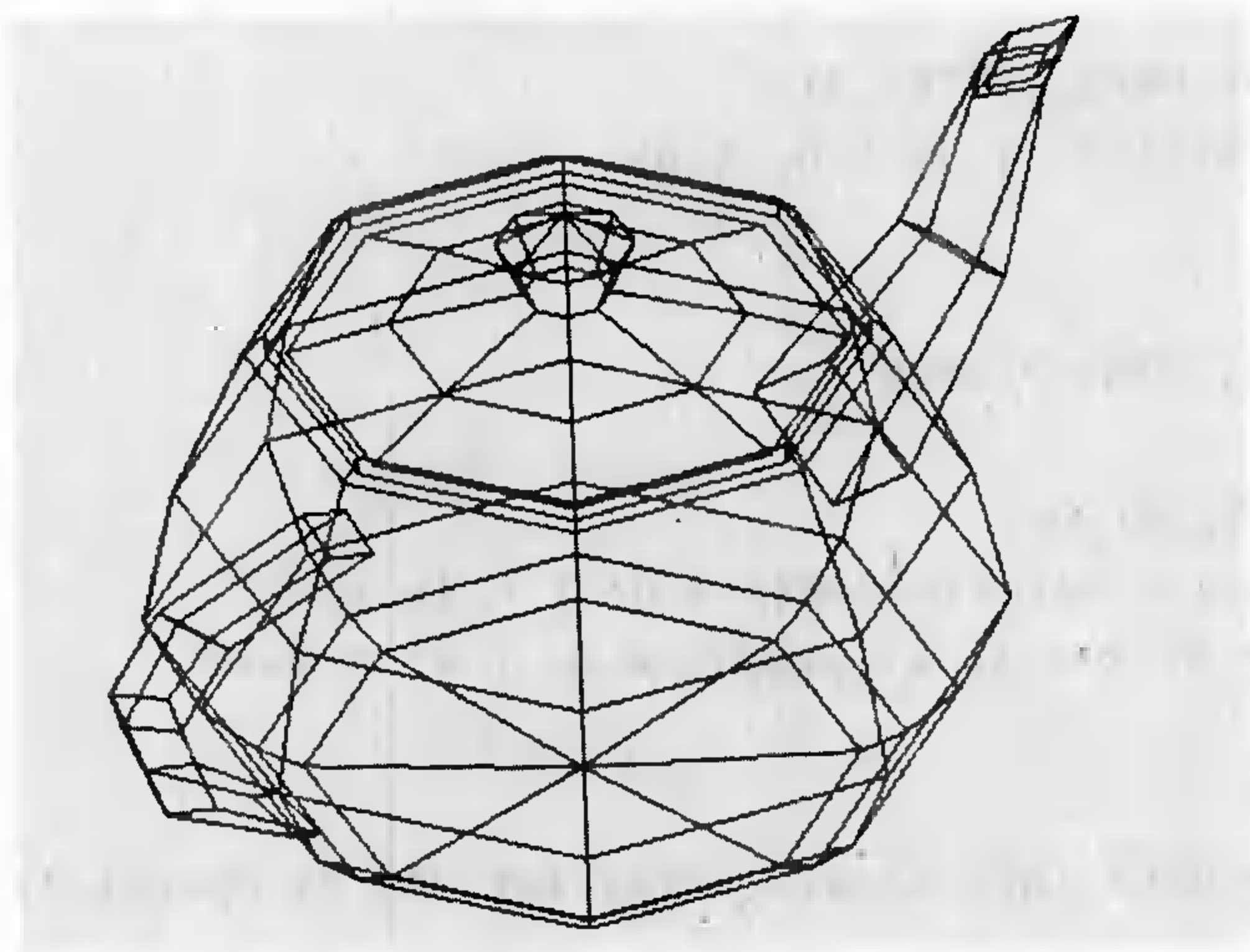


图 9.9 使用二维网格绘制的犹他壶

该图像所含的信息量不如前例的多，这是由于两个变量的取值间隔都比较大，而在前面的例子中，对于所绘制的每条多边形我们都使用了比较精细的分辨率。如果在每个方向上我们都使用更多数量的曲线，则会得到更平滑同时也更稠密的图像。例如，如果在 u 和 v 中使用 5 个间隔而非 2 个，则我们将得到图 9.10 所示的图像。

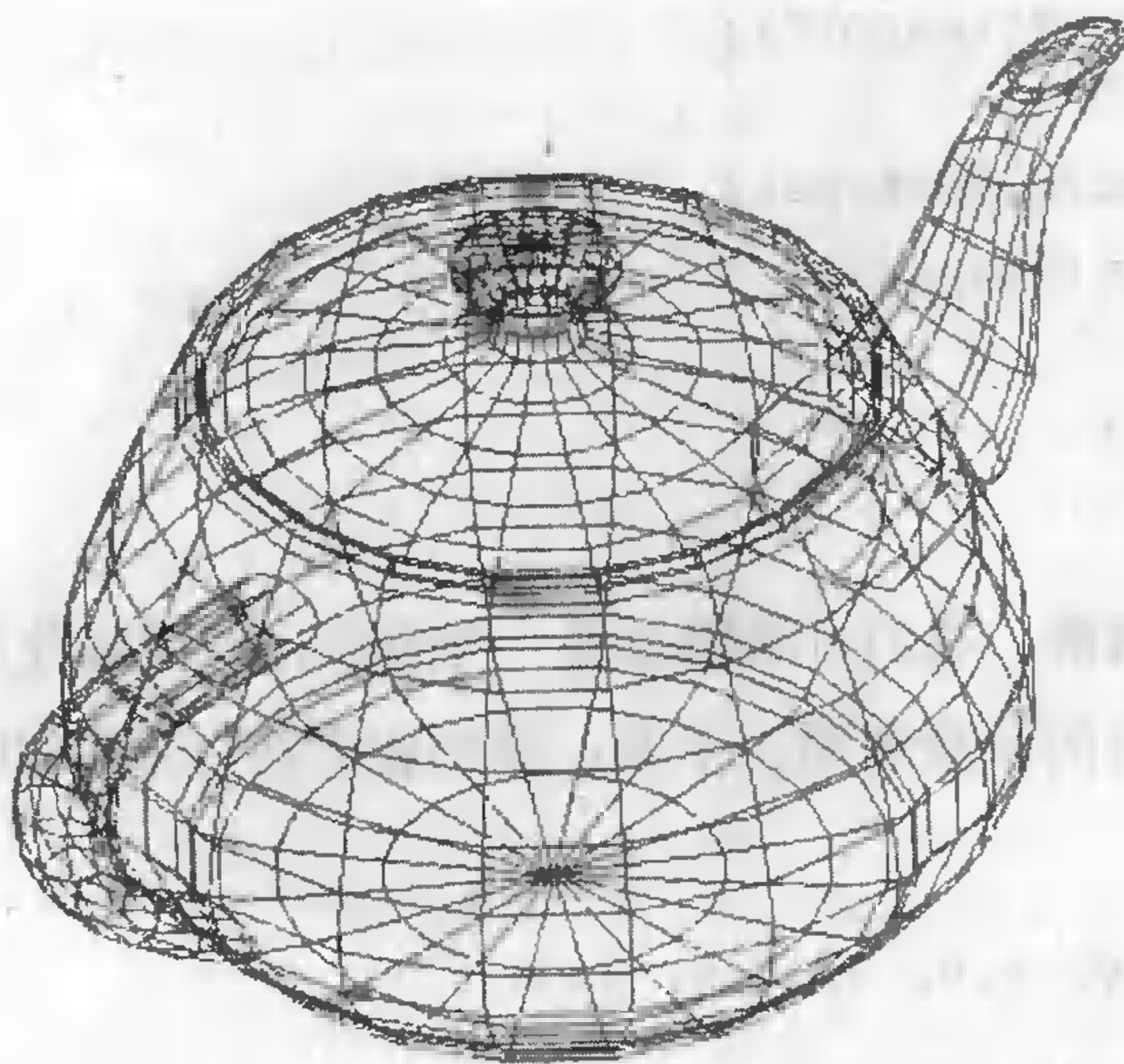


图 9.10 用二维网格绘制的更稠密的犹他壶

如果希望用线段来表现这个茶壶，虽想使用函数 `glEvalCoord*()` 会带来一些额外的函数调用开销，但可能还是一个比较好的选择。但是，如果我们希望用多边形(`GL_FILL`)而非线段(`GL_LINE`)来表现茶壶，使用函数 `glEvalMesh2()` 应该更好一些。如果选择前者，我们还必须仔细考虑如何对明暗进行处理。 【228~229】

9.9 法向量与明暗

希望使用明暗效果时，需要定义法向量。正如我们在第6章中所看到的，在 OpenGL 中，法向量在应用程序中通常是借助函数 `glNormal*()` 来定义的。如果希望获得最平滑的明暗效果，则应该在定义每个顶点之前首先定义一个新的法向量。对于曲面相应地应该在每次调用 `glEvalCoord*()` 之前定义一个新法向量。虽然理论上可以这样做，但实际上必须为在求值器中隐式定义的所有顶点计算法向量。这样的计算涉及一些参数曲面的数学知识，但我们并不能奢望所有的程序员都已掌握这些知识。对于规则网格，情况更糟，因为在这种情况下我们并不是在一个循环内对函数 `glEvalCoord*()` 多次调用，而是只对 `glEvalMesh2()` 进行一次调用，这就使得程序中完全没有定义法向量的机会。

OpenGL 通过自动为这些曲面计算法向量来解决这一问题。为了生成这些法向量，我们只需要用如下方式启用自动法线计算功能：

```
glEnable(GL_AUTO_NORMAL);
```

所以，为了生成一个具有明暗效果的茶壶，我们可以在构造茶壶的代码段一开始的位置使用函数 `glMapGrid2f()` 和 `glEvalMesh2()`。例如，如果我们希望每个面片都用 64 个多边形来逼近表示，则可以像下面这样做：

```
glMapGrid2f(8, 0.0, 1.0, 8, 0.0, 1.0);
glEvalMesh2(GL_FILL, 0, 8, 0, 8);
```

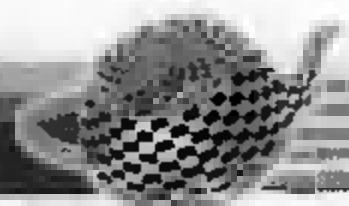
【229】

在函数 `main()` 或 `init()` 内部，我们可以启用自动法向量计算功能，并定义我们所期望的光照和材质。

例如，将光照和材质属性按如下方式设置将得到图 9.11 所示的结果。

```
GLfloat ambient[] = {0.2, 0.2, 0.2, 1.0};
GLfloat position[] = {0.0, 0.0, 2.0, 1.0};
GLfloat mat_diffuse[] = {0.6, 0.6, 0.6, 1.0};
GLfloat mat_specular[] = {1.0, 1.0, 1.0, 1.0};
GLfloat mat_shininess[] = {50.0};
```

```
glEnable(GL_LIGHTING);
```

```
glEnable(GL_LIGHT0);  
  
glLightfv(GL_LIGHT0, GL_AMBIENT, ambient);  
glLightfv(GL_LIGHT0, GL_POSITION, position);  
  
glMaterialfv(GL_FRONT, GL_DIFFUSE, mat_diffuse);  
glMaterialfv(GL_FRONT, GL_SPECULAR, mat_specular);  
glMaterialfv(GL_FRONT, GL_SHININESS, mat_shininess);
```



图 9.11 使用自动法向量计算功能绘制出的茶壶

如果仅用 16 个多边形来近似每个面片，我们将得到一幅略微粗糙的图像，如图 9.12 所示。注意，此时每个面片上的明暗过渡仍然非常平滑，但是图中外围的轮廓边缘还是比较明显，这正是由于表达每个面片时我们没有使用足够多的多边形所造成的。



图 9.12 用少于图 9.11 的多边形绘制出的茶壶

【230~231】

9.10 为表面映射纹理

在第8章中，我们通过使用 OpenGL 的自动纹理坐标生成功能介绍了如何对二次曲面对象进行纹理映射。可将同样的技术应用于贝塞尔曲面。也可以使用求值器来为明暗和纹理坐标计算生成法向量。于是，如果我们乐意接受由这种特殊机制决定的比例变换和扭曲变形，这个过程可以实现自动化。

通过使用函数 `glutSolidTeapot()`，并使用纹理，可以看到一个自动坐标生成的例子。在茶壶定义的内部，我们可以为顶点创建一个二维求值器：

```
glMap2f(GL_MAP2_VERTEX_3, 0.0, 1.0, 3, 4, 0.0, 1.0, 12, 4, &P[0][0][0]);
```

其中，数组 `p` 包含用于单个面片的顶点数据。第二个求值器通过如下方式创建：

```
glMap2f(GL_MAP2_TEXTURE_COORD_2, 0.0, 1.0, 2, 2, 0.0, 1.0, 4, 2,
&tex[0][0][0]);
```

其中，数组 `tex` 包含顶点的纹理坐标。注意，由于这些面片是三维的双三次曲面，而纹理是二维的，所以定义求值器时维数和跨度都不同。数组 `tex` 仅包含单位正方形 4 个顶点的坐标：

```
float tex[2][2][2] = {0.0, 0.0, 1.0, 0.0, 0.0, 1.0, 1.0, 1.0};
```

所以每个面片都被映射了完整的纹理。`glutSolidTeapot()`启用了自动法向量计算功能，所以我们便不再需要第三个求值器来对法向量进行插值。 【231~232】

该应用程序只需定义纹理图及其参数，启用纹理映射并绘制茶壶即可。若纹理图为棋盘纹理，我们将得到如图 9.13 所示的效果。茶壶上的每个方格尺寸不同，这是由于茶壶由 32 个独立的曲面片构成，每个面片都有自己的尺寸。在函数 `glutSolidTeapot()`中曲面的自动纹理生成功能为每个曲面片使用了相同范围的 s 和 t 。我们也可以通过启用纹理生成功能并像我们在第 8 章中那样为纹理坐标定义线性方程组而使用自动纹理坐标生成功能。

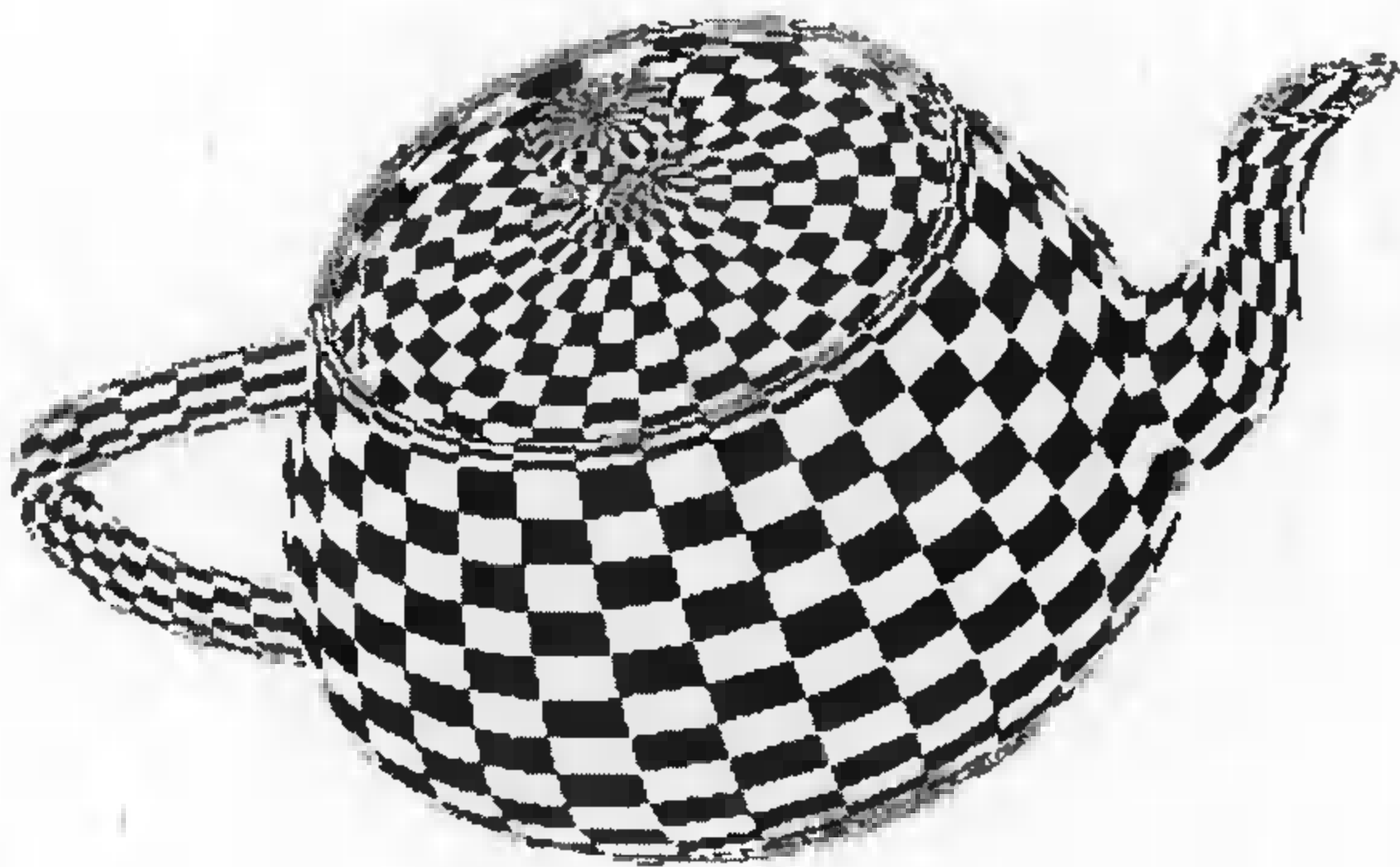
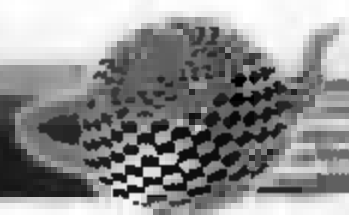


图 9.13 使用自动纹理坐标生成功能所得到的茶壶

【232】



9.11 编程练习

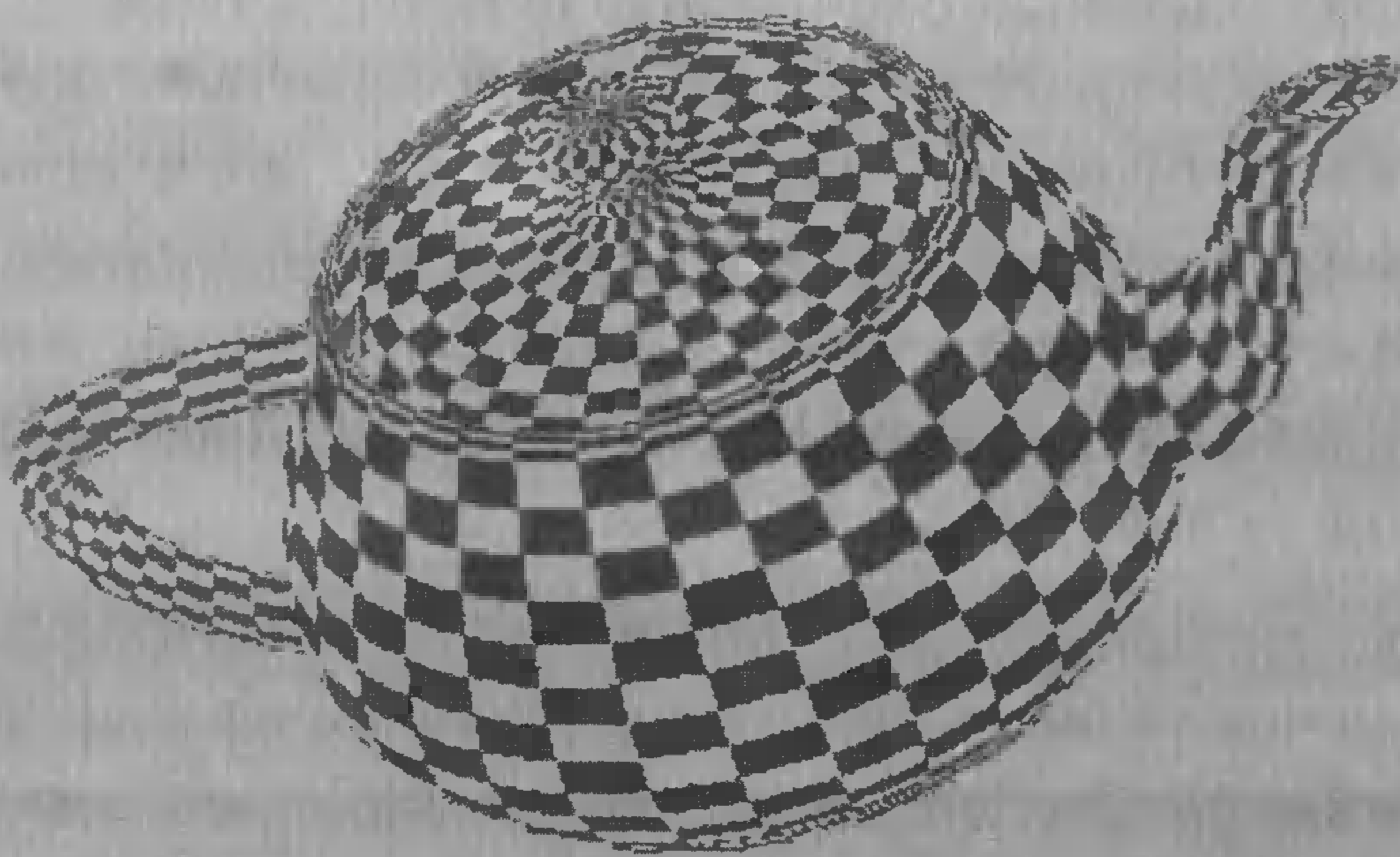
1. 对用于比较贝塞尔曲线、B样条曲线和插值曲线的交互程序进行扩展，以对相应的曲面进行比较。
2. 显示一个带有棋盘图案的茶壶，要求所有被映射的正方形的尺寸都近似相等。
3. 重新编写本章的曲线生成交互程序，使其能够生成三维曲线。请仔细考虑以何种交互方式输入控制点。
4. 从网上寻找带有高程(海拔高度)数据的地形数据。依据这些数据，用贝塞尔曲面片来表现这些数据。



第 10 章

OpenGL 着色语言

在本章中，我们将介绍图形系统的一个主要的新特性：可编程流水线。所有最新的显卡都允许程序开发人员在编程时同时使用集合处理器和片段处理器。比较新的 OpenGL 版本通过 OpenGL 着色语言(GLSL, OpenGL Shading Language)来支持这种强大功能。



10.1 对流水线的回顾

首先我们借助图 10.1 所示的简化框图站在更高的层次来重新审视一下几何流水线，该框图本质上与我们在第 1 章中介绍的流水线完全相同。

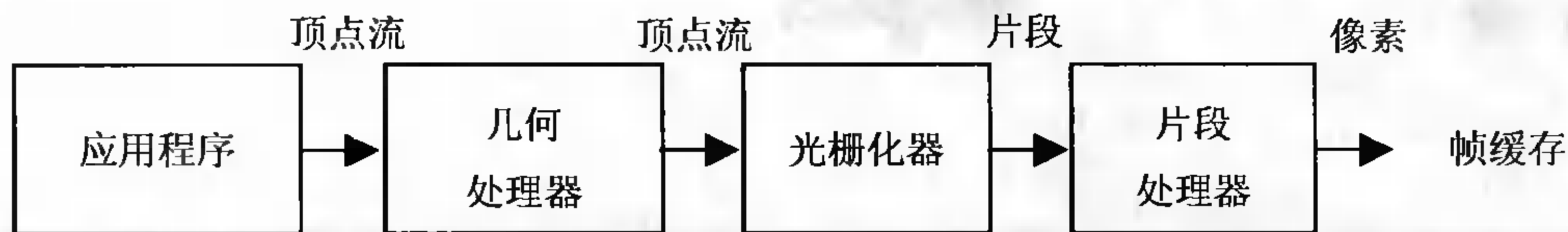


图 10.1 OpenGL 流水线

这里我们将重点放在流水线中每个模块的输入和输出上。应用程序可以对各种状态变量进行修改，这些状态变量在流水线的各个环节都可被获取，但为了创建可见的输出，应用程序要定义一系列封装在 `glBegin` 和 `glEnd` 之间的顶点，而这些顶点定义了一个或多个结合图元。几何处理器接受这些顶点，然后依据被启用的 OpenGL 的各项功能对这些顶点执行一系列运算，包括模型视图变换、投影变换、顶点光照计算以及裁剪等。几何处理器的输出是一个装配为几何图元的顶点流。每个顶点都与各种属性建立了关联，如颜色、明暗、法向量以及纹理坐标，这些属性可以通过指定或计算得到。【233~234】

光栅化器要完成两项任务。第一项任务是对于每个图元，必须确定帧缓存中的哪些像素将会受到影响，而且必须将这些可能的像素或片段输出。第二项任务是光栅化器必须计算出每个片段的性质或属性。这些属性对应于从几何处理器中输出的顶点属性。光栅化器依据各顶点的值进行插值以便得到每个片段的值。例如，为每个顶点定义的明暗和颜色都可通过对每个片段进行插值计算得到。

片段处理器接收一个片段流，然后分两阶段输出一个片段流。在第一个阶段中，由光栅化器生成的每个片段的颜色被确定。该颜色值可以是由光栅化器通过对顶点颜色进行插值而得到；或者也可以是从一幅纹理图中得到的颜色，或者是以上两种颜色来源的某种组合。如果启用了融合运算，则片段颜色还将受到颜色缓存中已有颜色的影响。一旦片段颜色计算完毕，片段在被放入当前颜色缓存(通常为后台缓存)之前，将经历一系列测试。其中一项测试就是判断一个片段是否位于另一个片段之后(隐藏面测试)。另外一项测试是判断片段是否被一幅位图作了掩膜处理。

虽然我们可将顶点处理器和片段处理器视为实现了某些算法的黑箱，但实际上所有的现代显卡中顶点处理器和片段处理器都是分别针对顶点流和片段流工作的专用计算机。虽然每种处理器在微型计算机上都有相应的组件，但由于每种处理器所负责的工作过于专业化，

它们之间的差异相当巨大。最重要的一点是目前这两种处理器都是可编程的，这就使得应用程序开发人员使用不同于默认 OpenGL 流水线完成任务的方式来处理顶点和片段。

接下来我们将讨论如何对这些处理器编程。您将惊奇地发现，答案竟来自于离线绘制领域。

10.2 着色器与着色语言

动画产业要求图像最终输出时具有较高的分辨率。一方面我们希望这些图像满足所需的分辨率；另一方面又希望能够在这些图像中添加一些特效，这就使得这些图像的实时生成难于实现。我们采取的策略是，不是让一个单个流水线处理整个计算机图像生成过程，而是将其分为两个阶段：建模阶段和绘制阶段。建模阶段的主要任务是创建几何对象并使其运动起来。在这一阶段，并不需要高分辨率的输出，也不需要高质量的绘制。这部分处理可以在标准工作站用商业显卡来完成，通常还需要利用一些基于 OpenGL 的软件。【234~235】

该过程的第二个阶段——绘制，通常是离线完成的，并无实时性要求。实际上，要绘制一帧包含许多对象和复杂绘制效果的图像，可能需要花费若干分钟甚至若干小时。该环节通常是在一簇通用处理器中完成。用于建模阶段和绘制阶段的软硬件是不同的。它们之间惟一的联系就是一个以文件形式存在的接口，该文件中保存着建模器的输出，并作为绘制器的输入。这样的文件包含几何对象及其运动的描述信息、属性(如表面属性)信息、光源描述信息、效果的指定以及一些可能存在于绘制器中的数据(如纹理)。Pixar 公司指定了这样一种文件格式，目前该格式已标准化，并成为其他形式的接口的基础。

10.2.1 RenderMan

在绘制器内部，我们必须能够以某种方式来描述如何以一种复杂的方式来对几何对象的表面上色，以便获得比修正的 Phong 模型更好的表面材质模拟效果，或者允许程序员创建新效果。负责完成这些任务的程序称为着色器(shader)，这是由于这些程序负责为表面或其他类型的对象赋予明暗效果的缘故。我们特别感兴趣的是蕴含在 Pixar 的 RenderMan 着色语言中的思想，这种着色语言赋予了程序员极大的灵活性，使其可以设计出更复杂的着色器。我们先来研究一下标准的 Phong 着色器，从中我们可以获得对这种着色语言基本要素的一些了解。标准的 Phong 着色器由三个组件：漫反射、镜面反射以及环境反射，都可通过使用材质属性、光源属性、顶点位置以及第 6 章介绍的 4 个向量独立计算得到。我们可以用标准的代数运算和标准的向量运算(如点积)来为 Phong 着色器编写代数表达式。大多数程序员都知道，标准的代数表达式可用树结构来描述。一旦我们意识到 Phong 着色器可用树型数据结构来描述，通过添加节点以及修改节点内容，我们就可轻而易举地将这个概念推广

到其他着色器中。这个着色树(shading tree)的概念是近年来人们对着色语言进行研究的基础。

除了着色树, RenderMan 还包含一个对所有着色语言都非常关键的概念——环境变量(environmental variable)。开发人员可以假定着色计算所需的某些变量,如法向量、纹理坐标以及光源信息等,都处在环境中,并可为着色器所使用。虽然在 RenderMan 发展初期,其功能仅限于离线绘制,但可编程流水线可用类似于 RenderMan 着色语言的语言形式实时处理许多表达式。【235】

10.2.2 Cg 与 GLSL

再回到图形流水线,我们可将顶点处理器和片段处理器看作是专门用途的处理器,其主要任务是分别计算顶点和片段的明暗度。因此,我们可将这些处理器分别称为顶点着色器(vertex shader)和片段着色器(fragment shader)。控制这些着色器的程序也被称为着色器。在可编程着色器的使用初期,它们只能用非常底层的汇编语言来编制,当程序运行时,这些着色器可被应用程序加载到硬件中。在这种层次上进行编程,就要求应用程序对所有的底层任务(通常都是与汇编语言相关的)进行跟踪,例如对寄存器的使用,而且需要了解着色器体系结构的各个细节。然而正是受 RenderMan 着色语言的启发,才有了另外两种可编程着色器高级语言:Cg 和 GLSL。

“Cg”代表“C for Graphics”,即用于图形程序的 C 语言,它是由 NVIDIA 和微软公司开发的,开发的初衷是为在 Windows 平台上通过 OpenGL 和 DirectX 来支持可编程硬件。Cg 实际上与微软的高级着色语言 HLSL 是等同的。OpenGL 着色语言(GLSL)是从 ARB 扩展中衍生而来的,该扩展于 2003 年被采用。这两种高级着色语言都是 C 风格的,同时具有一些 C++语言的特征。这两种高级着色语言都包括了二维、三维和四维矩阵和向量数据类型,并将其作为基本类型,此外还包括了一些对实现图形功能极其重要的内置函数,如反射和长度函数。这两种高级着色语言都为访问 OpenGL 状态变量以及与 OpenGL 应用程序进行通信的简单机制。本章中我们将只关注 GLSL。

10.3 GLSL

我们先来研究一个非常简单的顶点着色器以及一个非常简单的片段着色器。接着介绍这种语言的基础知识。之后我们将讨论应用程序与着色器之间的接口。这里我们将介绍一组 OpenGL 函数,借助该组函数我们就可以将着色器加载到流水线中,并在应用程序与着色器之间,以及顶点着色器和片段着色器之间进行值的传递。最后,我们将展示一些着色器的

例子。

10.3.1 执行模型

当顶点流和片段流通过 OpenGL 流水线时,着色器将对其执行某些操作。因此,顶点着色器实际上是一段对每一个到达顶点处理器的顶点都执行某些操作的程序。虽然每个顶点都由同样的程序(直到一段新程序被加载到处理器中)进行处理,该程序实际上在顶点到达处理器时,依其位置和 OpenGL 当前状态的不同,对不同的顶点操作的结果也可能不同。片段处理也是如此。光栅化器生成片段,每个片段都由片段着色器来进行处理。注意,必须特别关注各个变量在程序执行过程中发生的改变。例如,如果应用程序在指定两个顶点之间修改了一个变量,如纹理坐标或材质属性,对于顶点着色器是有意义的。但这样的修改对片段着色器毫无意义。我们将会看到,在 GLSL 中存在许多对变量进行限定的方式。【236~237】

我们可以将自己编制的顶点着色器或片段着色器加载到处理器中。我们可以只加载其中一种,也可以两种同时加载。如果我们没有使用自己的着色器,则 OpenGL 将用默认的着色器来完成标准 OpenGL 流水线中的所有运算。但是,一旦我们决定使用自己编写的着色器,该着色器就必须完成它所对应的处理阶段所应完成的所有操作。例如,即使我们想用顶点着色器来完成如设置某种固定颜色这样简单的操作,该着色器也必须完成坐标变换(通过模型视图变换和投影变换来实现)。幸运的是,着色器可以获取所有所需的信息。必须记住对这些信息。

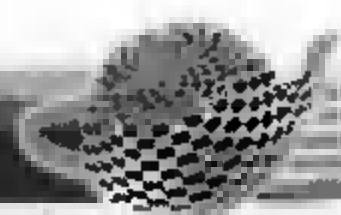
10.3.2 一个简单的顶点着色器

下面是一个比较简单的顶点着色器,尽管很简单,它还是展示了 GLSL 的许多主要特性:

```
const vec4 red = vec4(1.0, 0.0, 0.0, 1.0);

void main(void)
{
    gl_Position = gl_ModelViewProjectionMatrix * gl_Vertex;
    gl_FrontColor = red;
}
```

第一行用了一个 C++ 风格的构造函数来创建一个含有 4 个元素的浮点型向量 `vec4`。每次执行时,应用程序都会为该顶点着色器提供一个描述了顶点位置的四维向量 `gl_Vertex`。每个顶点程序必须将顶点位置保存在向量 `gl_Position` 中。注意,所有以 `gl_` 为前缀的变量都是 OpenGL 状态的一部分。在大多数情况下,我们希望自己定义的着色器能够完成模型视图矩阵和投影矩阵所做的工作。它们的乘积也是 OpenGL 状态的一部分,并保存在变量



`gl_ModelViewProjectionMatrix` 中。如果我们将到来的顶点与该矩阵相乘,我们将得到该顶点在裁剪坐标系中的正确位置。GLSL 用其基本类型来完成所需的矩阵运算。接下来的一行将前景色(一个四维向量)设为红色,这就是我们必须完成的所有任务。

该着色器的功能有些过于简单,因为我们基本上不可能希望所有的顶点都取同一种颜色。如果将最后一行去掉,不对前景色进行设置,该段程序仍然构成一个合法的顶点着色器,我们称之为流经着色器(pass-through shader, 亦称穿透式着色器),该着色器将确定每个片段颜色的任务交给了片段着色器来处理,颜色的确定有可能是利用一幅纹理图。如果我们用下面的代码取代最后一行:

```
gl_FrontColor = gl_Color;
```

就将应用程序中与每个顶点关联的颜色传递给光栅化器。在光栅化器中,顶点颜色集将对由这些顶点定义的图元进行插值,而无论该图元是何种类型。

10.3.3 一个简单的片段着色器

由光栅化器生成的每个片段都将引发片段着色器的执行。每个片段都由一个位置属性,即该片段在帧缓存中的位置,这个位置属性是不可修改的。片段着色器至少应该为每个片段生成一种颜色 `gl_FragColor`。下面是一个流经片段着色器:

```
void main(void)
{
    gl_FragColor = gl_FrontColor;
}
```

该着色器假定颜色的设置都是由应用程序或顶点着色器来负责完成的。这里的变量 `gl_FrontColor` 与前面顶点着色器中的 `gl_FrontColor` 并不完全相同,前者是由来自顶点着色器的 `gl_FrontColor`(它是由光栅化器自动生成的)经过插值计算得到的。我们又一次看到,OpenGL 已知的变量是内置的,且无需定义。

注意,顶点着色器和片段着色器看起来非常类似。二者都有一个主函数,而且使用的数据类型也很类似。就 GLSL 而言,二者是完全相同的,借助 GLSL 我们可用相同的语言编写出两种类型的着色器。这些着色器的使用方式也非常类似。例如,在光照计算时,我们既可以顶点为单位,也可以片段为单位,代码看起来都是很相似的。这些着色器也可以有较大差异。我们可能希望在片段着色器中以某种方式对纹理进行操作,而这种方式很可能对于顶点着色器来说毫无意义。真正将二者区分的标志是我们对着色器的编译方式以及每种着色器被加载到的处理器的类型。在介绍了 GLSL 的基础知识后,我们将对如何进行这些运算进行讨论。

10.4 GLSL 语言基础

在 GLSL 中,基本数据类型与 C 和 C++ 中的数据类型非常类似。由于目前显卡的体系结构对数值类型的支持还十分有限,所以 GLSL 只有三种基本类型: float, int 以及 bool。此外,虽然 GLSL 确实包含一些 C 风格的数组和结构,但不包括指针。

GLSL 具有二维、三维和四维的矩阵和向量类型,由于它们的基本类型,所以无需借助指针也可在函数之间进行复制。向量类型分为浮点型(vec2, vec3, vec4)、整型(ivec2, ivec3, ivec4)和布尔型(bvec2, bvec3, bvec4)。矩阵都是方阵,类型只有浮点型(mat2, mat3, mat4)。我们可用通常的 C 的下标索引方式来引用矩阵中的单个元素,例如:

```
vec3 a;
mat3 b;

a[0] = 1.0;
b[1][2] = 2.0;
```

【238~239】

我们可利用选择运算符来指定一个或多个元素。一个三维或四维的向量可用于指定某个位置、某个纹理坐标或某种颜色。在第一种情形中,我们倾向于使用 x、y、z、w 来表示各分量。而对颜色,我们一般用大家熟知的 r、g、b、a 来表示各颜色分量。对于纹理, GLSL 则使用 s、t、p、q。所以,如果 c 是一个向量,则 c[0], c.r, c.x 和 c.s 都是指 c 的第一个分量。我们可用选择运算符及 C++ 风格的构造函数来编写代码,例如:

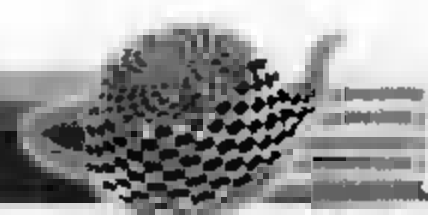
```
vec3 a = vec3(1.0, 2.0, 3.0);
vec3 b;
b.xy = a.yz;
b.z = 2.0;
```

10.4.1 限定符

在 GLSL 的函数中定义的简单变量的使用方式与 C 语言非常相似。我们可以定义类型为 float, int 以及 bool 的变量,也可定义这些类型的数组。我们也可在定义变量时使用限定符 const, 例如:

```
const vec4 red = vec4(1.0, 0.0, 0.0, 1.0);
```

于是,在定义该变量的函数中,该变量的值便不可被修改。如果一个变量同时在应用程序和着色器中出现,该变量发生变化时应如何处理则变得非常复杂。例如,如果在应用程序中定义了一个时间变量,我们可能不希望在指定一个图元期间对该变量发生变化,也不希



望该值被着色器修改。在应用程序中为不同的顶点指定其他变量(如颜色)时,则这些变量应该是可变的。

如果一个变量在定义时使用了限定符 `uniform`,则在 `glBegin` 和 `glEnd` 之间该变量不可被修改,所以对于一个图元的所有顶点,该变量的值都是相同的。在着色器内部,我们可在主函数的声明之前指定一个 `uniform` 变量,如下所示:

```
uniform float time;  
void main(void)  
{
```

下一节将讨论一个难度更大的问题,即如何将着色器中的 `uniform` 变量与应用程序中的变量进行匹配。 【239】

OpenGL 状态变量,如颜色、纹理坐标和法向量等在应用程序中为不同顶点指定时,是允许被修改的。虽然这些变量也是 OpenGL 状态的一部分,但实际上它们的用途是为顶点的属性赋值,所以我们称之为 `attribute` 限定变量。正如前面那个简单的例子,OpenGL 状态变量,(如 `gl_Vertex` 和 `gl_FrontColor`)可为着色器获取,而无需在代码中显式指定,所以这种变量称为内置变量。我们可能经常需要将一些与应用程序有关的变量以顶点为单位传入着色器中。例如,在一个流体模拟应用程序中,我们可能希望将每个顶点的速度都传入顶点着色器中。我们可以用一种类似于定义 `uniform` 变量的方式来定义这样一个 `attribute` 限定的变量,如下列代码段所示:

```
attribute vec3 velocity;  
  
void main(void)  
{
```

我们还需要一个能够将数据从顶点着色器传递到片段着色器的方法。在前面的简单示例中,我们看到顶点着色器中的 OpenGL 状态变量,(如 `gl_FrontColor`)是由光栅化器进行插值计算的,而且在片段着色器中可为每个片段所用。为了在两种着色器之间传递数据,GLSL 使用 `varying` 限定的变量,这些变量在顶点着色器中定义,然后被传递给光栅化器。光栅化器依据这些变量进行插值,并将每个片段的值传递给片段着色器。例如,假定在前面那个简单的顶点着色器示例中,我们用一个 `varying` 变量 `color_out` 来替换 `gl_FrontColor` 的使用,就得到如下所示的顶点着色器:

```
varying vec4 color_out;  
  
void main(void)  
{
```



```

    gl_Position = gl_ModelViewProjectionMatrix * gl_Vertex;
    color_out = gl_Color;
}

```

在上述代码中，我们并没有使用颜色常量，而是依据 OpenGL 状态赋给 `varying` 变量一定的颜色值。一个相应的流经片段着色器如下所示：

```

varying vec4 color_out;

void main(void)
{
    gl_FragColor = color_out
}

```

【240】

虽然这些着色器非常简短，但其中包含丰富的信息。应用程序中每个顶点的颜色都被传给光栅化器，然后光栅化器对其进行插值计算，就得到每个片段的颜色值。这些着色器将执行默认的平滑着色。

变量还可依据其作为函数参数的用途来进行限定。由于在 GLSL 中没有指针，所以在函数调用时参数直接复制给被调函数，而返回值也是通过复制而被返回。这些参数可用限定符 `in`、`out` 或 `inout` 来修饰。例如，让我们来考虑一个用如下方式定义的函数：

```
void func(in float a, out vec3 b, inout mat4 c, int d)
```

在该函数被调用时，`a` 的值将被复制进来，并可为 `func` 函数所用。但是任何对该值的改变都不会反传。参数 `b` 的值在进入函数 `func` 时处于不定状态，但是在 `func` 中对 `b` 所做的任何修改都将通过复制返回给调用函数。`c` 既可作为传入参数被复制给 `func`，也可作为返回值而返回给调用函数。任何没有限定符的参数，如 `d`，都被认为是 `in` 限定的变量。虽然没有指针，但由于矩阵和向量都是基本类型变量，所以如果被正确地限定，这些变量既可作为传入参数，也可作为返回参数。其他类型的数组可通过将其嵌入在 C 类型的结构中而被来回复制。

10.4.2 运算符

标准算术运算是 GLSL 的一部分。这些运算也适用于矩阵和向量类型，所以如下表达式是有意义的：

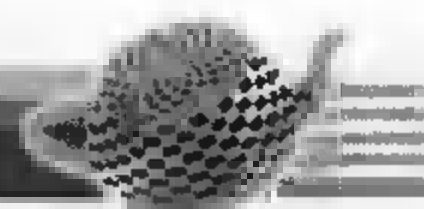
```

vec4 a, b, c;
c = a + b;

```

对于那些涵盖矩阵、向量和基本类型的运算，运算符将被重载。例如，考虑如下代码段：

```
vec4 a, b, c;
```

```
mat4 d;
```

```
c = m * a;
```

```
d = a * m;
```

GLSL 会将 c 作为 4×4 矩阵 m 与一个由 a 指定的 4×1 矩阵的乘积来计算,但会将 d 作为由 a 指定的 1×4 的矩阵与 4×4 矩阵 m 的乘积来计算。可见,为了使 GLSL 的表达式有意义, GLSL 以两种不同的方式来使用向量 a 。在两种情形下,计算结果的类型都是 `vec4`,它既非 1×4 的矩阵,也非 4×1 的矩阵,而是一个 GLSL 的基本类型。注意, c 和 d 的每个分量都不相同。

GLSL 包括一个选择运算符的变体,称为多重选择运算符(`swizzling`),该运算符允许我们以一种优雅的方式对向量和矩阵类型变量的分量进行选择 and 重新排列。我们可以像下面的例子那样使用多重选择运算符一次选定向量的多个分量。在该例中,我们定义一个 RGBA 颜色值,然后又形成一个 RGB 颜色:

```
vec4 red4 = vec4(1.0, 0.0, 0.0, 1.0);
```

```
vec3 red3;
```

```
red3 = red4.rgb;
```

【241~242】

或者我们也可像下列代码那样将某些系统中默认的 BGR 颜色转为 RGB 颜色:

```
vec3 red_in = vec3(0.0, 0.0, 1.0);
```

```
vec3 red_out;
```

```
red_out.rgb = red_in.gbr;
```

10.4.3 控制结构

GLSL 使用标准 C 的方法来进行程序流控制,即 `if`, `for`, `while` 以及 `do-while`。如果您的 GPU 比较低端,使用循环的着色器也许可以编译通过,但执行起来将异常缓慢,甚至可能无法运行。

10.4.4 内置函数

GLSL 包含标准的数学和三角函数,包括 `pow`(幂), `exp`, `abs`, `sqrt`, `max`, `min`, `sin`, `cos`, `tan`, `asin`(`sin` 的反函数), `acos` 以及 `atan`。大多数函数都是为向量和矩阵类型定义的。在如下代码中:

```
vec3 a = vec3(2.0, 3.0, 4.0);
```

```
vec3 b;
```



```
b = sqrt(a);
```

b 的每个分量都将是 a 中对应分量的平方根。其他内置函数，如 `matrixCompMult`，`lessThan` 以及 `equal` 都是逐分量进行计算的。所以，函数 `matrixCompMult` 实际上是将两个矩阵的对应元素分别相乘。函数 `lessThan` 和 `equal` 则分别对参与运算的两个向量中对应位置的每对分量分别应用 `<` 和 `==`，并生成一个布尔型的向量作为运算结果。

此外，GLSL 还有一些对图形学特别有用的内置函数。例如，函数 `dot` 和 `cross` 分别用于计算两个向量的点积和叉积。函数 `length` 负责返回输入向量的长度。`normalize` 函数返回与输入向量方向一致的单位向量。`reflect` 函数接收两个向量，一个是单位法向量，另一个是光源的方向，并返回沿全反射方向的一个向量。

【242】

10.4.5 采样器

着色器，尤其是片段着色器，主要用于对纹理进行处理。GLSL 支持 OpenGL 的高级纹理处理特性如多重纹理(multitexturing)、细节层次(LOD)以及立方纹理图(cube map)。这里我们将仅介绍 GLSL 中最重要的纹理处理功能。纹理是在应用程序内部指定的，可以是简单纹理，也可以是纹理对象，它包含一幅给定纹理图像的全部参数。如果启用纹理映射功能，则纹理就会被映射到片段上。所使用的纹理颜色是由光栅化器传递给片段着色器的插值纹理坐标所确定的。这些插值纹理坐标被片段处理器用来对纹理图像中正确的纹理元进行定位或采样(sample)，然后片段处理器依据指定的纹理参数进行滤波或箝位处理。

在 GLSL 中，采样器(sampler)是特殊类型的 uniform 变量。这种变量用来对应用程序中的纹理对象进行引用。在下一节中，我们将看到如何在应用程序中设置纹理和采样器。在着色器内部，我们可指定一维到四维的采样器。现在我们只关注片断着色器中的二维采样器。采样器可用如下方式指定：

```
uniform sampler2D my Texture;
```

我们用 GLSL 函数 `texture2D` 从与采样器关联的纹理对象中获取一些值(通常是颜色值)。但是，我们需要为该函数提供纹理坐标。纹理坐标通常是对每个顶点逐个指定的，所以我们必须使用一个 `varying` 变量来将插值纹理坐标传给片段着色器。我们可在顶点着色器中定义一个顶点的属性，然后将这些属性值赋给一个 `varying` 变量。于是有以下定义：

```
attribute vec2 texCoordST;
varying vec2 texcoord;
```

并在 `main()` 函数中进行如下赋值：

```
texcoord = texCoordST;
```


这样，坐标值就在应用程序中完成了设置，并通过 `texCoordST` 传递给顶点着色器，并通过 `texcoord` 传给光栅化器。对 `texcoord` 进行插值得到的结果可用于片段着色器中以对纹理进行采样，所以我们可能会看到类似这样的代码：

```
varying vec2 texcoord;
vec3 texColor;
uniform sampler2D myTexture;

texColor = texture2D(myTexture, texcoord);
```

这个颜色值现在可以用来确定片段的颜色或者是与由光照模型得到的颜色组合后再来确定片段的颜色。 【243】

完成该任务的另外一种方式就是借助 OpenGL 状态变量。OpenGL 支持一系列的纹理单元，其中单元 0 是默认值。所以，借助内置变量 `gl_MultiTexCoord0`，顶点着色器就可得到在应用程序中用函数 `glTexCoord` 指定的纹理坐标。在顶点着色器中，我们可用如下方式设置恰当的纹理坐标值：

```
gl_TexCoord[0] = gl_MultiTexCoord0;
```

对 `glTexCoord[0]` 进行插值得到的结果就可为片段着色器所用，因此我们可用与前面相同的变量来获取纹理颜色：

```
texColor = texture2D(myTexture, glTexCoord[0].st);
```

10.5 建立与应用程序之间的接口

在第 2 章中我们开始接触 OpenGL 时，不得不介绍大量的 OpenGL 和 GLUT 函数以及一个大多数应用程序都采取的具有一定通用性的结构。

对于使用可编程着色器的程序来说，虽然情况略微复杂一些，但结构上还是有许多共同点的。我们仍然需要在应用程序中进行例行的 OpenGL 初始化，而且必须特别留意应用程序与着色器之间的接口。此外，应用程序可能会使用多个着色器，所以程序应该具备在不同着色器之间进行切换的功能。

10.5.1 着色器对象的创建

开始处理时，我们需要通过函数 `glCreateProgram` 为着色器和其他可能在不同着色器之间共享的 GLSL 函数创建一个称为程序对象(program object)的容器。函数 `glCreateProgram` 将返回一个所创建的程序对象的标识符，例如：


```
GLuint myProgObj;
myProgObj = glCreatProgram();
```

GLuint glCreatProgram()

该函数负责创建一个空的程序对象，并返回该对象的标识符。

我们的着色器是由源代码构成的文本字符串，这些源代码通常保存在文件中。对于比较简短的着色器，我们甚至可以在应用程序中定义一个字符串，然后用着色器源代码来对其进行初始化，例如：

```
const * char myshader[] = {"void main()\n{\n...\n}"}
```

这样做虽然可行，但无法利用着色器的一个非常好的特性，即如果着色器位于一个独立的文件中，当我们的着色器的源代码进行修改后，再次运行该应用程序时无需重新编译。借助函数 `glCreateShader`，我们既可创建顶点着色器对象，也可创建片段着色器对象，该函数可为每个着色器对象返回一个标识符。例如，下面的代码段就创建了一个顶点着色器对象和一个片段着色器对象：

```
GLuint myVertexObj, myFragObj;
myVertexObj = glCreatShader(GL_VERTEX_SHADER);
myFragObj = glCreate(GL_FRAGMENT_SHADER);
```

【244~245】

GLuint glCreateShader(GLenum type)

该函数可用来创建一个 `type` 类型 (`GL_VERTEX_SHADER`、`GL_FRAGMENT_SHADER`) 的着色器对象，并为该对象返回一个标识符。

接下来，我们借助函数 `glAttachShader`，将着色器对象添加到程序对象中：

```
glAttachShader(myProgObj, myVertexObj);
glAttachShader(myProgObj, myFragObj);
```

void glAttchShader(GLuint progObj, GLuint shaderObj)

该函数将着色器对象 `shaderObj` 添加给程序对象 `progObj`。

10.5.2 着色器的读取和编译

现在我们来对三个空着色器对象进行填充。如果这些着色器位于独立的文件中，则需要一段代码来实现对该文件的读取，并创建一个以 `null` 结尾的字符串。下面是完成上述任务的一个函数：

```
static char* readShaderSource(const char * shaderFile)
{
    struct stat statBuf;
    FILE* fp = fopen(shaderFile, "r");
```



```

char* buf;

stat(shaderFile, &statBuf);
buf = (char*) malloc((statBuf.st_size + 1) * sizeof(char));
fread(buf, 1, statBuf.st_size, fp);
buf[statBuf.st_size] = '\\0';
fclose(fp);
return buf;
}

```

【245】

现在我们就可将着色器代码从文件中读取进来，顶点着色器所在文件和片段着色器所在文件的名称分别保存在字符串 `vShaderFile` 和 `fShaderFile` 中：

```

GLchar *vSource, *fSource;
GLchar* vShaderFile, const GLchar* fShaderFile;

vSource = readShaderSource(vShaderFile);
fSource = readShaderSource(fShaderFile);

```

现在我们可通过函数 `glShaderSource` 将着色器源代码放入着色器对象中。对于我们这个例子，两个着色器都被放入了以 `null` 结尾的字符串中，所以有：

```

glShaderSource(myVertexObj, 1, vSource, NULL);
glShaderSource(myVertexObj, 1, fSource, NULL);

```

```

void glShaderSource(GLuint obj, GLsizei num_strings,
                   const GLchar **source, const GLint len)

```

该函数负责将 `source` 中的着色器代码放入着色器对象 `obj` 中。如果字符串长度 `len` 被设为 `NULL`，`num_strings` 被设为 1，则 `source` 就是一个以 `null` 结尾的字符串。

10.5.3 编译和连接

最后，我们用函数 `glCompileShader` 来对着色器对象进行编译，并用函数 `glLinkProgram` 对程序对象进行连接：

```

glCompileShader(myVertexObj);
glCompileShader(myFragObj);
glLinkProgram(myProgObj);

```

```

void glCompileShader(GLuint shader)

```

该函数负责对着色器对象 `shader` 进行编译。

```

void glLinkProgram(GLuint prog)

```

该函数负责对程序对象 `prog` 进行连接。

在应用程序中可有多个程序对象，我们可用 `glUseProgram` 函数来选择我们希望使用的那个

程序对象：

```
glUseProgram(myProgObj);
```

【246】

```
void glUseProgram(GLuint prog)
```

该函数将程序对象 `prog` 设置为当前程序对象。

10.5.4 错误检查

还有另一个问题我们没有交待，即错误检查。例如，如果着色器编译失败会有什么后果？函数 `glGetShaderiv` 允许我们查询着色器参数。例如，我们可以通过如下方式检查顶点着色器是否已被编译：

```
GLint status;
glGetShaderiv(vShader, GL_COMPILE_STATUS, &status);
```

如果返回的状态值非零，则表明发生了一个错误，此时我们可以从应用程序退出。

```
void glGetShaderiv(GLuint obj, GLenum param, GLint *status)
```

该函数可对着色器对象的参数 `param` 进行查询，查询结果由 `status` 返回。

10.5.5 将数据送入着色器

最后一步是建立必要的联接以使数据能够从应用程序中传递给着色器中的 `uniform` 和 `attribute` 变量。将数据从着色器中返回给应用程序的机制也是类似的，只是沿这个方向的数据传递不太常用，对此我们不准备过多讨论。将数据从应用程序中传递到着色器中的基本思想是当程序对象被连接后，一个包含了着色器中 `attribute` 变量和 `uniform` 变量信息的表格就被创建了。应用程序可通过查询该表来对变量进行匹配。

不妨来考虑一个 `uniform` 型的时间变量，在应用程序中我们对其进行了更新。在顶点着色器中，我们将定义一个变量：

```
uniform float v_time;
```

并在着色器中使用它。假定在应用程序中，我们使用了变量 `a_time`，该变量可在空闲回调函数中进行更新。为了建立变量 `a_time` 和 `v_time` 之间的对应关系，在创建并连接好一个程序对象 `myProgObj` 后，我们借助函数 `glGetUniformLocation` 为 `attribute` 变量获取一个标识符：

```
GLfloat time_id;
time_id = glGetUniformLocation(myProgObj, "v_time");
```

该标识符可在应用程序中用来引用 `v_time`。现在空闲回调函数为：

```
float a_time = 0.0; /* global variable */
```



```

void myIdle()
{
    a_time += 0.01;
    glUniform1f(time_id, a_time);
    glutPostRedisplay();
}

```

【247~248】

这样,每次更新 `a_time` 时,通过函数 `GetUniform1f`, `v_time` 便相应地被更新。对于各种 GLSL 数据类型, `glUniform` 都有相应的形式:

GLint glGetUniformLocation(GLuint obj, const GLchar * name)

该函数将返回程序对象 `obj` 中 `uniform` 变量 `name` 的位置。

void glUniform<1234><if>(GLint loc, type aname)

该函数将类型为 `int` 或 `float` 的值 `aname` 赋给位置为 `loc` 的 `uniform` 变量。

`attribute` 变量的处理方式也很类似。假定在应用程序中我们希望计算出每个顶点的速度值,并通过一个 `attribute` 变量 `vertex_vel` 传给着色器。在顶点着色器中用如下方式对 `vertex_vel` 进行定义:

```
attribute vec 3 vertex_vel;
```

如果应用程序将顶点的速度保存在变量 `app_velocity` 中,我们可通过在应用程序中编写如下代码来建立这两个变量之间的关联:

```

float app_velocity[3];
GLint vel_id;

vel_id = glGetAttribLocation(myProgObj, "vertex_vel");

```

现在我们可在应用程序中用如下方式对着色器变量的值进行设置:

```
glVertexAttrib3fv(vel_id, app_color);
```

GLint glGetAttribLocation(GLuint obj, const GLchar * name)

该函数返回一个用于标识程序对象 `obj` 中的 `attribute` 变量 `name` 的位置的整数。

void glVertexAttribute<1234><if>v(GLint loc, type aname)

该函数该函数将类型为 `int` 或 `float` 的值 `aname` 赋给位置为 `loc` 的 `attribute` 变量。

【248】

10.6 顶点着色器示例

顶点着色器的一个主要用途是对顶点的位置进行修改。下面是一个用 `uniform` 变量来传入一个时间参数的简单示例。这个时间参数后被用来作为正弦函数的参数,并用所得的正弦值对位置进行比例变换。


```

uniform float time;

void main()
{
    gl_Position = gl_ModelViewProjectionMatrix * gl_Vertex;
    gl_Position.xyz =
        (1.0 + 0.1 * sin(0.001 * time)) * gl_Position.xyz;
    gl_FrontColor = gl_Color;
}

```

该应用程序中的初始化与 10.5 节中的相同。对于本例，我们可用 GLUT 在空闲回调函数中获知流逝的时间。如果 `timeParam` 是一个通过下列函数调用得到的标识 `time` 所在的位置的变量：

```
timeParam = glGetUniformLocation(program, "time");
```

则空闲函数为：

```

static void myIdle()
{
    glUniform1f(timeParam, glutGet(GLUT_ELAPSED_TIME));
    glutPostRedisplay();
}

```

只需做少量修改，我们就可用顶点着色器实现顶点绕 y 轴的扭曲。

```
uniform float time;
```

```

void main()
{
    float d;
    float twist = 0.1 * sin(0.001 * time);

    vec4 a;

    a = gl_ModelViewProjectionMatrix * gl_Vertex;
    d = length(a.xz);
    gl_Position.x = cos(d * twist) * a.x - sin(d * twist) * a.z;
    gl_Position.z = cos(d * twist) * a.z + sin(d * twist) * a.x;
    gl_Position.y = a.y;
    gl_Position.w = a.w;
    gl_FrontColor = gl_Color;
}

```

【249~250】

图 10.2 展示了一个扭曲的茶壶线框模型，该图是通过显示回调函数来显示的。


```

void mydisplay()
{
    glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);
    glPushMatrix();
    glTranslatef(0.0f, 0.0f, -5.0f);
    glutWireTeapot(1.0);
    glPopMatrix();
    glutSwapBuffers();
}

```

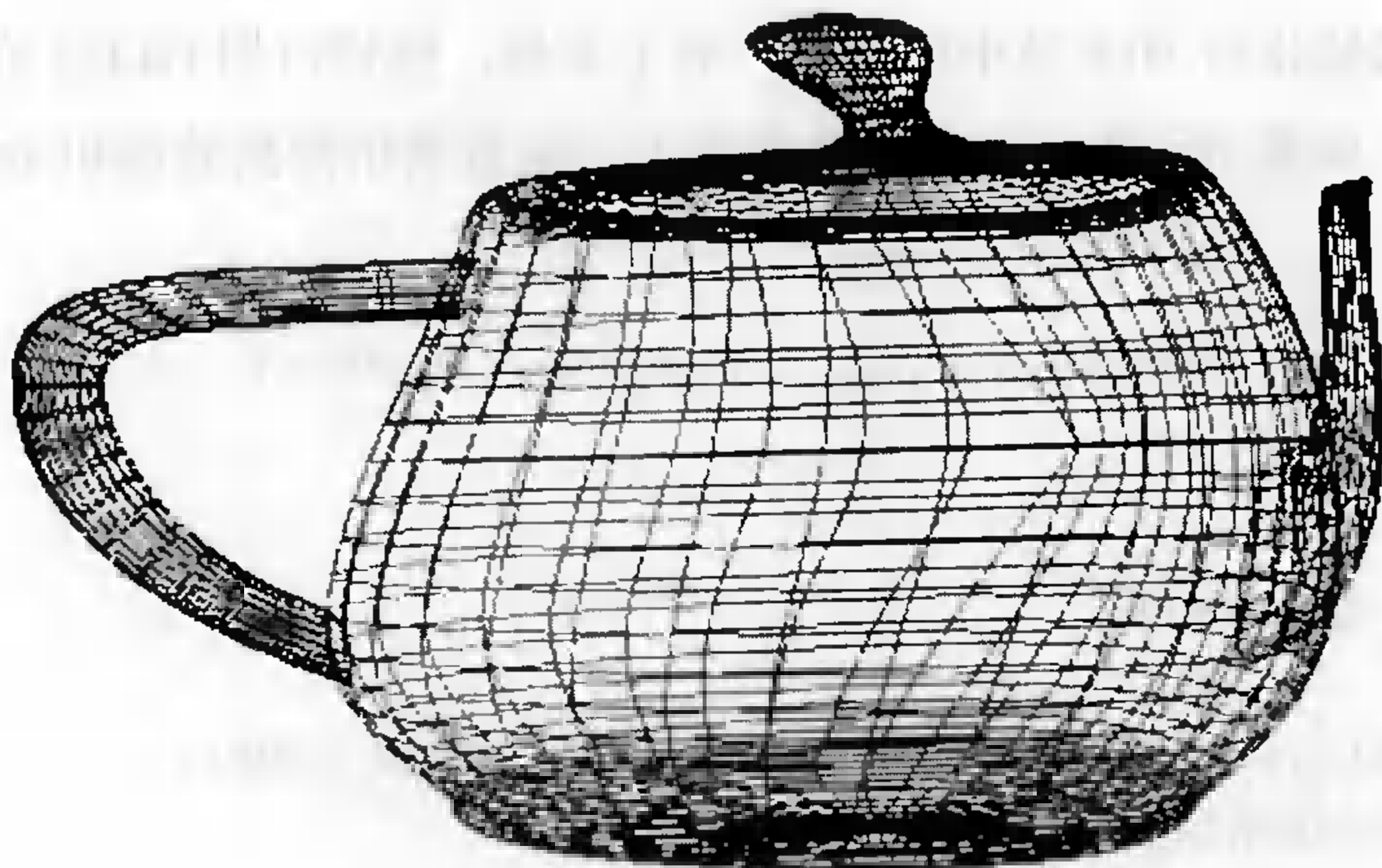


图 10.2 扭曲的茶壶

10.7 片段着色器示例

片段着色器的主要任务是确定颜色 `gl_FragColor`。我们可利用的信息来源包括 OpenGL 状态、由顶点处理器输出的以及由光栅化器插值得到的 `varying` 型变量、接收了来自应用程序中数据的 `uniform` 变量和来自纹理图中的数据。我们可能会使用默认的顶点着色器，或者我们可以使用自己编写的顶点着色器和片段着色器。

我们的第一个例子非常简单，无需我们专门编写一个相应的顶点着色器。着色器用片段的位置来确定片段的颜色值。如下面的代码段所示：

```

uniform float time;

void main()
{
    float d = length(gl_FragCoord.xy);
    gl_FragColor.r = 0.5 * (1.0 + sin(0.001 * time)) * gl_FragCoord.x/d;
    gl_FragColor.g = 0.5 * (1.0 + cos(0.001 * time)) * gl_FragCoord.y/d;
}

```



```

gl_FragColor.b = gl_FragCoord.z;
gl_FragColor.a = 1.0;
}

```

【250~251】

上述代码中有一个内置变量 `gl_FragCoord`，它表示片段在窗口中的位置，并可为任何片段着色器所用。 x 和 y 分量都以像素为单位。 z 分量表示片段的深度值，通常其取值范围为 $0.0 \sim 1.0$ 。 w 分量是透视分割(perspective division)的结果，但在我们的例子中不需要该分量。

在着色器中，片段颜色的红色分量和绿色分量由窗口中规范化的位置所决定，而蓝色分量是依据片段的深度值设定的。为了给输出增加一点趣味性，我们用一个 `uniform` 型的时间参数来以正弦规律改变红色分量和绿色分量的值。

虽然片段着色器比较简单，它却揭示了一些关于片段处理的关键点。第一点，片段着色器可以对应用程序生成的任意图元进行操作。第二点，该着色器阐释了片段着色器的主要使用方式之一。假定我们的应用程序生成了一个落在裁剪窗口内的，各边分别与 x 轴和 y 轴平行的二维的正方形，则相应的片段就形成了一个位于颜色缓存中的矩形数组，我们可通过片段处理器的运算(该计算可能与计算机图形学毫无关系)结果来对该数组中的每个值进行设置。图形处理单元(GLU)借助多个片段处理器就可实现并行计算(这种并行计算是可能的，因为对每个片段执行的都是同一段程序)，从而可以实现对 `RGBA` 各分量的并行处理。于是，片段处理器就变成了一个运算速度非常快的数值处理器。一个称为基于图形处理单元(GPU)的通用计算(GPGPU, General Purpose Processing on a Graphics Processing Unit)的新领域就是专门对这种思路进行研究的。

我们的第二个例子将介绍片段着色器的三个主要特性：顶点着色器与片段着色器之间的通信、用于逐像素光照计算的片段着色器的使用以及采样器和纹理图的使用。为了使得这个示例保持简短风格的同时又能充分展示这些特性，我们将仅考虑遵循朗伯定律的修正 Phong 模型中的漫反射分量。朗伯定律的主要内容是：来自点光源的漫射光强度与法向量 $\mathbf{N}$ 及到达光源的向量 $\mathbf{L}$ 之间夹角的余弦成正比。如果这两个向量都已规范化，则它们夹角的余弦等于这两个向量的点积($\mathbf{L} \cdot \mathbf{N}$)。我们还必须考虑材质属性，它可用一个包含三个反射系数的数组 $\mathbf{K}_d$ 来描述。其中每个分量的值都介于 0.0 与 1.0 之间，这些分量实际上反映的是入射光从该表面反射的比率。在第 6 章中，我们是在 `glMaterial` 中用一个漫反射项来对这些值进行设置的。相应的光源强度可在应用程序中作为漫射光源项用 `glLight` 函数来进行设置。我们可用如下公式来描述漫射光对颜色 C 的贡献：

$$C = (\mathbf{L} \cdot \mathbf{N}) \mathbf{K}_d \mathbf{I}_d$$

其中 $\mathbf{I}_d$ 包含了光源的 `RGB` 漫射分量，乘积 $\mathbf{K}_d \mathbf{I}_d$ 是逐分量进行的。如果启用了光照，则默

认流水线将逐顶点地计算该分量。所以，为了使光照计算能够逐个片段地进行，我们必须
在自己的着色器中使用上述方程。 【251~252】

我们可能也希望增加一幅纹理图，以便与漫反射颜色相乘，以模拟具有某种图案的材质或
照射该物体的某种结构光。

为了逐个片段地进行漫反射项的计算，我们需要一个逐片段的法向量以及一个逐片段的光
源向量。为了得到这些片段着色器所需的实体，一种方式是在顶点着色器中逐个顶点地将
其指定为 `varying` 型变量，这样这些值经过插值计算所得的结果就可为片断着色器所用。但
是我们必须注意在计算中所采用的坐标系。为每个顶点计算出的位置(`gl_Position`)是在裁剪
坐标系中定义的。光照计算通常在摄像机坐标系中进行。理由之一是虽然瓜果能源的位置
是在物体坐标系中用 `glLight` 定义的，但 OpenGL 要对这些位置应用模型视图矩阵，所以这
些值都是保存在摄像机坐标系中的。不妨考虑如下顶点着色器：

```
varying vec3 N;
varying vec3 L;

void main()
{
    gl_Position = gl_ModelViewProjectionMatrix * gl_Vertex;
    gl_TexCoord[0] = gl_MultiTexCoord0;
    vec4 eyePosition = gl_ModelViewMatrix * gl_Vertex;
    vec4 eyeLightPos = gl_LightSource[0].position;

    N = normalize(gl_NormalMatrix * gl_Normal);
    L = normalize(eyeLightPos.xyz - eyePositon.xyz);
}
```

在前面的代码中，我们为法向量和光源向量在摄像机坐标系中定义了两个 `varying` 型变量。在
`main` 函数内部，首先我们以常规模式计算出裁剪坐标系中的一个位置。然后建立与第一个纹
理单元对应的纹理坐标。OpenGL 状态中的所有光照参数都可通过结构变量 `gl_LightSource` 来
获取。第一个元素指向 `GL_LIGHT0`。光源的位置此时已位于摄像机坐标系中，但我们必须
使用模型视图矩阵来顶点在摄像机坐标系中的位置。当我们变换到摄像机坐标系中时，为
了保持法向量和光源向量之间的夹角不变，我们还必须将物体坐标系中的法向量与模型视
图矩阵的逆转置矩阵相乘。OpenGL 通过内置变量 `gl_NormalMatrix` 使得该矩阵可为着色器
所用。最后，为了进行光照计算，法向量和光源向量都必须规范化。 【252~253】

现在我们来研究一下该片段着色器：

```
varying vec3 N;
```



```

varying vec3 L;
uniform sampler2D texMap;

void main()
{
    vec3 Normal = normalize(N);
    vec3 Light = normalize(L);
    vec4 diffuse = max(dot(Normal, Light), 0.0) *
        gl_FrontMaterial.diffuse * gl_LightSource[0].diffuse;
    vec4 texColor = texture2D(texMap, gl_TexCoord[0].st);
    gl_FragColor = diffuse * texColor;
}

```

在计算漫反射项之前，由插值得到的法向量和光源向量都必须重新规范化。对于漫反射分量，我们计算了必要的点积，并确保其为非负，然后用来自 OpenGL 状态的光源强度和反射系数与之相乘。纹理项的获取则需要借助采样器和插值得到的纹理坐标。下面给出用棋盘纹理着色器来设置纹理对象的应用程序代码：

```

GLuint texHandle;
GLubyte image[64][64][3];
int i, j, c;
for(i = 0; i < 64; i++)
{
    for(j = 0; j < 64; j++)
    {
        c = (((i&0x8) == 0) ^ ((j&0x8) == 0)) * 255;
        image[i][j][0] = (GLubyte)c;
        image[i][j][1] = (GLubyte)c;
        image[i][j][2] = (GLubyte)c;
    }
}

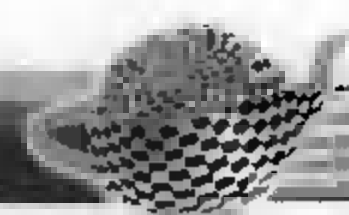
glGenTextures(1, &texHandle);
glBindTexture(GL_TEXTURE_2D, texHandle);
glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MIN_FILTER, GL_LINEAR);
glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MAG_FILTER, GL_LINEAR);
glTexImage2D(GL_TEXTURE_2D, 0, GL_RGBA, 64, 64, 0, GL_RGB,
GL_UNSIGNED_BYTE, image);
glEnable(GL_TEXTURE_2D);

```

【253~254】

我们用与前面例子相同的方法建立着色器。一旦对程序对象完成连接，将得到纹理图的位置：

```
texMapLocation = glGetUniformLocation(program, "texMap");
```

然后将该值设为纹理单元 0:

```
glUniform1i(texMapLocation, 0);
```

图 10.3 展示了应用程序绘制出的实体茶壶。

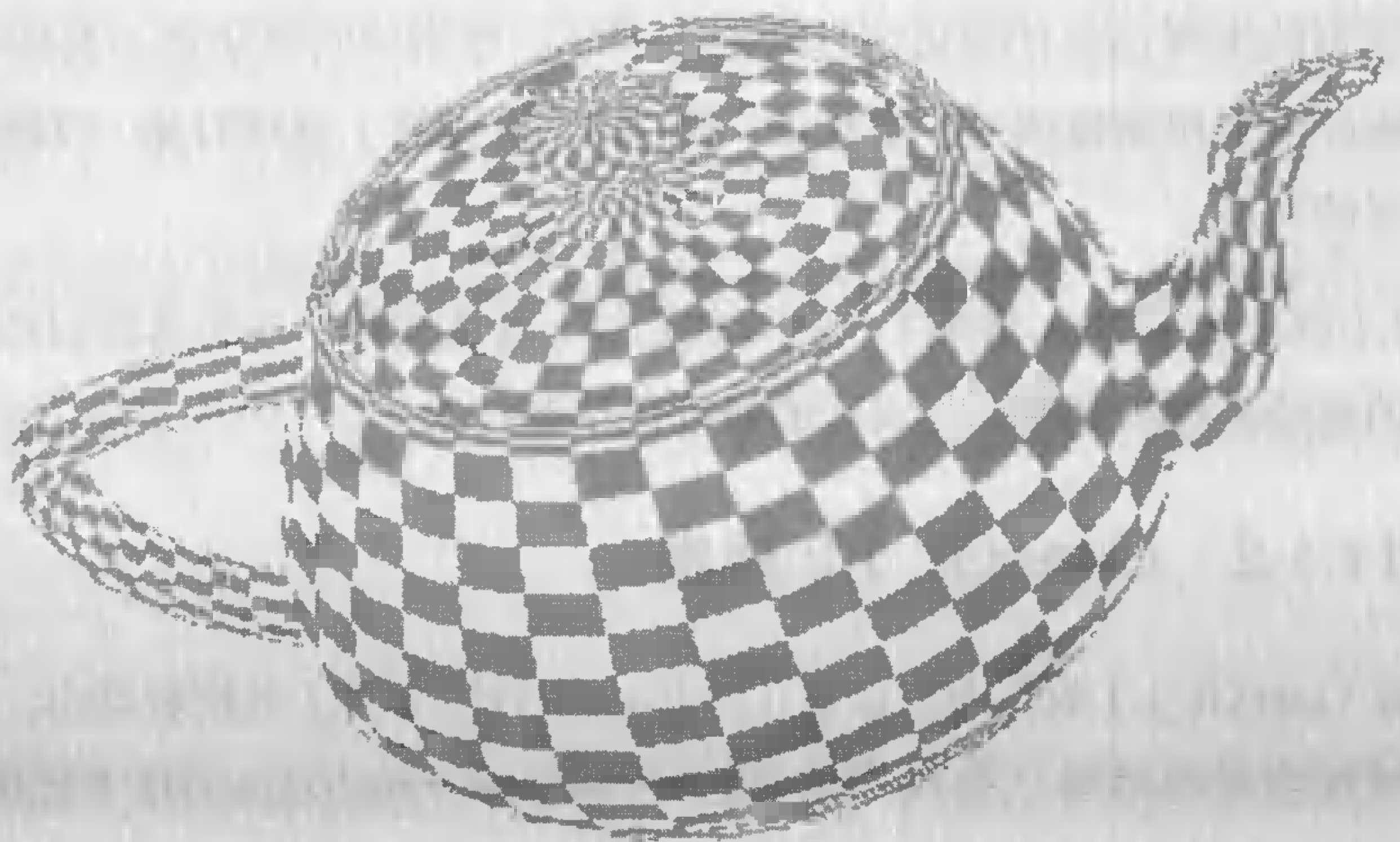


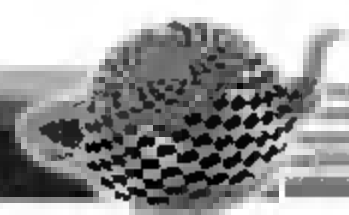
图 10.3 具有漫射光照和纹理的茶壶

第 11 章

总结与展望

在本章中，将介绍前面一些章节没有涉及的一些 OpenGL 特性。首先来回顾一下 OpenGL 的演化发展过程及其修改机制。





11.1 OpenGL 各版本及扩展

自从 1992 年 OpenGL 发布之日起, OpenGL 就一直随着图形硬件和软件的发展而不断发展。OpenGL 具有两种修改机制。一种是经过 OpenGL ARB(OpenGL 体系结构审核委员会)批准而发布的新版本。不过在 2006 年, ARB 已经被 OpenGL 工作组所取代, 该工作组是由 Kronos Group 财团所操纵的。OpenGL 目前最新的版本是 2.1(译者注: 本书翻译时, OpenGL 3.0 的规范正在讨论中)。所有的新版本都是向后兼容的, 所以用以前版本的 OpenGL 开发的程序无需修改便可直接移植到最新的版本中。

第二种修改机制是对当前版本进行扩展。与发布新版本不同, 扩展并不是以整体形式出现的, 也不是企图体现在所有的配置上, 它可能仅仅应用于某些系统, 例如选定的一组制造商的产品, 也可能建立在以前的扩展基础上。

11.1.1 OpenGL 1.1 版本

对 OpenGL 的第一次更新始于 1997 年, 即 OpenGL 1.0 发布 5 年之后。许多方面表明, 这一版是最重大的一次变革, 因为 1.1 版增加了许多 1.0 版所不具备的功能, 这些新功能直到用户对 1.0 版有了一些使用经验后才凸显出来。

在 1.1 版新增的功能中, 意义最重大的可能就是顶点数组。正如在第 4 章所见, 顶点数组能够显著地减少为显示几何对象所需函数调用的次数, 尤其是在用到光照和纹理的场合。顶点数组还为表示网格提供了一种简洁的机制, 在网格中每个顶点和边一般都被一个以上的多边形所共享。

1.1 版提供的第二个主要新功能是一种更好的纹理处理机制。在 OpenGL 问世时, 图形硬件取得的进展已经使得用纹理映射来表现一些用明暗多边形不易模拟的特效完全可行。应用程序需要快速地访问多个纹理。所以 1.1 版提供了纹理对象、纹理代理及从帧缓存中复制纹理的功能。

1.1 版还为 RGBA 增加了一些逻辑运算, 这就使得一些技术得以应用, 如在第 7 章中介绍过的通过逻辑绘制模式而实现的橡皮线绘制技术。

11.1.2 OpenGL 1.2 版本

在 OpenGL 1.1 版发布后仅过了一年, 在 1998 年 1.2 版便被推出。当时, 医学成像等应用领域逐渐发现借助三维纹理的体视化(volume-visualization)技术能够实现大数据集的实时显

示。OpenGL 1.2 版通过扩展 OpenGL 的纹理映射功能以集成三维纹理映射功能，在其 API 中对该技术提供了必要的支持。

另外一个主要的进展是图像子集的引入，该功能已在第 7 章介绍。从标准如何指定的角度看，图像子集是被区分对待的，因为虽然它是 OpenGL 的一部分，但标准却未要求所有的实现都提供该功能。

历史上，图像处理和计算机图形学是两个不同的领域。后者更关注几何，而前者关注的重点是对像素的操作。虽然硬件不断演化，图像子集也应运而生，因为此时硬件已经发展到能够在同一个硬件中既处理图形又处理图像的程度。OpenGL 预见到了这一点，并引入了一条并行像素流水线。但是，由于该流水线主要用于纹理映射，许多图像处理运算如查找表、直方图及卷积都无法发挥硬件的性能。虽然通过将图像读进处理器内存，并用软件来执行一系列运算的方式能够进行大多数标准图像处理运算，但这样做的开销令人望而却步，以至于开发人员宁愿使用硬件来完成处理任务。

11.1.3 OpenGL 1.3 版本

OpenGL 1.3 版发布于 2001 年。大多数新增功能的主要目的在于提供更强大的纹理处理能力。这些新增功能包括压缩纹理、多重纹理单元及立方体纹理(用于环境纹理映射中)。

1.3 版还包括矩阵转置功能。该新增功能使得应用程序可将像 C 语言那样按行存储的矩阵直接送入 OpenGL 中，而非 OpenGL 标准方法所要求的有些过时的、基于 Fortran 的按列存储的矩阵。

【256】

11.1.4 OpenGL 1.4 版本

OpenGL 1.4 版发布于 2002 年。虽然它为原先已经存在于扩展中的核心增加了一些功能，但其主要贡献在于提供了对顶点程序的 ARB 扩展。将在 11.2 节中讨论该扩展。从原理角度看，该版本揭示了一个主流趋势，因为人们已经意识到作为 OpenGL 基础的固定功能流水线已经变得有些过时了。所以，顶点程序扩展的增加为 OpenGL API 日后进行修订，以跟上现代硬件发展的步伐铺平了道路。

11.1.5 OpenGL 1.5 版本

OpenGL 1.5 版发布于 2003 年，这一版延续了 1.4 版的发展趋势。1.5 版只对核心作了少量变动，但是 OpenGL 着色语言(参见 11.5 节)作为一个主要扩展而被增加，以期成为未来向标准化迈进的基础。

11.1.6 OpenGL 2.0 版本

OpenGL 2.0 版发布于 2004 年, 该版本是对以前版本作出的一次重要修订, 但仍支持所有 1.x 系列版本的程序。这次升级中最重要的部分就是将 OpenGL 着色语言及其组成部分(着色器对象、顶点对象及片段对象)提升为 OpenGL 的核心规范。

其他的修订也从扩展成为了核心规范的一部分, 包括非 2 的整数次幂的纹理图、点精灵(point sprite)及多重绘制目标(multiple render target)。点精灵是点的推广, 它允许纹理被绘制在一个尺寸大于单个像素的点上。多重绘制目标允许在一个单个绘制路径上对多个缓存进行绘制。

11.1.7 OpenGL 2.1 版本

OpenGL 2.1 版本发布于 2006 年, 仅仅对 2.0 版作了少量的升级。在新增加的功能中, 最重要的可能就是像素缓存对象的引入, 该功能可用 GPU 内存加速数据的传输。此外, OpenGL 2.1 版还对以往版本进行了少量修订, 以取得与 GLSL 更好的兼容性。【257】

11.2 OpenGL 扩展

OpenGL 自身的演化过程反映了图形学界对何为图形学实现的核心这一问题达成的共识。这种共识反映的并非高端显卡中的可用功能, 也非只有少数制造商能够提供的特有功能。为了解决这个问题, OpenGL 提供了一种扩展机制, 该机制允许某些功能通过 OpenGL 来访问, 而无论这些特性是否为所有的实现所支持。

每个制造商也可以对扩展做出提议。如果这些提议被批准, 则该扩展的名称中将包含标识来源的一个子串。其他由一组制造商提议的扩展, 如果被 ARB 批准, 则该扩展的名称中将包含 ARB 子串。例如, 图像子集就是一个 ARB 扩展。在 OpenGL 2.0 版之前, 顶点和片段都属于扩展。比较新的商业显卡中可用的许多特性, 如离屏缓存(off-screen buffer)和高级纹理处理功能都只能通过扩展来访问。

11.3 一些附加的 OpenGL 特性

虽然我们没有覆盖 OpenGL 的全部功能, 但起码已经涉及了大部分功能。在我们略去的内容中最主要的就是 OpenGL 中一些附加缓存(将在 11.4 节进行介绍)及一些通过扩展或 GLU 可以利用的更丰富的功能。可能最重要的就是对 OpenGL 网格化器(tessellator)和 NURBS 的使用。这些工具都是通过 GLU 库而得到支持的。

借助网格化器，可以超越 OpenGL 只对凸多边形正确性保证的限制。网格化器中的代码可以处理一般多边形，即使是极复杂的多边形也没有问题。不幸的是，使用网格化器时必须指定许多参数和选项以告知网格化器我们希望多边形被处理的方式。不但需要指定一些决定多边形显示方式的选项，而且必须就如何处理复杂情况(如带有孔洞的表面)给予网格化器一些显式指示。鉴于这种复杂性，我们不再对网格化器做进一步讨论。

相较于贝塞尔曲线和曲面，NURBS 曲线和曲面具有更大的灵活性。所以，它在 CAD 和动画界更受欢迎。但是，与网格化器一样，灵活性是以要求用户定义许多的参数和选项为代价的。此外，使用 NURBS 时，要求用户对其中的数学原理具有较为深入的理解，远远超出了本书的深度。

11.4 其他缓存

OpenGL 提供了另外一些缓存，虽然这些缓存并不为所有的实现所支持。这些缓存包括累积缓存(accumulation buffer)和模板缓存(stencil buffer)。此外，还可能有一些附加的颜色缓存(辅助缓存)。所有这些缓存的初始化和清除方式都与颜色缓存和深度缓存相同。每种缓存都有一个相应的标识符(GL_ACCUM、GL_STENCIL、GL_AUX1 和 GL_AUX2)，并可像其他缓存一样被开启和清除，如下所示：

```
glutInitDisplayMode(GL_RGB | GL_DOUBLE | GL_STENCIL | GL_ACCUM);

glClear(GL_COLOR_BUFFER_BIT | GL_CLEAR_DEPTH_BUFFER_BIT
        | GL_ACCUM_BUFFER_BIT | GL_STENCIL_BUFFER_BIT);    【258~259】
```

辅助缓存的工作方式类似于前台缓存和后台缓存。所以，可以对这些缓存使用与第 7 章完全相同的读写函数。这些缓存的一种可能用途是多路径绘制(multipass rendering)，其中每个对象都被重复绘制多次。辅助缓存的一个典型应用是为沉浸式环境(immersive environment)生成图像。假定现有一个其每个面都有发光器的物理房间，希望为该房间的前、后、左、右，甚至天花板和地板都分别创建一幅图像。可以编写一个 OpenGL 程序，通过使用 6 个虚拟摄像机并分别对 6 个颜色缓存进行绘制而生成这些图像。

11.4.1 累积缓存

如果用户开始使用 OpenGL 来处理图像，在某些时候可能会遇到颜色缓存分辨率有限的问题。不妨考虑如下合成问题。假定有一组图像，我们希望将这组图像合成为一幅单个图像。一种显而易见的方法是使用在第 7 章中介绍的融合函数。假定我们所处的特定实现在物理

颜色缓存中为颜色的红色分量、绿色分量和蓝色分量各分配 8 位。如果将各幅图像的 RGB 值简单的对应相加,则有可能超过颜色分量的最大值 1.0。如果对颜色值进行比例缩放来防止越界,会降低每种颜色的分辨率,因为虽然 OpenGL 在计算中使用的是浮点数,但各颜色分量在物理上却受制于 8 位的限制。

对于这种类型的问题,解决方案之一是创建一个缓存——累积缓存,该缓存为颜色的每个分量分配更多的位。可通过一个比例因子将颜色缓存中的内容添加到累积缓存中。一旦对所有的图像完成累积,就可以读出一个累积缓存的缩放版本,并将其放入颜色缓存中。这样既保持了颜色分辨率,又不至于在颜色缓存中出现上溢。如果实现支持累积缓存,为了使用累积缓存,只需使用函数 `glAccum()`。

```
void glAccum(GLenum operation, GLfloat value)
```

该函数定义了由累积缓存执行的运算 `operation` (`GL_ACCUM`、`GL_LOAD`、`GL_RETURN`、`GL_ADD` 和 `GL_MULT`), 以及一个用于对运算进行比例缩放的浮点数 `value`。 【259】

运算 `GL_ADD` 允许用 `value` 对累积缓存中的颜色值进行比例缩放,并将结果与当前累积缓存的内容相加。运算 `GL_RETURN` 用 `value` 对累积缓存中的颜色值进行比例缩放,然后结果复制到当前绘制缓存中。

累积缓存为多路径绘制提供了一些新而有趣的功能。例如,假定我们要对同一场景重复绘制多次,每次绘制时摄像机都稍作移动(或轻微抖动),并将这些图像累加到累积缓存中。每幅图像的走样程度都略有差别,该差别在累积缓存中可以通过平均来达到平衡。所以,我们是对整个场景进行了反走样处理,而非逐行或逐多边形进行处理。

使用了累积缓存的应用程序运行速度可能会很慢,这是由于累积缓存通常是在处理器内存中实现的。较新的显卡已经具备了浮点数帧缓存。所以,许多使用累积缓存的程序现在都可以用片段程序来完成。

11.4.2 模板缓存

模板(stencil)是用来确定绘制位置的掩膜。模板缓存的一种典型用途是在视口的矩形绘制区域内创建任意形状的窗口。例如,通过构造一个全部元素为 1 的颜色缓存,然后将 0 放在一个圆形的区域中(即用 0 取代该区域中的 1),便可以创建一个圆形的观察孔。然后,在颜色缓存中进行绘制时,每个由光栅化所形成的像素都依据模板缓存进行测试,以判断该像素是否应被绘制到颜色缓存中。借助模板缓存,还可以创建更复杂的行为,因为我们可以依据由光栅化器生成的像素值对模板缓存中的值进行修改。

11.4.3 片段测试

我们已经看到,对于每个几何图元,绘制器将生成一个片段集合。每个片段都有一个与颜色缓存中像素对应的位置及一组属性(包括颜色、深度和纹理坐标等)。除了由片段处理通过一系列运算(如纹理映射)来确定像素的最终颜色外,片段在到达颜色缓存的过程中还将经历一系列测试(裁剪、alpha、模板和深度)和运算(融合、抖动和逻辑运算)。在前面的章节中,我们已经接触过一些测试和运算。例如,深度测试包括将片段的深度与深度缓存中的深度进行比较。融合和逻辑运算决定了一个可见像素如何与已经存在于颜色缓存中的颜色值进行整合。

通常这些测试和运算都是作为标准绘制过程中所要执行的常规内容。但是,也可直接对这些测试编程,以决定接收还是丢弃某个像素。例如,借助裁剪测试,就可以定义一个矩形区域,在该区域中所有的片段都将被接收,而所有位于该区域之外的片段将被丢弃。这样的运算和测试对于交互式程序极其重要。【260~261】

11.5 编写可移植、高效而又健壮的代码

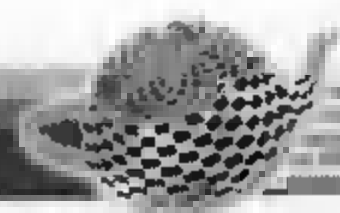
我们都希望自己编写的代码能够移植到其他系统后,无论硬件是否存在差别,程序无需修改便可运行。我们也希望我们的程序能够在所有的系统中高效地运行。当然,也希望程序的可读性好且易于修改。如果每个 OpenGL 程序都具有上述特征,则实在太完美了。

虽然 OpenGL 程序一般都比较清晰而且可移植性较好,但仍然存在一些问题。一个问题是一旦我们准备使用一些高级特性,如累积缓存和模板缓存,就将丧失可移植性,因为这些特性并不被所有的实现所支持。虽然可以在程序中检查该特性是否被支持,但是如果遇到自己希望的特性不被支持的情况,到底该如何处理(退出除外)?这个问题尚不明朗。

显示器属性之间的差异比较容易处理,因为可以使用查询函数,如 `glGetInterv()` 函数,来获取某个特定显示器的属性,并可在应用程序中对该属性做出调整。

效率是一个更为复杂的问题。首先,几乎毫无例外,底层算法对用户是隐藏的。我们不知道(通常也不必了解)一个 OpenGL 实现到底是如何填充多边形或进行纹理插值的。其次,不同的实现依据不同的准则进行了相应的优化。所以,即便购买了一个具有极快的三角形单元绘制速度的显卡,该显卡对于纹理映射可能并不具有同样的相对性能。对 PC 的显卡应特别注意这种情形,因为大多数 PC 显卡都对游戏中所需的运算进行了优化。

最后,由于图形硬件持续而快速的发展,许多应用程序的瓶颈不在于进行图形运算所需的



时间,而在于处理器内存和图形内存之间传递数据所需的时间。近来,我们已经看到许多新的体系结构涌现出来,这表明目前的一些制约在将来可能不再是问题。幸运的是,OpenGL提供了修改的机制,一切都表明,OpenGL将成为一种重要的图形API。

OpenGL 1.0版发布于1992年,其设计初衷是成为一种用于实时图形学的API。OpenGL非常稳定,这就使得用OpenGL编写的代码无需针对不同的硬件重新编写,且当核心标准发生变化时,原来的代码也无需改动。当回顾图形硬件在价格和性能上日新月异的巨大变化时,正好印证了在过去15年中OpenGL所做的工作是多么的出色。 【261】

索引

函数索引

GL Functions

- glAccum, 259
- glAttachShader, 245
- glBegin, 18
- glBindTexture, 204
- glBitmap, 160
- glBlendFunc, 150
- glCallList, 73, 96
- glCallLists, 73
- glClear, 19
- glClearColor, 21
- glColor, 21
- glColorPointer, 95
- glCompileShader, 246
- glCopyPixels, 168
- glCopyTexImage2D, 197
- glCopyTexSubImage2D, 197
- glCreateProgram, 244
- glCreateShader, 245
- glCullFace, 35
- glDeleteList, 73
- glDeleteTextures, 204
- glDepthMask, 151
- glDisable, 31
- glDisableClientState, 95
- glDrawBuffer, 169
- glDrawElements, 96
- glDrawPixels, 166
- glEdgeFlag, 38
- glEnable, 31
- glEnableClientState, 95
- glEnd, 18
- glEndList, 72
- glEvalCoord1, 214–215
- glEvalCoord2, 216
- glEvalMesh1, 216
- glEvalMesh2, 217
- glFlush, 19
- glFrontFace, 36
- glFrustum, 104
- glGenLists, 74
- glGenTextures, 203
- glGetAttribLocation, 248
- glGetBooleanv, 48
- glGetDoublev, 48
- glGetError, 49
- glGetFloatv, 48
- glGetIntegerv, 48
- glGetPointerv, 48
- glGetShaderv, 247
- glGetUniformLocation, 248
- glHint, 192
- glInitNames, 77
- glIsEnabled, 49
- glIsTexture, 203
- glLight, 137
- glLightModel, 146
- glLineStipple, 31
- glLineWidth, 30
- glLinkProgram, 246
- glListBase, 73
- glLoadIdentity, 26
- glLoadMatrix, 116
- glLoadName, 77
- glLogicOp, 163
- glMap1, 214

glMap2, 216
 glMapGrid1, 216
 glMapGrid2, 217
 glMaterial, 140
 glMatrixMode, 26
 glMultMatrix, 117
 glNewList, 73
 glNormal, 136
 glOrtho, 87
 glPixelMap, 177
 glPixelStore, 169
 glPixelTransfer, 177
 glPixelZoom, 179
 glPointSize, 29
 glPolygonMode, 35
 glPolygonOffset, 36
 glPolygonStipple, 33
 glPopAttrib, 51
 glPopMatrix, 50
 glPopName, 77
 glPushAttrib, 51
 glPushMatrix, 50
 glPushName, 77
 glRasterPos, 47
 glReadBuffer, 168–169
 glReadPixels, 166
 glRect, 33
 glRenderMode, 76
 glRotate, 113
 glScale, 114
 glSelectBuffer, 77
 glShadeModel, 37
 glShaderSource, 246
 glTexCoord, 188
 glTexEnv, 196
 glTexGen, 202
 glTexImage, 187–188
 glTexParameter, 190
 glTexSubImage2D, 197
 glTranslate, 109
 glUniform, 248
 glUseProgram, 247

glVertex, 18
 glVertexAttribute, 248
 glVertexPointer, 95
 glViewport, 24

GLU Functions

gluBuild2DMipmaps, 199
 gluCylinder, 100
 gluDeleteQuadratic, 99
 gluDisk, 101
 gluGetErrorString, 49
 gluLookAt, 90
 gluNewQuadratic, 98
 gluOrtho2D, 23
 gluPartialDisk, 101
 gluPerspective, 105
 gluPickMatrix, 78
 gluQuadricDrawStyle, 99
 gluQuadricNormals, 99
 gluQuadricTexture, 99, 200
 gluScaleImage, 180
 gluSphere, 100

GLUT Functions

glutAddMenuEntry, 66
 glutAddSubMenu, 67
 glutAttachMenu, 66
 glutBitmapCharacter, 46
 glutBitmapWidth, 47
 glutCreateMenu, 66
 glutCreateSubWindow, 68
 glutCreateWindow, 16, 68
 glutDestroyWindow, 68
 glutDisplayFunc, 17
 glutEntryFunc, 65
 glutGet, 49
 glutGetModifiers, 60
 glutIdleFunc, 55
 glutInit, 16
 glutInitDisplayMode, 20
 glutInitWindowPosition, 20
 glutInitWindowSize, 20

- glutKeyboardFunc, 59
- glutMainLoop, 17
- glutMotionFunc, 64
- glutMouseFunc, 61
- glutPassiveMotionFunc, 64
- glutPostRedisplay, 56
- glutPostWindowRedisplay, 68
- glutReshapeFunc, 53
- glutSetMenu, 66
- glutSetWindow, 68
- glutSolidCone, 102
- glutSolidCube, 88
- glutSolidDodecahedron, 103
- glutSolidIcosahedron, 103
- glutSolidOctahedron, 103
- glutSolidSphere, 102
- glutSolidTeapot, 103
- glutSolidTetrahedron, 103
- glutSolidTorus, 102
- glutSpecialFunc, 60
- glutStrokeCharacter, 47
- glutStrokeWidth, 47
- glutSwapBuffers, 58
- glutWireCone, 102
- glutWireCube, 88
- glutWireDodecahedron, 103
- glutWireIcosahedron, 103
- glutWireOctahedron, 103
- glutWireSphere, 102
- glutWireTeapot, 103
- glutWireTetrahedron, 103
- glutWireTorus, 102

主题索引

A

- accumulation buffers, 258, 259–260
- additive color model, 20
- affine transformations, 108
- agl, 14
- alpha test, 260
- alpha value, 148, 149
- ambient light, 135, 146
- ambient reflections, 135
- angle brackets, use of, 7
- antialiasing, 260
- application programmer's interface (API), 1–2
- Architectural Review Board (ARB), 5, 255, 258
- argument values, 6
- arrays
 - color, 95, 96
 - dynamic, 172
 - one-dimensional, 173
 - texel, 188
 - types, 95
- aspect ratio, 55
- at point, 89
- attribute-qualified variables, 240
- attributes
 - defined, 28
 - groups, 50
 - line, 30
 - point, 29
 - polygon, 33
 - popping, 50, 51
 - pushing, 50, 51

- state and, 28

- variables, 248

- automatic coordinate generation, 231

- automatic texture generation, 232

- auxiliary buffers, 259

B

- back buffer, 58

- back face, 35

- baseline, 159

- Bernstein polynomials, 213

- Bézier curves

- approximations to, 214

- defined, 212

- implementation, 213

- interactive example, 217–219

- line segment approximation, 214

- See also* curves

- bit block transfer operation (bitblt), 156

- bitmap characters

- defined, 46

- descender, 159

- bitmaps, 157–162

- defined, 156, 157

- displaying, 158–160

- fonts, 157

- masks, 157

- writing model, 162

- blending

- destination factor, 149

- source factor, 149

- texture and, 195

blending (*continued*)
 translucent polygons, 151
 writing model, 150

Bluebook, 8

B-splines, 220–224

See also curves

buffers

accumulation, 258, 259–260

auxiliary, 259

back, 58

clearing, 19

color, 18

depth, 97–98, 151

drawing, 169

frame, 18, 155, 156, 165

hit, 80

selecting, 168–169

stencil, 258, 260

C

callbacks

changing, 71

display, 16–17, 55, 56, 88, 124

GLUT, 17

idle, 55–56, 57

keyboard, 59, 144

menu, 67

motion, 64–65

mouse, 61–64, 78, 79

null, 67

reshape, 53–55, 71, 78, 164

cameras

eye point, 89

jittered, 260

locating, 89–92

moving, 92

orthographic, 87

projection, 83–84

specification, 85–86, 89

synthetic, 23

virtual, 3

center of projection (COP), 83

Cg, 236

checkerboard textures, 189, 200

clamping, 190

clients, 72

clipping

coordinates, 120

planes, 104, 105, 106

volume, 86, 92

clipping window, 78

defined, 24

points within, 29

color, 20–22

additive, 20

arrays, 95, 96

clear, 22

components, 20

current, 22

destination, 149

environmental, 196

index mode, 20, 21

interpolation, 37–46

masks and, 161

primary, 20

raster, 161

RGB (RGBA), 20–22, 135, 136,
 166, 252, 256

setting, 21

shades, 131

source, 162

state and, 22

subtractive, 20

table, 180

texture, 196

vertices, 37

color buffers, 18, 149, 168–169

See also buffers

compiling, 8–9, 72

concatenation

defined, 110

rotation, 113–114

translation, 110–111

cones, 101, 201

- context, window, 68
- control functions, 5
- control points, 212
- convex hull, 212
- convex objects, 34
- convolution, 180
- coordinate systems, 22–23
 - camera (eye), 25, 120
 - clip, 120
 - normalized device, 120
 - object, 25
 - right-handed, 111
 - screen, 25
 - texture, 184, 188–190
 - transformations and, 25–26, 120
 - window, 120
 - world, 25, 120
- cubic polynomials, 210
- culling, 35
- curves
 - Bézier, 212–214
 - B-spline, 220–224
 - interpolating, 210, 211, 221
 - NURBS, 224, 258
 - order of, 214
 - parametric, 209–211
 - segments, 211
 - spline, 220
- cylinders, 100, 123, 201

D

- data types, 6–7
- decal, 196
- degree of freedom, 85
- degrees, 58
- depth buffer, 97–98, 151
- depth mask, 155
- depth test, 260
- descender, 159
- destination blending factor, 149
- destination pixels, 162
- diffuse reflections, 133–134

- Direct X, 1
- direction of projection, 86
- directional light, 137, 142
- discrete primitives, 155–181
- disks, 100
- display callback, 55, 88, 124
 - dummy, 63
 - execution, 56
 - passing values to, 17
 - See also* callbacks
- display lists
 - creating, 72–73
 - defined, 72–75
 - executing, 73
 - identifiers, 73
 - multiple, 73–74
 - objects and, 75
 - text and, 74–75
- dithering
 - defined, 167
 - turning off, 168
- dodecahedron, 102
- double buffering, 58–59, 69
- drawing
 - bitmaps, 160
 - modes, 162–165
 - rectangles, 17–19, 33
- dynamic arrays, 172

E

- edge flags
 - setting, 38
 - tessellation and, 37–38
 - for triangles, 41
- efficiency, 261
- emissive light, 133, 135
- entry event, 64, 65
- error checking, 49
- evaluators, 213–217
- event queue, 16
- events
 - display, 16

events (*continued*)

- entry, 64, 65
- idle, 55–56
- keyboard, 16, 59
- loop, 79
- mouse, 61
- move, 64
- passive motion, 64
- processing, 16
- queue, 16
- reshape, 53–55
- exclusive OR operation, 163
- extensions, 5
- extensions, OpenGL, 258
- eye point, 89

F

- feedback mode, 76
- filled primitives, 32–33
- filtering, 180
- fixed point, 112
- flat shading, 37
- fonts
 - GLUT, 46, 74
 - monospaced, 47
 - proportionally spaced, 47
 - stroke, 47
- fragment processor, 4, 234
- fragments
 - defined, 4
 - tests, 260–261
- fragment shaders, 236, 237
 - examples, 250–254
 - function, 250
 - for per-pixel lighting, 251
 - See also* shaders
- frame buffers
 - defined, 18, 155
 - illustrated, 156
 - pixels in, 165
 - See also* buffers
- front buffer, 58

front face, 35

frustum

- defined, 86
- perspective projections, 104
- viewing, 86, 104

functions

- arguments, 6
- attribute, 5
- control, 5
- GLSL, 242
- input and window, 5
- libraries, 6
- primitive, 4–5
- query, 5
- viewing, 5
- See also* the Function Index

G

- geometric objects, 2
- GKS, 1
- GL (Graphics Library), 1, 6, 17
- GLSL, 236–254
 - built-in functions, 242
 - control structures, 242
 - execution model, 236–237
 - matrix types, 238–239
 - operators, 241–242
 - qualifiers, 239–241
 - samplers, 243–244
 - swizzling, 241–242
 - variables, qualifying, 237
 - vector types, 238–239
- GLU (Utility) library, 6, 14
 - cylinders, 100, 123
 - disks, 100
 - quadratics, 98–101
 - spheres, 99–100, 101, 123
- GLUT (OpenGL Utility Toolkit), 2, 8, 15–16
 - callback function, 17
 - cone, 101
 - coordinate system differences, 22–23

- defaults, changing, 20
- defined, 15
- error-reporting mechanism, 49
- fonts, 56, 74
- libraries, 15
- platonic solids, 102
- teapot, 103
- torus, 101
- GLX library, 14
- Gouraud shading, 148
- GPGPU (General Purpose Processing on Graphics Processing Unit), 251
- graphics servers, 72
- grids, 249

H

- hidden-surface removal, 97–98
- hierarchical models, 121–129
- histograms, 180
- hit buffer, 80
- hit lists, 77
- HLSL (High Level Shading Language), 236
- homogeneous coordinates, 108–109

I

- icosahedron, 102
- idle callback, 55–56, 57
- image processing, 180–181, 256
- imaging subset, 180
- immediate mode, 72
- input and window functions, 5
- instances, 120–121
 - defined, 121
 - transformation, 121
- interpolation
 - bilinear, 37
 - color, 37–46
 - control, 189
 - curve, 210, 211, 221
 - vertex properties, 37

J

- jitter, 260
- join points, 211
- joint angles, 121

K

- keyboard
 - callback, 59, 144
 - special keys, 60–61

L

- Lambert's law, 251
- leaves, 121
- left child right sibling structure, 127
- light

- ambient, 135, 146
- directional, 137, 142
- emissive, 133, 135
- material interactions, 131–132
- source positioning, 139
- source specification, 136–140
- transformations and, 138–140

lighting

- calculations, 146–147, 148, 252
- diffuse, 254
- global, 132
- OpenGL, 135–136
- per-pixel, 251
- two-sided, 146

lines, 28

- attributes, 30
- stippling, 31
- types, 30
- width, 31

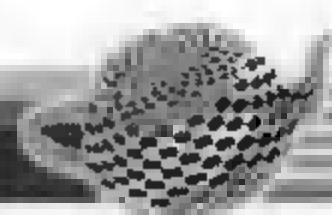
linux, 8, 14

- logical operation, 132
- logica test, 260
- luminance, 176–177

M

- Macintosh, 14
- Makefiles, 8

- masks
 - bitmap, 157, 161
 - colors and, 161
 - depth, 151
 - material
 - light interactions, 131–132
 - properties, 140–142
 - specification, 140–142
 - matrix
 - current, 111
 - GLSL types, 238–239
 - model-view, 25, 50
 - projection, 25, 50
 - rotation, 127
 - setting directly, 116–119
 - stacks, 50, 160
 - storage, 116
 - transformation, 109
 - menus, 65–67
 - callback, 67
 - pop-up, 65
 - submenus, 67
 - top-level, 66
 - Mercator projection, 199
 - Mesa, 9
 - Microsoft, 14, 236
 - minification, 191, 198
 - mipmapping, 186, 198
 - mipmaps, 197–199
 - building, 198, 199
 - loading, 199
 - modeling, 120–129, 234–235
 - model-view matrix, 25, 50
 - modifier keys, 60
 - modulation, 196
 - motion callback, 64–65
 - mouse
 - callback, 61–64, 78, 79
 - event, 61
 - motion, 64–65
 - move event, 64
 - multi-pass rendering, 259, 260
- N
 - name stack, 77
 - nodes
 - child, 121
 - dynamic, 128
 - parent, 121
 - root, 121
 - terminal, 121
 - normalized device coordinates, 120
 - normals
 - per-fragment, 252
 - scaling and, 148
 - shading and, 229–231
 - working with, 147–148
 - null callback, 67
 - NURBS (NonUniform Rational B-Spline), 224, 258
- O
 - objects
 - building, 92–96
 - convex, 34
 - coordinates, 25, 47
 - display lists and, 75
 - GLU, 98–101
 - GLUT, 101–103
 - light striking, 132
 - texture, 203–204
 - See also specific types of objects*
 - oblique projection, 118
 - octahedron, 102
 - opacity, 21, 148
 - OpenGL
 - API, 1–2
 - color, 20–22
 - contents, 4–5
 - coordinate systems, 22–23
 - data types, 6–7
 - extensions, 5, 258
 - features, enabling, 31–32
 - pipeline, 4
 - resources, 9–10



- Shading Language. *See* GLSL
- state machine, 3–4
- Utility Toolkit. *See* GLUT
- versions, 5, 255–257
- Working Group, 5
- operators, GLSL, 241–242
- orthographic projections
 - defined, 86
 - viewing volume, 87
 - See also* projections

P

- packing, 166
- painter's algorithm, 96
- parametric curves
 - defined, 209
 - illustrated, 210
 - interpolation, 210, 211
- parametric surfaces, 212–213
- passive motion event, 64
- pass-through shaders, 237, 240
- patches, 232
- perspective projections, 103–106
 - problem, 105–106
 - viewing frustum, 104
 - See also* projections
- PHIGS, 1
- Phong model, 132–135, 251
 - ambient reflections, 133, 135
 - basis, 132
 - diffuse reflections, 133–134
 - emissive light, 133, 135
 - specular reflections, 133, 134
 - vectors in, 133
- picking, 75–76
 - defined, 75
 - methods, 81
- pipelines, 233–234
 - default, 252
 - graphics, 4
 - imaging, 181
 - model, 4

- pixel, 165
- pixel mapping, 177–178
 - defined, 177
 - operations, 165
- pixels, 155–157
 - copying, 168
 - defined, 3, 155
 - destination, 162
 - packing, 166
 - pipeline, 165
 - preimage, 191
 - reading, 156, 167–168
 - replicating, 178, 179
 - source, 162
 - storage, 167
 - transfer operations, 165, 174
 - types, 157
 - unpacked, 165
 - writing, 166
- pixel store modes, 169–170
- pixel zoom, 178–180
- points, 28
 - sampling, 191
 - size, 29
 - types, 30
- point sprites, 257
- polygons, 28, 33–34
 - blending, 149
 - convex, 35, 258
 - display, 34
 - filling, 36
 - offset, 36
 - simple, 34
 - stipple, 33–34
 - tessellating, 38
 - types, 34–36
- polyhedra, 88
- pop operation, 50, 51
- posting redisplay, 56–57
- PPM (Portable Pixel Map)
 - defined, 170
 - header, 171, 174

PPM (Portable Pixel Map) (*continued*)

- identification, 171
- images, displaying, 170–176
- reader, 174
- window size and, 173, 174

- primary colors, 20

primitives

- attributes, 28
- defined, 28
- discrete, 155–181
- filled, 32–33
- geometric, 160
- line, 28, 29–31
- point, 28, 29
- polygon, 28, 33–34
- raster, 160
- rectangle, 17–19, 33
- types, 4–5

- program objects, 244–245

- projection matrix, 25, 50

projections

- center of, 83
- defined, 83
- illustrated, 84
- Mercator, 199
- oblique, 118
- orthographic, 86–87
- perspective, 103–106
- plane, 84, 85
- shadow, 119

- projective transformations, 108

- projectors, 83

- pseudocolor, 178

- push operation, 50, 51

Q

- quadrics, 98–101

- queries, 48

- query functions, 5

R

- raster color, 161

- rasterizer, 4, 234

- raster position, 47

rectangles

- drawing, 17–19, 33
- size, 78

- Redbook, 8

reflections

- ambient, 133, 135
- diffuse, 133–134
- specular, 133, 134

- refresh rate, 58

rendering

- defined, 2
- multi-pass, 259, 260
- requirements, 235
- Utah teapot, 225, 228, 229, 230

- RenderMan, 235

- replacement mode, 196

- replication, 179

- reshape callback, 71, 78, 164

- retained mode, 72

- RGB (RGBA) color model, 20–22, 135, 136, 166, 252, 256

- rigid-body transformations, 108

- Robins, Nate, 9

rotation

- concatenation, 113–114
- defining, 112
- fixed point of, 112
- matrix, 127
- positive direction, 112

S

- samplers, 243–244

sampling

- point, 191
- texel, 243
- texture, 189

- scaling, 114

- blocks of pixels, 178
- length of normals and, 148

- scissoring, 260

- segments
 - curve, 211
 - line, 29–30
 - selection mode
 - defined, 76
 - name stack and, 77
 - shaders
 - compiling, 245–246
 - defined, 235
 - error checking, 247
 - fragment, 236, 237, 238, 250–254
 - linking, 246–247
 - pass-through, 237, 240
 - Phong, 235
 - program objects, 244–245
 - reading, 245–246
 - sending data to, 247–248
 - vertex, 236, 237–238, 249–250
 - shades, 131
 - shading
 - calculation, 143, 145–146
 - flat, 37, 147
 - Gouraud, 148
 - normals and, 229–231
 - parameters, 144
 - physically correct, 132
 - smooth, 37, 146–147
 - tree, 235
 - shadows, 119
 - shininess coefficient, 134
 - silhouette edge, 230
 - silhouette mode, 99
 - smooth shading, 37, 146–147
 - source bits, 162
 - source blending factor, 149
 - source pixels, 162
 - specular reflections, 134
 - spheres, 123
 - approximation, 99
 - texture mapping for, 199–200
 - splines, 220
 - spotlights, 137
 - stacks
 - matrix, 50, 160
 - name, 77
 - state
 - attributes and, 28
 - changes, 22
 - color and, 22
 - defined, 49
 - machine, 3–4
 - queries, 48
 - saving, 49–51
 - variables, 244
 - stencil buffers, 258, 260
 - stencil test, 260
 - stippling
 - defined, 30
 - line, 31
 - polygons, 33–34
 - stroke characters, 46
 - subdivision, 39–46
 - submenus, 67
 - subtractive color model, 20
 - subwindows, 68
 - surfaces
 - applying texture to, 195–196
 - Bézier, 212–213
 - combining texture with, 185
 - computing normals for, 229
 - NURBS, 224, 258
 - parametric, 211–212
 - patch, 232
 - spline, 220
 - texturing, 231–232
 - swizzling, 241–242
 - symbols, 121
 - synthetic camera model, 23
- ## T
- teapot. *See* Utah teapot
 - tessellation
 - after twisting, 45
 - edge flags and, 37–38

- tessellation (*continued*)
 - number of triangles in, 46
 - subdivision and, 39–46
 - tessellator, 38
 - tetrahedron, 102
 - texels
 - array, 188
 - defined, 183
 - sampling, 243
 - text
 - display lists and, 74–75
 - generation, 46
 - texture coordinates, 184, 188–190
 - automatic generation, 199–203
 - current, 188
 - transformation, 190
 - texture mapping, 183–207
 - defined, 183
 - enabling, 186
 - in fragment processing, 190
 - for image manipulation, 204–207
 - sphere, 199–200
 - steps, 184–185
 - textures
 - applying to surfaces, 195–196
 - automatic generation, 232
 - blending and, 195
 - color, 196
 - combining with surface, 185
 - compressed, 256
 - dimensions, 187
 - memory, 187
 - objects, 203–204
 - parameters, 190–192
 - rotating cube, 192–195
 - sampling, 189
 - sizes, 196
 - three-dimensional, 185
 - two-dimensional, 183, 202
 - torus, 101
 - transformations
 - affine, 108
 - concatenation, 110
 - coordinate systems, 25–26, 120
 - defined, 107
 - hierarchical, 121–129
 - homogeneous coordinates, 108–109
 - instance, 121
 - light sources and, 138–140
 - line-preserving, 107–108
 - matrices, 25–26, 109
 - modeling with, 120–129
 - model-view, 25, 109–111
 - projection, 25, 108, 109–111
 - rigid-body, 108
 - rotation, 112–114
 - scaling, 114
 - shear, 121
 - texture coordinates, 190
 - translation, 109–111
 - translation, 109–111
 - translucent, 151
 - transparency, 148–152
 - tree traversal, 125
 - triangles
 - edge flags, 41
 - number in tessellation, 46
 - in three dimensions, 84
 - twists, 43, 45
- U**
- uniform-qualified variables, 239
 - unpacked pixels, 165
 - up vector, 90
 - Utah teapot, 103, 224–229
 - defined, 224
 - one-dimensional grid rendering, 225
 - rendering with auto normals, 230
 - texture mapped, 232
 - twisted, 250
 - two-dimensional grid rendering, 228, 229
- V**
- variables
 - attribute, 248

- attribute-qualified, 240
 - state, 244
 - uniform-qualified, 239
 - varying-qualified, 240
 - varying-qualified variables, 240
 - vectors
 - GLSL types, 238–239
 - per-fragment, 252
 - up, 90
 - versions, 5, 255–257
 - Version 1.1, 255–256
 - Version 1.2, 256
 - Version 1.3, 256–257
 - Version 1.4, 257
 - Version 1.5, 257
 - Version 2.0, 257
 - Version 2.1, 257
 - vertex arrays, 94–96, 255
 - vertex processor, 234
 - vertex shaders, 236
 - examples, 249–250
 - pass-through, 237, 240
 - simple, 237–238
 - See also* shaders
 - vertices
 - colors, 37
 - defined, 17–18
 - processing, 236
 - viewing
 - cubes, 87–88
 - frustum, 86
 - functions, 5
 - isometric, 90
 - oblique, 111
 - orthographic, 87
 - parallel, 86
 - parameters, 104
 - perspective, 103–106
 - rectangle, 23
 - two-dimensional, 23–24
 - volume, 86
 - window, 24
 - viewports
 - defined, 24
 - setting, 24
 - voxels, 183
- W**
- wgl, 14
 - widgets, 65
 - windows
 - callbacks, 69–70
 - clipping, 24, 29, 78
 - context, 68
 - coordinates, 25, 120
 - current, 68
 - multiple, 67–68
 - size, 62, 173, 174
 - Windows, Microsoft, 2, 14
 - wire frames, 87
- X**
- X Window System, 14
- Z**
- z-buffer algorithm, 97